

WQuestions

UNA GRAMÁTICA UNIVERSAL DE LA ARQUITECTURA DE DATOS

JOSÉ ABANTO MARÍN



LA TORRE DE BABEL

Cada industria, su propio lenguaje.
Datos fragmentados, sistemas
incompatibles, conocimiento aislado.

UN LENGUAJE COMÚN

Un estándar universal basado en
la lingüística y la ciencia formal
para conectar datos, sistemas
y personas.

**WQuestions propone un vocabulario universal
para estructurar, conectar y comprender cualquier tipo de información.**



DE LA TEORÍA A LA PRÁCTICA

Casos reales en Python
aplicados a sectores
comerciales y científicos.



UN MODELO ESCALABLE

Diseñado para funcionar
en cualquier dominio,
en cualquier industria,
en cualquier idioma.



EL MOMENTO ES AHORA

Los Modelos de Lenguaje (LLMs)
ofrecen la tecnología ideal
para llevar este lenguaje común
a escala global.

SI PODEMOS PREGUNTARLO, PODEMOS ESTRUCTURARLO.
SI PODEMOS ESTRUCTURARLO, PODEMOS COMPRENDERLO Y TRANSFORMARLO.

Introducción

La intuición que todos compartimos

Hay seis palabras que cualquier niño de cinco años usa como todo un experto: **quién, qué, dónde, cuándo, cómo y por qué**. Las utiliza muchísimo antes de saber qué cosa es un sustantivo o un verbo en la escuela. Las dice cuando todavía confunde el ayer con el mañana, cuando ni siquiera entiende que existen otros países al otro lado del océano, y cuando los números mayores a diez le suenan a puros cuentos de magia. El niño las usa para entender el mundo porque esas preguntas son, literalmente, las piezas con las que nuestro cerebro empieza a armar la realidad para que tenga sentido.

El contraste con la realidad industrial

Ahora, imagínate la siguiente escena en el mundo profesional. Es un martes por la tarde, estás frente a la pantalla revisando un script en Python —pongamos que se llama `leer_sunat.py`— y tu tarea es lograr que el sistema de facturación se comunique limpiamente con una plataforma estatal de salud, como SUSALUD. Tienes la información correcta frente a ti: sabes qué servicio se prestó, a quién y cuándo. El problema es que un sistema exige los datos bajo una estructura rígida y el otro utiliza un vocabulario completamente distinto. Terminas escribiendo algoritmos, diseñando puentes y exportando *datasets* a la medida, solo para que dos bases de datos se pongan de acuerdo sobre un hecho que en la vida real (y para ese niño de cinco años) es increíblemente simple.

Ese desgaste técnico, esa necesidad constante de construir traductores manuales para conectar sistemas que deberían entenderse por simple lógica, no es un gaje del oficio. Es una falla arquitectónica profunda en cómo la industria ha modelado la información durante décadas. Y es, precisamente, la clase de frustración que te obliga a dar un paso atrás y repensar la estructura desde los cimientos.

De qué va este libro

Este libro plantea, defiende y demuestra una sola idea. Es una idea tan simple de enunciar que resulta difícil creer en su potencia hasta que la ves ejecutándose en código: **las preguntas básicas de toda la vida —quién, qué, dónde, cuándo, cuánto, cuál y cómo— son estrictamente suficientes para organizar, con precisión técnica, la información de cualquier industria o dominio en el mundo.**

Quizá notes un pequeño desplazamiento respecto de las seis preguntas con las que abrimos. Es deliberado: al llevar esa intuición a la precisión, *cuánto* y *cuál* se revelan como preguntas propias que el genérico *qué* tenía escondidas, y *por qué* resulta no ser una coordenada más sino una **relación entre hechos** — la causa de algo es siempre otro hecho que lo explica. La intuición de la infancia no se reemplaza: se **afina** hasta volverse siete coordenadas exactas. (Al "por qué" le dedicamos, de hecho, un capítulo entero más adelante.)

No hay trucos ocultos ni capas de complejidad innecesaria. Pero, como ocurre con los verdaderos cambios de paradigma, cuando tomas esta premisa y la llevas hasta sus últimas consecuencias lógicas, el panorama entero cambia. Cambia la forma en que diseñas una base de datos. Cambia la manera en que construyes agentes de inteligencia artificial. Cambia cómo conectas sistemas que hoy son incapaces de hablarse entre sí. Incluso cambia la forma en que le explicas un negocio nuevo a un programador junior.

Lo hemos puesto a prueba. Funciona. Y el propósito de este texto es mostrarte la ingeniería exacta detrás de ese funcionamiento.

Por qué escribir un libro entero

Si la idea central cabe en un solo párrafo, la pregunta es válida: ¿hace falta escribir un libro entero para esto? Sí, por una razón fundamental. En la ingeniería de software, entre **enunciar** una propuesta teórica y **poder usarla** en un entorno de producción hay un abismo.

Para que una arquitectura de este tipo sea adoptada y no se quede en un mero ensayo académico, necesita cinco cosas:

1. **Una intuición clara:** Demostrar que esta idea no es un capricho de diseño, sino que se fundamenta en cómo procesa la información la mente humana, la lingüística y la ciencia formal.
2. **Un modelo formal:** Reglas escritas con precisión matemática, cerrando cualquier margen de ambigüedad para el programador que deba implementarlo.
3. **Una batería de pruebas de estrés:** Aplicar el modelo en dominios reales y complejos —nada de ejemplos de juguete— para que el lector escéptico compruebe que el sistema no colapsa cuando la realidad del negocio se vuelve exigente.
4. **Una conexión con el ecosistema actual:** Las propuestas que ignoran su contexto están condenadas al fracaso. Este modelo debe dialogar de frente con las ontologías existentes, los estándares de la industria y las herramientas modernas de IA.
5. **Una hoja de ruta honesta:** Definir con total transparencia qué problemas están resueltos y cuáles son los desafíos pendientes para que esto se convierta en una infraestructura de uso diario.

Cada uno de estos cinco requerimientos ocupa una sección de este libro. No hay relleno; cada capítulo cumple una función de carga en la estructura general.

Qué vas a encontrar en estas páginas

A medida que avancemos, vas a ir construyendo junto conmigo una arquitectura llamada WQuestions. Piensa en este modelo no como otra base de datos rígida, sino como un protocolo de normalización semántica. Lo que hace este motor es tomar las preguntas cotidianas y transformarlas en los ejes formales de un espacio geométrico.

Es un mapa de coordenadas para el conocimiento: las primeras seis preguntas (quién, qué, dónde, cuándo, cuánto y cuál) actúan como las dimensiones que fijan la ubicación exacta de cualquier hecho en el universo. La séptima coordenada (cómo) define la naturaleza o el modo de esa existencia, mientras que el por qué —como veremos en detalle— opera como la arista o el vector de conectividad que enlaza un hecho con otro. De este modo, cualquier evento del mundo real, por complejo o específico que sea, encuentra una ubicación matemática precisa y una relación lógica inequívoca dentro de esta estructura.

Al plantearlo como un estándar de interoperabilidad y no como una taxonomía impositiva, el panorama entero cambia. Cambia la forma en que diseñas una base de datos porque dejas de modelar tablas aisladas y empiezas a registrar flujos universales. Cambia la manera en que construyes agentes de inteligencia artificial, ya que dejas de entrenarlos en dialectos corporativos privados. Cambia cómo conectas sistemas que hoy son incapaces de hablarse entre sí. Incluso cambia la forma en que le explicas un negocio nuevo a un programador junior.

Y este libro **no se detiene en la teoría**. Va hasta el final del proceso de ingeniería:

- Define las reglas del modelo con rigor (utilizando notación de teoría de conjuntos cuando la precisión lo exige, y prosa clara cuando es suficiente).
- Somete la arquitectura a prueba en **ocho dominios industriales completos** —un spa comercial, un servicio de taxis de alta concurrencia, una historia clínica, un banco regional, un ERP multi-módulo, una universidad, una municipalidad y una operación minera— y, además, la enfrenta a **cuatro escenarios de alto estrés técnico**: composición musical, reacciones químicas, análisis de partidos de fútbol y la redacción de contratos legales.

- Se respalda en un **prototipo funcional escrito en Python**, garantizando que lo que se afirma en el texto es ejecutable: los ocho dominios pasan sus validaciones y los tests corren sin errores.
- Conecta la arquitectura con doce teorías y estándares fundamentales de la informática y la lingüística, asegurando que el modelo se integre al estado del arte y no nazca como un sistema aislado.

Por qué este libro aparece precisamente ahora

Si hubiéramos propuesto esta arquitectura hace cinco años, su adopción habría sido extremadamente difícil. Faltaba una pieza clave para que el sistema fuera escalable: una capa de inteligencia capaz de traducir con fluidez entre el modelo de datos estructurado y el lenguaje natural de los usuarios.

Hoy, esa pieza de hardware semántico **ya existe**: son los Grandes Modelos de Lenguaje (LLMs). La capacidad actual que tienen estos modelos para ejecutar herramientas de forma autónoma (*function calling*) y mantener flujos de trabajo dinámicos (*agentic workflows*) ha reescrito las reglas de la integración. Cuando un agente de IA actual necesita extraer datos de un servidor hospitalario o registrar una venta, los "cables" ya están tendidos: domina los formatos JSON, valida esquemas y entiende los tipos de datos.

Lo que le sigue faltando desesperadamente a la industria es un **vocabulario común** que viaje a través de esos cables. Actualmente, cada API expone sus datos en su propio dialecto privado. Como resultado, un agente de IA tiene que consumir tokens y tiempo de procesamiento intentando aprender un idioma distinto por cada herramienta a la que se conecta.

Si todas las aplicaciones de internet estructuraran su información basándose en los ejes de las preguntas fundamentales, un agente solo tendría que aprender a comunicarse **una sola vez**, obteniendo la capacidad inmediata de entender y operar cualquier sistema del planeta.

La tesis de este libro es que las preguntas son, por naturaleza, ese idioma universal. Y el momento tecnológico para implementar este estándar es ahora.

Para quién está escrito este libro

Diseñé este texto apuntando a tres perfiles de lectores simultáneos. Si el balance es el correcto, cada uno encontrará valor de alto impacto en estas páginas:

Para el lector con curiosidad estructural: Si te fascina entender cómo la inteligencia artificial intenta mapear el conocimiento humano, este texto te obligará a mirar con rigor analítico algo que solemos dar por sentado: la naturaleza de las preguntas. Cuando te detienes a analizarlas, descubres que la forma en que decidimos organizar la información altera radicalmente cómo aprendemos, cómo razonamos y, en última instancia, cómo diseñamos máquinas que pretenden imitar nuestra cognición.

Para el profesional técnico: Si tu trabajo diario es ser arquitecto de software, ingeniero de datos o desarrollador de IA, aquí tienes un *framework* concreto, testeado y desmenuzado a un nivel de detalle listo para ser implementado. Encontrarás comparativas directas con las tecnologías de integración actuales, esquemas de modelado en ocho industrias diferentes, decisiones de diseño justificadas y un diccionario base (*lexicon*) para arrancar tus propios proyectos.

Para el investigador o académico: Si tu campo de estudio es la semántica, las bases de datos o la lingüística computacional, notarás que cada propuesta está fuertemente anclada en la literatura científica. Aquí no reinventamos la rueda a escondidas: cada concepto se contrasta y se referencia con el trabajo de autores como Barwise y Perry [11], Davidson [12], Gärdenfors [13], Yang y Hu [9], y Mahmood [10]. Además, dialogamos directamente con estándares consolidados como CIDOC CRM [4], FrameNet [14], VerbNet [15], Universal

Schema [16], Biolink Model [5], Snodgrass [17] y QUDT [18]. Todo concepto heredado tiene su cita correspondiente, y toda divergencia tecnológica está argumentada lógicamente.

Para optimizar la lectura, la estructura del libro está **diseñada por capas**. Cada capítulo inicia planteando un escenario o analogía de fácil comprensión, y aumenta progresivamente su densidad técnica. Si solo necesitas la visión estratégica, el primer tercio de cada capítulo es suficiente. Si buscas el sustento matemático, los apéndices están a tu disposición. Y si tu objetivo es ver código y modelado aplicado, tu lugar de enfoque debe ser la Parte V.

El recorrido

El desarrollo del modelo se divide en seis bloques principales:

- **Parte I (Capítulo 1) — El problema:** Definimos, sin rodeos, el problema industrial que justifica todo el esfuerzo: la torre de Babel de los datos, donde la información existe pero está atrapada en esquemas incompatibles que no saben hablar entre sí. Y planteamos la apuesta del libro: que la solución es anterior y más simple que cualquier diseño de base de datos.
- **Parte II (Capítulos 2 al 6) — Las siete coordenadas:** Diseccionamos las siete coordenadas —quién, qué, dónde, cuándo, cuánto, cuál y cómo—, agrupadas por capítulo (los cuatro pilares van juntos en el primero), analizando la carga semántica de cada pregunta, las trampas lógicas al programarlas y las reglas para modelarlas limpiamente. Aquí también desmontamos por qué los intentos previos de la industria (5W1H, web semántica, ontologías) se quedaron a medias, y cerramos el bloque mostrando por qué son *estas* preguntas, y no otras: la convergencia independiente de la filosofía, el derecho, el periodismo y la lingüística.
- **Parte III (Capítulos 7 al 10) — Cómo funcionan juntas:** Pasamos al ensamblaje. Explicamos cómo un evento del mundo real se transforma en la unidad atómica de nuestra base de datos, cómo la intersección de estos ejes crea una geometría de datos coherente, y cómo se vinculan los eventos entre sí (incluyendo el complejo modelado de la causalidad o el "por qué").
- **Parte IV (Capítulos 11 al 13) — Del lenguaje a los hechos:** Construimos el puente entre la ambigüedad del lenguaje humano y la rigidez de los datos estructurados. Abordamos la función crítica de los verbos, el diseño del diccionario operativo del sistema (*lexicon*) y la resolución técnica de problemas lingüísticos severos (polisemia, frases compuestas y fricciones entre idiomas).
- **Parte V (Capítulos 14 al 23) — En la práctica:** Entramos a la sala de máquinas. Ejecutamos el modelado completo de ocho dominios industriales —spa, taxi, clínica, banca, ERP, universidad, municipalidad, minera— y enfrentamos el modelo a cuatro escenarios de alto estrés técnico (música, química, fútbol y redacción de contratos). Esta es la sección donde la teoría se convierte en código.
- **Parte VI (Capítulos 24 al 29) — IA, el futuro y el cierre:** Proyectamos cómo la arquitectura WQuestions potencia a los LLMs mediante *function calling*, exploramos las aplicaciones comerciales que se desbloquean al unirlos, sometemos el modelo a su prueba más exigente —describirse a sí mismo en una aplicación hecha de puras preguntas—, confrontamos la otra cara del grafo compartido —la seguridad y la privacidad cuando todo habla el mismo idioma—, definimos los retos pendientes para convertir esto en un estándar de infraestructura, y cerramos el ciclo volviendo al problema original.

Una nota metodológica

La creación de este libro siguió un proceso iterativo muy particular. Esta arquitectura no se diseñó en el vacío sobre una pizarra para luego forzar a los datos a encajar en ella. Todo lo contrario: el modelo se construyó de manera empírica, dominio tras dominio, chocando contra las fricciones reales que cada modelo de negocio exigía resolver. Ninguna regla de diseño se incluyó por puro amor a la elegancia teórica; cada decisión técnica está ahí porque un caso de uso real demandó su existencia.

Aunque suene paradójico, ese proceso empírico es la mayor garantía de robustez del sistema. La arquitectura actual emergió en el momento exacto en que comprendimos que **ocho industrias que no tenían relación alguna**

entre sí estaban demandando exactamente la misma solución estructural, solo que disfrazada con vocabularios distintos. Cuando ocho problemas independientes convergen matemáticamente en la misma respuesta, esa respuesta deja de ser una simple hipótesis.

Aprovechando este punto, conviene aclarar el tono de la obra: este libro **no peca de falsa modestia ni de tibieza**. La arquitectura se expone con la contundencia de quien la ha sometido a prueba. Del mismo modo, sus vulnerabilidades y límites se declaran de frente, porque ocultar una falla en ingeniería es una irresponsabilidad. Donde encontremos un problema que la arquitectura aún no resuelve de forma óptima, lo leerás sin rodeos. Y donde la solución demostró ser superior a los estándares actuales, la defenderemos con argumentos técnicos, sin diplomacias innecesarias.

Empecemos.

Capítulo 1 — La torre de Babel de los datos

Una sala de emergencias a las dos de la mañana

Para entender la magnitud del problema arquitectónico que enfrentamos hoy en la industria de los datos, no necesitamos mirar un servidor corporativo; basta con mirar una sala de emergencias a las dos de la mañana.

Un médico de guardia recibe a una paciente de cuarenta y dos años que ingresa con un dolor torácico agudo. Está consciente, pero experimenta sudoración fría y dificultad para respirar. En esos minutos críticos, el médico necesita acceder a tres piezas de información para tomar una decisión de vida o muerte: qué medicación de uso diario consume, si tiene alergias documentadas y si existe un historial reciente de episodios similares. Sin embargo, al introducir el nombre de la paciente en el sistema del hospital, la pantalla solo devuelve una ficha de tres líneas indicando que fue atendida por una bronquitis hace cuatro años.

La información vital que el médico busca no es un misterio; de hecho, está perfectamente documentada. Esta paciente tiene una endocrinóloga privada que gestiona su historial desde hace siete años; fue evaluada por un cardiólogo en otra ciudad hace unos meses; y ya pasó por una emergencia similar donde le realizaron un electrocardiograma completo. Los datos existen, están almacenados en discos duros, y ninguna de estas clínicas tiene la intención de ocultarlos.

El problema es puramente estructural: los datos están atrapados en esquemas incompatibles.

La endocrinóloga utiliza un software que guarda el diagnóstico bajo la variable `diagnostico_principal`. El sistema del cardiólogo, desarrollado por otra empresa, lo registra como `dx_p`. Cuando miramos los medicamentos, la primera clínica utiliza una estructura plana llamada `medicacion_actual` separando `nombre`, `dosis` y `frecuencia`. La segunda clínica, en cambio, diseñó una base de datos relacional con una tabla de `prescripciones` que requiere un identificador numérico (`farmaco_id`), los miligramos por toma (`mg_por_toma`) y las tomas diarias (`tomas_dia`).

Si eres ingeniero de software o analista de datos, sabes que conectar estas dos bases de datos no es ciencia ficción. Es posible, pero es un trabajo artesanal, costoso y sumamente frágil. Tienes que programar un puente de código (un *middleware*), mapear manualmente qué columna equivale a qué columna, y escribir excepciones para cada formato de fecha o número. Y lo peor de esta solución parche es su caducidad: el día en que la clínica actualice su sistema y cambie un solo nombre en su base de datos, tu puente de código colapsará silenciosamente.

Multiplica este pequeño esfuerzo técnico por miles de hospitales, decenas de proveedores de software y décadas de historiales clínicos. La integración de datos deja de ser un reto técnico y se revela como un problema de diseño a nivel mundial. La información existe, pero matemáticamente no podemos razonar con ella.

El mismo hecho, cuatro idiomas distintos

Para visualizar exactamente qué ocurre dentro de las bases de datos cuando ocurre un evento, analicemos algo mucho más cotidiano: *el instante exacto en que una persona compra una camiseta en una tienda física*.

Es un único hecho en la realidad. Sin embargo, si nos asomamos a los servidores de la empresa que vende la camiseta, descubriremos que sus distintos departamentos registran ese mismo segundo en el tiempo utilizando cuatro lenguajes radicalmente distintos:

En el sistema del **punto de venta (la caja registradora)**, la prioridad es registrar la transacción comercial y los actores involucrados. Por eso, el hecho se guarda como una fila en una base de datos relacional:

```
INSERT INTO ventas
(id, cliente_id, producto_id, monto, fecha, vendedor_id)
VALUES (74921, 1042, 88, 49.90, '2026-05-14', 17);
```

Para el sistema de **contabilidad**, la identidad del vendedor o el cliente es irrelevante. Su objetivo es mantener el equilibrio financiero de la empresa mediante el método de partida doble. Ese mismo evento se descompone así:

```
asiento: 2026-05-14
debe:  cuentas_por_cobrar      49.90
haber: ventas_brutas          42.29
haber: impuesto_al_consumo     7.61
```

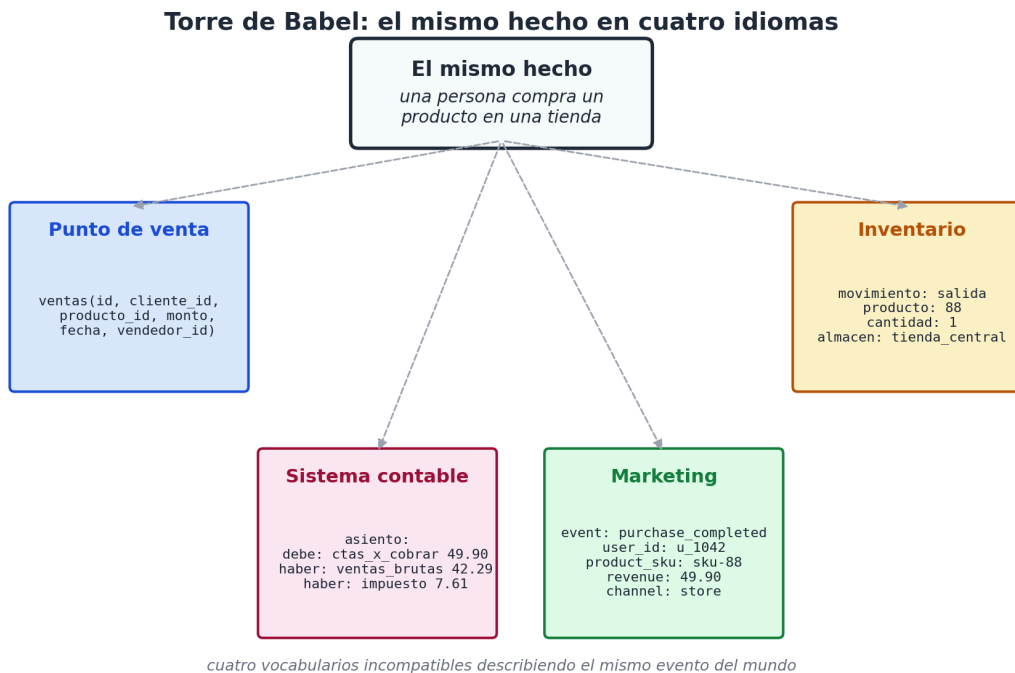
Si saltamos al departamento de **marketing**, su obsesión es el comportamiento del usuario y el origen de la compra para medir el éxito de sus campañas. Ellos estructuran la información como un evento de rastreo:

```
evento: compra_completada
id_usuario: u_1042
sku_producto: sku-88
ingresos: 49.90
canal: tienda_física
id_sesión: s_abcdef
```

Finalmente, para el sistema de **inventario o almacén**, lo único que importa es la logística: un objeto físico acaba de abandonar las instalaciones. El evento se reduce a un simple movimiento de existencias:

```
movimiento: salida
producto: 88
cantidad: 1
almacen: tienda_central
ref: vta-74921
```

Observa la desconexión. Tenemos cuatro departamentos operando con cuatro estructuras diferentes. Cada esquema es perfecto y coherente para el equipo que lo diseñó. El problema surge cuando la dirección de la empresa pide un análisis global. Si le entregamos estas cuatro piezas de código a una computadora, el sistema no tiene ninguna forma obvia de saber que todas están hablando del mismo suceso. El sistema contable no sabe quién vendió la camisa; marketing no sabe de qué almacén salió; y la caja registradora no sabe cómo separar los impuestos. Cada software capturó solo la "sombra" del evento que le interesaba y la guardó en su dialecto privado.



El mismo hecho del mundo registrado en cuatro vocabularios incompatibles, uno por cada sistema que lo procesa.

A esta fragmentación masiva es a lo que en este libro llamaremos **la torre de Babel de las ontologías**. En la ciencia de datos, la palabra *ontología* no es un concepto filosófico inalcanzable; es simplemente el catálogo oficial de conceptos que un sistema reconoce (como "cliente", "factura" o "producto") y las reglas lógicas que dictan cómo se conectan entre sí. El gran error de la industria tecnológica ha sido permitir que cada departamento, empresa y sector científico construya su propia ontología cerrada, aislándose sistemáticamente del resto del mundo.

La causa de todo el problema

La raíz estructural de esta torre de Babel —y el punto de partida de este libro— es que **hemos permitido que cada sector invente su propia forma de entender la realidad desde cero**.

La ingeniería médica no hereda ninguna estructura de la contabilidad comercial; el comercio no comparte bases con el derecho penal; y la educación diseña bases de datos que no se hablan con el urbanismo. Cada industria construye su pirámide desde la base utilizando planos distintos, y luego nos sorprendemos cuando resulta carísimo contratar programadores para tender puentes colgantes entre las cimas de esas pirámides.

¿Qué pasaría si la solución a este caos fuera anterior y mucho más simple que cualquier diseño de base de datos? ¿Qué pasaría si, mucho antes de que existieran los hospitales, los bancos o el comercio internacional, la humanidad ya hubiera desarrollado un esquema cognitivo universal que nadie puede evitar utilizar?

La apuesta de este libro

Regresemos a la compra de la camiseta. Si detuviéramos a cualquier persona en la calle —sin conocimientos de bases de datos ni de código— y le pidiéramos describir ese hecho, su respuesta natural sería algo muy parecido a esto:

*"Una clienta (**quién**) compró (**qué**) una camiseta (**qué**) en la tienda del centro (**dónde**) esta tarde (**cuándo**) por casi cincuenta dólares (**cuánto**)."*

Sin el menor esfuerzo, cualquier ser humano produce una descripción **estructurada, completa y combinable**. Nuestra mente no percibe el mundo como un bloque de texto ni como tablas de Excel: descompone automáticamente cualquier evento con un conjunto muy reducido de preguntas fundamentales.

Esa es la apuesta de este libro:

Existe un conjunto reducido y estable de preguntas primarias —siete— con capacidad descriptiva suficiente para modelar cualquier hecho factual, en cualquier dominio industrial y en cualquier idioma. Quién, qué, dónde, cuándo, cuánto, cuál y cómo. Al diseñar las bases de datos alrededor de estas coordenadas cognitivas —en lugar de las terminologías de turno de cada industria— se neutraliza de raíz la fragmentación semántica, y la torre de Babel se desmorona desde los cimientos. Como bono, cualquier inteligencia artificial entrenada con lenguaje humano entiende esta estructura de fábrica, porque es la misma con la que aprendió a leer.

No la tomes como un acto de fe. Estas preguntas no son un invento moderno: filósofos griegos, juristas romanos, escuelas de periodismo y hasta los niños que aprenden a hablar convergen, sin coordinarse y separados por siglos, en la misma lista. A esa convergencia —la prueba de *por qué estas preguntas y no otras*— le dedicamos un capítulo entero más adelante (Capítulo 6). Pero no hace falta esperar a la demostración para empezar a construir.

Lo que sigue es la ingeniería: diseccionar, una por una, las siete coordenadas operativas sobre las que se levanta el modelo. Empecemos por las cuatro más concretas.

Capítulo 2 — Quién, qué, dónde, cuándo: los cuatro pilares

Una oración sencilla

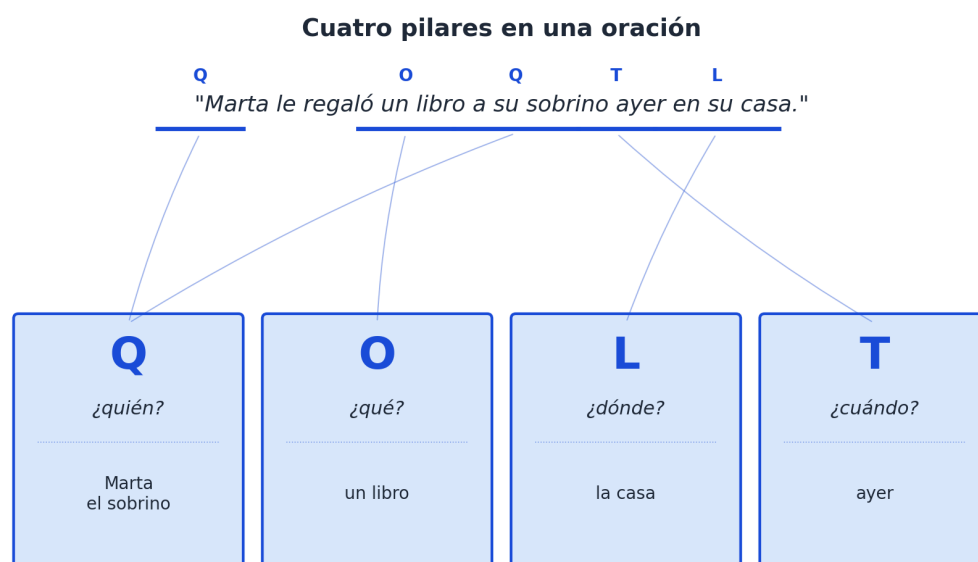
Empecemos este análisis por la oración más inocente y cotidiana que se nos pueda ocurrir:

Marta le regaló un libro a su sobrino ayer en su casa.

Cualquier persona que hable español entiende esta frase sin hacer el menor esfuerzo consciente. Sin embargo, esa comprensión aparentemente simple esconde un trabajo de procesamiento mental extraordinario. Al leerla, nuestro cerebro identifica de manera automática y en una fracción de segundo cuatro elementos totalmente distintos: **una persona que ejecuta la acción** (Marta), **una cosa que cambia de manos** (un libro, que involucra a un receptor, el sobrino), **un lugar físico donde todo ocurre** (la casa) y **un momento específico en el tiempo** (ayer).

No solo identificamos estas piezas al vuelo, sino que las encajamos en su lugar exacto. Nadie con un nivel básico de comprensión lectora confundiría la casa con el libro, ni pensaría que "ayer" es el nombre de la persona que recibe el regalo.

Esa capacidad de descomposición automática que tiene nuestra mente es, literalmente, la materia prima sobre la cual se construye el modelo de este libro. Esas cuatro preguntas que nuestro cerebro acaba de responder sin que nos demos cuenta —*quién, qué, dónde, cuándo*— conforman los **cuatro pilares**. Son las dimensiones fundamentales que aparecen en cualquier descripción de un hecho real, sin importar el idioma que hablemos o el nivel técnico del tema. Si falta alguna de estas piezas, sentimos intuitivamente que la historia está incompleta. Si las cuatro están presentes, tenemos, como mínimo, el esqueleto funcional de un hecho.



La mente identifica los pilares sin esfuerzo. El modelo solo los hace explícitos.

Cómo los cuatro pilares descomponen la oración. Cada segmento del enunciado encuentra su lugar en uno de los cuatro ejes de valor concretos.

En este capítulo vamos a recorrer estos pilares uno por uno. Todavía no los vamos a tratar como ejes matemáticos formales —eso lo haremos más adelante cuando toquemos la arquitectura—, sino que vamos a analizarlos como **preguntas con personalidad propia**. Cada una de ellas esconde rarezas, trampas de diseño y convenciones que solemos pasar por alto. Entender a fondo cómo se comportan ahora nos va a ahorrar muchísimos dolores de cabeza cuando empecemos a modelar bases de datos.

Q — Quién: la pregunta por el agente o actor

A simple vista, la pregunta "¿quién?" parece la más obvia e inofensiva de las cuatro. Queremos saber quién hizo algo, quién recibió la acción o sobre quién recae la responsabilidad. La respuesta estándar y canónica que esperamos encontrar aquí es **un agente capaz de ejecutar una acción**: una persona individual, un grupo de personas o una organización formal.

Si lo cruzamos con nuestros cuatro ejemplos de prueba, el resultado es bastante plano: - *Receta*: ¿quién prepara el risotto? El cocinero. - *Gol*: ¿quién marca? El delantero (y si queremos más detalle, quién da la asistencia). - *Canción*: ¿quién compone? El autor o el músico. - *Noticia política*: ¿quién anuncia la nueva medida? El ministro de turno.

Hasta aquí no hay mayor fricción. Pero esta pregunta empieza a volverse muy interesante apenas la sacamos de su zona de confort y la enfrentamos a la realidad de los datos.

Pensemos en un partido de fútbol. ¿Quién marca el gol cuando el balón rebota accidentalmente en un defensor rival y entra al arco? Las estadísticas oficiales de los torneos hablan de un "gol en contra" y, para efectos de la tabla de goleadores, le adjudican el tanto al delantero que pateó originalmente, no al defensor que desvió el balón. Entonces, a nivel de datos, ¿quién es el agente real de ese evento? El reglamento deportivo toma una decisión, por supuesto, pero estructurar esa pregunta en un sistema informático no es algo trivial.

O miremos la industria musical. ¿Quién compone exactamente una canción cuando un autor escribe la letra en su casa y otro músico compone la melodía en un estudio meses después? La industria tiene muy clara la distinción legal entre *letrista* y *compositor*, pero muchísimos sistemas de software simplemente meten a ambos en un campo genérico llamado `compositor`. Cuando una base de datos intenta compartir esa información con otra, si no sabemos qué convención interna usó cada una, los datos chocan.

En el caso de una receta clásica de fideos con tuco, la situación temporal lo complica todo: ¿quién "prepara" el plato si la receta original la escribió la abuela hace cincuenta años, pero la salsa la está cocinando el nieto esta noche? Existe una distinción fundamental entre el *autor* (quien diseñó el conocimiento) y el *ejecutor* (quien materializa la acción hoy). Ambas personas son respuestas totalmente legítimas a la pregunta "¿quién?", pero pertenecen a momentos y naturalezas de acción distintas.

Y aquí aparece un caso límite que merece mención explícita y cuidadosa. ¿Qué ocurre con el "quién" cuando el que ejecuta la acción no es una persona ni una empresa, sino un objeto inanimado? Un horno inteligente que se enciende solo al alcanzar cierta temperatura, el sistema VAR que interrumpe un partido para anular un gol, o un algoritmo de Spotify que decide sugerirte una canción. En nuestra forma natural de hablar, les otorgamos agencia sin pensarlo: *el horno se encendió, el VAR anuló la jugada, el algoritmo decidió*.

Sin embargo, a nivel de arquitectura de la información, estos objetos no pertenecen naturalmente al eje *quién*; su lugar de origen es el eje *qué*, ya que son sistemas o cosas. Lo que sucede aquí es que, **en situaciones muy específicas, un objeto asume temporalmente la capacidad de agencia**.

Esta observación aparentemente menor tiene un peso enorme en el diseño de software. La idea central es que la "agencia" (la capacidad de hacer algo) no es una propiedad estática que un objeto tiene siempre, sino una **propiedad puramente contextual**. Dependiendo de la situación, un objeto pasivo puede "vestirse" de agente. A

este principio lo llamaremos "agencia contextual", y será una de nuestras decisiones de diseño más fuertes cuando lleguemos a la Parte III.

O — Qué: la pregunta por la cosa

La pregunta "¿qué?" es profundamente engañosa porque en la gramática parece exigir una sola respuesta concreta, pero en la práctica termina abriendo la puerta a entidades completamente distintas. Si preguntas "¿Qué pasó?", esperas que te cuenten un evento. Si preguntas "¿Qué le regaló?", buscas un objeto material. Si preguntas "¿Qué situación atraviesa el equipo?", estás pidiendo un contexto complejo. Las tres preguntas son gramaticalmente correctas, las tres aterrizan en el eje *qué*, pero se refieren a estructuras de datos que no se parecen en nada.

Para mantener la cordura en nuestro modelo, adoptaremos un criterio funcional y amplio: **el eje *qué* va a alojar absolutamente todo lo que no sea un agente, un lugar o un momento temporal**. Y al hacerlo, debemos reconocer que dentro de este gran contenedor conviven al menos tres familias muy diferenciadas.

La primera familia son los **objetos**. Hablamos de cosas tangibles o intangibles que existen por sí mismas, que persisten a lo largo del tiempo y que pueden ser mencionadas en muchas situaciones distintas. El libro que regaló Marta, el balón oficial del partido, el documento en PDF donde se redactó el decreto, o el papel manchado donde está anotada la receta.

La segunda familia son los **eventos**. Estos son hechos que ocurren, que tienen un punto de inicio y un punto de finalización claros, y que suelen ser la materia prima de las noticias. El remate al arco, el anuncio oficial en el micrófono, el acto de estampar la firma en el decreto, o la sesión de grabación del disco.

La tercera familia son las **situaciones**. Se trata de marcos de referencia mucho más amplios y duraderos, dentro de los cuales ocurren múltiples eventos pequeños que están interconectados. El partido de fútbol en su totalidad, la conferencia de prensa completa, o la cena de aniversario.

Aquí hay una intuición de diseño muy sutil pero crítica: estas tres familias no están separadas por muros impenetrables. Un evento puede transformarse en un objeto cuando otro dato hace referencia a él (por ejemplo, cuando decimos "**el gol** fue revisado minuciosamente por el VAR"). Del mismo modo, una situación gigante puede ser vista como un evento simple si la miramos desde un contexto aún mayor ("**el partido** fue suspendido a causa de la lluvia torrencial"). El eje *qué* tiene una flexibilidad inherente respecto a la escala o la granularidad: lo que en un nivel de la base de datos es un evento activo, en un nivel superior es simplemente un objeto pasivo al que hacemos referencia.

Esa flexibilidad es una ventaja arquitectónica masiva. Nos permite tratar de forma uniforme cosas que la programación tradicional suele obligar a separar en tablas de bases de datos distintas y difíciles de cruzar. Pero, cuidado, también es una trampa mortal para los ingenieros de datos: hay que ser muy precisos para no perder la riqueza de un evento complejo al reducirlo a un simple objeto de texto, o viceversa. En la Parte III vamos a desmenuzar este problema bajo el concepto formal de **reificación**.

Por el momento, nos basta con asimilar esta intuición operativa usando nuestros dominios: - *Receta*: el texto de la receta es el objeto; el acto físico de cocinar es el evento; la cena familiar donde se come es la situación. - *Gol*: el balón es el objeto; el disparo a portería es el evento; los noventa minutos de partido son la situación. - *Canción*: la partitura musical es el objeto; la interpretación del cantante en la cabina es el evento; la gira de conciertos es la situación. - *Noticia política*: el papel del decreto es el objeto; la rúbrica del presidente es el evento; el mandato presidencial de cuatro años es la situación.

En cada caso, usamos el mismo eje *qué* tres veces, pero operando en tres registros distintos. El modelo no se confunde ni se rompe; simplemente aloja la información en su nivel correspondiente.

L — Dónde: la pregunta por el lugar

La pregunta "¿dónde?" suele ser la más física y concreta de los cuatro pilares, y precisamente por esa aparente simplicidad es la que los desarrolladores dan por sentada con mayor frecuencia. Sin embargo, detrás de ella se esconde una decisión semántica que, si no se toma de forma explícita, arruina los cruces de información.

En el uso diario del lenguaje, utilizamos dos tipos de "dónde" completamente distintos. El primero es el **lugar físico estricto**: una coordenada en un mapa, una dirección postal, una ciudad o un recinto. La casa de Marta, las gradas del estadio, la sala de grabación B, o el palacio presidencial.

El segundo es el **lugar organizacional o administrativo**: un departamento corporativo, un ministerio del gobierno, una sucursal de una franquicia, o un club deportivo. Hablamos de la selección nacional, el Ministerio de Salud, o un sello discográfico específico.

Ambos conceptos responden con total naturalidad a la pregunta "¿dónde?", pero tecnológicamente apuntan a realidades estructuralmente distintas. Una ciudad es un polígono geográfico; un ministerio es un organigrama de personas e intenciones. Si cometemos el error de mezclarlos en el diseño —si guardamos Lima y Ministerio de Salud indiscriminadamente en una misma columna genérica llamada `ubicacion`— perdemos para siempre la capacidad matemática de separar dónde está ocurriendo algo físicamente y a qué estructura administrativa pertenece.

La regla que vamos a adoptar para evitar este caos es **explicitar siempre la naturaleza** del lugar. El eje *dónde* se reservará de manera estricta para los lugares físicos. Los lugares organizacionales, cuando estén ejecutando una acción, se moverán al eje *quién* (actuando como agentes); y cuando funcionen meramente como contenedores administrativos, podrán aparecer en el eje *dónde*, pero irán etiquetados obligatoriamente con una marca de "organización". Esta distinción teórica tomará mucho más sentido cuando veamos el código aplicado; por ahora, es vital tenerla mapeada.

Veamos cómo conviven en nuestros cuatro ejemplos: - *Receta*: ¿Dónde se prepara físicamente? En la cocina. ¿Dónde se sirve? En la mesa del comedor. ¿Dónde tiene su origen histórico? En una región extensa como Sicilia o Yucatán. Tenemos tres "dóndes" con escalas físicas totalmente distintas conviviendo sin problemas. - *Gol*: ¿Dónde se produjo el impacto del remate? A cinco metros fuera del área penal. ¿Dónde se jugó el partido? En un estadio específico, dentro de una ciudad específica. De nuevo, convivencias de escalas. - *Canción*: ¿Dónde se grabaron las pistas? En un estudio de Londres. ¿Dónde se escribió la letra? A veces es un dato histórico conocido, a veces es un misterio. - *Noticia política*: ¿Dónde se firmó el documento? En el despacho presidencial (ubicación física). ¿Dónde tiene jurisdicción la ley? En todo el territorio nacional (ubicación política).

Vale la pena dejar asentada una observación fundamental: el pilar "dónde" está diseñado por naturaleza para soportar jerarquías o anidamientos. La cocina está dentro de la casa, la casa pertenece a un barrio, el barrio está contenido en la ciudad, y la ciudad en el país. A nivel lógico, todas esas respuestas son matemáticamente correctas para la pregunta "¿dónde ocurrió?" aplicada a un mismo hecho. Todo dependerá del nivel de "zoom" que requiera la consulta del usuario. El modelo que estamos construyendo tendrá que ser capaz de navegar por esas cuatro escalas hacia arriba o hacia abajo sin generar ambigüedades.

T — Cuándo: la pregunta por el tiempo

Llegamos al último de los cuatro pilares base, el cual solemos tratar con mucha ligereza en informática asumiendo que se soluciona con un simple formato de fecha (YYYY-MM-DD). La realidad es que el "cuándo" resulta ser el eje más rico y lleno de sutilezas lógicas de todo el modelo. El lenguaje natural tiene muchísimas formas de expresar el tiempo, y las fechas de calendario son apenas una de ellas.

Volvamos a la oración con la que abrimos el capítulo: *Marta le regaló un libro a su sobrino ayer en su casa*. La respuesta al "cuándo" es la palabra *ayer*. Como arquitectos de datos, ¿cómo abordamos esto? Si el sistema simplemente guarda el dato frío como `2026-05-13`, estamos destruyendo información valiosa: perdemos el contexto de que la acción ocurrió "ayer" en relación al momento exacto en que la persona habló. Por otro lado, si guardamos literalmente la cadena de texto "ayer", el motor de la base de datos se vuelve inútil, porque no puede realizar cálculos de antigüedad ni ordenar los eventos cronológicamente. La salida técnica razonable es guardar la fecha absoluta computacional y, en una capa complementaria, conservar el adverbio relativo ("ayer") como contexto asociado.

Pero el problema se vuelve más profundo cuando salimos del calendario estándar. En la notación musical, una nota tiene una duración de "una negra" o "una corchea"; el tiempo aquí no lo dicta un reloj de cuarzo, es un **tiempo musical** interno que se mide en compases y ritmo. En el análisis de una obra literaria o cinematográfica, decimos que una muerte ocurre "al final del segundo acto"; estamos frente a un **tiempo narrativo**, medido por la posición estructural dentro de un relato. En un entorno hospitalario, la ficha indica que el paciente sufrió una crisis "tres horas después de la ingesta del medicamento"; este es un **tiempo relativo anclado a un evento previo**. En el ámbito jurídico, un decreto entra en vigencia "a los treinta días hábiles de su publicación en el boletín oficial"; nos topamos con un **tiempo derivado por reglas legales**.

Si prestamos atención al uso natural de la información, descubrimos que la pregunta "cuándo" exige soportar, como mínimo, cinco tipos de tiempo distintos:

1. **Tiempo absoluto:** Las fechas y horas exactas atadas al calendario gregoriano.
2. **Tiempo relativo:** Expresiones relacionales como "antes de", "durante" o "inmediatamente después de" otro hecho comprobable.
3. **Tiempo de reloj corto:** Marcadores temporales internos de un evento cerrado (por ejemplo, el minuto 87 del partido de fútbol).
4. **Tiempo cíclico:** Patrones de repetición (todos los lunes hábiles, cada inicio de mes).
5. **Tiempo no-reloj:** Escalas abstractas como compases de una partitura, el número de página de un libro, o el paso número cuatro de un manual de instrucciones.

Cualquier eje *cuándo* robusto tiene que estar preparado para recibir estos cinco formatos y, más importante aún, tener la inteligencia para reconocer cuál de ellos se está utilizando en cada transacción. La estrategia que adoptaremos formalmente más adelante es la **pluralidad de tiempos**. No intentaremos forzar toda la información a pasar por un único reloj universal. En su lugar, el eje funcionará como un espacio donde conviven múltiples escalas temporales, todas válidas, operando cada una bajo su propio sistema de coordenadas.

Esa pluralidad plantea de inmediato una pregunta operativa: si conviven varios relojes, ¿cómo ordena el sistema los eventos sin aplastarlos a uno solo? La regla de normalización es deliberadamente humilde: **no traducir** el compás 17 a un horario de cuarzo —eso destruiría su naturaleza—, sino reificar cada instante no-reloj con tres datos estándar:

- `escala` — a qué sistema temporal pertenece (`musical`, `narrativo`, `paginado`, `reloj_absoluto`). Impide comparar peras con manzanas: el motor nunca ordena un compás contra una fecha por accidente.
- `posicion` — un ordinal numérico que lo vuelve **ordenable dentro de su propia escala** (compás 17 → 17 ; página 240 → 240 ; "final del segundo acto" → acto 2, 1.0). Es la clave de orden.
- `valor_nativo` — la expresión original, preservada tal cual ("compás 17", "p. 240", "final del segundo acto"), para no perder fidelidad.

Con esto, ordenar es trivial *dentro* de una escala (se ordena por `posicion` filtrando por `escala`) y honesto *entre* escalas (no se mezclan, salvo que exista un puente real). Y los puentes existen cuando el mundo los provee: la grabación de un concierto ata el `compás 17` (escala musical) a las `20:03:14` (escala absoluta) con un hecho explícito, no con una conversión inventada. El modelo conserva así la naturaleza del dato y, a la vez, gana la capacidad de secuenciar que el negocio necesita.

A esto se le suma una complicación técnica fascinante en la que no profundizaremos mucho ahora, pero que dejaremos planteada. Una propiedad o un dato puede ser completamente **válido** en un período de tiempo, y dejar de serlo en el siguiente. Marta vivió ininterrumpidamente en esa casa entre 2010 y 2025; pero si consultamos en 2026, su residencia es otra. La respuesta a "dónde vive Marta" no es un bloque de piedra inamovible; es un flujo temporal compuesto por una sucesión de valores, cada uno con su propia fecha de caducidad. Este concepto es lo que en la teoría de bases de datos avanzadas se conoce como **bitemporalidad**, y en la lingüística estructural se denomina **vigencia**. Será un componente explícito de nuestro diseño final.

Si aplicamos todo esto a nuestros casos de uso: - *Receta*: ¿Cuándo se ejecuta? Puede ser un tiempo absoluto (la preparé anoche), un patrón cíclico (es la comida de todos los domingos), o un tiempo relativo interno ("añadir la sal después de que rompa a hervir"). - *Gol*: ¿Cuándo ocurre? En el minuto 87, en la segunda mitad, o en el tiempo de descuento. Todo esto es tiempo de reloj corto, que solo tiene sentido en relación con el pitido inicial del árbitro. - *Canción*: ¿Cuándo debe entrar el coro instrumental? Exactamente en el compás 17. ¿Cuándo se grabó físicamente el disco? El 14 de mayo de 2026. Aquí vemos dos líneas de tiempo conviviendo en paralelo. - *Noticia política*: ¿Cuándo se estampó la firma presidencial? Ayer en la tarde. ¿Cuándo entra en vigor la ley para los ciudadanos? Dentro de treinta días. Un tiempo absoluto y un tiempo derivado operando sobre el mismo objeto.

Cuando un pilar se confunde: una anécdota de IA en estado salvaje

Antes de dar por cerrado este capítulo conceptual, resulta sumamente instructivo analizar un caso de la vida real. Es una anécdota que ilustra a la perfección qué ocurre cuando un sistema avanzado confunde los pilares fundamentales de un hecho. La historia circuló bastante en foros de tecnología hace un par de años y, debido a lo común del fallo, se ha convertido casi en un ejemplo de estudio clásico sobre las limitaciones de la inteligencia artificial sin estructura.

Imagina a un usuario que acaba de terminar su jornada laboral. Agarra su teléfono y le dicta lo siguiente a su asistente virtual:

Estoy en mi coche, que se me ensució muchísimo viniendo del trabajo. Lo tengo que llevar al autolavado, que me queda a solo dos cuadras de aquí. ¿Me recomiendas ir caminando o ir en carro?

La respuesta que devolvió el modelo de lenguaje fue desconcertante: *"Si te queda tan cerca, puedes ir caminando sin problemas. Es mucho más saludable y de paso ahorras combustible."*

Evidentemente, el consejo es un absurdo físico. Es imposible llevar un vehículo a un centro de lavado yendo a pie. Sin embargo, el error no nace de una "estupidez" del algoritmo, sino de un problema técnico muy específico: una **mala asignación de roles semánticos a los cuatro pilares**.

Analicemos la situación utilizando la notación que acabamos de establecer. Una lectura humana y estructuralmente correcta del escenario identifica las siguientes variables:

Q (quién)	: el usuario (agente)
O (qué, objeto que se mueve)	: el coche (paciente)
L (dónde, destino)	: el autolavado, ubicado a dos cuadras
T (cuándo)	: ahora
M (con_finalidad)	: lavar el coche

El verbo núcleo de toda la situación es *llevar*. El usuario funciona claramente como el agente de la acción, pero el **paciente** de ese verbo (la cosa física que sufre el transporte) es el coche. La finalidad ineludible de toda esta maniobra es someter ese coche a un proceso de lavado. Si eliminamos esa finalidad de la ecuación, la pregunta "¿caminar o en carro?" pierde absolutamente todo su sentido.

¿Qué fue lo que entendió el modelo de IA basado puramente en cálculo estadístico? Su asignación de roles probablemente se vio así:

```
Q (quién)           : el usuario (agente)
O (qué, objeto que se mueve) : el usuario (¡el mismo!)
L (dónde, destino)   : el autolavado, ubicado a dos cuadras
T (cuándo)          : ahora
M (con_finalidad)    : trasladarse al autolavado
```

Bajo esta lectura errónea, el coche desaparece de su rol legítimo como paciente y es empujado conceptualmente a un rol de **instrumento opcional** ("puedes ir en carro o no"). La finalidad real (lavar el vehículo) se diluye y es reemplazada por el mero acto de llegar a unas coordenadas geográficas. Como el modelo estadístico evalúa que la distancia ("dos cuadras") es mínima, sus pesos probabilísticos sugieren que caminar es la acción más eficiente y saludable.

El fallo no es una deficiencia matemática. Es una deficiencia de **modelado del mundo**. El sistema perdió el rastro de una asignación estructural básica: en este contexto específico, el coche **es** el objeto central en el eje **O**, no una herramienta accesoria. Apenas esa pieza se cae del tablero, toda la cadena de razonamiento posterior colapsa, y el modelo ni siquiera es consciente del error.

Es aquí donde la promesa central de este libro cobra fuerza industrial. Si pudiéramos ofrecerle los hechos del mundo a los asistentes de inteligencia artificial utilizando una estructura donde los cuatro pilares fueran **explícitos** (el agente siempre en la caja **Q**, el paciente en la caja **O**, el lugar en la **L** y el momento temporal en la **T**), esta clase de absurdos lógicos desaparecerían. Y no porque hayamos creado un algoritmo más inteligente, sino porque la propia **arquitectura estructural impide que ocurra la asignación incorrecta**.

Un sistema con bases firmes "sabe" por diseño que un verbo transitivo como *llevar* exige obligatoriamente la existencia de un paciente. Sabe que ese paciente tiene que estar registrado en el eje **O**. Si el paciente no está, el sistema detecta que la orden está incompleta y, en lugar de alucinar una respuesta, detiene el proceso para pedir aclaraciones. La estadística deja de gobernar a ciegas; la estructura toma el control y le da límites seguros. Esto es lo que la industria de la IA necesita con mayor urgencia en la actualidad.

Los cuatro pilares y lo que falta

Si hacemos un repaso de las cuatro preguntas exploradas —*quién, qué, dónde, cuándo*— y recordamos la oración sencilla del principio del capítulo, comprobamos que las cuatro piezas están ahí, ejecutando un trabajo de ordenamiento preciso. Marta (quién) le regaló un libro (qué) ayer (cuándo) en su casa (dónde). El escenario queda sólidamente descrito.

¿Son suficientes para modelar la realidad completa? Para frases como esa, sí lo son. Pero no pasará mucho tiempo en un entorno de producción antes de que nos topemos con registros de datos donde estas cuatro cajas se queden cortas.

Imaginemos una leve expansión de nuestra oración original:

Marta le regaló a su sobrino, por su cumpleaños, un libro de cuentos que costó treinta dólares.

El hecho base sigue siendo el mismo, pero el volumen de información ha escalado. De pronto, tenemos que procesar un **precio** (treinta dólares). Entró en juego un **motivo o finalidad** clara de la acción (por su cumpleaños). Además, tenemos una **clasificación mucho más específica** del objeto: no es un libro genérico, es un libro *de cuentos*. Y, de paso, la frase deja en claro que el sobrino pertenece a la categoría de *persona* (igual que Marta), un

detalle técnico que nuestros cuatro pilares aún no nos permiten declarar explícitamente, ya que el pilar **Q** sirve para guardar individuos concretos, no para definir categorías teóricas.

Conceptos como "*persona*", "*libro de cuentos*" o "*dólar*" no son objetos palpables. Son **clases** o arquetipos. Y, por definición, no encajan limpiamente en ninguno de los cuatro pilares que acabamos de estudiar.

La intuición que comienza a emerger es que los cuatro pilares son **estrictamente necesarios, pero arquitectónicamente insuficientes**. Necesitamos incorporar, como mínimo, una quinta pregunta —*cuál*, es decir, qué tipo de cosa es esto— que exija un eje propio y exclusivo: el zócalo categórico donde habitarán los tipos, las familias y los conceptos abstractos. A esto debemos sumarle una sexta pregunta para las métricas —*cuánto*— que lidiará con sus propias trampas de conversión (como las unidades de medida); y finalmente, una séptima pregunta —*cómo*— encargada de articular las conexiones y atributos que amarran todo el modelo.

De esas cuatro preguntas restantes, la primera y más delicada de todo el modelo es el **eje categórico**: el zócalo donde habitan los tipos, las familias y los conceptos abstractos. Es, paradójicamente, el eje que soporta la mayor carga de trabajo en los sistemas del mundo real, y es aquí donde la arquitectura de WQuestions se integra pacíficamente con los gigantes de la industria (Schema.org, QUDT, SNOMED, CIDOC CRM), utilizándolos como aliados en lugar de intentar competir contra ellos: el vocabulario común que nos asegura no volver a construir, nunca más, la torre de Babel de los datos.

Capítulo 3 — Cuál: el zócalo categorico (K)

Lo que los pilares dejan sin decir

Para entender por qué necesitamos un eje nuevo, lo mejor es volver por un momento a la oración con la que abrimos el capítulo anterior. Habíamos desarmado el hecho "*Marta le regaló un libro a su sobrino ayer en su casa*" utilizando nuestros cuatro pilares fundamentales: Q, O, L y T. Y, a primera vista, la descripción cerró perfectamente. Marta encontró su lugar en el "quién", el libro en el "qué", la casa en el "dónde" y el ayer en el "cuándo".

Pero si miramos esa estructura con ojos de ingeniero de datos o de arquitecto de software, nos topamos con un punto ciego masivo. Hay información crítica que esos cuatro pilares, por diseño, **no nos dejan decir**.

Sabemos que Marta está en el eje Q. Sabemos que el sobrino también está en el eje Q. Hasta ahí, todo bien. Sin embargo, un sistema informático que pretenda ser útil necesita saber algo mucho más profundo que simplemente dónde guardar los nombres: necesita saber *que ambos son personas*. Necesita la capacidad lógica de reconocer que `marta_001` y `sobrino_045` comparten una naturaleza; ambos pertenecen exactamente a la misma **categoría**.

El mismo problema ocurre con el libro. Ese objeto físico, con sus tapas de cartón y sus hojas impresas, vive correctamente en el eje O. Pero la palabra "libro" representa algo mucho más amplio que ese ejemplar específico que Marta compró en la tienda. "Libro" es **el tipo** de cosa a la que pertenece ese objeto. Cualquier sistema inteligente tiene que ser capaz de distinguir sin esfuerzo entre el ejemplar concreto (este libro puntual, con una mancha de café en la página tres) y la categoría universal a la que pertenece (el libro como concepto general).

La limitación estructural es clara: los pilares que hemos visto hasta ahora funcionan como un inventario de individuos del mundo. Sirven para catalogar a esta persona en particular, a este objeto, a este lugar exacto y a este momento en el reloj. Lo que nos falta es un inventario que defina **lo que esos individuos son**. Y esa definición de identidad no cabe en Q ni en O. Necesita y exige su propio eje: **K**, el eje de las clases.

Qué es K, exactamente

La letra K se eligió por convergencia lingüística (*Kind* en inglés, *Klasse* en alemán, *Kategoria* en lenguas latinas o griego). Es el eje que responde a la pregunta **¿cuál?**: cuál, de entre un conjunto acotado de tipos, estados o categorías, le corresponde a algo. Está diseñado exclusivamente para alojar **tipos, categorías y conceptos abstractos**. Es el territorio lógico donde viven ideas genéricas como *libro*, *persona*, *gol*, *receta*, *modelo de lenguaje*, *infarto agudo de miocardio*, *cliente*, *kilogramo*, *dólar*, *arquitectura transformer* o *receta vegetariana*.

Para visualizarlo mejor, podemos pensar en el modelo como un edificio sostenido por dos grandes fundaciones. Los pilares Q, O, L y T conforman el **primer zócalo**, el de lo concreto. Allí viven las cosas que existen materialmente en el mundo, entidades que tienen una identidad propia y, casi siempre, una ubicación en el espacio y en el tiempo.

K, por el contrario, es el **segundo zócalo**, el de lo categorico. Allí no hay cosas que puedas tocar. Hay nombres genéricos, patrones y moldes bajo los cuales agrupamos a los ejemplares del primer zócalo.

Esta distinción entre el individuo y su categoría es sutil, pero es la herramienta tecnológica más potente que tenemos. Un *cocinero* de carne y hueso vive en el eje Q (es un individuo llamado Juan, que tiene un número de identificación fiscal, una edad y un sueldo). Pero el concepto de *cocinero* —la profesión, el rol laboral, la categoría académica— vive en K. Una *llamada específica al modelo GPT* vive en O (ocurrió a las 14:00, consumió 500 tokens y tuvo una latencia de 2 segundos). Pero el concepto técnico de *llamada a un modelo de lenguaje* vive en K. La ciudad donde ocurrió un evento vive en L. La categoría abstracta de *ciudad capital* vive en K.

Separar las instancias (los individuos) de sus categorías es precisamente lo que le permite a un sistema **razonar con generalidad**. Es lo que hace posible que un motor de búsqueda hospitalario entienda la instrucción "busca a todos los pacientes con hipertensión", en lugar de obligar al operador a escribir el nombre de cada persona que sufre esa enfermedad.

Cuatro familias dentro de K

El eje K no es simplemente una bolsa plana donde arrojamamos etiquetas al azar. Es un entorno sumamente estructurado que aloja al menos cuatro familias distintas de entidades categóricas. Vale la pena detenerse a analizar cada una para entender su alcance.

1. Tipos de objetos y eventos. Cuando hablamos de conceptos como *coche*, *cliente*, *gol*, *receta*, *llamada_API* o *infarto*, estamos hablando estrictamente de tipos. Como dijimos, la taza física está en O; pero el molde teórico *taza* está en K. Para unir ambos ejes, utilizamos una relación matemática fundamental a la que llamaremos `instancia_de`:

(taza_007,	instancia_de, taza)	$\in M(O, K)$
(messi,	instancia_de, jugador_de_futbol)	$\in M(Q, K)$
(lima,	instancia_de, ciudad_capital)	$\in M(L, K)$
(gpt_x,	instancia_de, modelo_de_lenguaje)	$\in M(O, K)$

Cualquier individuo registrado en Q, O, L (y a veces en T) debe tener al menos una respuesta a la pregunta "¿de qué concepto eres instancia?". Ese hilo conector es lo que ata el grafo de hechos concretos con el diccionario abstracto del sistema.

2. Unidades de medida. Términos como *kilogramo*, *segundo*, *token*, *dólar*, *milisegundo* o *grado_Celsius* son categorías puras; no son entidades materiales. Nunca te vas a cruzar por la calle con "un kilogramo físico" flotando en el aire como si fuera una botella; lo que experimentas es el patrón o estándar de "ser un kilogramo" aplicado a la masa de una botella. Dado que son conceptos teóricos utilizados para medir, todas las unidades habitan en K. Afortunadamente, no tenemos que inventarlas. Existe un catálogo canónico universal llamado **QUDT** [18] (*Quantities, Units, Dimensions and Types Ontology*), que clasifica miles de unidades, define sus dimensiones físicas, permite conversiones matemáticas y les asigna identificadores estables. Estudiaremos esto en profundidad cuando abordemos el eje N (Números).

3. Estados y valores enumerativos. En el diseño de software, es muy común encontrarnos con atributos que solo pueden tomar un valor de una lista estricta y cerrada. Por ejemplo: *casado* / *soltero* / *viudo*, *aprobado* / *pendiente* / *rechazado*, *zurdo* / *diestro*, *encendido* / *apagado*. Esos valores predefinidos no son números, ni deberían tratarse como texto libre; son categorías de estado. Y como son conceptos genéricos aplicables a múltiples casos, viven en K.

(paciente_042,	estado_civil,	casado)	$\in M(Q, K)$
(prestamo_017,	estado,	aprobado)	$\in M(O, K)$
(llamada_042,	modo,	streaming)	$\in M(O, K)$

4. Conceptos abstractos y nomenclaturas. Todo dominio profesional serio cuenta con diccionarios controlados. Aquí entran los diagnósticos médicos oficiales (CIE-10, SNOMED), las categorías comerciales y códigos de producto (SKUs), los géneros musicales, los partidos políticos, las clasificaciones biológicas o las arquitecturas de inteligencia artificial. Cualquier nomenclatura controlada, sin excepción, aterriza en el eje K.

(diagnostico_017,	codigo_cie10,	"I21.9")	$\in M(O, K)$
(producto_088,	sku,	"coca_500ml")	$\in M(O, K)$
(cancion_yesterday,	genero,	pop_rock)	$\in M(O, K)$
(modelo_gpt_x,	arquitectura,	transformer)	$\in M(O, K)$

Si hay una línea en común que atraviesa a estas cuatro familias es esta: **todo lo que vive en K no es un ejemplar único en el mundo, sino un patrón conceptual que puede aplicarse a decenas, miles o millones de casos.**

La frontera resbaladiza: ¿instancia (O) o clase (K)?

El lector técnico choca tarde o temprano con un caso incómodo: ciertos objetos híbridos —un **VAR**, un **algoritmo de recomendación**, un **modelo de lenguaje**— parecen vivir en varios ejes a la vez. La confusión se disuelve apenas se entiende que **no es la cosa la que tiene un eje, sino cada uno de sus papeles**. Una misma palabra nombra cosas distintas según qué le estemos preguntando:

Si te referís a...	Eje	VAR	Algoritmo de recomendación
El tipo o concepto (la tecnología, la categoría)	K	<code>var</code> como clase de sistema arbitral	<code>filtrado_colaborativo</code> como técnica
El ejemplar concreto (un despliegue, una unidad)	O	el VAR instalado en el estadio Monumental	el motor de recomendación de Netflix
Una acción suya, reificada (un evento que ejecutó)	O	"el VAR revisó la jugada del minuto 34"	"el motor recomendó la serie a Ana"
Cuando actúa (ocupa el rol <code>agente</code>)	Q	el VAR como agente de <code>anular_gol</code>	el motor como agente de <code>recomendar</code>

Acá se ve la **reificación** en movimiento, y su parentesco con la agencia contextual (la regla D5 que formalizaremos en el Capítulo 9). El mismo VAR es un objeto pasivo en O cuando describimos su instalación; es el `agente` en Q cuando *anula* un gol; y ese "anular el gol" es, a su vez, una situación reificada que vive en O. La entidad **no se duplica** al cambiar de eje: es un solo individuo referido desde situaciones distintas. Y un evento activo en un nivel —la revisión del VAR— puede volverse el objeto pasivo de otro —"la revisión del minuto 34 fue impugnada por el club"—: eso es, exactamente, reificar.

La regla práctica para no perderse es preguntar, ante cada mención, *qué* se está nombrando:

- **¿De qué tipo es?** → K (la categoría).
- **¿Qué ejemplar, o qué evento, es?** → O (la instancia concreta o la acción reificada).
- **¿Quién actúa en esta situación?** → Q (el agente, por contexto, no por naturaleza).

La misma disciplina ordena a los demás híbridos del libro: *transformer* la arquitectura (K), *gpt_x* el modelo desplegado (O), "*gpt_x resumió la consulta*" el evento (O), y *gpt_x como redactor del informe* el agente (Q).

Por qué K necesita un eje propio

Existe una objeción sumamente válida que cualquier ingeniero de bases de datos plantearía al llegar a este punto: *¿Para qué molestarnos en crear todo un eje nuevo? ¿Por qué no tratar las categorías como simples cadenas de texto (strings) guardadas dentro de una columna? Que el atributo sea "estado_civil" y el valor sea la palabra "casado", y problema resuelto.*

A corto plazo, parece una solución pragmática. A nivel de arquitectura empresarial, es la receta perfecta para la fragmentación de datos. La objeción del texto plano no se sostiene por tres razones técnicas que se acumulan y terminan colapsando los sistemas tradicionales.

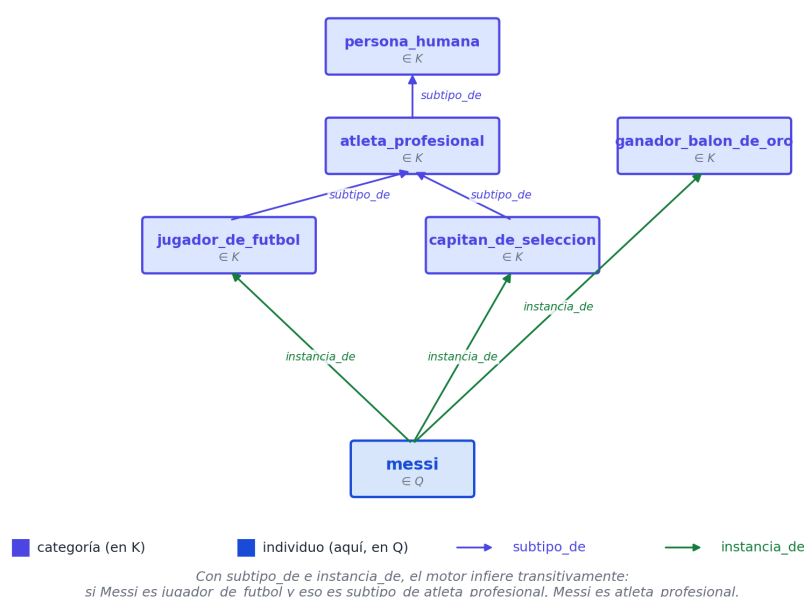
Razón 1: Las categorías tienen estructura interna. La palabra *casado* no es solo una secuencia de seis letras. Es un nodo semántico que posee una inmensa riqueza de propiedades. Tiene sinónimos en otros idiomas (*married*, *marié*, *verheiratet*). Tiene dependencias lógicas con otras categorías (el *matrimonio* es un tipo de *vínculo legal*). Tiene contextos de aplicabilidad (un estado civil no aplica a un objeto inanimado). Si guardamos "casado" como un

simple string en una tabla SQL, toda esa estructura es inaccesible para el motor de la base de datos; la palabra queda "ciega". En cambio, si la tratamos como un individuo con derechos propios dentro del eje K, esa categoría puede tener sus propios hechos conectados:

(casado, sinonimo_en_ingles, "married")	∈ M(K, K)
(casado, requiere, matrimonio_legal)	∈ M(K, K)
(casado, opuesto_de, soltero)	∈ M(K, K)
(casado, codigo_iso, "M")	∈ M(K, K)

Visto así, K no es un archivo plano; es una **red de conceptos interconectados**. Los individuos de K son ciudadanos de primera clase en nuestro modelo. Esta capacidad de vincular conceptos entre sí será la clave técnica cuando analicemos cómo WQuestions se integra con diccionarios de otras industrias.

K no es un saco plano: es una red de conceptos



K como red de conceptos: las relaciones subtipo_de arman la jerarquía categórica e instancia_de ata cada individuo concreto a sus tipos. El motor puede inferir transitivamente.

Razón 2: El vocabulario serio exige autoridad externa. Las categorías de alto nivel que utilizan los hospitales, los bancos o los gobiernos no se inventan en una lluvia de ideas de programadores; provienen de autoridades internacionales que las publican con identificadores únicos, conocidos como URIs. **QUDT** [18] regula las unidades de medida; **Schema.org** [30] provee los estándares para el comercio web; **SNOMED** dicta los códigos médicos; la **ICAO** define las siglas de los aeropuertos. Tratar una de estas categorías como un mero texto ignora por completo su peso legal e internacional. Al registrarla como un individuo formal en el eje K, podemos asignarle su URI canónica como un atributo permanente:

```

(qudt_milliseg, uri_canonica, "[http://qudt.org/vocab/unit/MilliSEC](http://qudt.org/vocab/unit/MilliSEC)
(snomed_infarto, uri_canonica, "[http://snomed.info/id/22298006](http://snomed.info/id/22298006)")
(cie10_I21, uri_canonica, "[http://id.who.int/icd/release/10/2019/I21](http://id.who.int/icd/release

```

Mira el caso del **CIE-10**, la *Clasificación Internacional de Enfermedades* en su décima edición, publicada por la Organización Mundial de la Salud. Es el ejemplo más nítido de por qué esto importa. Cuando un médico anota en una historia clínica el código **I21**, no está escribiendo una palabra cualquiera: está apuntando a una entrada formal, internacional y traducida a decenas de idiomas, que significa "*infarto agudo de miocardio*". Si tu sistema guarda el diagnóstico como el texto libre "infarto", pierdes esa conexión: un hospital en Alemania escribirá "Herzinfarkt", uno en Brasil "infarto do miocárdio", y nadie podrá cruzar datos. En cambio, si el sistema guarda la

categoría como un individuo de K con su URI del CIE-10 anexada, todos los hospitales del mundo hablan el mismo idioma sin esfuerzo. La estadística global de salud, los estudios epidemiológicos, las alertas sanitarias internacionales — todo eso descansa sobre este mecanismo.

Y lo mismo aplica para los demás dominios. Si dos sistemas hospitalarios hablan dialectos distintos (uno anota "milisegundo" y el otro "ms"), pero ambos apuntan internamente a la misma URI oficial, los sistemas se entienden de forma automática sin necesidad de que un programador escriba un traductor a mano.

Razón 3: Las consultas sobre categorías son el corazón del sistema. Las preguntas de negocio más valiosas y complejas siempre cruzan información categórica. Un analista no pide buscar un ID específico; pide *"todas las recetas vegetarianas que se preparen en menos de 30 minutos"*, o *"todos los modelos de lenguaje de arquitectura transformer entrenados después de 2024"*. Si las categorías estuvieran guardadas como texto libre, estas búsquedas serían un campo minado (un usuario pudo escribir "Vegetariana", otro "vegetariana" y otro "Veg"). Al convertir las categorías en nodos estructurados dentro de K, las consultas dejan de depender de la ortografía y pasan a depender de la matemática relacional. Se vuelven exactas, predecibles y combinables.

Dos relaciones canónicas: `instancia_de` y `subtipo_de`

Para que esta inmensa red de conceptos funcione y sea navegable, existen dos relaciones fundacionales que organizan la estructura interna de K, y es vital entender cómo operan.

La primera es `instancia_de`. Como adelantamos antes, es la relación de uso más intensivo en todo el modelo. Su trabajo exclusivo es actuar como puente, amarrando a los individuos del mundo físico (los pilares) con sus definiciones conceptuales (el eje K). Y un punto crucial aquí es que un individuo puede ser instancia de múltiples conceptos simultáneamente:

```
(messi, instancia_de, jugador_de_futbol)      ∈ M(Q, K)
(messi, instancia_de, capitan_de_seleccion)   ∈ M(Q, K)
(messi, instancia_de, persona_humana)         ∈ M(Q, K)
(messi, instancia_de, ganador_balon_de_oro)   ∈ M(Q, K)
```

Estas cuatro asignaciones no compiten entre sí ni generan errores lógicos. Coexisten en paralelo. Si el motor de la base de datos recibe una consulta filtrando por cualquiera de esas cuatro categorías, devolverá a Messi como resultado válido.

La segunda relación es `subtipo_de`. A diferencia de la anterior, este conector nunca sale de las fronteras de K. Se utiliza de forma exclusiva para enlazar categorías entre sí, creando jerarquías organizadas y árboles de conocimiento (taxonomías):

```
(jugador_de_futbol, subtipo_de, atleta_profesional)      ∈ M(K, K)
(atleta_profesional, subtipo_de, persona_humana)         ∈ M(K, K)
(modelo_de_lenguaje, subtipo_de, modelo_de_aprendizaje_automatico) ∈ M(K, K)
(modelo_transformer, subtipo_de, modelo_de_lenguaje)     ∈ M(K, K)
```

¿Por qué es esto tan importante a nivel tecnológico? Porque cuando declaras correctamente las reglas `instancia_de` y `subtipo_de`, le otorgas al sistema la capacidad de realizar **inferencias transitivas básicas**. Si la máquina sabe que Messi es un `jugador_de_futbol`, y por otro lado sabe que todo jugador es un `atleta_profesional`, el motor deduce de forma automática que Messi es un atleta profesional. En el mundo de la arquitectura de datos, a esta habilidad se le conoce como *cierre transitivo* (transitive closure). Es el mecanismo de razonamiento más simple, pero es el superpoder que evita que los programadores tengan que codificar miles de reglas lógicas a mano.

Lo que ya se intentó: tres puertas, ningún piso

Antes de ver cómo K aloja las ontologías del mundo, conviene entender por qué nadie lo había resuelto antes. No fue por falta de intentos: la industria abrió tres grandes puertas, y cada una se quedó a un paso.

La primera puerta: las 5W1H como extracción. A fines del siglo XX, varios investigadores [9] [10] vieron las seis preguntas periodísticas como una herramienta para extraer datos de texto: un programa lee una noticia y acomoda las respuestas en casilleros (*quién: el ministro de Salud; qué: anunció una campaña; cuándo: ayer*). El resultado luce ordenado, pero estalla apenas se intenta **almacenarlo y cruzarlo**: para la máquina, "el ministro de Salud" y "el titular de la cartera de salud" son dos cadenas de texto distintas. Sin una capa de tipos ni un vocabulario canónico, las 5W1H funcionan como un buen *checklist* para no olvidar nada, pero no como una arquitectura: les faltan justamente las dos piezas que este capítulo introduce —una estructura de tipos (K) y un vocabulario oficial.

La segunda puerta: la web semántica. En 2001, Tim Berners-Lee propuso la **Web Semántica** [31], con **RDF** [8] como pieza maestra: toda la información reducida a tripletas *sujeto–predicado–objeto*. La idea es de una elegancia impecable y sostiene proyectos titánicos como Wikidata [32] y DBpedia [33]. Pero RDF resolvió la *sintaxis* y dejó libre la *semántica*: un sistema escribe (McCartney, compuso, "Yesterday"), otro (McCartney, autor_de, "Yesterday"), un tercero (McCartney, escribió, "Yesterday") — las tres correctas, las tres incompatibles. Sin un diccionario mínimo común, la diversidad de lenguajes simplemente se mudó a otra capa.

La tercera puerta: las ontologías de dominio. Ante ese caos, el tercer enfoque eligió el control estricto: reunir a los expertos de cada industria y publicar un vocabulario oficial y obligatorio. Así nacieron obras de arte de la ingeniería como **CIDOC CRM** [4] (patrimonio), **Biolink** [5] (biomedicina), **HL7 FHIR** [6] (historias clínicas) y **Schema.org** [30] (web comercial). Cada una es excelente dentro de su perímetro: Schema.org/Recipe modela una receta a la perfección. El problema asoma apenas hay que **vincular profundamente** ramas distintas —la receta con la biografía del chef (Person) y el contexto histórico (Event)—: las ontologías crean los nodos, pero no estandarizan los cables entre ellos, y atarlos vuelve a recaer en código manual. Peor aún: como cada una se construyó aislada, todas tuvieron que modelar desde cero lo universal. Para decir "una persona", los museos usan E21_Person, la genética biolink:Agent, el comercio Person y las clínicas Patient: cuatro etiquetas incompatibles para un mismo ser humano físico.

A estas tres puertas se sumaron dos variantes que chocaron con el mismo muro: los **estándares de intercambio** (FHIR [6], EDI [20], ISO 20022 [21]), que funcionan como un servicio de mensajería —traducen para el transporte, no unifican—; y la **canonicalización a posteriori** (OpenIE [23], sistemas de limpieza de datos), que intenta reconciliar el caos *después* de que ocurrió, con un costo computacional que se vuelve inmanejable al cruzar decenas de sistemas.

El balance, llevado a los cuatro ejemplos que recorren este libro, es elocuente:

	1. 5W1H (heurística)	2. RDF / Web Semántica (libre conexión)	3. Ontología de dominio (diccionario estricto)
La Receta	Insuficiente: no entiende medidas exactas, listas ordenadas de pasos ni tiempos de horno.	Factible, siempre que los programadores no usen verbos distintos para "mezclar" o "hervir".	Schema.org/Recipe la modela a la perfección, pero dificulta cruzarla con datos históricos o geográficos.
El Gol de fútbol	Útil para leer la noticia deportiva, inútil para armar estadísticas del partido.	Posible vía Wikidata, aunque los términos para describir una jugada varían drásticamente entre bases.	Existe SportsEvent, pero no llega al detalle de con qué pierna se ejecutó el remate.
La Canción	Sirve para una noticia sobre la canción, no para modelar la obra musical.	MusicBrainz lo soporta bien, pero sus vocabularios chocan con otras fuentes.	Modelos robustos en la industria musical, pero operan como burbujas cerradas difíciles de integrar.
La Noticia política	Excelente: el dominio exacto para el que nació el modelo.	Funcional, pero la falta de acuerdo sobre cómo nombrar "promulgar" o "debatir" arruina las búsquedas.	Schema.org/NewsArticle estructura el "contenedor", pero no entiende qué dice adentro.

El patrón de las fallas es nítido: las 5W1H identificaron las dimensiones pero no construyeron arquitectura; RDF construyó la conexión pero sin vocabulario base; las ontologías construyeron vocabularios de lujo pero sin un piso común. Todas levantaron **techos** espléndidos; ninguna construyó el **piso**. Y un piso es justamente lo que el eje K aporta: una capa fundacional, por debajo de todos los vocabularios, lo bastante sutil para no estorbar la jerga de médicos o arquitectos y lo bastante firme como para que la información fluya sin traductores. Las preguntas cognitivas —que el Capítulo 1 postuló como universales y que el Capítulo 6 fundamentará en detalle— son ese piso. Veamos, entonces, cómo K abraza las ontologías existentes en lugar de competir con ellas.

Las tres puertas previas — ninguna terminó de cerrar

5W1H	RDF / Web Semántica	Ontologías de dominio
metodología periodística + extracción NLP	grafos abiertos de tripletas	vocabularios formalizados por área
Yang & Hu, Mahmood	RDF, OWL, Wikidata, DBpedia	CIDOC CRM, Biolink, Schema.org, FHIR
✓ qué resuelve	✓ qué resuelve	✓ qué resuelve
Identifica las dimensiones correctas (quién, qué, dónde, cuándo, por qué, cómo).	Sintaxis universal sujeto-predicado-objeto.	Profundidad rica dentro de un dominio.
Heurística probada en un siglo de uso real.	Escala a miles de millones de tripletas.	URIs estables, autoridad externa, comunidad activa.
✗ qué deja sin resolver	✗ qué deja sin resolver	✗ qué deja sin resolver
Sin estructura de tipos — las respuestas son texto.	Inventario de predicados libre — cualquiera puede inventar etiquetas.	Ningún piso común entre dominios.
Sin vocabulario canónico — cada noticia inventa.	Sin firmas tipadas: 'compuso' y 'autor_de' no se reconocen.	Cada nuevo dominio requiere otra ontología y otro puente.

Las tres puertas previas comparadas en qué resuelve cada una y qué deja sin resolver. Ninguna terminó de cerrar el problema porque cada una atacó un síntoma distinto y dejó el otro intacto.

K como zócalo para las ontologías existentes

Quizá la promesa de mayor impacto industrial que ofrece el eje K es que **no obliga a ninguna empresa a reinventar su terminología**. Los diccionarios masivos y sofisticados que ya rigen en el mundo (Schema.org, QUDT, SNOMED, CIDOC CRM, Biolink) se mapean dentro de K como subconjuntos de datos, manteniendo intactas todas sus relaciones internas. WQuestions no llega para competir o reemplazar a estas herramientas: su objetivo explícito es **abrazarlas**.

Esta integración pacífica ocurre a través de una estrategia de tres niveles técnicos:

Nivel 1: Importar URIs canónicas. En lugar de transcribir todo un diccionario, el sistema simplemente aloja los conceptos esenciales y ancla a cada uno su URI internacional como validador de identidad:

```
(infarto_miocardio_agudo) ∈ K
uri_snomed : [http://snomed.info/id/22298006] (http://snomed.info/id/22298006)
uri_icd10  : I21
etiqueta_es : "infarto agudo de miocardio"
etiqueta_en : "acute myocardial infarction"
```

El beneficio es inmediato: cuando el software de un laboratorio y el software de un hospital referencian la misma URI, la sincronización de los historiales clínicos es perfecta, por más que la base de datos esté en español y el otro sistema en inglés.

Nivel 2: Mapear el vocabulario de dominio (dialectos locales). En la práctica, las empresas se resisten a usar términos canónicos excesivamente largos o formales; prefieren su jerga interna. K resuelve esto permitiendo que cada organización defina alias (dialectos) que apunten al término oficial. Si en una clínica específica los médicos registran la enfermedad bajo las siglas "IAM", el sistema lo mapea así:

```
(IAM_clinica_norte, alias_de, infarto_miocardio_agudo) ∈ M(K, K)
```

Con esta instrucción, todas las búsquedas que el médico realice introduciendo "IAM" apuntarán limpiamente al concepto global de SNOMED. Es pertinente adelantar que este principio sentará las bases de una decisión de diseño fundamental (D9) que abordaremos en los capítulos sobre lingüística computacional: la idea rectora de que **el usuario final nunca debe verse obligado a tocar etiquetas canónicas**. El usuario utiliza su vocabulario natural, y la capa estructural del sistema se encarga de la traducción matemática. K es la maquinaria que hace esto posible.

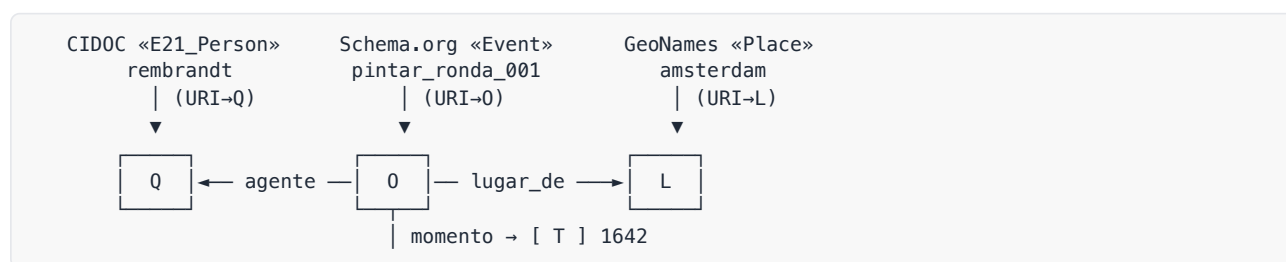
Nivel 3: Federar conceptos equivalentes. El caos semántico alcanza su punto máximo cuando dos entidades internacionales describen exactamente la misma enfermedad pero publican URIs distintas. K soluciona este conflicto permitiendo declarar una equivalencia explícita:

```
(snomed_infarto, equivalente_a, icd10_I21) ∈ M(K, K)
```

A lo largo del tiempo y a medida que el sistema se alimenta, K se transforma orgánicamente en una **red maestra de equivalencias** entre normas heterogéneas. Actúa como el puente de traducción universal que las ontologías de dominio nunca lograron (o no quisieron) construir entre ellas.

El enchufe: las ontologías ponen los nodos; WQuestions, los cables

El mecanismo no se limita a K. Una entidad de cualquier ontología externa se *enchufa* en el eje que le corresponde por naturaleza —una persona de CIDOC CRM (E21_Person) en Q, un evento de Schema.org en O, un lugar de GeoNames en L— conservando su URI canónica como ancla de identidad. Lo que ninguna de esas ontologías sabe hacer por sí sola —el problema que diseccionamos unas páginas atrás, en "Tres puertas, ningún piso"— es **vincular profundamente nodos de catálogos distintos**. Ahí entra WQuestions: las ontologías aportan los **nodos**; el eje M aporta los **cables** que cruzan de una a otra.



El hecho "*La ronda de noche fue pintada por Rembrandt (CIDOC) en Ámsterdam (GeoNames) en 1642*" queda como **un solo grafo**, sin el código-pegamento que cada ontología exigía para saltar de su perímetro al del vecino. Cada catálogo sigue siendo dueño de sus nodos y sus URIs; WQuestions solo añade la capa de predicados que los estándares dejaron sin estandarizar — y con eso neutraliza la fragmentación.

Cinco dominios, cinco vistas de K

Para terminar de materializar la función del eje categórico, resulta muy ilustrativo ver qué tipo de información específica almacena K cuando lo aplicamos a distintos sectores. Repasemos nuestros cuatro ejemplos recurrentes, sumando el entorno de la inteligencia artificial como caso de estudio adicional:

- **La Receta:** En K guardaríamos los tipos de plato (*entrada, plato_principal, postre*), las técnicas de cocción requeridas (*saltear, hervir, hornear*), las etiquetas de origen cultural (*siciliano, yucateco, japonés*) y los regímenes dietéticos (*vegetariano, vegano, sin_gluten*). Una buena parte de estos identificadores ya existe en el estándar Schema.org/Recipe, mientras que el léxico sobre técnicas o culturas suele importarse de Wikidata [32].
- **El Gol de fútbol:** El eje alojaría las taxonomías de los tipos de gol (*de cabeza, de tiro libre, de penal, en contra*), las clasificaciones de las partes del cuerpo utilizadas (*derecha, izquierda*) y las zonificaciones del campo de juego (*área grande, fuera_del_área*). Parte de este vocabulario emana directamente de la FIFA; el resto es jerga analítica desarrollada por empresas de estadística deportiva.
- **La Canción:** Aquí vivirían los grandes géneros musicales (*rock, pop, bossa_nova*), las escalas y tonalidades armónicas (*sol_mayor, re_menor*) y la clasificación de instrumentos (*guitarra_acústica, piano*). En este escenario, la base de datos abierta MusicBrainz proporciona catálogos excepcionalmente maduros, fácilmente consumibles vía API.
- **La Noticia política:** K se encargaría de clasificar la jerarquía de las instituciones (*ministerio, partido_político, congreso*), la tipología formal de los actos administrativos (*decreto, ley, resolución*) y la escala de las jurisdicciones de gobierno (*nacional, regional, municipal*).
- **La Llamada a un modelo de lenguaje (IA):** K tendría la responsabilidad de ordenar las arquitecturas tecnológicas subyacentes (*transformer, mamba, mixture_of_experts*), las familias comerciales (*GPT, Claude, Gemini, Llama*), los propósitos o tipos de tarea que el usuario solicita (*resumir, clasificar, traducir, generar_código*) y la modalidad de entrega de la respuesta (*síncrono, streaming, batch*). Este dominio es particularmente fascinante porque su vocabulario se encuentra en plena fase de expansión. Cada laboratorio de investigación inventa sus propios términos comerciales. K ofrece precisamente el sustrato organizativo donde todas esas convenciones dispares podrán converger a medida que la industria madure.

D1: La plantilla y la instancia

Para blindar la consistencia de la base de datos, existe una regla de diseño estricta que regula la frontera entre el eje abstracto K y los ejes del mundo físico (particularmente el eje O). Es un principio que debe respetarse sin excepciones — y, además, es la **primera decisión de diseño** que el libro registra formalmente:

D1 — En el eje K habitan exclusivamente los conceptos atemporales y categóricos (las plantillas); en el eje O (y en el resto de los pilares) habitan las entidades creadas, situadas geográficamente e instanciadas (los objetos derivados).

A lo largo del libro, las decisiones de diseño se numeran con la letra **D** seguida de un número (D1, D2, D3...). Cada una se enuncia formalmente cuando aparece por primera vez y luego se referencia por su número. Vas a ir encontrándolas en orden a medida que el modelo se construye.

La pregunta operativa que un desarrollador debe hacerse ante cualquier dato nuevo es directa: *¿Este individuo posee una fecha de creación concreta, una historia propia, o una ubicación geográfica comprobable?* Si la respuesta es afirmativa, su lugar ineludible es el eje O. Si carece de estas coordenadas, pertenece a K.

Observemos la diferencia en la práctica: - *La receta tradicional del risotto a la milanesa*: Es una fórmula, un patrón de conocimiento replicable. No caduca ni tiene una ubicación física. Su lugar es **K**. - *La preparación de ese risotto el domingo pasado en mi cocina*: Ocurrió un día puntual, se quemó ligeramente y dejó una sartén sucia. Es una instancia material basada en la fórmula. Su lugar es **O**. - *El concepto general de "modelo de lenguaje de arquitectura transformer"*: Es teoría informática. Su lugar es **K**. - *La versión ejecutable GPT-X-2026-05, lanzada públicamente el 14 de mayo*: Tiene una fecha histórica inamovible, consume energía y corre en servidores físicos. Es una instancia. Su lugar es **O**.

Esta dualidad produce un patrón arquitectónico extraordinariamente robusto: existe siempre una **plantilla teórica anclada en K**, a partir de la cual se genera una **instancia concreta en O**, unidas ambas por el cordón umbilical de la relación `instancia_de`.

La receta que escribe un chef vive abstractamente como un tipo. Pero cada vez que un cocinero prende los fuegos y ejecuta esa receta, el sistema crea un nuevo evento-instancia, al que se le añade un factor de escala (qué sustituciones se hicieron, si se duplicaron las porciones o el horario exacto del servicio).

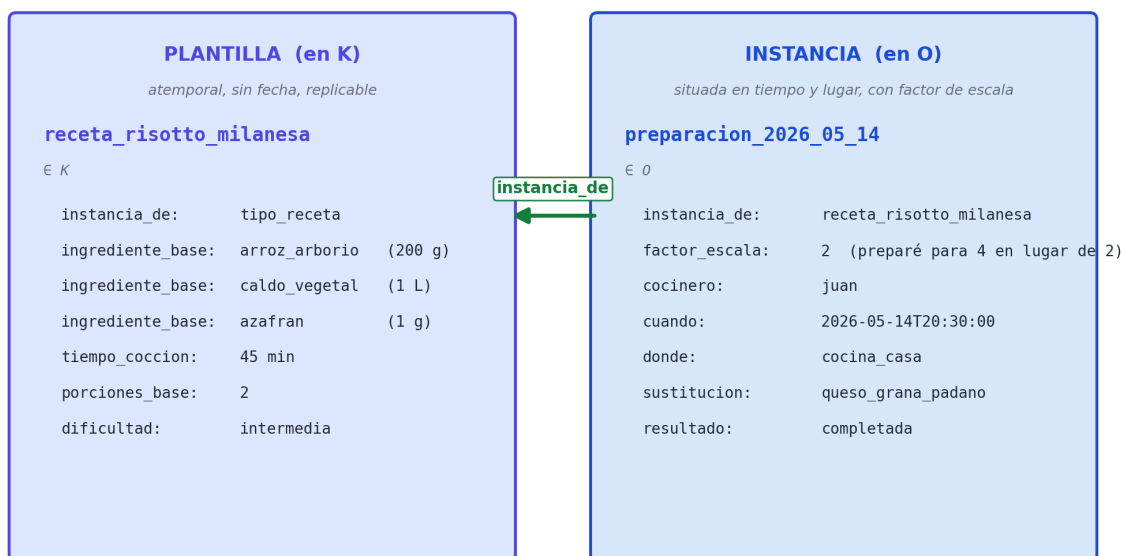
Codificado bajo la lógica del modelo, se leería de este modo:

```
(receta_risotto_milanese) ∈ K           // La plantilla teórica
  ingrediente_base : arroz_arborio (porción base 200 g)
  paso            : ...

(preparacion_2026_05_14) ∈ O           // El evento instanciado
  instancia_de    : receta_risotto_milanese
  factor_escalas : 2 (se cocinó para 4 personas en lugar de 2)
  cocinero       : juan
  cuando        : 2026-05-14T20:30:00
```

Esa danza constante entre la plantilla atemporal de K y la ocurrencia situada de O es uno de los patrones de modelado de datos más poderosos en aplicaciones comerciales a gran escala. Lo veremos reaparecer continuamente cuando lleguemos a los casos prácticos de la Parte V.

D4 — plantilla atemporal en K, instancia situada en O



La plantilla atemporal vive en K y se reusa.
Cada preparación concreta vive en O con su contexto: fecha, lugar, cocinero, escala.

D1 en acción: la receta abstracta vive como tipo en K; la preparación concreta del 14 de mayo vive como instancia en O, con factor de escala, fecha, lugar y cocinero específicos.

Trampa de programación: la categoría guardada como texto

El atajo de todos los días: `estado_civil = "casado"`, `estado = "aprobado"`, guardados como texto libre en una columna. Parece inofensivo hasta que el sistema crece: un operario escribe "Casado", otro "casado " con un espacio, un tercero "married", y la consulta "dame todos los casados" devuelve la mitad. No hay validación, no hay traducción, y no hay forma de saber que "IAM" y "infarto agudo de miocardio" son lo mismo. La categoría, tratada como string, **pierde su identidad**.

El eje K lo corrige tratando cada categoría como un **individuo formal** con su URI canónica: `casado` es un nodo único, no una cadena de caracteres. Las búsquedas son exactas, los alias ("IAM") apuntan al mismo concepto, los idiomas conviven, y los diccionarios internacionales (SNOMED, Schema.org) se enchufan sin reescribir nada. Un valor de una lista cerrada nunca debería ser texto libre — debería ser un punto en K.

Resumen del capítulo

El eje K opera como el segundo gran zócalo fundacional de nuestra arquitectura, actuando como el complemento intelectual y abstracto de los pilares físicos (Q, O, L, T). Hemos establecido que:

Se encarga de alojar **tipos conceptuales, unidades de medida, estados enumerativos y nomenclaturas oficiales**. Posee una rica **estructura interna**, lo que significa que los conceptos pueden vincularse entre sí generando redes semánticas complejas. Proporciona el terreno para que aterricen las **ontologías industriales existentes** (Schema.org, QUDT, SNOMED, CIDOC CRM, Biolink) importando su rigor, pero sin exigir que el usuario final hable en código. Dota al sistema de inteligencia habilitando la inferencia transitiva básica mediante el uso cruzado de las relaciones `instancia_de` y `subtipo_de`. Garantiza el ordenamiento arquitectónico mediante la regla de diseño frontera D1, que separa meticulosamente la **plantilla atemporal** de la **instancia situada**.

Con la formalización del eje K, el universo de individuos capaces de ser modelados por el sistema está prácticamente mapeado. Las cajas de Q, O, L, T y K conforman el grupo de cinco ejes primarios encargados de dar hogar a **todas las cosas materiales del mundo y a todas sus categorías teóricas**.

Sin embargo, para poder cuantificar la realidad de forma matemática, nos queda desvelar un último eje fundamental en este primer bloque: **N**, el eje de las magnitudes y los números.

Ese eje N esconde más complejidad de la que aparenta. La pregunta metodológica ya asomó de reojo cuando expandimos la oración original y dijimos "*un libro de cuentos que costó treinta dólares*": ¿cómo diseñamos un modelo que procese correctamente la cantidad? La respuesta está muy lejos de ser una simple discusión sobre tipos de datos numéricos, porque rige una regla física inviolable en el diseño de datos serios: **en el mundo real, un número desnudo no significa nada; todo número válido viene acoplado a una unidad de medida**. Y el único lugar donde pueden habitar las unidades es en el zócalo categórico de K. Es exactamente por esa interdependencia arquitectónica que el eje K tenía que presentarse con todo detalle antes de que nos atreviéramos a hablar de números.

Capítulo 4 — Cuánto: el eje cuantitativo y sus trampas

Una sonda que se estrelló y una IA que se desconcertó

En septiembre de 1999, la sonda espacial *Mars Climate Orbiter* de la NASA completó un viaje de nueve meses hacia Marte. Se preparó para la maniobra final de inserción orbital, encendió sus motores... y se desintegró por completo en la atmósfera marciana. La pérdida instantánea fue de 327 millones de dólares.

Cuando los ingenieros revisaron qué había salido mal, encontraron una de las fallas más absurdas en la historia de la exploración espacial: el software de la empresa constructora enviaba la fuerza de los propulsores medida en **libras-fuerza por segundo**, pero el software de navegación de la NASA recibía esos números creyendo que eran **newtons por segundo**. Esa pequeña discrepancia —un error de factor de 4.45— se fue acumulando en silencio durante meses. Para cuando la nave llegó a Marte, su trayectoria era tan baja que la atmósfera la hizo pedazos.

Nadie había hecho mal el cálculo matemático. Ningún código estaba roto. El problema fue que el número viajó "desnudo", sin su unidad de medida pegada a él.

Saltemos veinticinco años hacia el futuro. En agosto de 2024, un equipo de ingenieros en una startup tecnológica sufrió una versión en miniatura de esta misma historia. Estaban construyendo un asistente de Inteligencia Artificial para analizar documentos legales inmensos. Eligieron el modelo más potente del mercado porque presumía de una capacidad enorme: "*128.000 de ventana de contexto*". El primer usuario subió un contrato de 200 páginas, pero la IA empezó a comportarse de forma errática: cortaba las respuestas a la mitad, inventaba artículos de ley y alucinaba datos.

Cuando el equipo revisó los registros, entendieron el error: la capacidad de 128.000 era de **tokens** (fragmentos de palabras), no de **caracteres**. Un contrato en español de 200 páginas consume fácilmente 200.000 tokens. La Inteligencia Artificial, sencillamente, estaba borrando todo lo que no cabía en su memoria, y nadie en el equipo sabía hacer la conversión entre páginas, caracteres y tokens.

Estas dos historias no tratan realmente sobre matemáticas. Tratan sobre el mismo error de diseño fundamental: **un número sin su unidad de medida no es información; es ruido disfrazado de dígitos**.

Este capítulo se adentra en el eje encargado de alojar los números —al que llamaremos el eje **N**— y en la trampa mortal que esconde: fingir que un número se basta a sí mismo. Modelar correctamente la dependencia entre el número y su unidad es lo que diferencia a una base de datos profesional de una hoja de cálculo a punto de colapsar.

Por qué N exige tener su propio eje

Es una pregunta totalmente válida: ¿para qué crearle un eje exclusivo a los números? ¿No podríamos simplemente guardar los números dentro de la caja de los objetos (O), junto con las botellas, los contratos y los goles?

La respuesta es no, y el motivo queda claro apenas intentas forzar esa idea. Una botella es un individuo físico: tiene identidad propia, existe a lo largo del tiempo y puede aparecer en diferentes eventos. Lo mismo ocurre con un gol. Pero el número "87" no es una cosa física; es un **valor**. No existe un "87" guardado en un cajón del universo al que podamos hacer referencia.

Lo que sí existe son millones de hechos en el mundo que utilizan el valor 87 para describir algo: el minuto 87 del gol de Messi, el 87% de precisión de un algoritmo, o los 87 tokens descartados por una IA. Cada vez que usamos ese número, su aparición es totalmente independiente de las demás.

Esta diferencia filosófica tiene un impacto brutal en la arquitectura de software. En los ejes que ya vimos (Q, O, L y K), cada elemento necesita una **identidad propia e inconfundible**. Un sistema informático serio le asigna a cada persona, ciudad o concepto un código interno único (un UUID) para no perderlo de vista.

El eje **N** no necesita eso. Para un número, su valor es su identidad. Cuando un servidor anota que "la IA respondió en 340 milisegundos", el sistema no necesita inventarle un código secreto al número 340; el dato puro y crudo es suficiente. (Lo mismo ocurre con las fechas en el eje T: las fechas se comparan por su valor matemático, no por un código interno).

La regla es simple: **Q, O, L y K son ejes de entidades físicas o conceptuales; N y T son ejes de valor puro.**

Las unidades de medida nunca son neutrales

Volvamos a la sonda marciana y al caos con los tokens de la IA. Ambas anécdotas demuestran una regla de oro en la ingeniería de datos: **todo número extraído del mundo real trae una unidad implícita**, y confundir esa unidad es una de las formas más caras de destruir un proyecto.

La conclusión arquitectónica no admite excepciones: **un valor en el eje N nunca debe viajar solo**. Tiene que estar siempre escoltado por su unidad de medida. Y aquí es donde los ejes comienzan a trabajar en equipo. Como aprendimos en el capítulo anterior, las unidades (como *kilogramo*, *segundo*, *dólar* o *token*) no son objetos físicos; son categorías abstractas. Por lo tanto, esas unidades viven cómodamente en el eje **K**.

Nuestro trabajo al diseñar la base de datos es asegurar que cada vez que se registre un número, se tienda un "cable" que lo conecte con su unidad correspondiente en K.

En la notación formal del modelo se vería así:

```
(gol_messi_87, minuto, 87)      ∈ M(O, N)
(gol_messi_87, unidad_minuto, minuto) ∈ M(O, K)

(respuesta_017, latencia, 340)   ∈ M(O, N)
(respuesta_017, unidad_latencia, milisegundo) ∈ M(O, K)
```

A primera vista, esto podría parecer un exceso de burocracia. ¿Acaso no es obvio que el minuto de un partido se mide en minutos? Sí, es obvio hoy. Pero deja de serlo el día en que un proveedor de estadísticas decide enviar los datos del partido en segundos en lugar de minutos, y tu sistema se rompe por no saber en qué unidad estaba hablando la otra máquina.

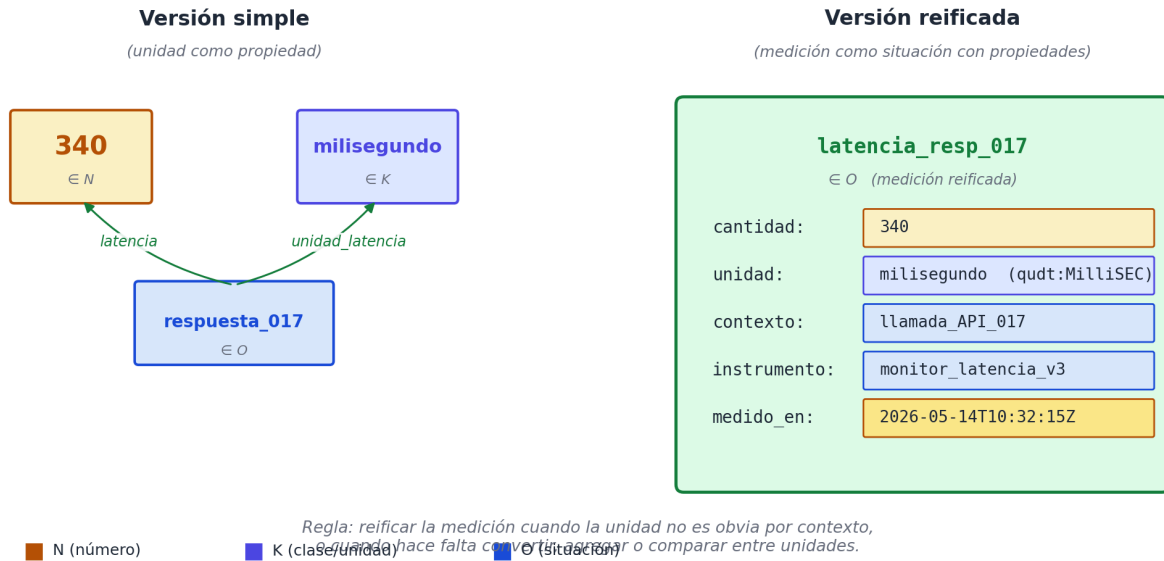
Cuando sabemos que un dato va a sufrir conversiones frecuentes, la estrategia de diseño más limpia es **reificar la medición**. Esto significa elevar la simple medida al estatus de un evento formal en el eje O, dándole sus propias propiedades.

En código se vería así:

```
(latencia_resp_017) ∈ O
  cantidad   : 340           → Va al eje N
  unidad     : milisegundo    → Va al eje K
  contexto   : llamada_API_017 → Va al eje O
```

Hacer esto es lo que separa a un sistema amateur que escupe un "340" sin contexto, de un sistema profesional que te dice exactamente: *"Esto es 340 ms, medidos sobre la llamada número 017 al modelo GPT-4, tal día y a tal hora"*. El primero es solo un número flotando en el vacío; el segundo es inteligencia accionable.

Un número nunca viaja solo: cantidad + unidad (+ contexto)



Dos formas de modelar una medición: la versión simple guarda el valor y la unidad como propiedades del sujeto; la versión reificada eleva la medición misma a entidad en O, con cantidad, unidad, contexto, instrumento y momento como sus propiedades.

El catálogo oficial: QUDT y las nuevas unidades de la IA

Una duda común al llegar a este punto es: *¿Tengo que sentarme a escribir un diccionario con todas las unidades del mundo para mi sistema?* La respuesta es un rotundo no. Este es uno de los pocos problemas que la industria mundial ya solucionó de forma definitiva.

Existe una ontología pública llamada **QUDT** (*Quantities, Units, Dimensions and Types*) [18] que cataloga miles de unidades reconocidas, sus dimensiones físicas, sus fórmulas de conversión exactas y sus enlaces oficiales (URIs). Si necesitas milisegundos, usas el código `qudt:MilliSEC`; si es un archivo de computadora, usas `qudt:Byte`; si es dinero, `qudt:USD`. Cuando el sistema de WQuestions adopta una unidad, lo hace enlazándose directamente a este catálogo oficial.

Sin embargo, hay industrias tan nuevas que QUDT todavía no tiene palabras para describirlas. El ecosistema de la Inteligencia Artificial es el mejor ejemplo. ¿Con qué unidad se mide la "calidad" de un texto generado por ChatGPT? ¿Qué unidad le ponemos a la "similitud" entre dos conceptos para una IA? Algunas de estas medidas ni siquiera tienen nombre oficial todavía, y otras dependen enteramente de pruebas de laboratorio (*benchmarks*).

Nuestra regla de diseño para estos casos es sencilla: 1. Si la unidad existe en el catálogo QUDT, se usa QUDT. 2. Si es un concepto inventado ayer, el desarrollador define una unidad nueva en el eje K, la describe, y si es posible, le programa una fórmula para convertirla a una unidad conocida.

Para el mundo de la IA, el eje K tendría que alojar unidades como estas:

K:token	– La unidad básica con la que la IA lee el texto.
K:parametro_modelo	– Unidad para medir el "cerebro" de la IA (ej: 7B, 70B).
K:precision_clasificador	– La tasa de acierto, medida entre 0 y 1.
K:USD_por_millon_tokens	– La unidad con la que OpenAI o Google te cobran la factura.

Esa última es sumamente interesante: es una "unidad compuesta" que mezcla dinero y capacidad de procesamiento. Los sistemas maduros permiten armar estas unidades híbridas y guardarlas en el eje K sin que la arquitectura

tiemble.

Conversiones y sumas: El fin de las matemáticas ciegas

Una vez que cada número de tu base de datos tiene su unidad explícitamente pegada a él, operaciones que antes eran peligrosísimas se vuelven seguras y automáticas.

La conversión: Imagina que tienes un panel de control que recibe la latencia de cuatro servidores distintos. Uno te manda el dato en milisegundos, dos en microsegundos y otro en segundos. Si tu sistema tiene las unidades mapeadas en K, la base de datos se encarga de convertir todo a una misma unidad antes de sumar. La regla matemática vive centralizada en K, y el programador no tiene que escribir traductores a mano.

La agregación: Si le pides a un sistema mal diseñado que saque el promedio de "340 ms, 1200 μ s y 0.7 s", la máquina va a sumar los números desnudos ($340 + 1200 + 0.7$) y dividirlos, entregándote un dato ridículo y totalmente falso. Con nuestro modelo, la máquina sabe que no puede mezclar peras con manzanas y homogeneiza los datos antes de operar. (Aunque parezca increíble, este error de suma ciega ocurre todos los días en paneles de análisis de datos corporativos).

La inflación y el tiempo: Los 100 dólares que costaba algo en 2020 no valen lo mismo que 100 dólares en 2026. Si estás modelando un contrato indexado por inflación, registrar "100 dólares" no es suficiente. Ese número tiene que llevar pegada la unidad (USD) y la **fecha exacta de denominación**. El valor del dinero es una propiedad que cambia con el tiempo, y nuestro modelo está diseñado para gestionar esto a través de la bitemporalidad (un concepto que abordaremos más adelante).

Rangos y porcentajes: Lidiando con la incertidumbre

En el mundo real, los números casi nunca son exactos. Cuando lees un estudio sobre un modelo de lenguaje, te dicen que su nivel de acierto es " $78.3\% \pm 2.1\%$ ". Cuando un servidor mide su latencia, no te da un número fijo, te da una campana de Gauss con promedios y percentiles.

Nuestro eje N está preparado para recibir la respuesta a "¿cuánto?" en tres formatos distintos:

1. **Valor puntual:** Un número fijo y simple (ej: 340 ms).
2. **Rango:** Un intervalo con piso y techo (ej: [300 ms, 400 ms]).
3. **Distribución:** Aquí la cosa se pone profesional. La distribución estadística se reifica; es decir, se convierte en una entidad dentro del eje O, y se le añaden propiedades como la media, la varianza y el tipo de curva.

Esto último es vital al trabajar con IA. Si le haces la misma pregunta dos veces a un modelo como GPT-4, te dará dos respuestas distintas. La "calidad" de la IA no es un número clavado en piedra; es una distribución de probabilidades. Cuando un estudio dice que un modelo tiene un 78.3% de eficacia, te está dando el promedio de 14.000 pruebas distintas. Modelar bien los datos significa que nuestro sistema sepa que ese 78.3% no es una métrica plana, sino la punta del iceberg de un evento estadístico complejo.

Los cuatro ejemplos pasan por el eje N

Veamos cómo se comporta el eje N cruzando nuestros escenarios habituales, sumándole el ecosistema de la Inteligencia Artificial:

La Receta: 200 gramos de harina, 30 minutos a 180 °C, rinde para 4 porciones. Tenemos tres números atados a tres unidades distintas (gramo , minuto , °C), más un número solitario que indica cantidad (4 porciones). **El Gol de fútbol:** Minuto 87, remate desde 22 metros, velocidad del balón a 105 km/h. Tres números, tres unidades físicas. (Nota: el "87" requiere contexto explícito en el sistema para saber si se cuenta desde el inicio del partido o

desde el descanso). **La Canción:** BPM 128, duración de 3 minutos y 24 segundos. (Ojo aquí: si decimos que la canción está en "Sol Mayor", eso no va a N, porque una nota musical no es un número matemático, es una categoría que pertenece al eje K). **La Noticia política:** Un decreto entra en vigor en 30 días, mueve un presupuesto de 50 millones de dólares y afecta a 3 ministerios. Tres números, tres unidades, siendo una de ellas altamente sensible a la inflación temporal. **La llamada a una IA:** Entrada de 4.500 tokens, salida de 1.200 tokens, latencia de 2.3 segundos, costo de 0.018 dólares y precisión de $78.3\% \pm 2.1\%$. Seis números, seis unidades, y dos de ellos incluyen márgenes de incertidumbre.

La verdadera magia ocurre cuando un usuario hace una consulta que cruza todos estos mundos. Por ejemplo: "¿Cuántos dólares y tokens me cuesta pedirle a una IA que analice un decreto gubernamental de 30 páginas sobre importación de comida?"

Es una pregunta perfectamente lógica para un negocio moderno. Mezcla unidades de páginas, tokens, dólares y tiempo. Y todas estas variables exigen que el sistema sepa exactamente qué unidad acompaña a cada número y cómo se convierten matemáticamente entre sí. Sin esa capa de inteligencia estructural, cualquier respuesta que dé la base de datos es una simple adivinanza disfrazada de cálculo.

Resumen del eje N

Antes de saltar al siguiente concepto, consolidemos lo que hemos construido con el eje cuantitativo:

1. **N es un eje de valor puro.** A diferencia de los objetos, los números son su propia identidad y no necesitan un código interno (UUID) para existir en la base de datos.
2. **La desnudez numérica está prohibida.** Todo número ingresa al sistema con una unidad de medida pegada a él. Esa unidad siempre habita en el catálogo del eje K.
3. **La medición como evento.** Si la unidad de medida no es obvia, o si los datos sufrirán conversiones matemáticas, la medición se reifica: pasa a ser un evento formal en el eje O.
4. **Estándares mundiales.** Utilizamos el catálogo oficial **QUDT** para no inventar unidades básicas. Para conceptos emergentes (como los tokens o la precisión de un LLM), creamos unidades personalizadas en K.
5. **Tolerancia estadística.** El eje soporta la incertidumbre modelando rangos o distribuciones estadísticas completas, no solo promedios planos.

Trampa de programación: el número que viaja desnudo

El error clásico, el que aparece en producción y cuesta dinero real: guardar `100` en una columna `monto` y dar por hecho que "ya se sabe" que son dólares. Un día el sistema integra un proveedor que factura en yenes, otro operario carga `100` pensando en soles, y la suma `100 + 100` arroja `200` de una moneda que no existe. Es la misma falla que, a otra escala, le costó a la NASA el Mars Climate Orbiter en 1999: un módulo entregaba libras-fuerza y el otro leía newtons. El número, solo, **miente por omisión**.

La regla del eje N lo previene de raíz: una magnitud nunca se almacena como un escalar pelado, sino como el par **valor + unidad**, y la unidad vive en K con su URI canónica (QUDT). El motor que intente sumar `100 USD` con `100 JPY` se detiene en seco, igual que se negaría a sumar peras con tornillos. La cantidad deja de ser un número y pasa a ser un **hecho completo y autoexplicativo**.

El inventario de cajas está completo

Con el pilar N instalado, nuestro universo de catalogación está formalmente completo. Ya contamos con los cuatro pilares del mundo físico (quién, qué, dónde, cuándo), tenemos el zócalo intelectual para las categorías (K) y acabamos de asegurar el terreno para las magnitudes matemáticas (N). En resumen, **ya tenemos los cajones listos para alojar cualquier tipo de valor existente**.

Lo que nos falta no son más cajones. Lo que nos falta ahora son los **cables** que conecten todos estos valores entre sí. Necesitamos etiquetas que le expliquen a la máquina *cómo* un agente interactúa con un objeto.

Esos cables lógicos viven en un eje adicional: el eje **M** (*cómo*), el de los predicados o conectores que le dan vida al modelo. Y la diferencia entre "tener una propiedad" y "estar relacionado con algo" es, a nivel informático, muchísimo más delgada de lo que dicta nuestro sentido común: es solo cuestión de cardinalidad.

Capítulo 5 — Cómo: los predicados (M)

El universo está completo, pero las cosas siguen sueltas

Hagamos un balance de lo que hemos construido hasta ahora en los capítulos anteriores. Nuestro modelo ya cuenta con seis "cajas" gigantes (los ejes de valor) donde podemos alojar cualquier elemento del mundo:

Q guarda a las personas y agentes que actúan. **O** guarda los objetos físicos y los eventos. **L** guarda los lugares. **T** guarda las fechas y los momentos. **N** guarda los números y las magnitudes. **K** guarda los conceptos abstractos y las categorías.

Seis ejes, seis tipos distintos de individuos. En teoría, cualquier cosa que exista o pase en el universo puede encontrar un lugar en alguna de estas cajas.

Pero si nos quedáramos solamente con esto, nuestro sistema sería inútil. Imagina que tienes una base de datos donde "Marta" vive en **Q**, "Buenos Aires" vive en **L**, "1984" vive en **T**, el número "45" vive en **N** y el concepto "persona" vive en **K**. Tienes las piezas del rompecabezas, pero... ¿cómo sabe la computadora que esas piezas están conectadas? ¿Cómo le explicamos a la máquina que *Marta vive en Buenos Aires*, que *nació en 1984*, que *tiene 45 años* y que *es una persona*?

Los seis ejes que ya vimos son perfectos para alojar individuos aislados, pero no sirven para guardar los **enlaces**. Nos faltan los "cables" lógicos que le dicen a la máquina qué tiene que ver cada cosa con cada cosa.

Esos enlaces son los protagonistas de este capítulo. En el mundo de la lógica y la programación se les conoce como *predicados*. Y en nuestro modelo todos viven en un único eje estructural: **M** (de *Modos*), el eje que responde a la pregunta **¿cómo?** —cómo se conecta cada cosa con cada cosa.

Con este último eje cerramos oficialmente el inventario de las siete coordenadas. Pero antes de darlo por sentado, este capítulo nos obliga a hacernos una pregunta técnica fascinante: en el fondo de una base de datos, *¿qué diferencia real hay entre una propiedad y una relación?* La respuesta —que ambas son el mismo tipo de cable, y que lo único que las separa es su cardinalidad— se convertirá en nuestra decisión de diseño **D2**, y nos ahorrará miles de horas de programación innecesaria.

Una pregunta que parece de sentido común

Arranquemos con un ejemplo para calentar motores. Imagina que tenemos estos cinco datos sueltos sobre un paciente y sobre un modelo de Inteligencia Artificial:

1. Al paciente_042 le corresponde la edad de 42.
2. Al paciente_042 le corresponde la fecha_nacimiento de 1984-03-17.
3. El paciente_042 vive_en Buenos Aires.
4. El modelo_gpt_4 tiene parámetros por 175.000.000.000.
5. El modelo_gpt_4 fue entrenado_con el corpus de texto C4.

Si le pedimos a un programador junior que divida estos cinco enlaces en "propiedades" (atributos propios de la cosa) y "relaciones" (vínculos con otra cosa externa), lo haría sin dudar: Diría que la *edad*, la *fecha de nacimiento* y la cantidad de *parámetros* son claramente **propiedades** del sujeto. Por otro lado, argumentaría que *vive en* y *entrenado con* son **relaciones**, porque conectan al sujeto con otra entidad separada.

Esta intuición suena muy lógica: *propiedad* = un atributo interno; *relación* = un puente hacia afuera.

Sin embargo, esa lógica se rompe apenas la sometemos a un poco de presión arquitectónica:

- Decimos que *el paciente vive en Buenos Aires* es una relación externa. Pero ¿no podríamos diseñar la tabla diciendo que el paciente tiene un campo interno llamado `ciudad_residencia` que equivale a Buenos Aires? Bajo esa óptica, se vuelve una propiedad.
- Decimos que *el modelo tiene 175 mil millones de parámetros* es una propiedad interna. Pero, matemáticamente, podríamos leerlo como que el modelo "está conectado o se relaciona" con el número gigante 175.000.000.000 en el eje N. Bajo esa óptica, se vuelve una relación externa.
- Decimos que *la fecha de nacimiento* es una propiedad. Pero el valor "1984" es un individuo con derechos propios que vive en el eje T. El paciente, en realidad, se está relacionando con un punto en el tiempo.

Si analizamos estos cinco enlaces con frialdad informática, descubrimos una verdad incómoda: **la diferencia entre propiedad y relación no existe en la realidad material**. Es solo una diferencia gramatical sobre cómo los humanos redactamos las oraciones. A veces usamos verbos posesivos ("tiene 42 años") y a veces verbos de lugar ("vive en Buenos Aires").

Pero para el disco duro de la computadora que guarda el dato, la estructura técnica es idénticamente la misma: hay un sujeto de partida, un cable conector y un objeto de destino.

Esa observación técnica es el punto de partida de nuestra regla D2. Pero antes de formalizarla, miremos cómo es esa "estructura técnica idéntica" que comparten todos los cables.

La anatomía del cable: la signatura tipada

Todo enlace dentro de nuestro modelo —no importa si lo llamas propiedad o relación— obedece matemáticamente a una misma forma de tres partes:

hecho = (sujeto, predicado, objeto)

Pero el modelo esconde un truco de seguridad brutal: cada enlace o cable (el predicado) no es un texto ciego; viene de fábrica con una **signatura**. Esta signatura es como la etiqueta de un enchufe que le indica a la computadora de qué eje tiene que venir obligatoriamente el sujeto, y a qué eje debe ir a conectarse el objeto.

Si lo vemos como funciones matemáticas, los cinco enlaces anteriores tienen estas etiquetas de seguridad:

```
edad           : va desde Q hacia N (De Personas a Números)
fecha_nacimiento : va desde Q hacia T (De Personas a Fechas)
vive_en        : va desde Q hacia L (De Personas a Lugares)
parametros     : va desde O hacia N (De Objetos a Números)
entrenado_con   : va desde O hacia O (De Objetos a otros Objetos)
```

La signatura es la magia que convierte un montón de datos desordenados en una estructura **predecible y validable**. Si un programador comete un error de código e intenta guardar esto: `(paciente_042, edad, "color rojo")`, el sistema lo bloquea automáticamente. ¿Por qué? Porque la signatura del cable `edad` advierte que el destino tiene que estar en la caja **N** (los números), y "color rojo" es un texto.

Esta capacidad de bloquear errores absurdos desde el momento en que se ingresan los datos es la ventaja comercial más grande de este modelo. Es lo que separa a la arquitectura sólida de WQuestions del caos que generan los grafos libres (como RDF), donde cualquiera puede conectar a un paciente con un color usando el verbo "edad" sin que el sistema detecte la falla.

La verdadera diferencia: ¿funcional o múltiple?

Visto que la forma del cable es siempre la misma (`sujeto - cable - objeto`), ¿existe alguna razón técnica para distinguir entre "propiedades" y "relaciones"? Sí, y la diferencia es estrictamente matemática. Se trata de la **cardinalidad**.

Vuelve a mirar nuestras cinco signaturas. Cuatro de ellas se comportan como lo que en matemáticas llamamos **funciones cerradas**: dado un sujeto específico, el cable solo puede conducir a *un único* destino posible en ese instante. Un paciente solo puede tener *una* edad matemática hoy; solo puede tener *una* fecha de nacimiento y solo tiene *una* ciudad donde reside principalmente. Un modelo de IA tiene *un solo* conteo exacto de parámetros.

El quinto cable — `entrenado_con` — rompe esa regla. Un modelo de IA puede haber sido entrenado usando el corpus C4, la Wikipedia y libros de dominio público al mismo tiempo. Es decir, un mismo sujeto puede disparar varios cables hacia múltiples objetos simultáneos, y todos son correctos.

Esta es la única y verdadera diferencia estructural en la base de datos entre lo que solemos llamar "propiedades" y "relaciones":

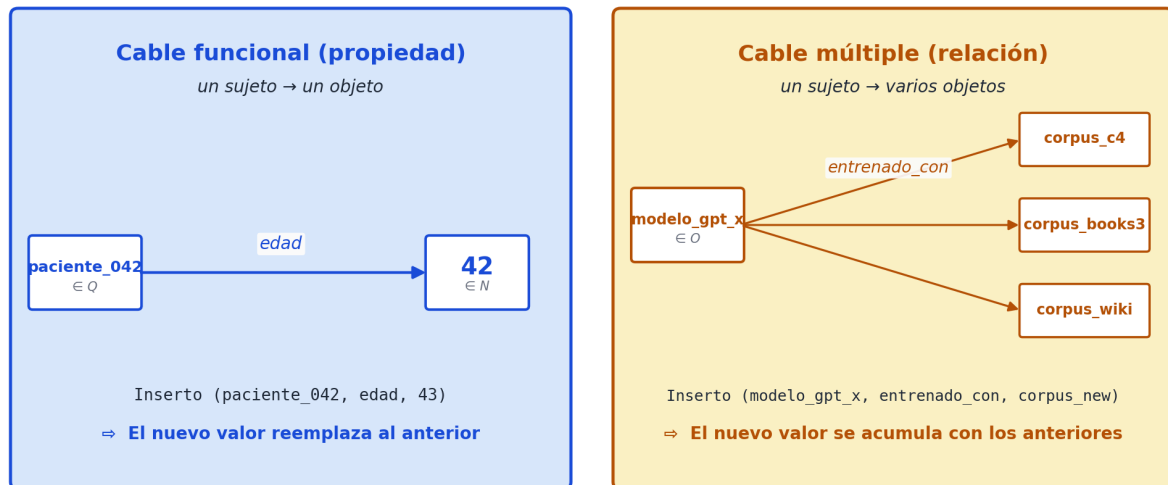
- Los cables **funcionales** (los que solo admiten *un destino único* por sujeto) son los que solemos llamar **propiedades**.
- Los cables **no funcionales** (los que permiten que el sujeto apunte a *múltiples destinos* a la vez) son los que solemos llamar **relaciones**.

Pero ambos son el mismo tipo de cable y viven en el mismo eje (**M**, *cómo*). La cardinalidad no es un eje aparte: es un **atributo de cada cable**, declarado en su signatura. Y lo único que ese atributo le indica a la base de datos es **cómo comportarse cuando el dato se actualice**. Si el cable es funcional (único) y llega información nueva, el sistema **borra y reemplaza** lo anterior. Si el paciente cumple años, borramos el 42 y ponemos un 43; no acumulamos edades. En cambio, si el cable es múltiple y llega información nueva, el sistema simplemente la **agrega a la lista**. Si el modelo de IA es entrenado con una biblioteca nueva, ese texto se suma al historial de entrenamiento sin borrar a la Wikipedia.

Esa diferencia de cardinalidad es lo único que distingue una propiedad de una relación. Pero, y aquí viene el alivio para el programador: **el motor de búsquedas del sistema no hace diferencias**. Si tú le preguntas a la base de datos "*¿Qué edad tiene el paciente 042?*" o "*¿Con qué fue entrenado el modelo X?*", la máquina ejecuta exactamente el mismo código de búsqueda. Solo que en el primer caso te devuelve una sola respuesta, y en el segundo te devuelve una lista.

Propiedades y relaciones: la única diferencia es la cardinalidad

D3: misma forma sujeto-predicado-objeto; solo cambia cuántos objetos puede haber



Cardinalidad de los cables: misma forma sujeto-predicado-objeto. Un cable funcional reemplaza el valor anterior; uno múltiple se acumula con los previos.

La regla de diseño D2: La unificación matemática

Esta revelación nos lleva a establecer de manera formal una de las decisiones de diseño más elegantes del modelo:

D2 — Las propiedades y las relaciones se unifican bajo el mismo concepto: son simplemente cables (predicados) del eje cómo (M) con etiquetas de seguridad (signaturas tipadas). La única diferencia entre ellas es la cardinalidad —si aceptan uno o varios destinos simultáneos—, y esa cardinalidad es un atributo de la signatura de cada cable, no un eje aparte. Le indica a la base de datos cuándo debe "borrar y reemplazar" (cable funcional) y cuándo debe "acumular" (cable múltiple) nueva información.

Entender la regla D2 significa comprender que el enorme abismo técnico que se nos enseña en la universidad —donde nos obligan a tratar a los atributos internos como "columnas en una tabla" y a los vínculos externos como "tablas conectadas o JOINS"— es, en realidad, **una ilusión pedagógica**. Es una complicación innecesaria de la programación tradicional. A nivel matemático, todo es simplemente un sujeto, un cable tipado y un destino. Y esa simplicidad es la gasolina que hace que este modelo vuele.

¿Qué gana una empresa unificando esto?

Adoptar la regla D2 tiene cuatro beneficios gigantescos y directos en entornos de producción:

1. **Un motor de búsqueda unificado:** El sistema ya no necesita tener un "lenguaje para consultar datos internos" y otro "lenguaje para consultar cruces de tablas". El código se reduce a una sola instrucción maestra: "dado este sujeto y este cable, devuélveme el destino". Le decimos adiós a los enredados *SELECT* y *JOIN* del SQL tradicional.
2. **Facilita el trabajo de la IA (JSON universal):** Cuando un agente de Inteligencia Artificial (LLM) solicita datos a través de *function calling*, el sistema le devuelve todo empaquetado en exactamente la misma

estructura limpia y predecible. La IA no gasta tokens tratando de averiguar si la edad viene en un formato y el listado de amistades viene en otro. La uniformidad abarata la integración corporativa.

3. **Extensibilidad a prueba de balas:** Si mañana la empresa quiere registrar un dato nuevo sobre sus clientes, los programadores no tienen que reunirse a debatir si crear una columna nueva en la tabla o construir una tabla intermedia entera. Simplemente añaden el nuevo "cable" al diccionario, deciden si la respuesta es única o múltiple, y el motor de la base de datos ya sabe qué hacer.
4. **Habilitan el procesamiento del lenguaje natural:** En capítulos posteriores comprobaremos que cada verbo que usamos los humanos (como "vender", "comer", "firmar") exige ciertos roles. Cuando le decimos al sistema: *"María le vendió un libro a Juan"*, la máquina registra al vendedor, al comprador y a la mercancía usando esta estructura uniforme. Al no hacer diferencias entre propiedades y relaciones, el sistema lee frases humanas como si leyera código de máquina.

Tres dominios bajo el microscopio

Para terminar de consolidar este concepto, veamos cómo se reparten los cables funcionales y múltiples cuando los aplicamos al diseño de tres mundos totalmente distintos.

1. El mundo de la gastronomía: Una receta

```
Cables funcionales (respuesta única):
tiempo_preparacion : va de 0 hacia N (15 minutos)
tiempo_coccion     : va de 0 hacia N (45 minutos)
porciones          : va de 0 hacia N (4)
dificultad         : va de 0 hacia K (intermedia)

Cables múltiples (acumulan):
ingrediente        : va de 0 hacia K (varios ingredientes por receta)
paso               : va de 0 hacia O (muchos pasos en la preparación)
utensilio          : va de 0 hacia K (varios utensilios a ensuciar)
```

La lógica se cumple: lo que el sentido común considera "atributos estáticos" del plato es funcional (un solo valor); lo que son "piezas que se suman" es múltiple.

2. El mundo del software: Una llamada a la Inteligencia Artificial

```
Cables funcionales (respuesta única):
tokens_entrada     : va de 0 hacia N (4500)
latencia_ms        : va de 0 hacia N (2300)
costo_usd          : va de 0 hacia N (0.018)
modelo_usado       : va de 0 hacia K (gpt-4-turbo)

Cables múltiples (acumulan):
fuente_documento   : va de 0 hacia O (la IA pudo consultar 5 PDFs distintos)
```

Mismo patrón de fondo: el consumo exacto en dinero es uno solo (funcional), pero los documentos analizados durante la consulta pueden ser docenas (múltiple).

3. El mundo del deporte: Un partido de fútbol

```
Cables funcionales (respuesta única):
  resultado_final      : va de 0 hacia K  (victoria_local, empate)
  asistencia           : va de 0 hacia N  (52.000)

Cables múltiples (acumulan):
  partes_jugando       : va de 0 hacia Q  (son siempre 2 equipos, es múltiple)
  gol_anotado          : va de 0 hacia 0  (puede haber varios goles)
  tarjeta_amarilla     : va de 0 hacia 0  (puede haber docenas)
```

Aquí la distinción requiere una mirada afilada. El cable `partes_jugando` (los dos equipos) engaña: podrías pensar que es **funcional** porque el número de equipos en un partido *siempre* es exactamente dos, un número fijo. Sin embargo, es **múltiple**. ¿Por qué? Porque la regla no trata sobre si el número es fijo o infinito, la regla dicta que un cable funcional solo admite **una única respuesta posible**. Al haber dos respuestas legítimas para el mismo partido (el equipo A y el equipo B), matemáticamente es un cable múltiple.

Trampa de programación: la relación disfrazada de columna

El error que se paga caro al escalar: modelar una relación como una **columna** de la tabla. Una persona tiene teléfono, así que ponemos una columna `telefono`. Funciona hasta que alguien tiene dos números — y empieza el parche: `telefono2`, `telefono_alt`, un campo de texto con comas, una tablita improvisada a las apuradas. La columna asumió, sin decirlo, una **cardinalidad** (un solo valor) que el mundo no respeta.

El eje M evita el problema poniendo la cardinalidad **en la signature del predicado**, no en la forma de la tabla: `telefono` se declara de antemano como funcional (un valor) o múltiple (varios), y el modelo lo respeta sin rediseños. Distinguir "tener un atributo" de "estar relacionado con algo" deja de ser una decisión rígida de esquema y pasa a ser un dato más — un cable con su cardinalidad escrita en la firma. Pasar de "un teléfono" a "muchos" es cambiar una marca en M, no migrar una tabla.

Cierre de la Parte II: El tablero está listo

Con la formalización del eje *cómo*, hemos terminado de construir todo el andamiaje del modelo. El inventario de las siete coordenadas maestras está oficialmente cerrado y listo para operar:

- **Q (Quién)** — Las personas y agentes.
- **O (Qué)** — Los objetos físicos, eventos y situaciones.
- **L (Dónde)** — Las locaciones.
- **T (Cuándo)** — El flujo del tiempo.
- **N (Cuánto)** — Los números y magnitudes matemáticas.
- **K (Cuál)** — Los conceptos abstractos y las categorías (tipos, unidades, estados, vocabularios).
- **M (Cómo)** — Los cables o predicados que conectan los ejes de valor (funcionales o múltiples).

Seis cajas gigantes para guardar "cosas", entrelazadas por una red de cables estructurales (el eje *cómo*, M). Cualquier fenómeno, suceso, registro bancario o reporte médico que ocurra en el mundo cabe, sin distorsiones, dentro de esta matriz de siete ejes. Cumplimos con la promesa inicial del libro: hemos demostrado que el universo de los datos puede ser mapeado usando únicamente las preguntas fundamentales del cerebro humano.



Los siete ejes de WQuestions: seis ejes de valor (Q, O, L, T, N, K) alrededor del universo V, conectados por el eje estructural de predicados (M, cómo).

El diagrama bautiza con una letra al conjunto de esas seis cajas: **V**, *el universo de valores*. No es un séptimo eje ni una caja nueva: es, simplemente, **la unión de los seis ejes de valor** (Q, O, L, T, N, K) tomados juntos — todo lo que puede ser *valor* de un hecho, frente a M, que son los cables y no los valores. Lo nombramos desde ya porque más adelante hará falta: cuando una relación no se ate a un eje concreto sino que admita *cualquiera* de los seis, diremos que su dominio o su rango es V (lo veremos al definir las signaturas, en el Capítulo 11).

Pero tener el plano del motor en la mano no es lo mismo que encenderlo. Hasta aquí, hemos visto los ejes por separado, como piezas sueltas de un reloj. Antes de armarlo y verlo girar, conviene cerrar una pregunta que dejamos abierta en el primer capítulo: *¿por qué estas preguntas, y no otras?* Es una pausa breve —el próximo capítulo— y enseguida entramos en la etapa operativa: el diseño en movimiento, observando **cómo interactúan todos los ejes juntos** para estructurar escenarios del mundo real de altísima complejidad.

Capítulo 6 — Por qué estas preguntas, y no otras

Ya tenemos el modelo sobre la mesa. Hemos diseccionado los cuatro pilares concretos (Q, O, L, T), el zócalo categórico (K), el eje de las magnitudes (N) y la urdimbre de relaciones (M) que los teje. En este punto, un lector escéptico tiene todo el derecho a frenar y preguntar lo más incómodo: *¿por qué estas siete preguntas y no otras seis, u ocho? ¿No será este reparto un capricho elegante del autor?*

Es la pregunta correcta, y merece una respuesta que no se apoye en la autoridad de nadie. La respuesta es que **no elegimos estas preguntas: las encontramos**. Cuatro grandes áreas del conocimiento humano —el periodismo, el derecho, la filosofía y la lingüística—, trabajando de forma independiente y separadas por siglos, llegaron exactamente a la misma conclusión estructural. Vale la pena recorrer esa convergencia con calma, porque es la prueba de que el cimiento del modelo no es de diseño, sino de fábrica.

Un aula de 1917

Imagina por un momento que estás en un aula de periodismo de cualquier universidad norteamericana, alrededor del año 1917. El profesor entra, toma una tiza y escribe seis letras mayúsculas en la pizarra: *W, W, W, W, W, H*. Debajo de cada una, anota su significado en inglés: *who, what, where, when, why, how* (quién, qué, dónde, cuándo, por qué y cómo). La consigna para los alumnos es simple pero estricta: para aprobar el ejercicio de esa noche, deben leer un recorte de periódico y subrayar, con colores distintos, las respuestas exactas a esas seis preguntas. Si el texto omite una sola de ellas, la noticia se descarta por estar incompleta.

Este ejercicio no era una ocurrencia pasajera del profesor. El manual *Newspaper Writing and Editing*, publicado apenas cuatro años antes por Willard Bleyer [3], había formalizado esto como la regla de oro de la profesión: cualquier noticia bien construida debía responder a estas seis interrogantes, idealmente en su primer párrafo. Esta regla metodológica, bautizada mundialmente como las **5W1H**, sobrevivió intacta a todo el siglo XX, superó la transición hacia el periodismo digital y se sigue enseñando hoy como el estándar básico en las escuelas de comunicación de todo el planeta.

Lo verdaderamente interesante de esta anécdota no es preguntarnos por qué a Bleyer se le ocurrió la regla, sino por qué funcionó con tanta eficacia. Cuando un académico inventa una regla por capricho, la industria suele olvidarla en la siguiente generación. Pero cuando una norma sobrevive más de cien años en un entorno tan cambiante, es porque ha logrado capturar una verdad estructural profunda. Las 5W1H no sobrevivieron por ser un "invento" brillante de los periodistas; sobrevivieron porque fueron el redescubrimiento moderno de un patrón cognitivo que aparece una y otra vez a lo largo de la historia de la humanidad.

Veinte siglos antes de Bleyer

Bleyer probablemente no era un erudito en historia antigua, pero la matriz analítica que acababa de plasmar en su manual ya llevaba más de dos mil años circulando. Si rastreamos esta idea hacia el pasado, nos topamos de frente con la retórica de la antigua Roma, y específicamente con Cicerón.

A mediados del siglo I antes de Cristo, Cicerón redactó un tratado fundamental para enseñar a hablar en público y argumentar en los tribunales, llamado *De inventione* [2]. En este texto, rescatando conceptos de un filósofo griego llamado Hermágoras, propuso que cualquier análisis serio sobre los actos de una persona debía estructurarse alrededor de siete *circumstantiae* (circunstancias):

quis, quid, ubi, quibus auxiliis, cur, quomodo, quando. Quién, qué, dónde, con qué medios, por qué, cómo, cuándo.

Para Cicerón, esta lista no era un recurso retórico para sonar elegante. Era una herramienta de ingeniería legal. Si un abogado pretendía defender a un acusado argumentando que actuó por necesidad, o buscaba incriminarlo demostrando malicia, tenía la obligación de reconstruir la historia paso a paso utilizando esas siete dimensiones. Si el abogado olvidaba responder a una sola de esas preguntas, su argumento presentaba una falla estructural por donde la contraparte podía destruir su caso.

Un siglo más tarde, otro romano llamado Quintiliano perfeccionó esta metodología en su obra *Institutio Oratoria* [25]. A partir de ahí, la fórmula viajó intacta a través del tiempo hasta llegar a la Edad Media. En su monumental *Suma Teológica*, Tomás de Aquino retomó exactamente estas mismas circunstancias, esta vez para evaluar si una acción humana era moralmente buena o mala [26].

La línea de transmisión histórica es innegable: los juristas romanos le pasaron el esquema a los teólogos medievales, estos se lo entregaron a los pensadores modernos y, para finales del siglo XIX, la misma lista reapareció (ya despojada de sus citas en latín) como un manual de uso práctico para las redacciones de noticias en Norteamérica.

Visto en retrospectiva, resulta casi cómico que Bleyer creyera estar patentando la rueda periodística cuando solo la estaba desenterrando. Sin embargo, la lección que nos importa sacar de aquí es estrictamente arquitectónica. Si la misma lista exacta de preguntas reemerge de forma natural en entornos tan dispares como un tribunal romano, una iglesia medieval y un periódico moderno, se debe a que existe una fuerza de gravedad que obliga a esa lista a regresar. Y esa fuerza no es la tradición cultural —las tradiciones se diluyen o se olvidan—; la lista vuelve a aparecer porque es la única solución lógica a un problema de procesamiento de información universal: ¿cómo hacemos para describir un evento de la realidad sin dejar puntos ciegos?

Aristóteles, todavía más atrás

Y la historia no comienza con Cicerón. Trescientos años antes de que los romanos sistematizaran el derecho, el filósofo griego Aristóteles ya había ejecutado este mismo ejercicio analítico, aunque utilizando otro vocabulario.

En su célebre tratado *Ética a Nicómaco* [1], Aristóteles se enfrentó al problema de cómo diferenciar una acción realizada a propósito de una ocurrida por accidente. Para resolverlo, trazó un inventario de las variables que una persona debe conocer al momento de actuar para que la sociedad pueda considerarla plenamente responsable. Traducido a un lenguaje contemporáneo, el planteamiento de Aristóteles sostenía lo siguiente:

Para determinar si alguien actuó en completa ignorancia, debemos verificar si desconocía: (a) quién es él mismo, (b) qué es lo que está haciendo, (c) a quién o sobre qué está recayendo la acción, (d) con qué instrumento lo hace, (e) por qué lo hace, o (f) cómo lo hace (por ejemplo, si aplicó fuerza moderada o violencia extrema).

Si observamos el desglose, la estructura es idéntica a la que venimos rastreando: identifica al agente, la acción, el paciente, el instrumento, el motivo y el modo. Es cierto que falta la variable temporal ("¿cuándo?"), pero su ausencia en este párrafo específico es totalmente lógica: a la hora de juzgar si alguien es moralmente responsable de haber golpeado a otro, el momento exacto en el reloj es un dato casi siempre irrelevante. No obstante, el "cuándo" reaparece con fuerza en otros textos de Aristóteles cuando teoriza sobre el tiempo y el cambio físico.

En resumen, la filosofía clásica ya estaba atravesada por una intuición metodológica brillante: para lograr comprender cualquier fenómeno del mundo real, es imperativo desensamblarlo en sus dimensiones naturales. Y esas dimensiones, sin falta, terminan mapeándose sobre nuestras preguntas básicas.

El testimonio de la gramática

Hasta este punto de la argumentación, podría parecer que estamos frente a un sesgo cultural, una simple herencia del pensamiento occidental europeo. Sin embargo, la evidencia empírica más contundente de que este esquema es universal proviene de una disciplina estrictamente técnica: la lingüística comparada.

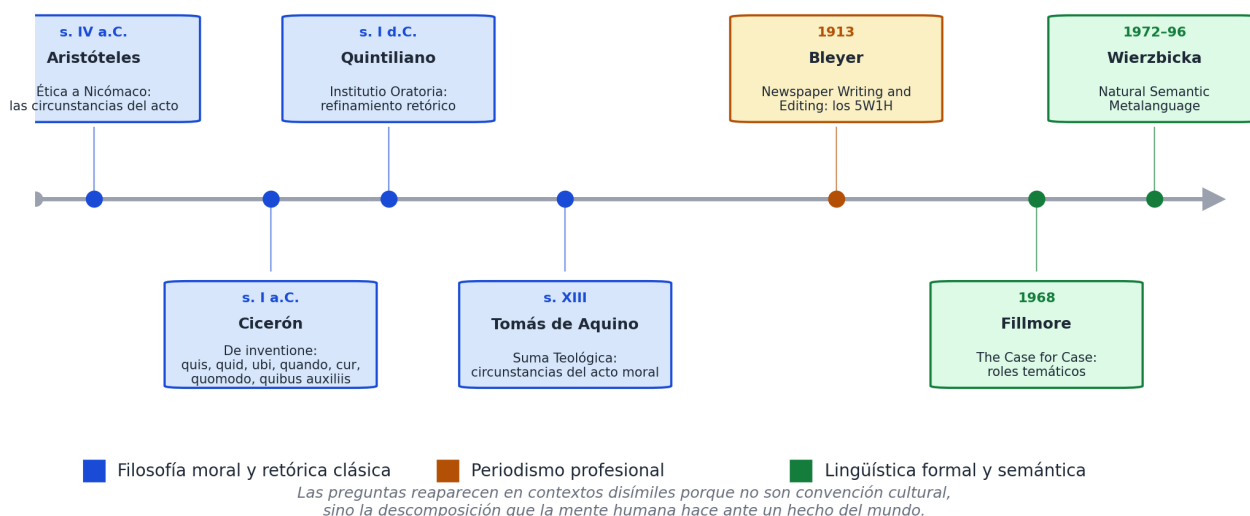
Si analizamos cualquier idioma vivo del planeta —sea quechua, chino mandarín, swahili, euskera, árabe o español— y observamos la mecánica que utilizan sus hablantes para interrogar la realidad, encontraremos invariablemente el mismo conjunto compacto de palabras: *quién*, *qué*, *dónde*, *cuándo*, *cómo*, *por qué*, *cuál* y *cuánto*. Cada lengua tiene, por supuesto, sus propias rarezas gramaticales. Algunas poseen pronombres distintos para decir "quién" en singular o en plural; otras utilizan partículas específicas para diferenciar entre "dónde estoy" y "hacia dónde voy". Pero, a nivel semántico, el núcleo de preguntas base es asombrosamente idéntico en toda la especie humana.

A mediados del siglo XX, la investigadora Anna Wierzbicka lideró un proyecto monumental buscando identificar conceptos que significaran exactamente lo mismo en todas las culturas humanas. El resultado fue la formulación del **Metalenguaje Semántico Natural** [28]. En su catálogo final de conceptos universales aparecieron, sin la menor sorpresa, términos como: alguien (quién), algo (qué), dónde, cuándo y por qué. Wierzbicka no incluyó estas palabras por conveniencia teórica; se ganaron su lugar en la lista tras superar pruebas de traducción exhaustivas en decenas de lenguas pertenecientes a comunidades aisladas que jamás tuvieron contacto con Occidente.

Casi en paralelo, la comunidad de la lingüística formal le otorgó a este fenómeno un nombre técnico que todavía se utiliza en ciencias de la computación: **roles temáticos** [24]. En 1968, Charles Fillmore postuló que la mente humana, al procesar cualquier verbo, asigna automáticamente "roles" específicos a las entidades que lo rodean. Estos roles son fijos: existe un *agente* (quién), un *tema* o paciente (qué), una *locación* (dónde), una *temporalidad* (cuándo), un *instrumento* (con qué) y un *beneficiario* (para quién). Dependiendo de la escuela lingüística, el modelo puede contar con ocho o doce roles, pero el chasis estructural coincide milimétricamente con el análisis que Aristóteles hizo veintitrés siglos atrás.

Contamos, entonces, con cuatro disciplinas distintas operando bajo cuatro nomenclaturas diferentes. Aristóteles hablaba de las *circunstancias del acto*; Cicerón las bautizó como *circumstantiae*; las escuelas de periodismo las empaquetaron como las *5W1H*; y la lingüística formal las catalogó como *roles temáticos*. Pero al tabularlas y compararlas, la conclusión es ineludible: todos descubrieron exactamente la misma estructura base de la información.

Cuatro tradiciones independientes, mismo descubrimiento



Línea de tiempo de las cuatro tradiciones independientes que convergen, sin coordinarse, en el mismo conjunto reducido de preguntas.

El niño que pregunta

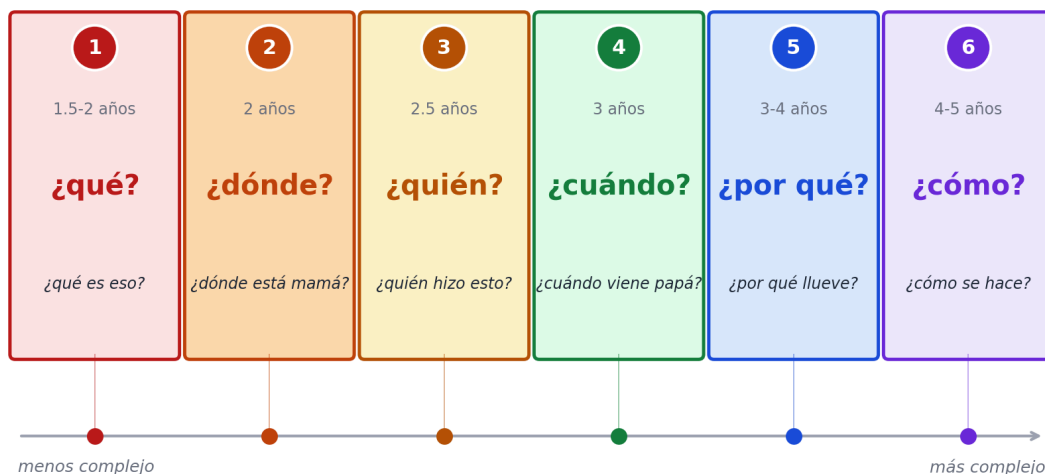
Nos queda una última pieza de evidencia, y resulta ser la más profunda a nivel biológico. Este inventario de preguntas no solo reside en tratados de filosofía o en la estructura de idiomas milenarios; se manifiesta de forma espontánea durante el desarrollo cognitivo temprano de cualquier niño, y lo hace en un orden de aparición increíblemente estricto sin importar su país de nacimiento.

En 1973, el investigador Roger Brown llevó a cabo un estudio longitudinal clásico [27] sobre la adquisición de la lengua materna. Documentó un patrón fascinante que posteriormente fue replicado y confirmado en niños hispanohablantes, franceses, japoneses y hebreos: los infantes no asimilan todas las preguntas al mismo tiempo. Su cerebro las desbloquea siguiendo una secuencia inalterable:

1. La primera herramienta en aparecer es el **qué** ("¿Qué es eso?"). Es el mecanismo de escaneo más básico para construir vocabulario e identificar el entorno material.
2. Le sigue el **dónde** ("¿Dónde está mamá?"). Emerge cuando el niño adquiere la noción de permanencia, entendiendo que los objetos existen aunque desaparezcan de su campo visual.
3. A continuación se activa el **quién** ("¿Quién hizo esto?"). Es el momento en que el cerebro logra distinguir entre un objeto pasivo y un agente capaz de ejercer fuerza y voluntad.
4. El **cuándo** tarda mucho más en llegar (generalmente cerca de los tres años de edad). Esto obedece a que el cerebro primero necesita madurar la abstracción mental necesaria para diferenciar el pasado del presente y proyectar un futuro.
5. El **por qué** aparece casi a la par, desatando la conocida "etapa de los porqués", donde el niño busca entender la causalidad mecánica de las cosas.
6. Finalmente, el **cómo** suele ser la última métrica en consolidarse. Es una interrogante altamente compleja, ya que exige que el individuo sea capaz de conceptualizar un proceso secuencial completo, en lugar de un simple hecho aislado.

Orden de adquisición de las preguntas-W en niños

Observado en hablantes de inglés, español, francés, mandarín, japonés, hebreo



fuelle: Brown, R. (1973) y estudios comparados posteriores

Las preguntas-W aparecen en los niños en un orden estable a través de los idiomas, lo cual sugiere que el orden refleja complejidad cognitiva del concepto, no del idioma.

La belleza arquitectónica de este hallazgo radica en que la secuencia de aprendizaje no está dictada por las normas culturales ni por la dificultad gramatical de cada idioma. Este orden inalterable nos demuestra que las dimensiones factuales poseen distintos niveles de exigencia computacional para el cerebro. Identificar un objeto tangible ("¿qué

es eso?") requiere menos carga cognitiva que ubicar una acción en un continuo espacio-temporal ("¿cuándo ocurrió?").

Si los seres humanos, independientemente de su origen, despliegan estas interrogantes en el mismo orden secuencial, la conclusión tecnológica es contundente: estas preguntas no son una convención inventada por la sociedad. Son el firmware de fábrica; el mecanismo primario de procesamiento de datos con el que nuestra mente logra darle sentido a la entropía de la realidad.

Las preguntas como invariantes cognitivos

Con este panorama empírico sobre la mesa, ya podemos enunciar explícitamente la tesis que sostiene en silencio todo lo que el modelo construyó en los capítulos anteriores:

Las preguntas fundamentales (quién, qué, dónde, cuándo, cómo...) no son un capricho cultural ni un artefacto inventado por una disciplina académica. Constituyen "invariantes cognitivos": las dimensiones estructurales universales mediante las cuales el cerebro humano desensambla y almacena la realidad. Operaban con total precisión mucho antes de la invención del derecho, la estadística, las bases de datos relacionales, o de que un niño sea siquiera capaz de estructurar una oración fluida.

Aunque esta sea una afirmación audaz, la validación empírica sostiene sus pilares:

- **¿Aparecen sistemáticamente en el análisis humano?** Sí. Filósofos griegos, juristas romanos, periodistas americanos y lingüistas modernos convergieron en el mismo vector dimensional sin haber cruzado información metodológica.
- **¿Son independientes del idioma?** Sí. Los pronombres interrogativos fungen como una clase léxica universal, manteniendo su núcleo semántico en lenguas sin ninguna relación etimológica.
- **¿Emergen de forma innata?** Sí. Su aparición en el desarrollo infantil sigue un cronograma rígido, transversal a todas las culturas y lenguajes.
- **¿Preceden a la especialización científica?** Sí. Ningún niño puede ser instruido en taxonomías biológicas o principios de física si antes no domina la operatoria básica de la pregunta "¿qué?".

Nos enfrentamos a cuatro predicciones estructurales, y las cuatro ofrecen confirmaciones sólidas. Si bien las ciencias cognitivas continúan debatiendo detalles granulares de estos procesos, el volumen de evidencia apunta unívocamente hacia un mismo centro de gravedad.

El cimiento de diseño más robusto

Llegados a este punto, un arquitecto de software pragmático podría preguntarse: *"Todo este recorrido antropológico y filosófico es fascinante, pero ¿de qué me sirve a mí si mi único objetivo es diseñar una base de datos escalable o programar el backend de un agente de Inteligencia Artificial?"*

La utilidad es inmensa y estrictamente **arquitectónica**. Si acabamos de demostrar con evidencia histórica y biológica que estas preguntas *son* la estructura nativa mediante la cual la mente procesa y clasifica la información, entonces el modelo que venimos construyendo no descansa sobre una preferencia de diseño, sino sobre **el cimiento más robusto posible para edificar cualquier sistema informático**.

Elegir este cimiento cognitivo es infinitamente más seguro que anclar un sistema a los vocabularios de turno de cada industria, porque la jerga corporativa muta año tras año. Es más estable que apostar por los "estándares" de diseño de bases de datos, porque las metodologías tecnológicas se vuelven obsoletas cada década. Resulta ser más

perdurable, incluso, que las propias lenguas maternas, ya que, aunque el euskera y el español suenen incomprensibles entre sí, la matemática subyacente de sus preguntas es la misma.

Y la ventaja operativa de tomar esta decisión en el contexto actual es insuperable: cualquier Inteligencia Artificial moderna —entrenada devorando millones de textos redactados por cerebros humanos— va a asimilar este sistema de datos de forma nativa e inmediata. No hará falta invertir meses en enseñarle a la red neuronal cómo navegar las intrincadas tablas de tu base de datos corporativa; la IA ya comprenderá la topología de la información porque estará organizada bajo la misma lógica inmutable con la que aprendió a leer.

Con esto cerramos la Parte II. Sabemos *cuáles* son las coordenadas y, ahora, *por qué* son esas y no otras. Lo que sigue es ver cómo se ensamblan entre sí: cómo un evento del mundo real se convierte en la unidad atómica del sistema, y cómo la intersección de los ejes crea una geometría de datos coherente.

Capítulo 7 — El hecho atómico

La unidad mínima que hace funcionar todo

A lo largo de los capítulos anteriores nos hemos dedicado a presentar nuestros siete ejes (Q, O, L, T, N, K, M) como si fueran piezas sueltas de un enorme rompecabezas (que la cantidad de piezas no nos abrume, recordemos que toda la información está codificada por millones de ceros y unos, solo necesitamos reglas claras y válidas). Es hora de dejar de mirar las piezas por separado y empezar a armarlo. Para que este rompecabezas de datos tenga sentido y empiece a describir el mundo real, necesitamos una pieza central: **lo que construimos al conectar esos ejes**.

A esta pieza maestra la llamaremos **hecho atómico**. Su estructura informática es tan simple que parece engañosa:

hecho = (sujeto, predicado, objeto)

Solo tres campos. El `sujeto` y el `objeto` son individuos que sacamos de alguna de nuestras cajas de valor (como Personas, Lugares o Números). El `predicado` es el cable conector del eje M (*cómo*); puede ser funcional (un solo valor por sujeto) o múltiple. Y eso es todo. **Absolutamente toda la información que el sistema sabe del mundo se construye, bloque a bloque, utilizando esta misma forma exacta.**

Sé que afirmar esto suena a exageración técnica. ¿De verdad todo cabe aquí? ¿Las recetas, los goles, las canciones de Los Beatles, las leyes de un gobierno, las historias clínicas y las llamadas a ChatGPT? Sí, todo. Este capítulo se va a encargar de demostrarlo. Vamos a ver por qué aferrarnos a esta estructura atómica lo cambia todo, cómo logramos describir cosas súper complejas acumulando piezas simples, y por qué —casi por arte de magia— esta estructura resulta ser exactamente la forma natural en la que los humanos y la Inteligencia Artificial hablan. Para ser honesto, al empezar este libro solo tenía la intuición de que las 7 preguntas podían modelar la información en un multiverso al que se accedía con un par de coordenadas, eje_a vs eje_b, pero la tripleta es el resultado de someter al modelo a las pruebas de laboratorio, más adelante veremos cómo.

La forma inalterable

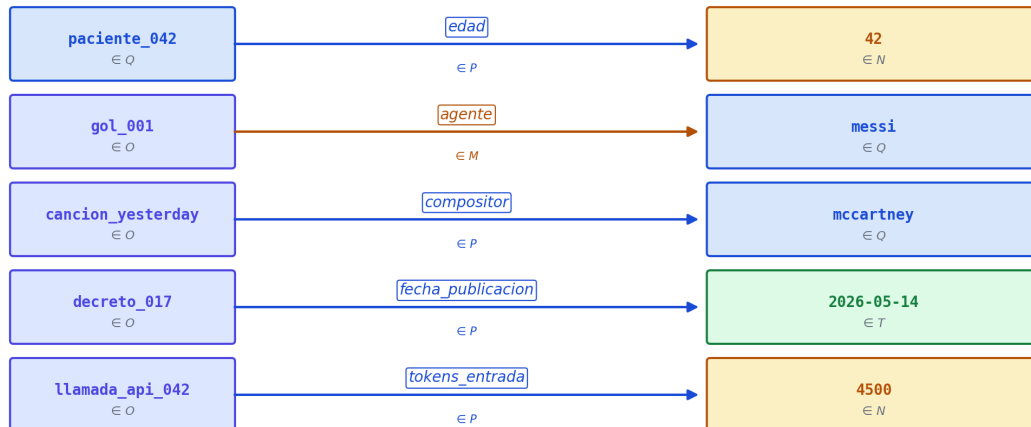
Como dijimos, un hecho atómico es siempre una frase de tres partes (una tripleta). Para visualizarlo, tomemos un ejemplo rápido de cada uno de los escenarios que venimos estudiando:

(receta_risotto,	tiempo_coccion,	45)	∈ M(0, N)
(gol_001,	agente,	messi)	∈ M(0, Q)
(cancion_yesterday,	compositor,	mccartney)	∈ M(0, Q)
(decreto_017,	fecha_publicacion,	2026-05-14)	∈ M(0, T)
(llamada_api_042,	tokens_entrada,	4500)	∈ M(0, N)

Tenemos cinco hechos atómicos. Pertenecen a cinco industrias totalmente distintas y utilizan cinco cables diferentes. Pero, y esto es vital, todos mantienen **exactamente la misma forma estructural**. Ese es el mensaje más potente de este capítulo: esta uniformidad no es un capricho para que el código se vea bonito, es la propiedad arquitectónica que hace que el sistema no se rompa nunca.

El hecho atómico: sujeto · predicado · objeto

Misma forma para todo dominio. Lo que distingue cada hecho es la *signatura tipada* del predicado.



Cada hecho atómico es independiente, tipado y componible.

Las cosas complejas se describen acumulando hechos sobre sujetos compartidos.

Cinco hechos atómicos en cinco dominios distintos: misma forma sujeto-predicado-objeto, distintos tipos de individuos en cada eje. Las flechas azules son predicados funcionales; las ámbar, múltiples.

Veamos por qué esto es tan poderoso.

La regla de diseño D3: El átomo de la información

Ha llegado el momento de escribir esto en piedra. Se trata de nuestra **tercera decisión de diseño** (después de la D1 y D2 que ya estudiamos), y probablemente es la regla más pesada de todo el libro, porque todo lo demás se apoya sobre ella:

D3 — Cualquier hecho de la realidad se va a representar siempre como una "tripleta atómica" de la forma (sujeto, cable, objeto) . Este cable debe estar tipado (saber qué cajas conecta) para que el sistema valide si el dato es lógico o absurdo. Las descripciones de cosas complejas jamás se construyen creando estructuras raras o tablas gigantes; se construyen apilando decenas de estas tripletas simples sobre un mismo sujeto.

Todo lo que leas a partir de aquí es la consecuencia directa de habernos atrevido a tomar esta decisión.

Tres pruebas de fuego que el átomo debe superar

Para que esta "unidad mínima" realmente funcione como el cimiento de cualquier base de datos mundial, tiene que cumplir tres reglas muy estrictas:

Exigencia 1: Tiene que estar "Tipada" (Protección contra el caos). Cada cable lleva pegada su propia regla de seguridad (su signatura). El cable `tiempo_coccion` *sabe de fábrica* que debe conectar un Objeto (eje O) con un Número (eje N). Si un programador cansado intenta ingresar el dato `(receta_risotto, tiempo_coccion, "muy rojo")`, el sistema lo rechaza al instante: la palabra "rojo" no vive en la caja de Números. Esta protección es lo que convierte a nuestra tripleta en un bloque de información seguro y validable en tiempo real.

Exigencia 2: Tiene que ser Independiente. Cada hecho atómico tiene que tener sentido por sí solo, sin depender de nada más. La frase `(gol_001, agente, messi)` significa exactamente lo mismo aunque se te olvide anotar el minuto en que ocurrió el gol. Esta independencia estructural nos da un superpoder: nos permite guardar estos datos en archivos de texto, en bases de datos relacionales, o enviarlos por internet sin que pierdan su significado original.

Exigencia 3: Tiene que ser Componible (Apilable). Aunque los hechos son independientes, su verdadera magia se desata cuando los "apilas" uno sobre otro usando el mismo sujeto para describir realidades complejas. Por ejemplo, una receta de cocina no se describe en una sola frase gigantesca. Se describe con una lluvia de hechos atómicos que comparten a `receta_risotto` como sujeto:

<code>(receta_risotto, instancia_de,</code>	<code>receta)</code>	<code>∈ M(0, K)</code>
<code>(receta_risotto, tiempo_coccion,</code>	<code>45)</code>	<code>∈ M(0, N)</code>
<code>(receta_risotto, unidad_tiempo,</code>	<code>minuto)</code>	<code>∈ M(0, K)</code>
<code>(receta_risotto, porciones,</code>	<code>4)</code>	<code>∈ M(0, N)</code>
<code>(receta_risotto, ingrediente,</code>	<code>arroz_arborio)</code>	<code>∈ M(0, K)</code>
<code>(receta_risotto, ingrediente,</code>	<code>caldo_vegetal)</code>	<code>∈ M(0, K)</code>
<code>(receta_risotto, ingrediente,</code>	<code>azafran)</code>	<code>∈ M(0, K)</code>
<code>(receta_risotto, dificultad,</code>	<code>intermedia)</code>	<code>∈ M(0, K)</code>

Ocho pequeñas piezas de Lego se han apilado para dibujar el plato completo. Cada pieza se entiende por separado, pero juntas forman la receta.

Lo increíble viene cuando el restaurante quiere ser más preciso y añadir el peso exacto de cada ingrediente. En una base de datos tradicional, esto obligaría a destruir la tabla de ingredientes y crear una nueva con más columnas (rompiendo todo el sistema). En nuestra arquitectura, **la estructura jamás cambia**. Simplemente agregamos un par de tripletas más, convirtiendo el uso del ingrediente en un evento físico (algo que ya aprendimos a llamar *reificación*):

<code>(uso_arroz_001, parte_de,</code>	<code>receta_risotto)</code>	<code>∈ M(0, 0)</code>
<code>(uso_arroz_001, ingrediente,</code>	<code>arroz_arborio)</code>	<code>∈ M(0, K)</code>
<code>(uso_arroz_001, cantidad,</code>	<code>320)</code>	<code>∈ M(0, N)</code>
<code>(uso_arroz_001, unidad,</code>	<code>gramo)</code>	<code>∈ M(0, K)</code>

Esta es la ventaja más espectacular del modelo: **el sistema escala acumulando bloques de información, no obligando a los programadores a rediseñar la base de datos.**

El universo de los datos es solo una pizarra gigante

A medida que tu empresa empieza a guardar miles y millones de estos hechos atómicos, se empieza a formar una red inmensa a la que llamamos un *grafo*. Técnicamente hablando, tu base de datos WQuestions no es más que **un conjunto gigante de oraciones de tres partes**. Ni más, ni menos.

Esto es liberador porque significa que "el estado de la realidad" para el sistema se reduce a contar cables. No hay archivos ocultos, ni arquitecturas secretas que solo entienden los creadores del software. Si le pides a la computadora *"¿quién compuso la canción Yesterday?"*, la máquina solo busca cables que arranquen en `cancion_yesterday` y que usen la etiqueta `compositor`.

Si pides algo complejo como *"búscame a todos los compositores que hicieron canciones después de 1960"*, el sistema hace un trabajo muy tonto pero muy rápido: busca todos los cables de `año_composicion` donde el número final sea mayor a 1960; luego mira qué canciones están pegadas a esos cables, y finalmente rastrea quién es su `compositor`. Cada paso es simplemente saltar de una tripleta a otra. Fin del misterio.

Cuando tres palabras no son suficientes: La Reificación en acción

A veces, la realidad es demasiado compleja para resumirla en una sola tripleta. Piensa en la siguiente narración deportiva:

"Messi le pasó el balón a Di María en el minuto 87 con un toque de pierna izquierda."

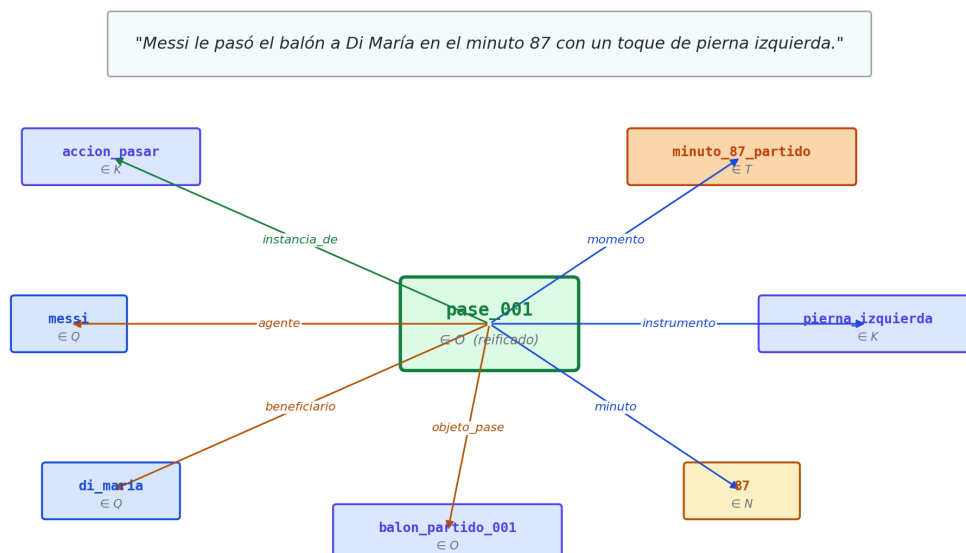
Esta frase es una mina de oro de información: tiene un autor (Messi), un receptor (Di María), un objeto (el balón), un tiempo (el minuto 87) y un instrumento (la pierna izquierda). Si intentamos guardarlo como una tripleta simple (messi, paso_el_balon_a, di_maria), estamos botando a la basura los otros tres datos vitales. ¿Cómo resolvemos esto sin romper la regla D3?

Aquí es donde entra al rescate la **reificación** (de la que ya hablamos antes). El truco es simple: elevamos ese pase de balón al estatus de "Evento Oficial" dentro del eje O. Le damos un nombre propio (pase_001) y conectamos a todos los participantes usando tripletas separadas que apunten a ese evento central:

(pase_001, instancia_de,	accion_pasar)	∈ M(0, K)
(pase_001, agente,	messi)	∈ M(0, Q)
(pase_001, beneficiario,	di_maria)	∈ M(0, Q)
(pase_001, objeto_pase,	balon_partido_001)	∈ M(0, O)
(pase_001, minuto,	87)	∈ M(0, N)
(pase_001, instrumento,	pierna_izquierda)	∈ M(0, K)

Lo que era un nudo lingüístico imposible de guardar en una tabla tradicional, se desarmó elegantemente en seis hechos atómicos independientes.

Eventos n-arios: cuando una tripleta no alcanza



Una oración n-aria se descompone en seis hechos atómicos sobre el evento reificado. Cada uno verificable, consultable y componible por separado.

El pase de Messi reificado como individuo en O y sus seis participantes conectados por roles canónicos: agente, beneficiario, objeto, instrumento, minuto, momento. La estructura n-aria se preserva sin abandonar la forma de tripleta.

La regla práctica que debes tatuarte como ingeniero de datos es esta: **reifica solo cuando te falte espacio**. Si la relación es entre dos sujetos y nada más, usa una tripleta directa. Si el hecho involucra a más personas o tiene detalles propios (como tiempo o lugar), elévalo a Evento Oficial y engánchale sus tripletas correspondientes.

El modelo y el lenguaje humano son hermanos gemelos

Hay un fenómeno asombroso que ocurre cuando ponemos este modelo frente a la Inteligencia Artificial moderna, y es algo que conviene resaltar de inmediato.

Cuando tú le pasas un artículo larguísimo de noticias a un modelo de lenguaje (como ChatGPT o Claude) y le pides: *"Describe paso a paso quién hizo qué en esta noticia"*, el modelo de IA te responde *casi siempre* con una lista estructurada, sin que nadie se lo haya pedido:

- El ministro firmó el decreto.
- El decreto entra en vigor en 30 días.
- El decreto asigna 50 millones de dólares.
- La medida afecta a tres ministerios.

Nos arroja cuatro líneas, cuatro hechos limpios. Y cada línea tiene su Sujeto, su Verbo y su Objeto. La IA no responde así porque conozca nuestro libro de WQuestions; responde así porque **así es exactamente como funciona el lenguaje humano cuando intentamos describir la realidad**. Las oraciones simples que aprendemos en la escuela (Sujeto + Verbo + Predicado) son idénticas, molécula a molécula, a nuestros hechos atómicos tipados.

Esta coincidencia tiene un impacto brutal en el negocio. Si tu base de datos guarda la información usando tripletas, y la Inteligencia Artificial piensa usando tripletas, **la comunicación entre la IA y tu empresa se vuelve totalmente automática**. No necesitas programar costosos algoritmos de comprensión del lenguaje natural (NLP) ni construir capas de traducción complicadas. Existe un "enchufe" directo entre lo que dice la IA y lo que guarda tu sistema.

Esa simplicidad es la llave maestra para lograr lo que tanto busca el mercado hoy: un agente de Inteligencia Artificial capaz de conversar, auditar y guardar información en cualquier empresa del mundo, sin obligar al robot a memorizar la estructura interna de las bases de datos de cada cliente.

Hacer una consulta es simplemente "tapar un hueco"

Sabiendo que toda la información se guarda como tripletas, el proceso de buscar información en la base de datos se vuelve un juego de niños. Hacer una consulta es, literalmente, escribir una tripleta pero tapando uno de los tres campos con un signo de interrogación (como si fuera un hueco que el sistema debe llenar).

Mira estos tres patrones de búsqueda:

```
(messi, ?, ?) → "Dime todo lo que sepas sobre Messi"
(?, compositor, mccartney) → "¿Qué cosas tienen a McCartney como compositor?"
(?, agente, messi) → "Búscame todas las acciones donde Messi fue el protagonista"
```

Incluso, si necesitas hacer una búsqueda digna de un analista avanzado, lo único que haces es escribir varias "tripletas con huecos" juntas, pidiéndole al sistema que encuentre los datos que encajen en todas ellas a la vez:

```
(?gol, agente, messi)
(?gol, parte_de, ?partido)
(?partido, fecha, [2026-01-01, 2026-12-31])
→ Traducción: "Encuentra todos los goles hechos por Messi en partidos del año 2026"
```

Así de simple. No importan los diagramas complejos ni las consultas eternas de código SQL. **Consultar información es simplemente decirle a la máquina que busque un patrón de cables dentro de la gran**

red. La operación matemática es idéntica en cualquier industria y a cualquier escala. Justamente lo que la antigua "Torre de Babel" de datos nos impedía hacer.

El contraste con el mundo relacional es brutal. Esa misma pregunta —*goles de Messi en 2026*—, sobre un esquema SQL clásico, obliga a encadenar tablas que ni siquiera fueron pensadas para conversar:

```
SELECT j.nombre, g.minuto
FROM goles g
JOIN jugadores j ON j.id = g.jugador_id
JOIN partidos p ON p.id = g.partido_id
WHERE j.nombre = 'Messi'
AND p.fecha BETWEEN '2026-01-01' AND '2026-12-31';
```

Y cuando un hecho necesita viajar entre sistemas, se serializa sin perder un solo eje. Así se ve una venta —una situación reificada— como JSON:

```
{
  "id": "venta_74921",
  "instancia_de": "venta",
  "agente": "vendedor_17",
  "cliente": "cliente_1042",
  "objeto": "camiseta_88",
  "monto": { "valor": 49.90, "unidad": "Currency:USD" },
  "momento": "2026-05-14T16:32:00-05:00"
}
```

Cada clave es un cable; cada valor, un individuo en su eje. El mismo átomo que dibujamos como grafo es, sobre el cable, un objeto que cualquier sistema —o cualquier IA— lee sin manual.

Los cuatro ejemplos de siempre, pasados por el átomo

Para cerrar el capítulo con los pies en la tierra, veamos cómo este sistema de hechos atómicos mapea nuestros cinco escenarios principales.

La Receta: Ya la analizamos. Bastan ocho tripletas para describir la idea general del plato, y si requerimos precisión en los gramos de cada ingrediente, reificamos el paso sumando nuevas tripletas sin tocar el diseño base.

El Gol de fútbol:

```
(gol_001, instancia_de, gol_jugada_abierta)
(gol_001, agente, messi)
(gol_001, asistente, di_maria)
(gol_001, parte_de, partido_arg_per_2026)
(gol_001, minuto, 87)
(gol_001, pierna, zurda)
```

Seis simples líneas. Suficiente detalle táctico para las casas de apuestas deportivas, sin reducir el evento a una aburrida columna de Excel.

La Canción:

```
(cancion_yesterday, compositor, mccartney)
(cancion_yesterday, año_creacion, 1965)
(cancion_yesterday, album, help)
(cancion_yesterday, tonalidad, fa_mayor)
(cancion_yesterday, duracion_seg, 125)
```

Cinco hechos que son oro puro para un algoritmo de Spotify. Bastan y sobran para cruzar datos como: "muéstrame canciones hechas antes de 1970 en tono mayor".

La Noticia política (el decreto):

```
(decreto_017, instancia_de, decreto_ejecutivo)
(decreto_017, firmante, ministro_017)
(decreto_017, fecha_firma, 2026-05-14)
(decreto_017, entra_en_vigor, 2026-06-13)
(decreto_017, presupuesto, 50_000_000)
(decreto_017, moneda, dolar)
(decreto_017, afecta_a, ministerio_salud)
(decreto_017, afecta_a, ministerio_educacion)
(decreto_017, afecta_a, ministerio_trabajo)
```

Nueve cables para definir un acto de Estado. Si alguien le pregunta al sistema "*¿qué leyes firmó el ministro 017 este año?*", la máquina cruza las dos primeras tripletas y responde de inmediato.

La Llamada a una IA:

```
(llamada_042, instancia_de, llamada_modelo_lenguaje)
(llamada_042, modelo, gpt_x_2026_05)
(llamada_042, agente_usuario, juan)
(llamada_042, tokens_entrada, 4500)
(llamada_042, tokens_salida, 1200)
(llamada_042, latencia_ms, 2300)
(llamada_042, costo_usd, 0.018)
(llamada_042, temperatura, 0.7)
(llamada_042, herramienta, busqueda_web)
(llamada_042, herramienta, lectura_archivo)
```

Diez hechos atómicos. Contienen toda la telemetría que un programador moderno exige: qué usuario fue, qué IA respondió, cuánto costó el chiste y si el robot tuvo que buscar en internet. Es cien veces más limpio que enviar reportes inentendibles.

Resumen del capítulo

El hecho atómico es la piedra fundacional de nuestra arquitectura. Siempre tiene una forma rígida de tres partes: (*sujeto, cable, destino*). Posee protección interna para evitar que se mezclen cosas absurdas, y tiene la capacidad de combinarse para describir cosas increíblemente complejas sin exigir cambios de programación. En nuestro sistema, la base de datos es literalmente un océano de estos hechos atómicos.

Lo hermoso de este hallazgo no es que hayamos inventado un código nuevo, sino que descubrimos que esta forma matemática coincide milimétricamente con la forma en que los seres humanos (y la Inteligencia Artificial) hablan. El modelo no intenta obligar a las empresas a aprender un idioma robótico; simplemente **toma la estructura natural del lenguaje humano y la convierte en tecnología**.

Hasta aquí hemos tratado cada hecho atómico como una pieza independiente, viviendo por su cuenta. Pero cuando se amontonan todos juntos, se forma un **espacio geométrico**: no una metáfora, sino una herramienta matemática que permite hacer consultas masivas aplicando restricciones sobre coordenadas, como si jugáramos a la batalla naval.

Capítulo 8 — El espacio multidimensional

Una hoja de cálculo imposiblemente grande

Para visualizar cómo funciona este modelo en su totalidad, imagina por un momento la hoja de cálculo de Excel más grande que hayas visto en tu vida. Esta tabla tiene exactamente ocho columnas, una por cada letra de nuestro sistema: **Q, O, L, T, N, P, M, K**.

Cada fila de esta tabla representa un único "hecho" que ha ocurrido en el mundo. En cada celda de esa fila, anotamos el valor que le corresponde a esa columna. Ahora bien, en la inmensa mayoría de las filas, habrá varias celdas vacías. Esto es normal: un hecho cotidiano (como "Juan compró pan") solo involucra a dos o tres columnas (quién, qué), no a las siete a la vez. Pero lo importante es que todas las columnas existen y están siempre disponibles, esperando a que cualquier hecho necesite usarlas.

A nivel de arquitectura de datos, esa inmensa hoja de cálculo es lo que llamamos **el espacio multidimensional de WQuestions**. Tiene 8 dimensiones (las columnas), y los hechos de la realidad son simplemente puntos dentro de ese espacio (las filas). En este espacio hay zonas con muchísima actividad —hechos muy detallados que llenan casi todas sus columnas— y otras zonas casi desiertas. Pero todos, sin excepción, viven bajo las mismas leyes de la física geométrica.

Este capítulo está dedicado a entender esa geometría. Y no lo hacemos por amor a las matemáticas teóricas; lo hacemos porque ver la base de datos como un "espacio geométrico" cambia por completo la forma en que los programadores escriben el código de búsqueda y, sobre todo, cambia cómo los sistemas de Inteligencia Artificial auditan nuestra información corporativa. Veámoslo de cerca.

La metáfora formal (Sin complicarnos demasiado)

Si quisiéramos ponernos estrictamente matemáticos, diríamos que nuestro universo de datos (al que llamaremos **V**) es simplemente la suma de todos los individuos que viven en nuestras cajas físicas y conceptuales: $V = Q \cup O \cup L \cup T \cup N \cup K$. A eso le sumamos nuestros dos tipos de cables conectores (**P** y **M**). Al multiplicar todo esto, creamos el "espacio total" de nuestro sistema.

Pero en la práctica, no necesitamos pensar en fórmulas. Como decíamos, la mayoría de los hechos no usan todas las cajas al mismo tiempo. Un hecho como "*Messi marcó un gol*" solo toca la caja **Q** (Messi), la caja **O** (el gol) y usa un cable de la caja **M** (agente). Las cajas de Lugar, Tiempo o Números se quedan vacías o **sin asignar**.

Por eso es mucho más útil y realista pensar en este espacio como una **estructura parcial**: cada evento de la vida ocupa únicamente las dimensiones que necesita. El sistema no te obliga a inventar datos para rellenar los huecos vacíos. La promesa central de este modelo no es que todo en la vida sea hipercomplejo; la promesa es que *cualquier* hecho, por raro que sea, **puede encontrar su ubicación exacta** en este mapa de siete ejes, dejando en paz las dimensiones que no le importan.

Cuando una empresa empieza a guardar millones de estos hechos parciales, se produce la magia: el sistema se convierte en una nube densa de puntos conectados. Si dos hechos distintos comparten el mismo valor en la caja **O** (porque ambos hablan del mismo gol de Messi, aunque uno hable del minuto y el otro de la pierna que usó), el sistema los conecta automáticamente. Si tres hechos comparten el mismo valor en la caja **T** (porque ocurrieron el mismo día), también se conectan. Esta telaraña de conexiones automáticas es lo que le da forma y poder a nuestro espacio.

Tres diferencias clave con los espacios matemáticos clásicos

Si eres analista de datos o programador, probablemente la idea de un "espacio multidimensional" te suene a algo que ya conoces, como los Cubos OLAP o los espacios vectoriales. Aunque comparten la idea de usar "dimensiones", nuestro espacio WQuestions tiene tres diferencias radicales que lo hacen único:

Diferencia 1: Nuestro espacio acepta el vacío. En la geometría matemática pura, cada punto en un mapa tiene que tener obligatoriamente un número en cada eje (X, Y, Z). En nuestro modelo, un hecho normal tiene datos en dos o tres ejes y el resto se queda vacío. Si un usuario busca *"todos los goles del torneo"*, no le importa si esos goles no tienen registrada una temperatura o un lugar exacto; el sistema simplemente omite las columnas vacías. En la programación tradicional, cuando falta un dato, el sistema te obliga a meter un engorroso valor `NULL` o "Desconocido" para que la tabla no se rompa. En nuestro espacio, la dimensión que no aplica simplemente no existe para ese hecho. Es limpio y eficiente.

Diferencia 2: Aceptamos valores múltiples en una misma caja. En la matemática tradicional, un punto en el mapa solo puede tener una coordenada X. En WQuestions, gracias a nuestros cables múltiples (el eje **M** que vimos en el capítulo anterior), un mismo hecho puede disparar varios valores hacia la misma caja. Un partido de fútbol tiene **dos** equipos; una llamada a ChatGPT puede haber utilizado **cinco** documentos distintos como fuente. En la matemática pura esto rompería las reglas, pero en la arquitectura de datos reales, esta flexibilidad "multivaluada" es obligatoria.

Diferencia 3: Cada eje tiene personalidad. En un gráfico matemático escolar, todos los ejes son iguales: todos son líneas de números infinitos. En WQuestions, cada eje es de una "especie" totalmente distinta: uno guarda humanos (**Q**), otro fechas (**T**), otro magnitudes numéricas (**N**) y otro conceptos teóricos (**K**). Esto significa que el espacio es **tipado** (protegido). El sistema sabe que es un absurdo lógico intentar sumar a una persona (**Q**) con una fecha (**T**), por lo que protege a la base de datos de cometer errores ridículos al cruzar información.

La hoja de cálculo dispersa en acción

Para que no quede ninguna duda de cómo se ve esto en la vida real, imaginemos nuestra "hoja de cálculo gigante" con cuatro hechos sacados de nuestras cuatro industrias de ejemplo.

Míralo en formato de tabla:

El Hecho que ocurrió	Quién (Q)	Qué (O)	Dónde (L)	Cuándo (T)	Cuánto (N)	El Cable (P/M)	Clase (K)
Receta cocinada	juan	preparacion_017	cocina_casa	2026-05-14T20:00	—	cocinero	risotto
Gol marcado	messi	gol_001	estadio_lima	2026-10-14T20:23	87	agente	gol_jugada_abierta
Llamada a una IA	—	llamada_042	—	2026-05-14T10:32	4500	tokens_entrada	llamada_modelo_lenguaje
Decreto firmado	ministro_017	decreto_007	sede_gobierno	2026-05-14	50_000_000	firmante	decreto_ejecutivo

Las cuatro filas, a pesar de ser de mundos que no tienen nada que ver entre sí, conviven pacíficamente en la misma hoja. La llamada a la Inteligencia Artificial tiene vacía la columna del Lugar (**L**) porque ocurre en la nube; también tiene vacía la columna del Quién (**Q**) porque la hizo un proceso automatizado.

Pero fíjate en lo importante: **la estructura es exactamente la misma para todos**. Cualquier analista de datos, e incluso cualquier modelo de IA, puede leer esta tabla y entender inmediatamente qué dice cada fila sin necesidad de ser un experto en fútbol o en leyes.

El espacio WQuestions como hoja de cálculo dispersa

Cuatro filas, cuatro dominios distintos, mismas ocho columnas. Las celdas vacías son la norma, no la excepción.

Hecho	Q	O	L	T	N	P/M	K
Receta cocinada	juan	preparacion_017	cocina_casa	2026-05-14T20:00	—	cocinero	risotto
Gol marcado	messi	gol_001	estadio_lima	2026-10-14T20:23	87 (minuto)	agente	gol_jugada_abierta
Llamada API	—	llamada_042	—	2026-05-14T10:32	4500 (tokens)	tokens_entrada	llamada_modelo_lenguaje
Decreto firmado	ministro_017	decreto_007	sede_gobierno	2026-05-14	50.000.000	firmante	decreto_ejecutivo

Las celdas marcadas con — son dimensiones no aplicables a ese hecho.
El espacio es parcial por diseño: ningún hecho ocupa los siete ejes a la vez.

Una consulta filtra por ciertas columnas y devuelve las filas que satisfacen las restricciones.

Cuatro hechos de cuatro dominios distintos viven en la misma estructura tabular. Las celdas vacías son la norma: el espacio es parcial por diseño.

Esta uniformidad milagrosa es lo que hace que buscar datos sea un paseo en el parque. Si el gerente de la empresa pide ver *"absolutamente todos los eventos que pasaron el 14 de mayo de 2026"*, el código solo tiene que filtrar la columna **T** por esa fecha. El sistema le devolverá instantáneamente la receta, la llamada a la IA y el decreto firmado. No hace falta programar conectores complejos entre el servidor de cocina, el de informática y el legal. Todo está en la misma matriz.

Consultar es jugar a "Hundir la flota"

Si entendemos que todos los hechos son puntitos flotando en este gran espacio de siete ejes, entonces hacer una búsqueda en la base de datos no es más que **poner restricciones geométricas sobre las coordenadas**, casi como si jugaras al clásico juego de mesa de los barquitos (*Hundir la flota* o *Batalla Naval*).

En la práctica, las empresas hacen tres tipos básicos de "cortes" o búsquedas en este espacio:

- Fijar un punto y dejar el resto libre:** *"Muéstrame todo lo que involucre a Messi"*. El sistema clava una estaca en la caja **Q = messi** y extrae todos los hechos conectados a él, sin importar en qué año, estadio o país ocurrieron.
- Acotar un intervalo de tiempo o cantidad:** *"Dame todos los eventos ocurridos entre el 1 y el 31 de mayo"*. El sistema pone una regla de medición sobre el eje **T** y corta todo lo que esté fuera de esas fechas, dejando libres a las personas y a los lugares involucrados.
- Hacer cortes cruzados (la intersección):** *"Encuentra todos los goles de Messi jugando en Argentina en el año 2026"*. El sistema corta por la coordenada **Q = messi**, luego corta por **K = gol**, y finalmente por **T = 2026**. El resultado que el usuario ve en pantalla es el pequeño cuadrito donde esos tres cortes geométricos se cruzan.

La belleza técnica de esto radica en que el motor de la base de datos siempre ejecuta exactamente la misma operación de "corte geométrico", sin importarle si está buscando goles, diagnósticos médicos o fraudes financieros. Esta es la promesa cumplida del modelo WQuestions: un único lenguaje universal para hacerle preguntas al universo entero.

Para los técnicos: Cómo se compara con otros sistemas modernos

Si eres un profesional del sector, es útil contrastar rápidamente nuestro espacio WQuestions con tres tecnologías multidimensionales famosas para entender sus límites:

1. Cubos OLAP (Business Intelligence): Los analistas financieros usan "cubos" que tienen dimensiones (tiempo, país, tienda) para sumar ventas o gastos. *La diferencia:* un cubo OLAP está cerrado en su propio mundo (un cubo para ventas, otro cubo para recursos humanos). WQuestions es **un único cubo maestro para toda la empresa**, donde la "industria" es solo un filtro más dentro de la categoría **K**. Pasamos de tener mil cubos pequeños a un solo universo integrado.

2. Los Espacios Conceptuales (Gärdenfors): El investigador Peter Gärdenfors [13] propuso que el cerebro humano entiende los conceptos agrupándolos en espacios geométricos continuos (como organizar colores por tono y brillo). *La diferencia:* el modelo de Gärdenfors se usa para explicar cómo el cerebro *aprende* un concepto general; WQuestions se usa para archivar **los hechos reales y concretos** que usan esos conceptos. Ambas teorías se dan la mano pacíficamente.

3. Vectores y Embeddings (La mente secreta de la IA): Cuando ChatGPT "lee" una palabra, la transforma en una lista gigante de números (un vector denso) para medir qué tan cerca está esa palabra de otras (por ejemplo, "rey" está matemáticamente cerca de "reina"). *La diferencia:* el espacio de los embeddings es una caja negra; es **opaco**. Los humanos no podemos leer esos millones de números para saber por qué la IA conectó dos palabras. El espacio WQuestions, en cambio, es totalmente **transparente y auditable**: cada columna tiene una etiqueta clara que podemos leer. La IA usa los embeddings para pensar rápido en secreto, pero usa WQuestions para guardar el resultado de forma segura y explicable para los humanos. ¡Ambos sistemas se necesitan mutuamente!

Espacios multidimensionales: cómo se diferencia WQuestions

Cubos OLAP <i>BI / Data Warehouse</i>	Espacios Conceptuales <i>Gärdenfors</i>	Espacios de embeddings <i>modelos de lenguaje</i>	WQuestions <i>este libro</i>
Dimensiones Dimensiones específicas (tiempo, geografía, producto, cliente, vendedor...).	Dimensiones Dimensiones continuas de cualidad (matiz, tono, temperatura, intensidad).	Dimensiones Cientos o miles de dimensiones opacas, emergidas del entrenamiento.	Dimensiones Siete ejes universales (Q, O, L, T, N, K, M).
Elementos Hechos por celda con medidas agregables.	Elementos Conceptos = regiones convexas; objetos = puntos.	Elementos Puntos densos con distancia semántica (coseno).	Elementos Hechos atómicos como puntos parciales, multi-valuados, tipados.
WQ se distingue por Un cubo por dominio. WQ usa un único cubo universal de siete ejes.	WQ se distingue por Continuo y geométrico. WQ es discreto y simbólico: los conceptos viven en K.	WQ se distingue por Opaco — cada dimensión es desconocida. WQ es transparente y se complementa.	Lo distintivo Pensado desde el inicio para consumo por LLMs + federación de dominios.

No son competidores: cada espacio sirve para distintas operaciones.
Un LLM razona internamente con embeddings y consulta hechos estructurados en WQuestions.

Cuatro espacios multidimensionales comparados. Cubos OLAP, espacios conceptuales y embeddings comparten con WQuestions la idea de varias dimensiones, pero difieren en granularidad, transparencia y propósito.

Una ventaja inesperada: La "Densidad Emergente"

Hay algo casi mágico que ocurre en la vida real cuando una empresa implementa este sistema. No es una regla matemática, sino un fenómeno orgánico: a medida que el sistema empieza a chupar millones de datos todos los días, **ciertas regiones del espacio geométrico se empiezan a llenar muchísimo más que otras**. Esto es lo que llamamos "densidad emergente".

Si aplicamos el modelo en un hospital gigante, veremos que la zona del espacio donde se cruzan los pacientes (Q), las citas médicas (O), las fechas (T) y las enfermedades (K) se vuelve un nodo densísimo de información, brillando en el mapa.

La maravilla corporativa ocurre cuando a ese mismo hospital le instalas un sistema nuevo (por ejemplo, un chatbot de citas por WhatsApp). El nuevo chatbot empieza a registrar sus propios hechos: llamadas (O), latencias (N) y tipos de IA usados (K). El chatbot empieza a poblar su propia esquina vacía del espacio, **sin estorbar ni romper un solo dato del sistema hospitalario**. Y el día en que un usuario pregunta por WhatsApp: "*¿Qué día me toca ir al médico?*", ambos sistemas se cruzan en un punto del mapa de manera natural, sin necesidad de programar complejos túneles de integración entre el servidor médico y el de WhatsApp.

Como beneficio extra, esta densidad te sirve para auditar empresas. Si llegas a una compañía y ves un mapa donde la zona de "ventas" está densamente poblada pero la de "reclamos" está misteriosamente vacía, sabes de inmediato dónde están los puntos ciegos de su negocio, sin tener que leer una sola línea de documentación.

Tres cosas que este espacio NO es (Para evitar confusiones)

Para mantener las expectativas bajo control, es sano aclarar tres cosas que la metáfora de la "geometría espacial" no hace:

1. **No mide distancias físicas:** Que dos hechos compartan la misma fecha no significa que uno esté "cerca" del otro a nivel de afinidad. Este modelo sirve para cruzar conexiones exactas, no para calcular qué tan parecido es un paciente a otro. (Para eso existen los embeddings de la IA).
2. **No te obliga a usar un servidor raro:** Este "espacio de 7 ejes" es una forma de pensar y organizar el código. A la hora de guardarlo en un disco duro físico, el programador puede usar la tecnología que más le guste: filas en PostgreSQL, documentos en MongoDB, o grafos en Neo4j. La filosofía funciona en todas.
3. **No es una prisión de datos:** Si tú decides mapear todo el sistema de Recursos Humanos en WQuestions, no significa que te prohíba mapear Contabilidad mañana. Todo lo que modelos aterriza seguro en este espacio; lo que decidas no modelar, simplemente no se ve. Es un sistema modular e infinito.

El modelo y los Agentes de Inteligencia Artificial

Cerremos este capítulo conectando la teoría con el futuro del mercado.

Cuando un agente autónomo de Inteligencia Artificial se conecta a la base de datos de tu empresa para hacer un trabajo, lo que está haciendo matemáticamente es **navegar como un dron a través de esta geometría espacial**. Cuando usa una herramienta para "agregar un cliente nuevo", simplemente está dibujando un puntito nuevo en el mapa. Cuando el agente quiere investigar un fraude, simplemente "recorta un cuadrado" en la geometría y lee lo que hay adentro.

Y aquí viene el golpe maestro: este mapa de siete ejes (quién, qué, dónde, cuándo...) **es el mismo mapa cognitivo que la Inteligencia Artificial ya usaba en su cabeza para entender el lenguaje humano**. Al presentarle los datos corporativos estructurados de esta manera, el agente de IA no tiene que aprender un ecosistema nuevo; **se encuentra con un universo digital que funciona bajo las mismas leyes físicas que su propio entrenamiento neuronal**.

Esta compatibilidad absoluta entre la estructura de la mente de la IA y la estructura de los discos duros de la empresa es lo que hace que todo el esfuerzo valga la pena.

Pero en la vida real las cosas no siempre son tan simples. En el eje O viven unos individuos muy especiales, los "nudos gordianos" de la base de datos: las **situaciones reificadas**, eventos gigantes (como un partido entero o una guerra) donde miles de hechos pequeños convergen y se agrupan porque hablan de lo mismo. Es ahí donde el

sistema demuestra si tiene los músculos para manejar eventos extremadamente complejos a lo largo del tiempo, y desatar esos nudos sin que la base de datos explote.

Capítulo 9 — Situaciones reificadas: los puntos articuladores

Por qué algunas acciones merecen ser sustantivos

Si te fijas con cuidado, el idioma español tiene un truco gramatical fascinante y muy útil: nos permite convertir cualquier verbo en un sustantivo sin el menor esfuerzo. El verbo *correr* se transforma en *la carrera*. *Vender* se vuelve *la venta*. *Operar* pasa a ser *la operación*. *Decidir* se convierte en *la decisión*.

Esos sustantivos nuevos no nombran cosas físicas que puedas tocar, como una taza de café o un escritorio; nombran **acciones, eventos o procesos**. Pero al volverlos sustantivos, les hemos otorgado algo vital: identidad propia. Los hemos convertido en "cosas" de las que podemos hablar.

Este cambio no es solo un adorno literario; en la arquitectura de datos, es la diferencia entre un sistema que entiende el mundo y uno que se ahoga en detalles.

Piénsalo así: decir "*María le vendió un libro a Juan*" no es lo mismo que decir "*la venta de María a Juan ocurrió ayer y quedó registrada bajo el código 14*". En la primera oración, "vender" es solo un acto pasajero que conecta a María con el libro. En la segunda oración, **la venta se ha convertido en una entidad en sí misma**. Ahora tiene un código, una fecha, y lo más importante: podemos añadirle nuevos datos en el futuro. Podemos decir que "la venta se canceló", "se rectificó" o "se pagó en cuotas".

A este truco tecnológico de tomar un evento fugaz y convertirlo en un objeto de primera clase con identidad propia, nuestro modelo lo llama **reificación** (del latín *res*, que significa 'cosa': literalmente, "hacer cosa" un evento). Y a la entidad que nace de ese proceso la llamamos **situación reificada**.

Estas situaciones reificadas van a vivir siempre en el eje 0. Son los "nudos" o **puntos articuladores** de nuestra red de datos. Son los conectores maestros donde docenas de pequeños hechos convergen porque todos hablan exactamente de lo mismo. Sin la reificación, nuestro modelo solo podría procesar oraciones simples. Con la reificación, el modelo puede entender y guardar cualquier evento humano, por gigantesco e intrincado que sea.

La regla de oro: ¿Cuándo conviene reificar?

Llegados a este punto, la duda lógica es: *¿Debo convertir absolutamente todo en una situación reificada? ¿Si Juan es alto, debo crear "la situación de la altura de Juan"?*

La respuesta es un rotundo no. Reificar cuesta dinero y recursos. Cada vez que conviertes un hecho en una "situación", estás creando un objeto nuevo en tu base de datos, tienes que asignarle un código de identidad único (un UUID) y el sistema tardará un milisegundo extra en buscarlo. Si reificas todo por costumbre, vas a inflar tu base de datos con basura abstracta.

Para mantener la base de datos veloz y limpia, nuestro modelo sigue una regla de oro: **solo se reifica cuando se cumple al menos una de estas cuatro condiciones**.

1. **El evento tiene demasiados detalles.** Si el hecho tiene propiedades que no caben en una oración simple (como la hora exacta, el lugar, el instrumento usado o un agente secundario), reifícalo. Un gol no es solo (messi, anota, gol). Un gol profesional tiene un minuto exacto, una pierna utilizada, un pase previo y una distancia. Esos detalles no tienen de dónde "colgar" a menos que conviertas el gol en una entidad propia.
2. **Participan más de dos personas o cosas.** Una transacción de venta tiene un vendedor, un comprador, un producto, un precio y una fecha. Como vimos en el capítulo 7, nuestra "tripleta atómica" solo puede

conectar dos cosas a la vez (Sujeto y Objeto). Reificar la venta es el mecanismo estándar para conectar a cinco actores distintos sin romper nuestras reglas matemáticas.

3. **Vas a necesitar hablar de este evento más tarde.** Si sabes que en el futuro vas a recibir actualizaciones sobre este hecho, debes reificarlo. Datos como *"la venta fue anulada"*, *"el diagnóstico fue apelado"* o *"el partido fue suspendido por lluvia"* exigen que la venta, el diagnóstico o el partido existan previamente como objetos independientes en la base de datos para poder recibir esas actualizaciones.
4. **El dato va a cambiar en el futuro y necesitas guardar el historial.** El dato "dónde vive Marta" es temporal. Hoy vive en una ciudad, mañana vivirá en otra. Si solo guardamos el dato actual, perdemos todo su historial. Si convertimos su residencia en una "situación" y le ponemos una fecha de `inicio` y un `fin`, conservamos la historia intacta. A esto lo llamaremos la regla de **vigencia temporal**, y la veremos más abajo.

Si un dato no cumple ninguna de estas cuatro reglas, **no lo reifiques**. Dejarlo como un hecho atómico simple (`juan`, `estatura`, `1.80`) es perfecto y mantiene el sistema rápido.

Para que no queden dudas, esta será la **cuarta decisión de diseño** de nuestra arquitectura:

D4 — Un evento o relación solo se transforma en una "situación reificada" (guardada en el eje O) si cumple al menos uno de cuatro requisitos: (1) tiene atributos propios como lugar o modo, (2) participan más de dos actores, (3) será referenciado por otros eventos en el futuro, o (4) su valor cambiará y necesitamos conservar un registro histórico. Si no cumple ninguna, usamos una tripleta simple.

Autopsia de una situación reificada

¿Cómo se ve uno de estos "nudos" por dentro? Abramos uno para revisarlo. Tomemos un evento cotidiano pero complejo: la consulta médica de la paciente María Gonzales del 14 de mayo de 2026 con la Dra. Torres.

A nivel humano, sabemos que una consulta médica incluye a un paciente, un doctor, una fecha, un consultorio, un motivo de visita, un diagnóstico oficial y, finalmente, un pago.

En nuestro sistema, lo primero que hacemos es "hacer cosa" la consulta, creándola en el eje O:

```
(consulta_2026_05_14) ∈ O
```

Ese código, `consulta_2026_05_14`, se convierte en nuestro punto articulador. Todos los detalles de la visita médica se van a enganchar a este nodo central utilizando nuestros hechos atómicos de tres partes:

(consulta_2026_05_14, instancia_de,	accion_consultar)	∈ M(0, K)
(consulta_2026_05_14, agente,	dra_torres)	∈ M(0, Q)
(consulta_2026_05_14, paciente,	maria_gonzales)	∈ M(0, Q)
(consulta_2026_05_14, momento,	2026-05-14T10:30:00Z)	∈ M(0, T)
(consulta_2026_05_14, lugar_de,	consultorio_03)	∈ M(0, L)
(consulta_2026_05_14, motivo,	control_rutinario)	∈ M(0, K)
(consulta_2026_05_14, diagnostico,	hipertension_grado_1)	∈ M(0, K)
(consulta_2026_05_14, estatus_factual,	real)	∈ M(0, K)

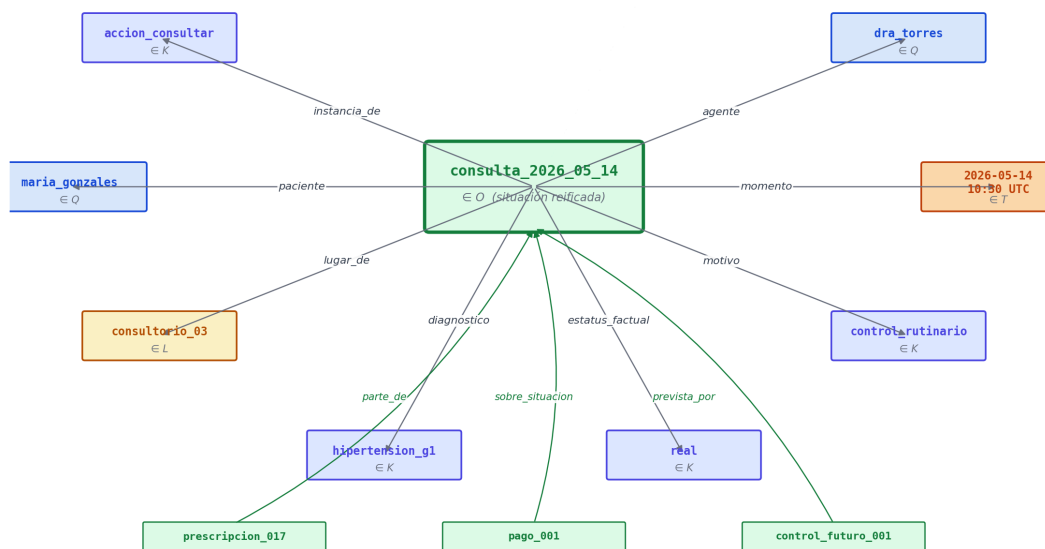
Hemos usado ocho hechos atómicos para dibujar la consulta completa. Y aquí viene la recompensa de haberlo reificado: como `consulta_2026_05_14` ahora es una entidad con identidad propia, otros eventos posteriores pueden apuntar directamente a ella:

```
(prescripcion_medica_017, parte_de, consulta_2026_05_14)
(pago_tarjeta_001, sobre_situacion, consulta_2026_05_14)
(control_mes_siguiente, prevista_por, consulta_2026_05_14)
```

Fíjate en lo que acabamos de hacer. La prescripción, el pago y la futura visita son, cada una de ellas, **nuevas situaciones reificadas** que se conectan al nodo maestro original. La red de la base de datos crece de forma natural, sin cambiar jamás su formato de "tres columnas".

Anatomía de una situación reificada

La consulta médica como punto articulador del grafo



Anatomía de una situación reificada: la consulta médica como nodo central con todos sus participantes, momentos, lugares y propósito colgando como hechos atómicos. Las situaciones derivadas (prescripción, pago, control) se conectan al nodo por relaciones canónicas.

D5: El agente no siempre es un humano

Hagamos una pausa para oficializar algo que los ejemplos ya nos venían gritando. En la consulta médica, el rol de agente se lo dimos a una persona de carne y hueso (la Dra. Torres). Sin embargo, si miramos otros mundos, el "agente" de una acción puede ser cualquier cosa: el algoritmo de Uber que te asigna un chófer, el servidor bancario que te cobra mantenimiento, o un modelo de IA que redacta un contrato.

Esto significa que nuestro modelo **no exige que el protagonista de la acción sea humano**.

D5 — Agencia contextual. El rol de agente puede ser ocupado por humanos, corporaciones, algoritmos de software o sensores físicos. Todo depende del verbo de la acción. Hay verbos que exigen un agente (como "vender") y otros que no necesitan a nadie ("ocurrir", "llover").

Esta regla (D5) es la que nos permite auditar sistemas industriales, plantas químicas o plataformas de Inteligencia Artificial utilizando exactamente el mismo lenguaje que usaríamos para registrar ventas humanas.

Los límites de la agencia: el rol, no la voluntad

Conviene fijar el límite, porque la pregunta es legítima: si un horno inteligente o un algoritmo de recomendación pueden "vestirse" de agente, ¿cuándo deja un objeto de ser una *herramienta* y pasa a ser un *actor*? La tentación es

buscar la frontera en la **voluntad** o la **conciencia** del objeto — y por ahí no hay salida limpia, porque ni un algoritmo ni una empresa "quieren" nada en sentido fuerte.

El modelo esquiva esa trampa filosófica: el criterio **no es la voluntad, es el rol que la entidad ocupa en una situación concreta**. Una misma cosa puede ser las dos cosas en momentos distintos. El bisturí con el que opera un cirujano es un **instrumento** (un objeto en O) de la situación *operar* — el agente es el cirujano. Pero "el robot Da Vinci ejecutó la sutura" pone al robot en el rol de **agente** (Q) de la situación *suturar*, donde no hay otro candidato a protagonista. El mismo aparato es herramienta o actor según **qué verbo lo invoque y qué rol llene**, no según una propiedad que lleve puesta.

La regla operativa, entonces, es simple: una entidad entra a Q como agente cuando **ocupa el rol agente de un verbo que exige uno** (vender, asignar, redactar, suturar); se queda como instrumento en O cuando es solo el medio a través del cual otro actúa. La agencia la determina la **gramática de la situación**, no la metafísica del objeto. Por eso el modelo puede auditar a un algoritmo que niega un crédito con el mismo rigor con que audita a un gerente que lo niega — sin tener que resolver primero si el algoritmo "tiene voluntad".

El Estatus Factual: Eventos que todavía no han pasado

Una situación en nuestro sistema no tiene que ser obligatoriamente un hecho consumado. A menudo, las empresas necesitan guardar situaciones que están planeadas, reuniones que se cancelaron, diagnósticos que son meras hipótesis o compras que fueron devueltas.

Para no tener que inventar etiquetas duplicadas para todo (como tener un cable `agente_real` y otro cable `agente_planeado`), la solución más limpia que encontramos fue obligar a todas las situaciones a declarar abiertamente su **estatus factual**:

```
estatus_factual : de 0 hacia K
  Opciones permitidas: real | planeado | confirmado | hipotético | cancelado | rectificado
```

Si agendas una consulta médica para el mes que viene, la guardas en el sistema exactamente igual que si fuera una consulta real, pero le añades la propiedad `estatus_factual: planeado`. Si el paciente asiste a su cita, el sistema no borra el plan; crea una *nueva* situación con el estatus `real` y la conecta a la antigua diciendo que la "cumplió". Si el paciente llama para cancelar, creamos una situación de cancelación.

```
(futuro_control_001) ∈ 0
  instancia_de   : accion_consultar
  agente        : dra_torres
  momento       : 2026-06-04T10:00:00Z
  estatus_factual : planeado                ← ¡La clave está aquí!
```

Este pequeño truco asegura la **inmutabilidad** de la base de datos: el pasado jamás se reescribe. A la vez, nos permite mapear toda la red de expectativas, promesas y fallos que componen el día a día de cualquier negocio real.

Vigencia y la Regla D6: Datos que caducan con el tiempo

Llegamos a uno de los trucos de diseño más inteligentes de todo el modelo.

Hay propiedades que simplemente no duran para siempre. El monto del alquiler que pagas hoy cambiará el año que viene por la inflación. El medicamento que receta el doctor hoy, se suspenderá en tres semanas. Si una base de datos antigua recibe un precio nuevo, borra el precio viejo y pone el nuevo encima, destruyendo el historial corporativo para siempre.

Nosotros evitamos ese desastre utilizando la reificación para dominar el flujo del tiempo. Esta es nuestra sexta decisión de diseño:

D6 — Las propiedades que cambian con el paso del tiempo no se guardan directamente. Se reifican convirtiéndolas en situaciones, y se les añade una fecha de inicio y una de fin (su rango de vigencia).

En lugar de hacer la barbaridad de reescribir un dato así:

```
(marta, vive_en, ciudad_a) ← (Y si se muda, borrar este texto y poner ciudad_b)
```

Lo que hacemos es crear una situación histórica:

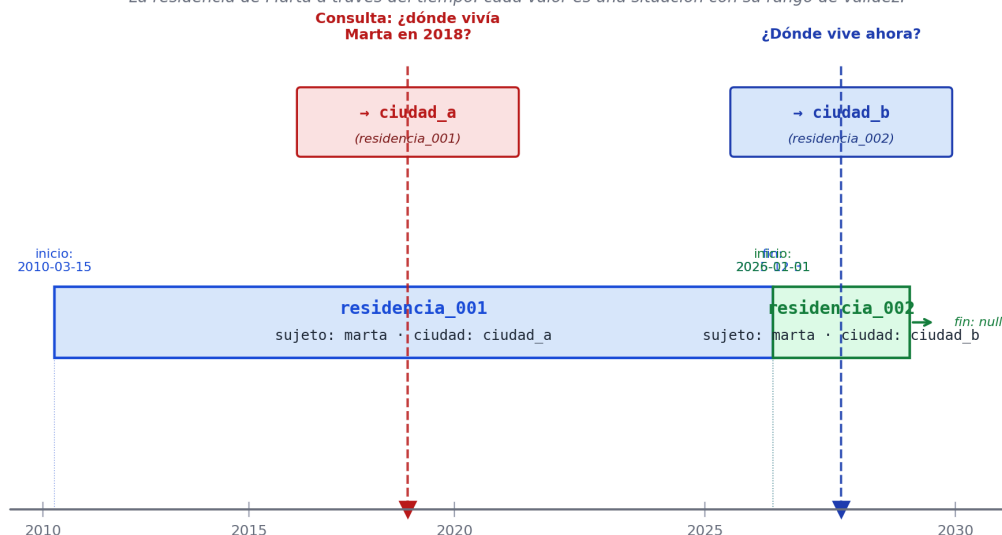
```
(residencia_historial_001) ∈ 0
  sujeto      : marta
  ciudad      : ciudad_a
  inicio      : 2010-03-15
  fin         : 2025-12-31

(residencia_historial_002) ∈ 0
  sujeto      : marta
  ciudad      : ciudad_b
  inicio      : 2026-01-01
  fin         : null      ← Al estar vacío, el sistema sabe que esta es la actual.
```

Gracias a esto, cuando le preguntes al sistema "¿dónde vive Marta?", la máquina te pedirá una fecha de referencia. Si pides el año 2024, el motor leerá los rangos de tiempo y te devolverá de forma automática la `ciudad_a`.

D9 — vigencia temporal mediante reificación

La residencia de Marta a través del tiempo: cada valor es una situación con su rango de validez.



Sin D9: el valor nuevo sobrescribe al anterior. Con D9: ambos rangos coexisten y la consulta temporal recupera el correcto según el momento solicitado.

D6 en acción: una propiedad que cambia en el tiempo se modela como una sucesión de situaciones reificadas, cada una con su rango de vigencia. La consulta temporal recupera la situación válida en el momento solicitado.

Esta técnica (llamada *bitemporalidad*) vuelve ridículamente fáciles consultas que antes volvían locos a los analistas de datos, tales como: "¿Qué versión exacta del algoritmo de IA estaba corriendo en producción el día que se reportó el fallo el año pasado?". Sin la regla D6, responder eso es casi imposible.

¿Cómo se conectan las situaciones entre sí?

Para evitar que la base de datos se convierta en una selva inmanejable de eventos sueltos, hemos definido un grupo de cables oficiales (relaciones canónicas) que se encargan de organizar cómo interactúan estas situaciones reificadas entre sí. Son pocos, pero cubren todos los escenarios:

1. Jerarquía (parte_de y su inverso contiene): Sirve para meter eventos pequeños dentro de eventos gigantes. Una compra con tarjeta es parte_de las ventas del mes. Un gol es parte_de un partido.

```
(gol_001,           parte_de,  partido_arg_per_2026)
(compra_tarjeta,    parte_de,  cierre_mes_julio)
```

2. Secuencia lógica (precede y su inverso sigue_a): Organiza el flujo de los procesos. No importa tanto la hora del reloj, sino qué paso obligatoriamente va antes que otro.

```
(examen_sangre_001, precede,  diagnostico_medico)
```

3. Acción y reacción (cumple, cancela, modifica, rectifica): Define cómo un evento nuevo impacta o anula a un evento del pasado. Un pago de nómina cumple la obligación mensual. Una boleta de devolución cancela la compra anterior.

```
(transferencia_julio, cumple,  obligacion_pago_julio)
(boleta_rectificativa, rectifica, boleta_original_007)
```

4. El grupo del "Por qué" (causado_por, motivado_por, justificado_por): Los cables que explican los motivos humanos o mecánicos detrás de las situaciones. A estos les dedicaremos un análisis profundo más adelante.

El Zoom de la reificación: Tú decides cuánto detalle quieres

Cerramos con una aclaración vital para cualquier arquitecto de software: reificar la información no es un interruptor que simplemente se enciende o se apaga. Es como hacer zoom con una cámara. Un mismo evento puede modelarse de tres formas distintas en la base de datos, dependiendo de cuán obsesionada esté tu empresa con los detalles.

Imagina que queremos guardar el "consumo eléctrico mensual de un edificio corporativo". Podemos hacerlo en tres niveles de profundidad:

Nivel 1 — El brochazo gordo (Tripleta plana):

```
(edificio_017, consumo_octubre, 1240)
(edificio_017, unidad_consumo,   kWh)
```

Es rápido, barato, y suficiente si el gerente solo quiere ver el total gastado en el mes.

Nivel 2 — Reificación corporativa:

```
(consumo_oct_017) ∈ 0
edificio   : edificio_017
cantidad  : 1240
unidad    : kWh
periodo   : 2026-10
medido_por : empresa_distribuidora_luz
```

Aquí "hicimos cosa" al recibo de luz. Es necesario si el departamento de contabilidad exige auditorías sobre qué proveedor emitió el cobro y en qué periodo exacto.

Nivel 3 — Reificación extrema (Microdatos):

```
(consumo_oct_017) ∈ 0
  edificio    : edificio_017
  total      : 1240 kWh
  ...
(consumo_oct_017_dia_03) ∈ 0    ← (Y decimos que esto es "parte_de" consumo_oct_017)
  fecha      : 2026-10-03
  cantidad   : 42 kWh
```

Aquí desarmamos el recibo gigante en 30 "situaciones de consumo diario" individuales. Solo harías esto si el edificio tiene paneles solares y necesitas calcular tarifas hora por hora para ahorrar energía.

¿Cuál de los tres niveles es el correcto? **Los tres son perfectamente legales.** Todo depende del presupuesto de tu empresa y de lo que quieras lograr. Y lo más hermoso de esta arquitectura es que **saltar del nivel 1 al nivel 3 no rompe tu base de datos**. Si mañana necesitas más detalle, simplemente inyectas nuevas situaciones y las conectas a las antiguas usando el cable `parte_de`. El sistema jamás se detiene.

Las situaciones reificadas son los semáforos y las rotondas por donde circula el tráfico de nuestra base de datos. Pero para que el mapa esté completo, falta pavimentar un tipo de conexión muy especial que une a estas rotondas: los cables del **"por qué"**. Saber que ocurrió una venta o un accidente médico es útil, pero saber *qué lo causó, qué normativa lo justificó o qué finalidad oculta perseguía* es lo que eleva una base de datos al nivel de inteligencia de negocios. Esas cuatro conexiones canónicas (*causado por, motivado por, con finalidad, justificado por*) son las que nos permiten construir una estructura capaz de dar explicaciones razonadas.

Capítulo 10 — El "por qué" no es una pregunta más

El niño que no dejaba de preguntar

Cualquiera que haya pasado tiempo con un niño de tres o cuatro años conoce perfectamente la fase del "por qué". Es una etapa agotadora pero fascinante. El niño te mira y dispara: "*¿Por qué se rompió el vaso?*". Tú le respondes que se cayó al suelo. Él ataca de nuevo: "*¿Por qué llora la abuela?*". Tú le dices que está emocionada. Sigue la ráfaga: "*¿Por qué vamos al médico?*". Y remata con: "*¿Por qué no puedo comer tierra?*".

Para el niño, la palabra es exactamente la misma en todos los casos. Utiliza la misma herramienta gramatical. Sin embargo, tú, como adulto, haces un esfuerzo mental inconsciente y le das cuatro tipos de respuestas completamente distintas. Al vaso le das una explicación **física** (la gravedad). A la abuela le das una explicación **psicológica** (los sentimientos). Al médico le das una explicación orientada al **futuro** (para curarnos). Y a la tierra le das una explicación **normativa** (porque es una regla de higiene y salud).

Esa misma trampa lingüística en la que caen los niños es la que ha confundido a los ingenieros de bases de datos durante décadas. En el mundo del software empresarial, tomamos preguntas que parecen idénticas y tratamos de guardarlas en el mismo cajón.

Observemos esto en un entorno profesional:

¿Por qué se cayó el servidor? — Porque hubo un cortocircuito. *¿Por qué Marta vendió las acciones?* — Porque tenía miedo de perder dinero. *¿Por qué el cirujano abrió esta vía?* — Para acceder al pulmón izquierdo. *¿Por qué se canceló el contrato?* — Porque la cláusula 14 lo exige en caso de impago.

Si nos quedamos en la superficie, todas estas frases se responden con un "porque" o un "para". Pero si analizamos qué tipo de información estamos entregando, son cuatro universos distintos: una **causa mecánica**, un **motivo mental**, una **finalidad u objetivo**, y una **justificación legal**.

Esto plantea un problema arquitectónico gigantesco para nuestro modelo: si quisiéramos crear un eje exclusivo para el "por qué", **¿qué demonios guardaríamos adentro?**

Por qué no existe el eje "Por qué"

Hagamos memoria de cómo funcionan las "cajas" (los ejes) que ya hemos construido. La caja **Q** guarda personas o agentes con intención. La caja **T** guarda fechas. La caja **K** guarda conceptos. Cada eje es ordenado y homogéneo: todo lo que vive adentro está hecho de la misma sustancia.

Para que existiera un eje llamado "Por qué", todo lo que metiéramos ahí debería ser similar. Pero acabamos de ver que no lo es: La respuesta a "*¿por qué se cayó el servidor?*" es **un evento del pasado** (un cortocircuito, que vive en la caja **Q**). La respuesta a "*¿por qué vendió las acciones?*" es **una emoción** (un miedo, que también vive en **Q** pero es un estado mental). La respuesta a "*¿por qué abrió esa vía?*" es **un evento del futuro** (alcanzar el pulmón, algo que todavía no ocurre). Y la respuesta a "*¿por qué se canceló el contrato?*" es **una ley** (un documento normativo).

Intentar forzar todas estas cosas en un solo eje sería como meter en la misma bolsa un cortocircuito, la tristeza, un plan a futuro y un documento legal. Si le pides a una computadora que trate todo eso por igual, la condenas a la confusión. Si un gerente pregunta "*¿qué causó este error?*", la máquina podría contestarle mostrándole una ley en lugar de una falla mecánica.

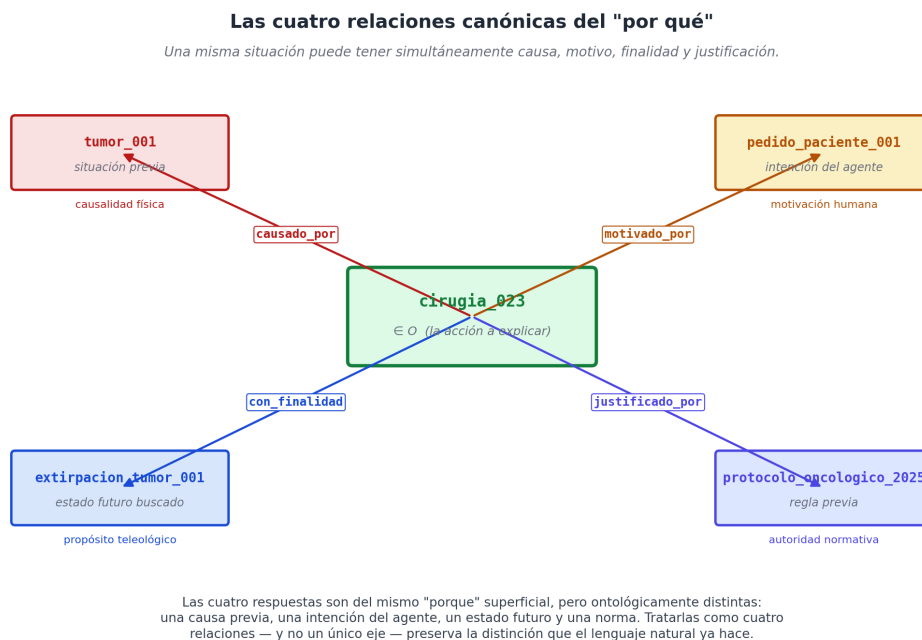
Aquí entra nuestra **séptima decisión de diseño (D7)**, y es mejor dejarla por escrito formalmente:

D7 — El modelo no tiene un eje "por qué". En su lugar, el sistema toma el "porque" del lenguaje natural y lo divide en cuatro cables distintos (relaciones canónicas) que conectan a las situaciones entre sí. Esos cables son: `causado_por` (para la física), `motivado_por` (para la intención humana), `con_finalidad` (para el propósito futuro) y `justificado_por` (para las reglas y leyes).

En la práctica, esto significa que no hay una caja nueva. Simplemente usamos nuestros cables de relación múltiple (`M`) para conectar eventos (situaciones en la caja `0`) con otros eventos:

```
causado_por      : conecta 0 con 0      (La física de causa y efecto)
motivado_por     : conecta 0 con 0      (La intención en la mente de alguien)
con_finalidad    : conecta 0 con 0      (Un propósito proyectado al futuro)
justificado_por  : conecta 0 con 0      (La autoridad de una norma o regla)
```

Tenemos cuatro relaciones precisas, no un eje confuso. Desambiguamos la pregunta *antes* de guardarla, no después.



Las cuatro relaciones canónicas del "por qué": cada una conecta una situación con otra de naturaleza distinta — causa previa, intención del agente, estado futuro buscado, regla que la autoriza.

Las cuatro relaciones, diseccionadas una por una

1. `causado_por` — El efecto dominó (La causa eficiente)

Usamos este cable para describir conexiones mecánicas y ciegas entre dos eventos: el primer evento empuja al segundo, sin que haya ningún deseo, pensamiento o ley de por medio. Es el "porque" de la física, del dominó que cae y tumba al siguiente.

```
(caida_vaso_001,    causado_por, empujon_001)
(incendio_edif_017, causado_por, cortocircuito_001)
(infeccion_001,     causado_por, bacteria_001)
```

Este cable es el favorito de los diagnósticos médicos y los ingenieros. Tiene una propiedad interesantísima: **no necesita agentes humanos**. Un terremoto causa un derrumbe sin que haya intención de por medio. Por eso, el sistema no te exige que vincules a una persona cuando usas este cable.

2. `motivado_por` — La chispa mental (El motivo intencional)

Este cable se conecta cuando queremos explicar una acción a través del estado mental (deseo, miedo, creencia) de la persona que la ejecutó. Es la respuesta perfecta cuando la pregunta no es *"¿qué fuerza movió esto?"*, sino *"¿qué pasaba por la cabeza de este sujeto para actuar así?"*.

```
(venta_casa_002,      motivado_por, deseo_mudarse_marta)
(renuncia_001,       motivado_por, conflicto_con_jefe_017)
(consulta_medica_046, motivado_por, dolor_persistente_001)
```

Ojo aquí, la diferencia con el cable de causa física parece sutil, pero es vital. Decir *"el incendio causó el derrumbe"* es muy distinto a decir *"el incendio motivó la evacuación"*. El derrumbe no es algo que el edificio *quisiera* hacer, simplemente ocurrió. Pero la evacuación sí fue una decisión tomada por alguien asustado. Las causas viven en el mundo físico; los motivos viven en las mentes de la gente.

3. `con_finalidad` — El viaje al futuro (El propósito teleológico)

Este es el cable más raro de la familia, porque no conecta un hecho con algo del pasado, sino que **lo conecta con un resultado que se busca en el futuro**. Es el cable del objetivo a alcanzar.

```
(incision_001,      con_finalidad, acceso_pulmon_izquierdo_001)
(rebajar_precio_001, con_finalidad, aumento_ventas_q3_2026)
(reunion_004,       con_finalidad, decision_inversion_001)
```

Seguramente te preguntas: *¿Cómo guardo en la base de datos un evento que todavía no ocurre?* Fácil: lo guardas hoy como una "situación" más en el eje `0`, pero le pones una etiqueta de estado (la propiedad `estatus_factual` que vimos en el capítulo anterior) indicando que es `previsto`, `intencionado` o `hipotético`. El evento futuro ya vive en tu disco duro; lo que apunta hacia adelante es simplemente su etiqueta de "aún no realizado".

4. `justificado_por` — El sello de aprobación (La autoridad normativa)

Este cable aparece cada vez que una acción no se explica por un choque físico, ni por un deseo emocional, sino por **una regla, protocolo o ley que te da permiso para hacerla**. Es el cable indispensable para el derecho, los procedimientos hospitalarios y las auditorías de calidad.

```
(rescicion_contrato_017, justificado_por, clausula_14_contrato_017)
(despacho_radioactivo_001, justificado_por, protocolo_iaea_2022)
(rechazo_devolucion_001, justificado_por, politica_30_dias_tienda)
```

Lo que justifica la acción no es un evento del mundo natural, es una **norma**. Y como veremos más adelante, las normas se guardan en el sistema como objetos con su fecha de creación, su condición y su emisor. Este cable es el que hace que cualquier sistema regulado sea 100% auditable.

Las cuatro respuestas en una sola acción

Lo maravilloso de tener estos cuatro cables bien definidos es que un mismo evento puede tenerlos todos conectados al mismo tiempo, enriqueciendo la historia de forma espectacular. Piensa en una cirugía oncológica real:

(cirugia_023, causado_por,	tumor_001)	← La realidad física que la obliga.
(cirugia_023, motivado_por,	pedido_paciente_001)	← El deseo mental de sanar.
(cirugia_023, con_finalidad,	extirpacion_tumor_001)	← El resultado futuro que se busca.
(cirugia_023, justificado_por,	protocolo_oncologico_2025)	← La regla médica que ampara al doctor.

Cuatro hechos atómicos simples dando cuatro explicaciones distintas a la misma cirugía. Si cometiéramos el error de unificar el "por qué" en un solo campo, toda esta riqueza explicativa se aplastaría.

El camino de migas de pan: Cadenas explicativas

En la vida real, los porqués casi nunca vienen solos. Se enganchan unos con otros formando una historia completa, a lo que llamamos una **cadena explicativa**.

Veamos un escándalo de noticias políticas: Un ministro renuncia. ¿Por qué? Porque enfrenta una investigación. ¿Y por qué hay investigación? Porque un periodista sacó una nota. ¿Y por qué sacó la nota? Porque alguien le filtró datos. ¿Y por qué filtraron los datos? Porque un funcionario estaba enojado.

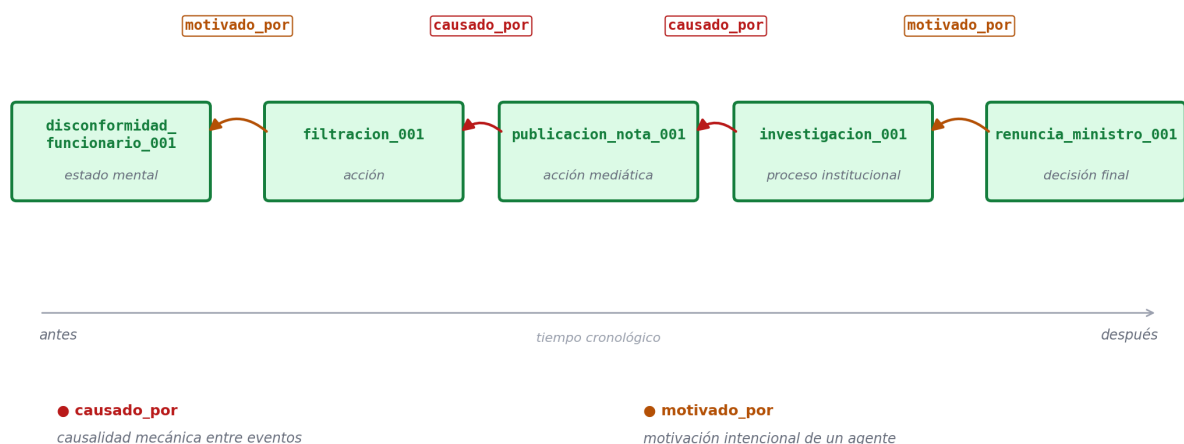
En nuestro sistema, esa historia se ve como un camino de migas de pan perfectamente enlazado:

```
(renuncia_ministro_001,      motivado_por, investigacion_001)
(investigacion_001,          causado_por,  publicacion_nota_001)
(publicacion_nota_001,       causado_por,  filtracion_001)
(filtracion_001,             motivado_por, disconformidad_funcionario_001)
```

Esto no es un relato literario; es un mapa navegable para la computadora. Si le pides a la base de datos: *"¿qué eventos contribuyeron a la renuncia del ministro?"*, el motor simplemente se agarra del primer evento y empieza a trepar por los cables hacia atrás. Y lo mejor es que puedes pedirle a la máquina cosas súper precisas como: *"recorre la historia hacia atrás, pero muéstrame solo las motivaciones humanas, ignora las causas mecánicas"*.

Cadena explicativa: cómo las relaciones del "por qué" reconstruyen una historia

Cuatro situaciones conectadas: dos motivos psicológicos y dos causas físicas, distinguidos en el grafo.



Una cadena explicativa: cómo cuatro relaciones canónicas del "por qué", encadenadas, reconstruyen una historia entera desde un evento final hasta sus orígenes lejanos.

Convertir las reglas en objetos de la base de datos

Hagamos un paréntesis rápido: si el cable `justificado_por` se conecta a una "regla", ¿cómo guardamos una regla en la base de datos?

Nuestro sistema trata a las reglas como **situaciones reificadas**, pero de un tipo especial. Toda buena regla tiene una condición, una consecuencia, una fecha de inicio y una autoridad que la emitió. En código, se ve así:

```
(regla_devolucion_001) ∈ 0
  instancia_de      : norma_comercial
  emisor           : tienda_central
  vigencia_desde    : 2025-01-01
  condicion         : situacion_compra con dias_desde_compra ≤ 30
  consecuente       : devolucion_autorizada
```

Una vez que guardas la regla como si fuera un objeto más, la magia ocurre. Puedes modificarla en el futuro, puedes citarla y puedes cruzarla con otras. Cuando un cliente logra que le devuelvan el dinero, la base de datos simplemente conecta la acción con la regla:

```
(devolucion_solicitada_017, justificado_por, regla_devolucion_001)
```

Esto hace que tu empresa sea auditable al instante. Si hay un problema, la base de datos te dice exactamente qué ley autorizó la acción y quién la había aprobado. Además, puedes inventar reglas nuevas mañana y la base de datos no se rompe; simplemente asimila el nuevo objeto.

(Aclaración técnica: el modelo guarda la regla de maravilla para que puedas consultarla, pero no es un robot que toma decisiones por sí solo. Para que la máquina vea la regla y decida sola devolverle el dinero al cliente, necesitas conectarle un evaluador o una IA por encima. WQuestions te da la información perfecta; el razonamiento automático lo construyes encima).

Lo que se gana destruyendo el cajón de "Observaciones"

Repasemos por qué hemos hecho todo esto, comparándolo con las prácticas mediocres que plagan el software de hoy en día.

La opción más perezosa (y la más común) es ponerle al sistema un campo de texto gigante llamado "*Observaciones*" o "*Justificación*", donde la gente escribe lo que quiere. Esto convierte a la base de datos en un cementerio: la computadora no puede leer eso, no puede auditarlo, y es imposible seguir una cadena de eventos.

La otra mala opción sería usar un solo cable general llamado `por_que`. Mejoraría un poco, pero perderíamos la frontera entre la física, la mente, el futuro y la ley. Un reporte corporativo mezclaría cortocircuitos eléctricos con miedos de empleados como si fueran lo mismo.

Al usar nuestra familia de cuatro conectores (`causado_por`, `motivado_por`, `con_finalidad`, `justificado_por`), logramos atrapar el detalle exacto que el cerebro humano usa para entender el mundo. Usamos simples hechos atómicos de tres partes y los conectamos a la misma maquinaria de búsqueda. Sin escribir código complejo, nuestra base de datos dejó de ser un simple cajón de archivos y se convirtió en una **red de explicaciones lógicas y recorribles**.

Cierre de la Parte III

Con este capítulo cerramos la Parte III, el corazón arquitectónico del libro: el hecho atómico, el espacio geométrico que forman al juntarse, las situaciones reificadas que amarran eventos complejos y los cables del "por qué" que le dan a la base de datos la capacidad de explicar sus propias acciones. La maquinaria conceptual está lista, armada y engrasada.

Pero nos falta una pieza crucial. Al final del día, nadie en tu empresa va a sentarse a escribir "tripletas atómicas" a mano en un teclado; la gente quiere hablar y escribir de forma natural. La **Parte IV** se encargará de resolver el último gran obstáculo: cómo construimos un traductor (un *Lexicon*) para que la base de datos entienda perfectamente el desordenado, hermoso y ambiguo lenguaje humano.

Capítulo 11 — El verbo: La firma de contrato de la oración

Lo que descubrió Davidson (Y por qué nos importa)

En 1967, un filósofo llamado Donald Davidson publicó un artículo de veinte páginas que le voló la cabeza a los lingüistas y lógicos de su época [12]. El problema que planteó parecía un juego de niños, pero escondía un abismo matemático. Tomemos una oración simple:

Juan pateó la pelota.

Si le pedimos a un programador clásico que guarde esto, probablemente escriba una función rápida de dos variables: `Patear(juan, pelota)`. Funciona perfecto. Pero observemos qué pasa si empezamos a añadirle detalles naturales a la frase:

Juan pateó la pelota con fuerza, en el patio, a las tres de la tarde, para asustar al perro.

Si seguimos la lógica del programador, ¿cómo escribimos esto ahora? ¿Será `Patear(juan, pelota, con_fuerza, patio, 15:00, asustar_perro)`? ¿Una función gigante de seis variables? ¿Y si luego decimos "con el pie izquierdo"? ¿Se vuelve de siete? Bajo esa lógica, el verbo "patear" tendría que cambiar su código interno cada vez que a una persona se le ocurra añadir un adjetivo. Eso es absurdo y computacionalmente inviable.

La genialidad de Davidson fue proponer algo distinto: sugirió que en cada oración de acción existe un protagonista fantasma, un argumento oculto que no pronunciamos pero que está ahí. Ese protagonista es **el evento en sí mismo**.

La oración no dice simplemente que Juan conectó su pie con una pelota; lo que la oración afirma realmente es que **existió un evento oficial llamado "patear"**. Y a ese evento, Juan le prestó su rol de *agente*, la pelota su rol de *objeto*, el patio su rol de *lugar*, y las tres de la tarde su rol de *tiempo*. El verbo se mantiene siempre igual; los modificadores son simplemente "cables" que se enchufan a ese evento central.

Esto es exactamente lo que vimos en el Capítulo 9 cuando hablamos de la **reificación**. Nuestro modelo convierte las acciones en objetos del eje 0. Y luego, usa hechos atómicos (tripletas) para enchufarle el agente, el lugar y el tiempo.

La lingüística lleva sesenta años demostrando que el lenguaje humano funciona "reificando" eventos de forma natural. Y nosotros, desde la ingeniería de datos, acabamos de descubrir que esa es la única forma de construir una base de datos universal que no colapse. No es coincidencia. Es la demostración de que la estructura del cerebro y la estructura del software, cuando están bien diseñadas, son exactamente la misma.

El verbo no es una acción, es un molde

El primer concepto que debemos tatuarnos como ingenieros es este: cuando el idioma español usa un verbo, ese verbo no representa una acción individual que ocurrió en la vida real. El verbo es simplemente un **molde** (un tipo o categoría).

Verbos como *patear*, *vender*, *consultar* o *llover* no viven en el eje de las cosas físicas (0). Viven en el eje de los conceptos (K). Cada molde viene de fábrica con una lista de requisitos: te dice qué tipo de participantes y qué circunstancias acepta para funcionar.

Cuando alguien habla y usa un verbo en una oración concreta, ocurren dos cosas matemáticamente en milisegundos:

1. **Se "fabrica" un evento nuevo:** El sistema crea una situación fresca en la caja 0 (le da un código único de identidad).
2. **Se etiqueta:** El sistema conecta ese evento nuevo con su molde en K para saber de qué trata.

```
(dar_001) ∈ 0  
(dar_001, instancia_de, accion_dar) ∈ M(0, K)
```

Si sabes programar, esto te resultará idéntico a crear clases y objetos. El verbo "dar" es la función principal que vive en la biblioteca de código; cada vez que alguien habla, el sistema *llama* a esa función y crea una ejecución particular en la memoria, anotando qué variables se le pasaron.

El verbo exige una "Firma de Contrato"

Para entender por qué esto vuelve invencible a nuestro modelo, miremos qué pasa cuando "llamamos" a un verbo. El verbo *dar* no es un cajón de sastre que acepta cualquier basura. Exige reglas estrictas: necesita un *agente* (alguien que entrega), un *tema* (la cosa entregada) y un *beneficiario* (el que recibe). Opcionalmente, puede aceptar un *momento*, un *lugar* o una *finalidad*.

Pero la exigencia va más allá. El verbo exige que cada rol provenga de un eje específico: el *agente* tiene que venir de la caja de personas (Q); el *lugar* tiene que venir de la caja de locaciones (L).

A esta lista estricta de requisitos la llamamos **signatura tipada** (la firma del contrato). Se ve exactamente igual a la declaración de una función en código moderno:

```
dar(agente: Q, tema: 0, beneficiario: Q,  
    momento?: T, lugar?: L, con_finalidad?: 0) → Fabrica una situación
```

Cada verbo del idioma tiene su propia signatura. Algunas son minúsculas y solitarias:

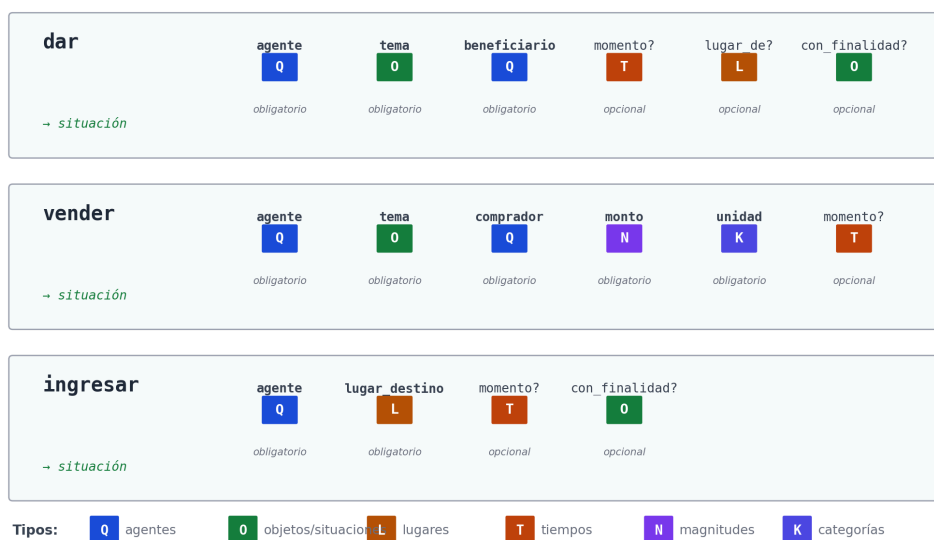
```
llover(momento?: T, lugar?: L) → Fabrica una situación    // Ojo: la lluvia no exige agente  
soñar(experimentador: Q, tema: 0, momento?: T) → Fabrica una situación
```

Otras son bestias complejas de facturación:

```
vender(agente: Q, tema: 0, comprador: Q, monto: N,  
        unidad: K, momento?: T, lugar?: L) → Fabrica una situación
```

El verbo como signature tipada

Cada verbo declara qué roles acepta, cuáles son obligatorios y de qué eje viene cada uno.



Tres verbos del español como signatures tipadas: cada verbo declara qué roles obligatorios y opcionales acepta, y de qué eje viene cada uno. Igual que una función en un lenguaje de programación.

Tratar a los verbos como "contratos de código" nos regala tres superpoderes automáticos:

- 1. Protección Anti-Tonterías:** Si un usuario escribe "el estadio de fútbol pateó la pelota", el sistema aborta la operación antes de guardarla. ¿Por qué? Porque la signature del verbo *patear* exige que el *agente* sea una persona (**Q**), y el estadio es un lugar (**L**). El sistema no necesita saber de fútbol para bloquear el error; la matemática de la signature lo frena en seco.
- 2. Flexibilidad real:** Decir "llovió" es una frase perfecta. No hace falta obligar al usuario a llenar los campos de "dónde" y "cuándo" si no le interesan. La signature del verbo ya marca que esos campos son opcionales (**?**), dándole a la base de datos la capacidad de tragar información incompleta sin romperse.
- 3. Autocontrol de modificadores:** El verbo *patear* acepta que le conectes un *instrumento* (con el pie) o una *finalidad* (para hacer un gol). El verbo *llover* acepta *intensidad* o *duración*. Si alguien intenta escribir "pateó con intensidad de tres milímetros por hora", el sistema explota de inmediato. La signature del verbo controla qué adjetivos son legales y cuáles son ridículos, sin que tengas que programar reglas manuales para cada caso.

La regla D8: El menú oficial de roles

A estas alturas te habrás dado cuenta de que palabras como *agente*, *tema*, *beneficiario* o *instrumento* no nos las estamos inventando al azar en cada ejemplo. Vienen de un **menú oficial y cerrado** que nuestro sistema trae de fábrica.

Oficialicemos esto como nuestra octava decisión de diseño:

D8 — El modelo trabaja con un catálogo oficial, cerrado y muy reducido de roles (cables conectores). Los protagonistas siempre se llamarán agente, tema, beneficiario, lugar_de, momento, etc. Cada rol tiene una regla que dicta de qué eje debe sacar la información. Los

programadores no pueden inventar roles nuevos a menos que sea un caso de extrema especialización industrial.

Nuestro sistema base funciona con apenas 38 roles oficiales. Parece un número bajísimo, ¿verdad? Pues resulta que con esas 38 palabras alcanza y sobra para modelar bases de datos de fútbol, medicina, banca, petroquímica y contratos legales. ¿El motivo? Porque el cerebro humano, sin importar su profesión, utiliza ese mismo número reducido de roles para procesar toda la realidad que lo rodea.

El catálogo completo: las 38 piezas del Lego

A continuación va el catálogo entero, agrupado por familias. Para cada rol verás su nombre canónico, su signatura (qué cajas conecta) y para qué sirve en la práctica. La notación $O \rightarrow Q$ se lee como "el sujeto del hecho vive en O y el valor del hecho vive en Q ". Cuando un rol es **funcional** (`func`), un sujeto solo puede tener un valor (una consulta médica tiene un solo paciente, una sola fecha). Cuando es **no funcional** (`mult`), un sujeto admite múltiples valores (una receta tiene varios ingredientes; un evento puede tener varias causas).

Familia 1: Estructurales — los cables que arman el esqueleto

Son los cables más básicos: sin ellos, no hay grafo. Permiten conectar instancias con sus categorías y formar jerarquías de pertenencia.

Rol	Signatura	Para qué sirve
<code>instancia_de</code>	$V \rightarrow K$ (<code>mult</code>)	Marca que un individuo —de cualquier eje— es un ejemplar de una categoría. "esta consulta (O) es una instancia de <code>accion_consultar</code> "; "Messi (Q) es una instancia de <code>jugador_de_futbol</code> ".
<code>subtipo_de</code>	$K \rightarrow K$ (<code>mult</code>)	Construye jerarquías de categorías. " <code>jugador_de_futbol</code> es subtipo de <code>atleta_profesional</code> ".
<code>parte_de</code>	$O \rightarrow O$ (<code>mult</code>)	Indica que algo forma parte de un evento o entidad mayor. "esta prescripción es parte de la consulta del 14 de mayo".
<code>contiene</code>	$O \rightarrow O$ (<code>mult</code>)	El inverso del anterior. "la consulta contiene esta prescripción".

El comodín V . Quizá notaste que `instancia_de` dice $V \rightarrow K$ y no $O \rightarrow K$. Es deliberado: **clasificar es universal**. No solo los objetos (O) pertenecen a una categoría — también un agente (Messi, en Q), un lugar (Lima, en L) o un instante (T) responden a la pregunta "¿de qué concepto sos instancia?". V es el comodín de signatura que significa *cualquier eje de valor*: es el **universo V** del Capítulo 5 — los seis ejes tomados como conjunto. Cuidado de no confundirlo con K : **K es un eje** —el lugar donde viven las categorías—; **V es "todos los ejes" a la vez** —un cuantificador, no un sitio donde algo vive—. Por eso `instancia_de` es $V \rightarrow K$: *cualquier individuo hacia una categoría*. El mismo comodín sirve en el rango cuando un rol necesita admitir varios ejes a la vez: por ejemplo, generalizar `partes` o `tema` a $O \rightarrow V$ para que acepten lo mismo un objeto que una categoría (lo retomamos en el Capítulo 28).

Familia 2: Participantes — el "quién" y "qué" del verbo

Estos son los protagonistas de cualquier acción. Vienen directamente de la lingüística (los famosos "roles temáticos") y son la columna vertebral de todo el modelo.

Rol	Signatura	Para qué sirve
agente	$O \rightarrow Q$ (func)	Quien ejecuta la acción de forma intencional. <i>"María vendió"</i> $\rightarrow$ agente es María.
paciente	$O \rightarrow Q$ (func)	Quien recibe o sufre la acción. <i>"el médico atendió al paciente"</i> .
tema	$O \rightarrow O$ (func)	El objeto o sub-situación sobre el que recae la acción. <i>"prescribió un medicamento"</i> .
beneficiario	$O \rightarrow Q$ (func)	El destinatario o quien se beneficia. <i>"le dio un libro a María"</i> $\rightarrow$ María es beneficiaria.
experimentador	$O \rightarrow Q$ (func)	Quien vive un estado mental. <i>"Juan teme"</i> , <i>"Marta cree"</i> . Distingue al sentidor del agente activo.
instrumento	$O \rightarrow O$ (func)	El objeto que media la acción. <i>"pateó con la pierna izquierda"</i> , <i>"escribió con un bolígrafo"</i> .
comprador	$O \rightarrow Q$ (func)	Comprador en una venta — alias especializado de beneficiario.
cliente	$O \rightarrow Q$ (func)	Cliente de un servicio — alias frecuente en negocios.

Familia 3: Lugar — el "dónde"

Rol	Signatura	Para qué sirve
lugar_de	$O \rightarrow L$ (func)	Lugar donde ocurre la situación. <i>"la consulta fue en el consultorio 03"</i> .
origen	$O \rightarrow L$ (func)	Punto de partida en movimientos o transferencias.
destino	$O \rightarrow L$ (func)	Punto de llegada. <i>"el taxi va al aeropuerto"</i> .
lugar_destino	$O \rightarrow L$ (func)	Alias de destino, útil cuando el verbo lo exige explícito (<code>ingresar</code>).

Familia 4: Tiempo — el "cuándo"

Rol	Signatura	Para qué sirve
momento	$O \rightarrow T$ (func)	Instante puntual. <i>"firmó el contrato el 14 de mayo a las 10:30"</i> .
inicio	$O \rightarrow T$ (func)	Comienzo de un rango temporal. <i>"el préstamo vigente desde junio"</i> .
fin	$O \rightarrow T$ (func)	Fin de un rango temporal. Junto con <code>inicio</code> arma la bitemporalidad (D6).

Familia 5: Cantidad — el "cuánto"

Rol	Signatura	Para qué sirve
monto	$O \rightarrow N$ (func)	Cantidad numérica con unidad. <i>"costó 49.90 dólares"</i> .
cantidad	$O \rightarrow N$ (func)	Alias de monto, más natural en contextos físicos (gramos, litros).
por_cuanto	$O \rightarrow N$ (func)	Precio o medida unitaria asociada. <i>"a 12 dólares la docena"</i> .
unidad	$O \rightarrow K$ (func)	Unidad de medida — siempre apunta a una categoría QUDT.

Familia 6: Clasificatorios — el "modo" del hecho

Estos cables permiten al modelo distinguir hechos del mundo real de hechos hipotéticos, planeados, prohibidos o negados.

Rol	Signatura	Para qué sirve
estatus_factual	$O \rightarrow K$ (func)	Marca si la situación es <code>real</code> , <code>intencionada</code> , <code>planeada</code> , <code>cancelada</code> , <code>hipotetica</code> .
modalidad	$O \rightarrow K$ (func)	Tipo de modalidad: volitiva (deseo), deóntica (deber), alética (necesidad), epistémica (conocimiento).
polaridad	$O \rightarrow K$ (func)	Afirmativa o negativa. Permite registrar <i>"el paciente NO tiene fiebre"</i> como un hecho positivo.
calificacion	$O \rightarrow K$ (func)	Atributo cualitativo libre. <i>"clasificación de riesgo: alto"</i> .

Familia 7: Las cuatro relaciones del "por qué" (D7)

Ya las estudiamos en el capítulo 10, pero las incluimos aquí porque son parte oficial del catálogo.

Rol	Signatura	Para qué sirve
causado_por	$O \rightarrow O$ (mult)	Causalidad mecánica/física. <i>"el incendio causó el derrumbe"</i> .
motivado_por	$O \rightarrow O$ (mult)	Motivo intencional. <i>"vendió la casa motivada por el deseo de mudarse"</i> .
con_finalidad	$O \rightarrow O$ (mult)	Propósito teleológico — apunta a un estado futuro. <i>"abrió la vía con finalidad de acceder al pulmón"</i> .
justificado_por	$O \rightarrow O$ (mult)	Autoridad normativa. <i>"la rescisión está justificada por la cláusula 14"</i> .

Familia 8: Inter-situacionales — eventos que se hablan entre sí

Permiten que las situaciones formen secuencias, se anulen, se corrijan o se contradigan entre ellas.

Rol	Signatura	Para qué sirve
precede	$O \rightarrow O$ (mult)	Orden lógico o temporal. <i>"la aceptación precede al traslado"</i> .
sigue_a	$O \rightarrow O$ (mult)	Inverso de precede.
cumple	$O \rightarrow O$ (mult)	Una situación cumple una obligación o regla previa.
cancela	$O \rightarrow O$ (mult)	Deja sin efecto una situación anterior. <i>"la cancelación cancela la reserva"</i> .
rectifica	$O \rightarrow O$ (mult)	Corrige una situación previa sin borrarla (nunca borramos: rectificamos).
contrasta_con	$O \rightarrow O$ (mult)	Relación adversativa, equivalente al "pero" del lenguaje. <i>"el paciente mejoró, pero contrasta con la elevación de la presión"</i> .

Familia 9: Atributos del sujeto Q

Los únicos roles que no tienen un sujeto en O. Sirven para describir a los agentes mismos.

Rol	Signatura	Para qué sirve
nombre	$Q \rightarrow K$ (func)	Nombre legible de un agente. <i>"el agente q_447 tiene nombre María Gonzales"</i> .
identificador	$Q \rightarrow K$ (func)	Identificador documental (DNI, RUC, número de empleado).

¿Son solo 38? ¿Puedo agregar más?

La respuesta corta es: **sí, puedes agregar más, pero con criterio**. El catálogo canónico está pensado para mantenerse pequeño y estable, exactamente como el inventario de roles temáticos que la lingüística viene refinando hace medio siglo. Sin embargo, el sistema implementa una política deliberadamente flexible que el código llama **"política liberal"**:

- Si un rol está en el catálogo canónico, el sistema **valida** que los ejes coincidan con la signatura.
- Si un rol **no** está declarado, el sistema lo **acepta sin validar**, en lugar de rechazarlo.

¿Por qué este criterio? Porque ningún catálogo central puede anticipar todas las palabras técnicas de todos los dominios. Si un químico necesita el rol `reactivo:  $O \rightarrow N$` , o un musicólogo necesita `obra_interpretada:  $O \rightarrow K$` , deben poder usarlos sin pedir permiso. El catálogo no es una camisa de fuerza; es un **núcleo estable** alrededor del cual cada dominio extiende.

La regla práctica que el libro defiende es:

1. **Usa primero los 38 canónicos.** Cubren la gran mayoría de los casos.
2. **Si tu dominio necesita un rol específico** (un médico necesita `principio_activo`, un banco necesita `cuenta_origen`), agrégalo como rol de dominio. El sistema lo aceptará sin protestar.
3. **Cuando un rol de dominio aparece tres veces en tres dominios diferentes**, es candidato a promoverse al catálogo canónico. Ese es el mecanismo evolutivo del catálogo.

Los capítulos de la Parte V muestran este patrón en acción: el dominio musical agrega `obra_interpretada`, el químico agrega `insumo` y `reactivo`, el bancario agrega `cuenta_origen` y `cuenta_destino`. Ninguno toca los 38 canónicos; cada uno extiende lo que necesita y deja documentado el patch.

Cinco situaciones inventadas: el catálogo en acción

Para que el catálogo no se quede como una lista abstracta, vamos a aterrizarlo con cinco escenarios completamente nuevos — situaciones que no aparecen en el resto del libro. La idea es que veas cómo los mismos 38 roles canónicos cubren contextos absolutamente dispares sin esfuerzo extra. Si el modelo es universal de verdad, debería poder absorber estos casos exactamente con las piezas que ya tiene.

Escenario 1 — Una familia adopta a un perro de un refugio

El sábado por la mañana, Carolina y Diego fueron al refugio "Patitas del Norte" y adoptaron a una perrita llamada Lola. La razón principal fue que su hija de seis años, Mariana, llevaba meses pidiendo una mascota.

Modelado completo:

(adopcion_001, instancia_de,	accion_adoptar)	∈ M(0, K)
(adopcion_001, agente,	carolina)	∈ M(0, Q)
(adopcion_001, agente,	diego)	∈ M(0, Q)
(adopcion_001, tema,	lola)	∈ M(0, O)
(adopcion_001, beneficiario,	mariana)	∈ M(0, Q)
(adopcion_001, lugar_de,	refugio_patitas_del_norte)	∈ M(0, L)
(adopcion_001, momento,	2026-05-16T10:00:00Z)	∈ M(0, T)
(adopcion_001, motivado_por,	pedido_mariana_001)	∈ M(0, O)
(adopcion_001, estatus_factual,	real)	∈ M(0, K)

Y la situación que la motivó vive aparte, con su propia identidad:

(pedido_mariana_001, instancia_de,	accion_pedir)	∈ M(0, K)
(pedido_mariana_001, agente,	mariana)	∈ M(0, Q)
(pedido_mariana_001, tema,	mascota_familiar)	∈ M(0, K)
(pedido_mariana_001, inicio,	2026-01-15)	∈ M(0, T)

Lo interesante: aparecieron dos agentes en una misma situación (Carolina y Diego como pareja). El modelo lo soporta sin queja porque `agente` es un rol multi-valor (M, no funcional). La perra Lola es **tema** (O→O), porque es el objeto de la acción; Mariana es **beneficiaria**, porque la adopción es "para ella". Y el motivo se modela como otra situación reificada conectada por `motivado_por`.

Escenario 2 — Una multa de tránsito por exceso de velocidad

El oficial Ramírez detectó con un radar Doppler que la camioneta del señor Mendoza circulaba a 92 km/h en

una zona limitada a 60. Le aplicó una multa de 180 dólares según el artículo 138 del Código de Tránsito.

(multa_001, instancia_de,	accion_multar)	∈ M(0, K)
(multa_001, agente,	oficial_ramirez)	∈ M(0, Q)
(multa_001, paciente,	sr_mendoza)	∈ M(0, Q)
(multa_001, instrumento,	radar_doppler_zona_norte)	∈ M(0, O)
(multa_001, tema,	vehiculo_mendoza_abc123)	∈ M(0, O)
(multa_001, lugar_de,	av_central_km_12)	∈ M(0, L)
(multa_001, momento,	2026-05-12T08:43:00Z)	∈ M(0, T)
(multa_001, monto,	180)	∈ M(0, N)
(multa_001, unidad,	dolar_estadounidense)	∈ M(0, K)
(multa_001, causado_por,	infraccion_velocidad_001)	∈ M(0, O)
(multa_001, justificado_por,	articulo_138_ct)	∈ M(0, O)

Y la infracción que disparó la multa también se reifica:

(infraccion_velocidad_001, instancia_de,	evento_exceso_velocidad)	∈ M(0, K)
(infraccion_velocidad_001, agente,	sr_mendoza)	∈ M(0, Q)
(infraccion_velocidad_001, tema,	vehiculo_mendoza_abc123)	∈ M(0, O)
(infraccion_velocidad_001, monto,	92)	∈ M(0, N)
(infraccion_velocidad_001, unidad,	km_por_hora)	∈ M(0, K)

Lo interesante: aquí trabajan al mismo tiempo las cuatro grandes familias. El **agente** (oficial) y el **paciente** (conductor) son humanos diferentes. El **instrumento** (radar) es un objeto físico; el **tema** (vehículo) también, pero ocupa otro rol. El **monto** + **unidad** registran el dinero de la multa, y existen *dos* relaciones del "por qué" simultáneamente: **causado_por** apunta a la causa fáctica (la infracción ocurrió antes), y **justificado_por** apunta a la regla legal que faculta al oficial a multar. El mundo real exige las dos.

Escenario 3 — Una devolución en una tienda online

El 14 de mayo, Sofía devolvió a la tienda online "EcoMarket" un cargador inalámbrico que había comprado dos semanas antes, porque el producto llegó dañado. La tienda le reembolsó 28.50 dólares a su tarjeta. La devolución estaba permitida por la política de "30 días sin preguntas".

(devolucion_001, instancia_de,	accion_devolver)	∈ M(0, K)
(devolucion_001, agente,	sofia)	∈ M(0, Q)
(devolucion_001, tema,	cargador_inalambrico_X7)	∈ M(0, O)
(devolucion_001, beneficiario,	ecomarket)	∈ M(0, Q)
(devolucion_001, momento,	2026-05-14T14:22:00Z)	∈ M(0, T)
(devolucion_001, causado_por,	evento_producto_danado_001)	∈ M(0, O)
(devolucion_001, justificado_por,	politica_30_dias_ecomarket)	∈ M(0, O)
(devolucion_001, cancela,	compra_001)	∈ M(0, O)
(reembolso_001, instancia_de,	accion_reembolsar)	∈ M(0, K)
(reembolso_001, agente,	ecomarket)	∈ M(0, Q)
(reembolso_001, beneficiario,	sofia)	∈ M(0, Q)
(reembolso_001, monto,	28.50)	∈ M(0, N)
(reembolso_001, unidad,	dolar_estadounidense)	∈ M(0, K)
(reembolso_001, sigue_a,	devolucion_001)	∈ M(0, O)
(reembolso_001, justificado_por,	politica_30_dias_ecomarket)	∈ M(0, O)

Lo interesante: nota cómo **ecomarket** (una empresa, no una persona) aparece tanto como **agente** del reembolso como **beneficiario** de la devolución. La caja Q acepta entidades jurídicas sin problema (D5 en acción). También aparecen tres relaciones inter-situacionales muy potentes: **cancela** (la devolución anula la compra), **sigue_a** (el reembolso ocurre después de la devolución) y **causado_por** (el daño físico provocó la devolución).

Escenario 4 — Una consulta médica por telemedicina con asistente de IA

El doctor González atendió por videoconferencia a su paciente Renata Ojeda el 10 de mayo a las 16:00. Antes de iniciar, un asistente de IA (Claude Medical) había resumido para el doctor los últimos análisis y antecedentes de la paciente.

Tenemos dos situaciones encadenadas:

(resumen_ia_001, instancia_de,	accion_resumir)	∈ M(0, K)
(resumen_ia_001, agente,	claudio_medical)	∈ M(0, Q)
(resumen_ia_001, tema,	historia_clinica_ojeda)	∈ M(0, O)
(resumen_ia_001, beneficiario,	dr_gonzalez)	∈ M(0, Q)
(resumen_ia_001, momento,	2026-05-10T15:58:00Z)	∈ M(0, T)
(resumen_ia_001, con_finalidad,	consulta_001)	∈ M(0, O)
(consulta_001, instancia_de,	accion_consultar)	∈ M(0, K)
(consulta_001, agente,	dr_gonzalez)	∈ M(0, Q)
(consulta_001, paciente,	renata_ojeda)	∈ M(0, Q)
(consulta_001, instrumento,	plataforma_telemed_zoom)	∈ M(0, O)
(consulta_001, lugar_de,	ubicacion_remota)	∈ M(0, L)
(consulta_001, momento,	2026-05-10T16:00:00Z)	∈ M(0, T)
(consulta_001, modalidad,	remota_sincrona)	∈ M(0, K)
(consulta_001, estatus_factual,	real)	∈ M(0, K)
(consulta_001, precede,	prescripcion_001)	∈ M(0, O)

Lo interesante: el agente de la primera situación es un **modelo de IA** (Claude Medical) — D5 hecho realidad. El sistema no distingue entre humanos, organizaciones y software; cualquiera que actúe entra en Q. Además, el **instrumento** (Zoom) muestra cómo los objetos digitales pueden mediar acciones físicas. El **con_finalidad** que conecta el resumen con la consulta posterior es interesante: el resumen no causó la consulta (estaba programada antes), pero tenía como propósito prepararla.

Escenario 5 — Un accidente sin agente humano: un cable suelto en una obra

El miércoles 7 de mayo a las 11:40, un cable eléctrico mal anclado de la grúa torre 3 se desprendió y golpeó al obrero Juan Vázquez, causándole una fractura en el brazo derecho. La investigación posterior determinó que el protocolo de seguridad N°14 había sido incumplido en la última inspección.

(accidente_001, instancia_de,	evento_accidente_laboral)	∈ M(0, K)
(accidente_001, paciente,	juan_vazquez)	∈ M(0, Q)
(accidente_001, instrumento,	cable_suelto_grua_torre_3)	∈ M(0, O)
(accidente_001, lugar_de,	obra_residencial_norte_03)	∈ M(0, L)
(accidente_001, momento,	2026-05-07T11:40:00Z)	∈ M(0, T)
(accidente_001, causado_por,	desprendimiento_cable_001)	∈ M(0, O)
(accidente_001, estatus_factual,	real)	∈ M(0, K)
(desprendimiento_cable_001, instancia_de,	evento_fisico)	∈ M(0, K)
(desprendimiento_cable_001, tema,	cable_suelto_grua_torre_3)	∈ M(0, O)
(desprendimiento_cable_001, momento,	2026-05-07T11:40:00Z)	∈ M(0, T)
(desprendimiento_cable_001, causado_por,	inspeccion_omitida_001)	∈ M(0, O)

Lo interesante: este es el caso donde **D5 brilla en su forma más extrema** — no hay ningún agente en la situación principal. El accidente *le ocurrió* a Juan; nadie lo *hizo*. El catálogo lo acepta sin marcar error porque agente es un rol opcional, no obligatorio. El instrumento es un objeto físico inanimado (un cable), y la cadena causal se reconstruye hacia atrás encadenando varios **causado_por**: el accidente fue causado por un

desprendimiento, que a su vez fue causado por una inspección omitida. El grafo permite recorrer hacia atrás toda la cadena de responsabilidad sin necesidad de un agente humano explícito.

Si miras los cinco escenarios juntos, descubres algo: usamos prácticamente todos los roles del catálogo y, sin embargo, **no inventamos ninguno nuevo**. Los mismos 38 cables, organizados de formas distintas, capturan una adopción, una multa, una devolución, una consulta médica con IA y un accidente sin culpable. Esa es la prueba operativa de que el catálogo no es arbitrario: es un alfabeto suficiente.

De oración humana a código informático (Paso a paso)

Para ver la magia en acción, desarmemos la oración de manual que usamos antes, esa que estaba cargada de detalles:

Juan le dio un libro a María ayer en su casa para sorprenderla.

Veamos cómo el sistema convierte esto en bases de datos con un procedimiento mecánico, casi tonto de tan simple:

Paso 1 — Detectar el molde. El verbo principal es *dio* (de *dar*). El sistema busca la signatura de *dar* en su diccionario.

Paso 2 — Reificar la situación. El sistema crea un evento vacío en el eje *0* y le pone un código de barras. Llamémoslo *dar_001*.

Paso 3 — Anclar el molde. Le decimos al sistema de qué trata el evento:

`(dar_001, instancia_de, accion_dar) ∈ M(0, K)`

Paso 4 — Enchufar los cables (Roles). El sistema lee la oración y reparte los pedazos en los enchufes que la signatura del verbo *dar* dejó abiertos:

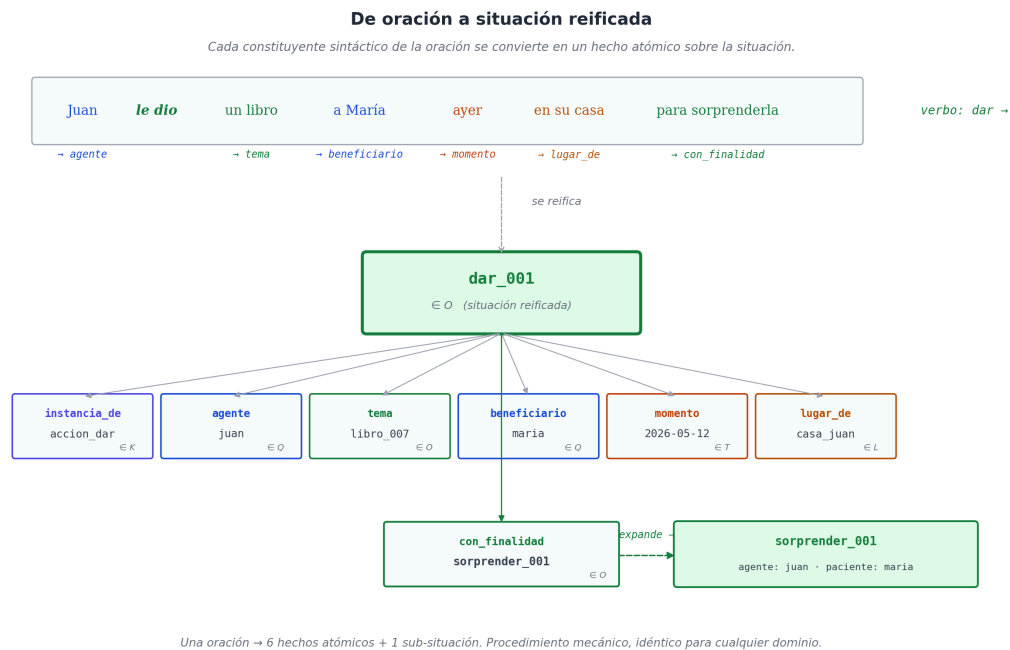
Juan	→ El que da	→ agente	→ va al eje Q
un libro	→ La cosa dada	→ tema	→ va al eje Q
a María	→ La que recibe	→ beneficiario	→ va al eje Q
ayer	→ El tiempo	→ momento	→ va al eje T
en su casa	→ El sitio	→ lugar_de	→ va al eje L
para sorprenderla	→ El objetivo	→ con_finalidad	→ va al eje O

Cada línea de arriba se convierte automáticamente en un hecho atómico (una tripleta) pegado al evento central *dar_001*:

```
(dar_001, agente,      juan)
(dar_001, tema,        libro_007)
(dar_001, beneficiario, maria)
(dar_001, momento,     2026-05-12)
(dar_001, lugar_de,     casa_juan)
(dar_001, con_finalidad, sorprender_001)
```

Paso 5 — El bucle (Inception). Resulta que el pedazo *para sorprenderla* tiene un verbo oculto adentro ("sorprender"). El sistema no entra en pánico; simplemente repite los 4 pasos anteriores para crear un "sub-evento" de sorpresa y lo engancha al principal.

Resultado final: **una frase humana se transformó en diez líneas de código perfecto y validado**, sin perder un solo matiz, sin usar campos de "texto libre" y sin necesidad de que un programador construya tablas especiales.



De una oración en lenguaje natural a una situación reificada con sus hechos atómicos: el verbo determina el tipo, el sujeto se vuelve agente, los complementos llenan roles canónicos. Una sub-oración (la finalidad) genera una sub-situación.

Y esto no es teoría: el prototipo que acompaña al libro lo ejecuta tal cual. Afirmar un hecho valida su signature; consultar es buscar un patrón de cables:

```
from wq import Axis, Individual, Universe, Catalog, Pattern, count, category

u = Universe(name="demo", catalog=Catalog())

# "Messi marcó un gol en el minuto 87" - una situación reificada
gol = u.add_individual(Individual(id="gol_min87", axis=Axis.0, label="gol del minuto 87"))
messi = u.add_individual(Individual(id="messi", axis=Axis.Q, label="Messi"))
u.add_individual(category("evento_gol"))

# Cada hecho es (sujeto, rol, valor); la signature del rol se valida al afirmar.
u.assert_fact(gol, "instancia_de", u.ind("evento_gol")) # M(0, K)
u.assert_fact(gol, "agente", messi) # M(0, Q)

# Consultar = buscar un patrón: "¿cuántos goles marcó Messi?"
goles = count(u, Pattern(fixed={"agente": messi},
                        type_constraint=u.ind("evento_gol")))
print(goles) # -> 1
```

Funciona igual para el SPA de Ana

Para que veas que esto no es un truco arreglado, apliquemos este mismo proceso "tonto y mecánico" a una industria totalmente distinta. Una clienta en un Spa:

Ana ingresó a la cámara de vapor a las seis y media para relajarse.

El molde es *ingresar*. Su signatura exige un agente, un destino y un momento. Aplicamos la reificación y los cables:

```
(ingresar_017, instancia_de, accion_ingresar)
(ingresar_017, agente, cliente_ana)
(ingresar_017, lugar_destino, camara_vapor_1)
(ingresar_017, momento, 2026-05-15T18:30Z)
(ingresar_017, con_finalidad, relajacion_001)

(relajacion_001, instancia_de, estado_relajacion)
(relajacion_001, experimentador, cliente_ana)
```

El verbo *ingresar* no se parece en nada al verbo *dar*, tienen reglas distintas. Pero **la tubería matemática para procesarlos fue exactamente la misma**. Esta es la prueba definitiva de que nuestro modelo puede absorber los datos de una clínica, un banco o un restaurante sin obligar a los ingenieros a reescribir el motor de la base de datos cada vez que abren una empresa nueva.

Cuando las preguntas de la vida son simplemente "Huecos en el código"

La belleza de este diseño cierra su círculo cuando nos damos cuenta de qué pasa cuando en lugar de afirmar algo, *hacemos una pregunta*.

Las palabras que usamos los humanos para preguntar (*quién, qué, dónde, cuándo*) no son palabras mágicas; **son simplemente los cables de nuestro sistema a los que les hemos borrado el destino**.

Si tomamos el evento `dar_001` de arriba, mira cómo cualquier pregunta que haga un humano se traduce a una tripleta donde tapamos un hueco con un signo de interrogación:

```
"¿Quién le dio el libro a María?"
  { Busco: accion_dar, tema: libro, beneficiario: maria, agente: ? } → (El sistema buscará en Q)

"¿Qué le dio Juan a María?"
  { Busco: agente: juan, beneficiario: maria, tema: ? } → (El sistema buscará en O)

"¿Cuándo se lo dio?"
  { Busco: el evento de dar, momento: ? } → (El sistema buscará en T)

"¿Para qué se lo dio?"
  { Busco: el evento de dar, con_finalidad: ? } → (El sistema buscará en O)
```

Para la computadora, leer un documento y guardar los datos es exactamente el mismo proceso matemático que buscar en el archivo para responder una duda. Son las dos caras de la misma moneda.

(Dato curioso: Las palabras que usamos para preguntar en español descienden directamente de las reglas judiciales de Cicerón en Roma que vimos en el Capítulo 6. Los filósofos modernos lo redescubrieron hace poco, pero la mente humana ya lo traía de fábrica).

El efecto Matrioshka: Cuando alguien habla sobre otra cosa

Hay un nivel de dificultad que tumba a casi todos los sistemas de bases de datos: ¿Qué hacemos cuando una oración habla sobre otra oración?

El cliente dijo que la cámara seca estaba muy caliente.

El verbo principal es *decir*. Y fíjate que la "cosa dicha" no es un objeto que puedas tocar (como un libro); la cosa dicha es **otra situación completa**. Nuestro sistema ni parpadea ante este reto, porque para nosotros, todas las situaciones viven como objetos iguales en el eje 0.

Creamos el evento de "hablar" y le metemos el "reporte" por dentro, como muñecas rusas (Matrioshkas):

```
(decir_023, instancia_de, accion_decir)
(decir_023, agente, cliente_ana)
(decir_023, tema, estado_camara_001) ← ¡El tema es otro evento!

(estado_camara_001, instancia_de, estado_temperatura)
(estado_camara_001, sujeto, camara_seca)
(estado_camara_001, calificacion, muy_caliente)
(estado_camara_001, estatus_factual, reporte_de_cliente) ← ¡Atención aquí!
```

Ese detalle final (`estatus_factual: reporte_de_cliente`) es oro puro. Le dice a la base de datos que la máquina no tiene pruebas físicas de que la cámara esté caliente; simplemente está guardando el "chisme" de que un cliente lo afirmó. Gracias a esto, podemos procesar frases gigantescas como *"María dijo que Juan piensa que el servidor se cayó"*, construyendo una torre de eventos sin tener que programar reglas nuevas.

El verbo es el contrato maestro

Cerremos este capítulo fijando un concepto en piedra. Cuando metemos un verbo en nuestra base de datos, no estamos guardando una palabra de diccionario; estamos metiendo un **contrato de seguridad**.

El verbo actúa como un abogado de aduana: declara qué información es obligatoria para pasar, qué información es opcional, y de qué caja (`Q`, `T`, `L...`) tiene que venir cada dato. Si intentas meter basura, el verbo la bloquea en la puerta. Si metes datos limpios, el verbo ejecuta un proceso de ensamblaje en cinco pasos y archiva todo perfectamente.

Pero claro, para que todo este sistema de aduanas funcione, nuestra base de datos necesita un documento oficial donde estén escritos todos y cada uno de los contratos de los verbos del idioma español. Ese documento maestro se llama el **Lexicon**.

No es un aburrido apéndice gramatical, sino **el motor traductor definitivo** que le permite a una Inteligencia Artificial hablar directamente con los discos duros de tu empresa usando lenguaje humano.

Capítulo 12 — El Lexicon: El traductor maestro

El problema de la mano (O por qué los verbos nos mienten)

Piensa en estas dos oraciones tan comunes en español:

- Pedro le dio un regalo a María.
- Pedro le dio la mano a su jefe.

Si aplicamos al pie de la letra las reglas matemáticas que aprendimos en el capítulo anterior, nuestra base de datos leería el verbo "dar" y crearía automáticamente un evento de tipo `accion_dar`. El sistema creería que Pedro (el agente) le transfirió algo (el objeto) a un beneficiario.

En la primera oración, todo funciona perfecto: María ahora es dueña de un regalo. Pero en la segunda oración... hay un gran problema. El jefe de Pedro no salió de la oficina con una mano ensangrentada en el bolsillo. No hubo ninguna "transferencia" física de nada. *Dar la mano* es una expresión idiomática de nuestro idioma que simplemente significa **saludar formalmente**. Es decir, la segunda oración no debería guardarse como una `accion_dar`, sino como una `accion_saludar`.

Y el verbo *dar* esconde aún más trampas: * *Pedro dio una conferencia el martes*: No transfirió nada, simplemente **realizó** un evento de exposición (`evento_exposicion`). * *El reloj dio las tres*: Es una metáfora, el reloj no regala tiempo, simplemente **suen**a (`accion_sonar`). * *La noticia le dio asco*: No hay regalo físico, es una **experiencia sensorial** interna (`experiencia_asco`).

Un mismo verbo, cinco tipos de situación completamente distintos. A este fenómeno —cuando el significado del verbo depende de la palabra que lo acompaña— los lingüistas lo llaman **polisemia léxica**. Y es la regla de oro de la comunicación humana, no la excepción.

Nuestra base de datos **necesita resolver** qué tipo de situación está ocurriendo realmente antes de empezar a conectar cables. Porque la regla de seguridad del verbo `dar(agente, regalo, beneficiario)` no tiene la misma firma que `dar_la_mano(agente, persona_saludada)` ni que `dar_conferencia(agente, tema, audiencia)`.

Para que la máquina no colapse, necesitamos construir un **diccionario** que no mapee palabras sueltas, sino frases completas (el verbo + su complemento clave). A este diccionario maestro lo llamaremos **El Lexicon**.

El Lexicon es la pieza más visible del proyecto

Antes de bajar al código, tenemos que entender la importancia arquitectónica de este diccionario.

En el capítulo 11 dijimos que el sistema funciona con un catálogo oficial de roles o cables (como `agente`, `tema` o `beneficiario`). Ahora vamos a presentar una decisión de diseño paralela, nuestra **novena regla (D9)**:

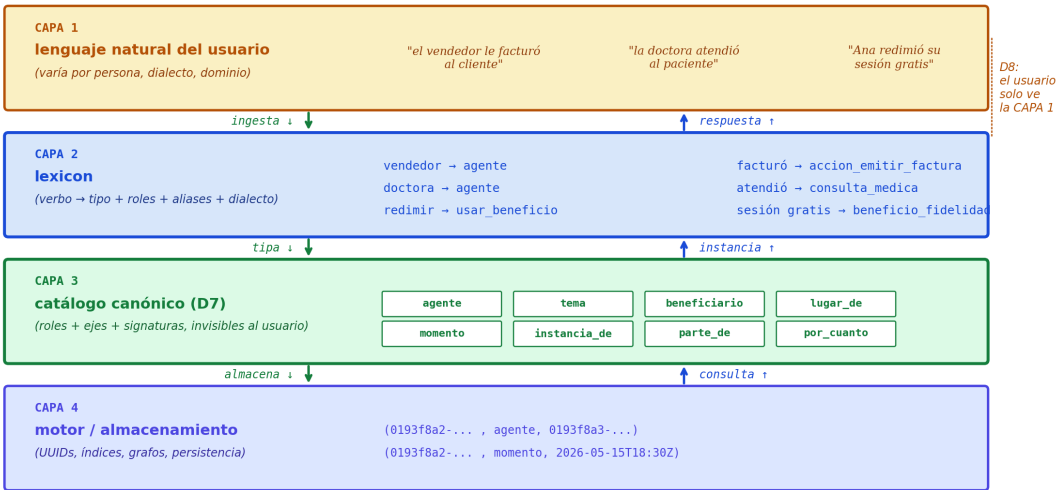
D9 — El catálogo oficial de roles es completamente invisible para el usuario final. Un humano jamás debe ser obligado a escribir palabras como "agente" o "beneficiario" para que el sistema funcione. Los usuarios deben poder usar su idioma natural ("vendedor", "comprador", "cliente"), y el sistema debe traducirlo silenciosamente.

¿Quién hace esa traducción? El Lexicon. D8 estableció **qué hay debajo del capó**; el Lexicon establece **cómo se maneja el coche desde arriba**.

Sin este diccionario, nuestro modelo sería una librería interna maravillosamente diseñada, pero inútil para la gente. Con el Lexicon, se vuelve una interfaz amigable. Y lo que es mejor: D9 garantiza que los ingenieros puedan cambiar la arquitectura interna del software (agregar roles, cambiar códigos) sin romper la experiencia de los clientes, porque la única "cara" que el usuario ve es la del Lexicon. Ese es el trato: el Lexicon es estable hacia afuera; los engranajes por dentro pueden moverse.

El lexicon como capa de traducción (D8)

El catálogo canónico (D7) es invisible al usuario: el lexicon hace la traducción en ambas direcciones.



Cualquier capa puede evolucionar sin romper las demás: el lexicon estabiliza la interfaz hacia el usuario y desacopla el catálogo interno del vocabulario externo.

La arquitectura en capas del lexicon (D9): el usuario habla con su vocabulario natural; el lexicon traduce a los roles canónicos del catálogo D8; el almacenamiento opera con identificadores internos. Cada capa puede cambiar sin afectar a las demás.

Autopsia de una entrada en el diccionario

Veamos cómo se ve una "página" de este diccionario por dentro. Tomemos el caso de `vender`. Así lo escribiría un ingeniero:

```
verbo: vender
tipo_situacion: accion_vender
roles:
  agente:
    canónico: agente
    alias: ["vendedor", "el que vende", "quien vende"]
  tema:
    canónico: tema
    alias: ["producto", "lo vendido", "ítem", "mercancía"]
  beneficiario:
    canónico: beneficiario
    alias: ["comprador", "cliente", "el que compra"]
  por_cuanto:
    canónico: por_cuanto
    alias: ["precio", "monto", "costo"]
obligatorios: [agente, tema, beneficiario, por_cuanto]
opcionales: [momento, lugar_de, moneda, instrumento]
ejemplo: "María le vendió el libro a Juan por 20 dólares"
```

Desarmemos este bloque. Tiene seis piezas y todas tienen un trabajo muy claro:

1. verbo : Es la unidad que activa esta regla. Puede ser una sola palabra (`vender`) o un patrón compuesto (`dar [la mano]`). **2. tipo_situacion** : Es el código interno exacto al que se anclará el evento en la caja `K` . Es el motor oculto que el usuario no ve. **3. roles y sus aliases**: Aquí está la magia de la traducción. El rol oficial es `agente` , pero la lista de *aliases* le enseña al sistema que si un humano dice "*vendedor*" o "*quien vende*", significa lo mismo. Así, "*el vendedor le dio una factura al cliente*" y "*el agente proveyó un comprobante al beneficiario*" generan exactamente el mismo registro en la base de datos. **4. obligatorios y opcionales** : Son las reglas de seguridad de la firma. Dictan qué datos no pueden faltar para que la base de datos acepte la información. **5. ejemplo** : Una frase humana real. No es decorativa: el motor de pruebas (o la Inteligencia Artificial) lee esta frase para aprender cómo se usa este verbo en la vida real.

Vista así, una entrada del Lexicon no es solo una definición; es **una declaración de función de código con su propio manual de uso**.

El Lexicon habla el idioma de la Inteligencia Artificial (Function Calling)

Los modelos de lenguaje más potentes del mundo actual (GPT-4, Claude, Gemini) han adoptado una forma estándar de conectarse con el software corporativo. Se llama *Function Calling* (o "Uso de Herramientas").

Funciona así: la empresa le entrega a la IA un catálogo con formato JSON que describe todas las herramientas disponibles y sus reglas. Cuando un humano le pide algo a la IA, ella lee el catálogo, elige la herramienta correcta y envía una orden estructurada a la base de datos.

Lo fascinante es que el formato JSON que exigen estas IAs... **es estructuralmente idéntico a una entrada de nuestro Lexicon**. Tomemos la entrada de `vender` y traduzcámosla al formato que lee una IA:

```
{
  "name": "vender",
  "description": "Registrar una venta: transferencia de un bien con compensación monetaria",
  "parameters": {
    "type": "object",
    "properties": {
      "agente": {"type": "Q", "description": "vendedor, el que vende"},
      "tema": {"type": "O", "description": "producto, ítem, mercancía"},
      "beneficiario": {"type": "Q", "description": "comprador, cliente"},
      "por_cuanto": {"type": "N", "description": "precio, monto, costo"},
      "momento": {"type": "T", "description": "cuándo ocurrió"},
      "lugar_de": {"type": "L", "description": "dónde ocurrió"}
    },
    "required": ["agente", "tema", "beneficiario", "por_cuanto"]
  }
}
```

Las correspondencias son uno a uno. El `tipo_situacion` se vuelve el `name`. Los *aliases* se convierten en la `description` para que la IA sepa qué significan. Los ejes (`Q`, `O`, `T`) se vuelven los tipos obligatorios. **Nuestro Lexicon es, literalmente, un catálogo de herramientas listo para que un LLM lo invoque.**

Esto significa que cuando un usuario de un spa le escribe a un bot: "*Ana se inscribió ayer en el plan mensual*", el modelo de IA no tiene que inventar cómo estructurar eso. Revisa el Lexicon, ve qué función debe llamar (`inscribirse`), qué roles llenar y qué datos espera tu base de datos. Esta convergencia no es suerte; es la demostración de que la estructura de la lingüística formal y la inteligencia artificial moderna llegaron exactamente al mismo puerto.

En la práctica, exponer el lexicon a un LLM convierte una frase suelta en un hecho estructurado sin que nadie escriba un parser. El prompt es casi todo catálogo:

SISTEMA. Eres un extractor de hechos WQuestions. Dada una frase en español, devuelve SOLO un JSON con la situación y sus roles canónicos.

Catálogo (verbo → situación · roles obligatorios):
inscribirse → inscripcion · [agente, tema]
tomar → servicio_spa · [cliente, lugar_de]

Frase del usuario: "Ana se inscribió ayer en el plan mensual"

Y el modelo responde con la situación lista para asentar en el grafo:

```
{  
  "situation_type": "inscripcion",  
  "agente": "ana",  
  "tema": "plan_mensual",  
  "momento": "2026-06-06"  
}
```

Resolviendo el caos de la "Mano" (Polisemia)

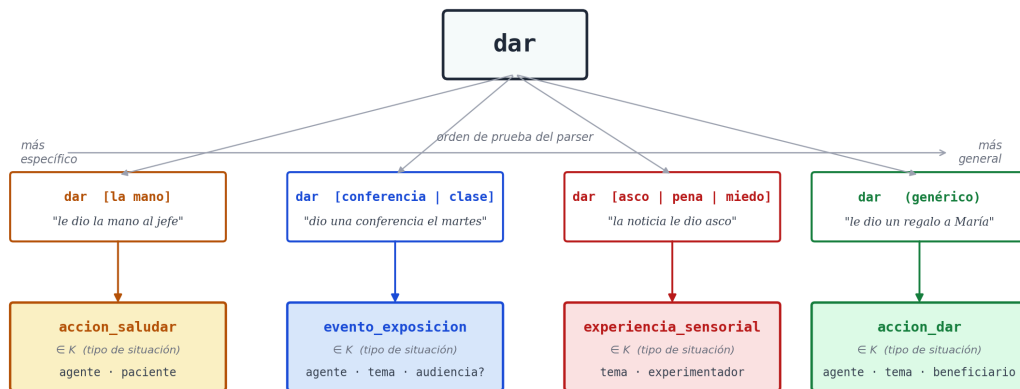
Volvamos al verbo `dar`. ¿Cómo lo trata el Lexicon para no confundirse? La respuesta es brutalmente directa: **crea una entrada distinta para cada significado**. El Lexicon no intenta que una sola regla mágica cubra todos los usos. En su lugar, hace una lista de patrones, cada uno apuntando a su propio evento:

```
dar  
  tipo_situacion: accion_dar  
  obligatorios: [agente, tema, beneficiario]  
  ejemplo:      "Pedro le dio un regalo a María"  
  
dar [la_mano]  
  tipo_situacion: accion_saludar  
  obligatorios: [agente, paciente]  
  ejemplo:      "Pedro le dio la mano a su jefe"  
  
dar [conferencia | clase | charla]  
  tipo_situacion: evento_exposicion  
  obligatorios: [agente, tema]  
  opcionales: [audiencia, lugar_de, momento, duracion]  
  ejemplo:      "Pedro dio una conferencia el martes"  
  
dar [las_horas]  
  tipo_situacion: accion_sonar  
  obligatorios: [agente, tema]  
  ejemplo:      "El reloj dio las tres"  
  
dar [asco | pena | miedo]  
  tipo_situacion: experiencia_sensorial  
  obligatorios: [tema, experimentador]  
  ejemplo:      "La noticia le dio asco"
```

El procedimiento de resolución es el mismo que usan los compiladores de código: el sistema intenta encajar la frase **desde el patrón más específico hacia el más general**. Si la oración contiene *dar la mano*, se activa la entrada de saludo. Si la oración dice *dar un regalo*, el sistema no encuentra un patrón específico para "regalo", así que aplica la regla genérica de `accion_dar`.

Resolución de polisemia: el verbo "dar"

Cuatro unidades léxicas distintas; el parser elige por patrón de complemento, del más específico al más general.



El parser intenta los patrones específicos primero ("dar la mano", "dar conferencia") y solo cae al patrón genérico "dar" si ninguno coincide.

Polisemia tratada como sobrecarga de funciones — declarativa, mecánica, extensible.

Resolución de polisemia: el verbo "dar" se desambigua por su patrón de complemento. Cuatro unidades léxicas distintas apuntan a cuatro tipos de situación distintos en K. El parser elige el patrón más específico que coincida.

La complejidad del lenguaje no desaparece, simplemente se traslada al lugar correcto: al diccionario, donde un modelador de datos puede registrar nuevos significados sin tener que alterar el núcleo de la base de datos.

Dialectos de dominio (La jerga corporativa)

Existe otro nivel de traducción en el Lexicon que es clave para los negocios. Hasta ahora vimos *alias* por rol ("vendedor" en lugar de agente). Pero hay palabras que aplican transversalmente a toda una empresa, sin importar qué verbo se esté usando.

Un sistema de salud siempre habla de "DNI", "paciente" y "diagnóstico". Un spa habla de "cliente", "plan mensual" y "promoción". Para que nuestro motor universal funcione en todos estos lugares sin obligar a la gente a cambiar su forma de hablar, el Lexicon permite instalar **dialectos de dominio**.

Es un pequeño archivo de traducción que se ve así:

```
dominio: spa_oasis
alias_de_dominio:
  cliente: agente
  sesion: servicio_spa
  plan_mensual: contrato_servicio
  sesion_gratuita: beneficio_fidelidad
  promo: aplicacion_de_promocion
  redimir: verbo_usar_beneficio
```

Con este dialecto cargado, un empleado del spa puede escribir: "El cliente redimió su sesión gratuita". El sistema, automáticamente, traduce "cliente" a agente, "redimió" al verbo usar_beneficio y "sesión gratuita" a la categoría beneficio_fidelidad. El sistema central sigue intacto, pero la clínica, el aeropuerto y el spa sienten que el software fue hecho a su medida.

A hombros de gigantes: Precedentes industriales (FrameNet, VerbNet, PropBank)

Es crucial entender que nosotros no nos estamos inventando estas estructuras desde cero. La lingüística computacional lleva treinta años construyendo inmensos catálogos con esta misma lógica, y cualquier implementación seria de nuestro Lexicon debe alimentarse de ellos. No tenemos que reinventar la rueda.

FrameNet [14] : Iniciado en la Universidad de Berkeley en los años noventa, este proyecto clasifica el idioma en *escenarios conceptuales* (frames). Su escenario de "Comercio" tiene roles predefinidos como **Buyer** (Comprador), **Seller** (Vendedor) y **Goods** (Mercancía), casi idéntico a nuestra entrada de **vender**. Sus más de 1.200 escenarios son una mina de oro para poblar nuestra caja **K** de forma profesional.

VerbNet [15] : Creado en la Universidad de Pensilvania, agrupa miles de verbos en inglés que comparten la misma estructura lógica (por ejemplo, verbos de transferencia o verbos de encuentro).

PropBank: Es un proyecto que prioriza la velocidad, etiquetando masivamente textos reales con roles genéricos (**Arg0** a **Arg5**). Es la base de datos favorita para entrenar modelos de Inteligencia Artificial modernos.

WQuestions no busca competir ni reemplazar a estas obras maestras. Al contrario, WQuestions actúa como una **capa de agregación**. Podemos extraer las signatures de FrameNet, alinearnos con VerbNet y usar los datos de PropBank para entrenar a nuestro propio sistema. El camino ya está pavimentado.

Un buen puñado de entradas del Lexicon (En acción)

Para que esta teoría pase a la realidad, es necesario mostrarte cómo se ve un catálogo abundante de entradas del Lexicon aplicadas a múltiples industrias.

Mientras lees esta inmensa lista, quiero que notes tres cosas fundamentales: 1. **Reciclamos los mismos roles siempre**. Observa cómo **agente**, **tema** y **beneficiario** se repiten en hospitales, bancos y escuelas. El catálogo base de 38 roles (D8) basta para todo. 2. **Los eventos "meteorológicos" y los dolores caben perfecto**. Verás verbos como *llover* o *doler* que funcionan a la perfección simplemente dejando en blanco la necesidad de un "agente". 3. **El poder de los alias**. Nota cómo las palabras en español cambian según la industria, pero el esqueleto de datos por debajo es siempre el mismo.

(Cada bloque contiene al final un ejemplo entre comillas que ilustra cómo el sistema leería la frase).

Comercio y e-commerce

```

verbo: comprar
tipo_situacion: accion_comprar
roles:
  agente: ["comprador", "el que compra", "cliente"]
  tema: ["producto", "artículo", "ítem", "mercancía"]
  beneficiario: ["vendedor", "tienda", "el que vende"]
  por_cuanto: ["precio", "monto", "valor"]
obligatorios: [agente, tema, beneficiario, por_cuanto]
opcionales: [unidad, momento, lugar_de, instrumento]
ejemplo: "Sofía compró tres camisetas en Zara por 89 dólares con tarjeta"

verbo: pagar
tipo_situacion: accion_pagar
roles:
  agente: ["pagador", "el que paga", "deudor"]
  beneficiario: ["receptor", "acreedor", "el que cobra"]
  monto: ["importe", "cantidad", "monto"]
  instrumento: ["medio de pago", "tarjeta", "transferencia"]
  tema: ["concepto", "factura", "deuda", "servicio"]
obligatorios: [agente, beneficiario, monto]
ejemplo: "Juan le pagó 250 dólares al plomero por la reparación"

verbo: facturar
tipo_situacion: accion_facturar
roles:
  agente: ["proveedor", "vendedor", "el que emite"]
  beneficiario: ["cliente", "el que recibe la factura"]
  tema: ["servicio prestado", "producto entregado"]
  monto: ["importe facturado", "total"]
obligatorios: [agente, beneficiario, tema, monto]
ejemplo: "El estudio jurídico le facturó 1.200 dólares a la constructora"

verbo: reclamar
tipo_situacion: accion_reclamar
roles:
  agente: ["reclamante", "el que reclama", "afectado"]
  paciente: ["empresa demandada", "destinatario del reclamo"]
  tema: ["motivo del reclamo", "defecto", "problema"]
  con_finalidad: ["lo que se pide", "compensación", "devolución"]
obligatorios: [agente, paciente, tema]
ejemplo: "Mariana reclamó al banco una transferencia mal procesada para que se la devolvieran"

verbo: devolver
tipo_situacion: accion_devolver
roles:
  agente: ["devolvedor", "el que devuelve", "cliente"]
  tema: ["producto devuelto", "ítem"]
  beneficiario: ["tienda", "vendedor"]
  causado_por: ["motivo de devolución", "defecto", "razón"]
obligatorios: [agente, tema, beneficiario]
ejemplo: "Pedro devolvió la cafetera defectuosa a la tienda online"

```

Banca y finanzas

```

verbo: transferir
tipo_situacion: accion_transferir
roles:
  agente: ["ordenante", "remitente"]
  beneficiario: ["destinatario", "el que recibe"]
  monto: ["importe", "cantidad"]
  origen: ["cuenta_origen", "de qué cuenta"]
  destino: ["cuenta_destino", "a qué cuenta"]
obligatorios: [agente, beneficiario, monto, origen, destino]
ejemplo: "Lucía transfirió 500 dólares de su cuenta corriente a la cuenta de su madre"

verbo: depositar
tipo_situacion: accion_depositar
roles:
  agente: ["depositante"]
  monto: ["importe", "cantidad depositada"]
  destino: ["cuenta", "destino"]
  instrumento: ["efectivo", "cheque", "transferencia recibida"]
obligatorios: [agente, monto, destino]
ejemplo: "El comerciante depositó 3.200 dólares en efectivo en la cuenta del negocio"

verbo: prestar
tipo_situacion: accion_prestar
roles:
  agente: ["prestamista", "banco"]
  beneficiario: ["prestatario", "el que recibe"]
  monto: ["capital", "importe del préstamo"]
  inicio: ["fecha de desembolso"]
  fin: ["fecha de vencimiento"]
  con_finalidad: ["destino del préstamo", "para qué"]
obligatorios: [agente, beneficiario, monto]
ejemplo: "El banco le prestó 50.000 dólares a Ramiro para la compra de su vivienda"

verbo: cobrar
tipo_situacion: accion_cobrar
roles:
  agente: ["acreedor", "el que cobra"]
  paciente: ["deudor", "el que paga"]
  monto: ["importe cobrado"]
  tema: ["concepto", "factura", "comisión"]
obligatorios: [agente, paciente, monto]
ejemplo: "El banco le cobró 18 dólares de comisión por mantenimiento de cuenta"

```

Salud

```

verbo: consultar
tipo_situacion: accion_consultar
roles:
  agente: ["médico", "doctora", "profesional"]
  paciente: ["paciente", "persona atendida"]
  motivo: ["motivo de consulta", "queja principal"]
  lugar_de: ["consultorio", "sala", "centro médico"]
obligatorios: [agente, paciente]
ejemplo: "La Dra. Vega consultó a Renata el martes por dolor lumbar"

verbo: prescribir
tipo_situacion: accion_prescribir
roles:
  agente: ["médico", "prescriptor"]
  paciente: ["paciente"]
  tema: ["medicamento", "principio activo"]
  cantidad: ["dosis", "miligramos"]
  duracion: ["por cuántos días"]
  con_finalidad: ["objetivo terapéutico"]
  justificado_por: ["diagnóstico", "protocolo"]
obligatorios: [agente, paciente, tema]
ejemplo: "El cardiólogo prescribió enalapril 10mg diarios para controlar la hipertensión"

verbo: diagnosticar
tipo_situacion: accion_diagnosticar
roles:
  agente: ["médico", "diagnosticador"]
  paciente: ["paciente"]
  tema: ["diagnóstico", "enfermedad", "condición"]
  causado_por: ["evidencia", "síntomas", "estudio"]
obligatorios: [agente, paciente, tema]
ejemplo: "El pediatra diagnosticó otitis media aguda basándose en la otoscopia"

verbo: operar
tipo_situacion: accion_operar
roles:
  agente: ["cirujano", "equipo quirúrgico"]
  paciente: ["paciente"]
  tema: ["zona operada", "órgano"]
  instrumento: ["instrumental", "equipo"]
  con_finalidad: ["objetivo de la intervención"]
  duracion: ["duración de la cirugía"]
obligatorios: [agente, paciente, tema]
ejemplo: "El Dr. Larrea operó la rodilla de Marcelo durante tres horas para reparar el menisco"

verbo: vacunar
tipo_situacion: accion_vacunar
roles:
  agente: ["enfermera", "personal sanitario"]
  paciente: ["paciente", "vacunado"]
  tema: ["vacuna", "antígeno"]
  con_finalidad: ["enfermedad prevenida"]
obligatorios: [agente, paciente, tema]
ejemplo: "La enfermera vacunó a 47 niños con la dosis triple viral el sábado"

```

Educación

```
verbo: inscribirse
tipo_situacion: accion_inscribirse
roles:
  agente: ["estudiante", "inscripto", "postulante"]
  tema: ["curso", "programa", "carrera"]
  momento: ["fecha de inscripción"]
  con_finalidad: ["objetivo académico"]
obligatorios: [agente, tema]
ejemplo: "Diana se inscribió en la maestría en ciencia de datos para mejorar su perfil"

verbo: examinar
tipo_situacion: accion_examinar
roles:
  agente: ["examinador", "profesor"]
  paciente: ["examinado", "estudiante"]
  tema: ["contenido evaluado", "materia"]
  momento: ["fecha del examen"]
  calificacion: ["nota", "resultado"]
obligatorios: [agente, paciente, tema]
ejemplo: "El tribunal examinó a Andrés sobre cálculo integral y le puso un 8.5"

verbo: graduarse
tipo_situacion: accion_graduarse
roles:
  agente: ["graduado", "el que se gradúa"]
  tema: ["título obtenido", "carrera"]
  lugar_de: ["institución", "universidad"]
  momento: ["fecha de graduación"]
obligatorios: [agente, tema]
ejemplo: "Camila se graduó de ingeniera civil en la Universidad Nacional en 2026"

verbo: enseñar
tipo_situacion: accion_enseñar
roles:
  agente: ["docente", "profesor", "instructor"]
  beneficiario: ["alumno", "estudiante", "audiencia"]
  tema: ["materia", "contenido", "concepto"]
  instrumento: ["método", "material didáctico"]
obligatorios: [agente, beneficiario, tema]
ejemplo: "El profesor Méndez enseñó programación a 25 alumnos durante tres meses"
```

Legal y gobierno

```

verbo: firmar
tipo_situacion: accion_firmar
roles:
  agente: ["firmante", "el que firma"]
  tema: ["documento", "contrato", "acuerdo"]
  momento: ["fecha de firma"]
  lugar_de: ["lugar de firma"]
obligatorios: [agente, tema]
ejemplo: "Eduardo firmó el contrato de alquiler el lunes en la escribanía"

verbo: denunciar
tipo_situacion: accion_denunciar
roles:
  agente: ["denunciante", "el que denuncia"]
  paciente: ["denunciado", "acusado"]
  tema: ["hecho denunciado", "delito", "irregularidad"]
  beneficiario: ["autoridad receptora", "fiscalía", "policía"]
obligatorios: [agente, paciente, tema]
ejemplo: "La asamblea denunció al constructor por daño ambiental ante la fiscalía"

verbo: multar
tipo_situacion: accion_multar
roles:
  agente: ["autoridad", "oficial", "ente regulador"]
  paciente: ["multado", "infractor"]
  monto: ["importe de la multa"]
  causado_por: ["infracción cometida"]
  justificado_por: ["norma aplicada", "artículo legal"]
obligatorios: [agente, paciente, monto, causado_por, justificado_por]
ejemplo: "El SUNAT multó a la empresa con 8.500 dólares por declaración tardía"

verbo: votar
tipo_situacion: accion_votar
roles:
  agente: ["votante", "elector"]
  tema: ["candidato", "opción", "moción"]
  lugar_de: ["mesa de votación", "centro electoral"]
  momento: ["fecha de la elección"]
obligatorios: [agente, tema]
ejemplo: "Mariana votó por la lista 14 en la mesa 087 el domingo en la mañana"

```

Trabajo y empleo

```

verbo: contratar
tipo_situacion: accion_contratar
roles:
  agente: ["empleador", "empresa contratante"]
  beneficiario: ["empleado", "contratado"]
  tema: ["puesto", "rol", "función"]
  inicio: ["fecha de inicio del contrato"]
  monto: ["salario", "remuneración"]
obligatorios: [agente, beneficiario, tema]
ejemplo: "La empresa contrató a Federico como ingeniero con un salario de 4.500 dólares"

verbo: despedir
tipo_situacion: accion_despedir
roles:
  agente: ["empleador"]
  paciente: ["despedido", "ex empleado"]
  causado_por: ["causa", "motivo del despido"]
  justificado_por: ["cláusula contractual", "ley laboral"]
  momento: ["fecha de despido"]
obligatorios: [agente, paciente]
ejemplo: "La empresa despidió a Roberto en marzo por incumplimiento del reglamento interno"

```

Tecnología y operaciones

```
verbo: desplegar
tipo_situacion: accion_desplegar
roles:
  agente: ["devops", "el que despliega", "pipeline"]
  tema: ["aplicación", "versión", "release"]
  lugar_destino: ["entorno", "servidor", "producción"]
  momento: ["timestamp del deploy"]
obligatorios: [agente, tema, lugar_destino]
notas: "El agente puede ser un humano o un algoritmo automatizado (Regla D5)"
ejemplo: "El pipeline desplegó la versión 3.4.1 en producción el martes a las 9:14"

verbo: fallar
tipo_situacion: evento_fallo
roles:
  paciente: ["sistema afectado", "componente que falló"]
  causado_por: ["causa raíz", "trigger"]
  momento: ["timestamp del incidente"]
  duracion: ["downtime"]
obligatorios: [paciente]
notas: "Verbo sin agente. Los sistemas caen sin intervención intencional."
ejemplo: "El servicio de pagos falló durante 47 minutos a causa de un overflow"

verbo: invocar
tipo_situacion: accion_invocar_funcion
roles:
  agente: ["llamador", "cliente API", "LLM"]
  tema: ["función invocada", "endpoint"]
  instrumento: ["protocolo", "HTTP"]
  con_finalidad: ["objetivo de la invocación"]
obligatorios: [agente, tema]
ejemplo: "Claude invocó la función registrar_venta con los parámetros extraídos"
```

Estados mentales (experimentador)

```
verbo: temer
tipo_situacion: experiencia_emocional
roles:
  experimentador: ["el que teme", "persona asustada"]
  tema: ["lo temido", "objeto del miedo"]
  inicio: ["desde cuándo"]
obligatorios: [experimentador, tema]
notas: "No hay agente. La persona no actúa, sufre un estado interno."
ejemplo: "Andrés teme a las alturas desde su accidente de niñez"

verbo: gustar
tipo_situacion: experiencia_preferencia
roles:
  tema: ["lo que gusta", "objeto del gusto"]
  experimentador: ["a quién le gusta"]
obligatorios: [tema, experimentador]
notas: "Verbo invertido en español. El sujeto de la frase es la cosa que gusta."
ejemplo: "A Sofía le gusta el jazz brasileño"

verbo: doler
tipo_situacion: experiencia_dolor
roles:
  tema: ["lo que duele", "zona dolorida"]
  experimentador: ["a quién le duele"]
  intensidad: ["nivel del dolor"]
obligatorios: [tema, experimentador]
ejemplo: "A Marta le duele la rodilla derecha desde el lunes con una intensidad 7 sobre 10"
```

Sin agente: Meteorología y eventos espontáneos


```

verbo: llover
tipo_situacion: evento_meteorologico
roles:
  lugar_de: ["dónde llueve", "región", "ciudad"]
  momento: ["cuándo"]
  intensidad: ["leve", "moderada", "intensa", "torrencial"]
  duracion: ["por cuánto tiempo"]
obligatorios: []
notas: "Verbo totalmente impersonal. No tiene roles obligatorios."
ejemplo: "Llovió torrencialmente en Lima durante seis horas el sábado"

verbo: ocurrir
tipo_situacion: evento_generico
roles:
  tema: ["lo que ocurre", "evento"]
  lugar_de: ["dónde"]
  momento: ["cuándo"]
  causado_por: ["disparador del evento"]
obligatorios: [tema]
ejemplo: "Ocurrió un sismo de magnitud 5.4 en la zona costera a causa del rozamiento de placas"

verbo: vencer
tipo_situacion: evento_vencimiento
roles:
  tema: ["lo que vence", "contrato", "plazo"]
  momento: ["fecha de vencimiento"]
obligatorios: [tema, momento]
notas: "Las cosas vencen por el simple paso del tiempo, sin agente humano."
ejemplo: "La licencia comercial vence el 31 de diciembre de 2026"

```

Verbos con polisemia clásica

```

verbo: correr
  tipo_situacion: accion_correr_fisica
  roles:
    agente: ["corredor", "el que corre"]
    distancia: ["distancia recorrida"]
    duracion: ["tiempo invertido"]
    lugar_de: ["dónde"]
  obligatorios: [agente]
  ejemplo: "Daniela corrió 10 km en el parque en 52 minutos"

verbo: correr [riesgo]
  tipo_situacion: estado_de_riesgo
  roles:
    experimentador: ["el que corre el riesgo"]
    tema: ["lo que se arriesga", "el riesgo"]
  obligatorios: [experimentador, tema]
  notas: "Verbo de soporte: correr = estar expuesto a."
  ejemplo: "La empresa corre el riesgo de perder el contrato más grande del año"

verbo: correr [el rumor | la noticia]
  tipo_situacion: difusion_informacion
  roles:
    tema: ["la información que circula"]
    lugar_de: ["dónde se difunde"]
  obligatorios: [tema]
  ejemplo: "Corre el rumor de que la oficina central va a mudarse"

verbo: cerrar
  tipo_situacion: accion_cerrar_fisica
  roles:
    agente: ["el que cierra"]
    tema: ["lo cerrado", "puerta", "ventana"]
  obligatorios: [agente, tema]
  ejemplo: "El conserje cerró la puerta principal"

verbo: cerrar [trato | negocio | acuerdo]
  tipo_situacion: accion_acordar
  roles:
    agente: ["las partes que acuerdan"]
    tema: ["el acuerdo cerrado"]
    beneficiario: ["contraparte"]
  obligatorios: [agente, tema]
  ejemplo: "Las dos empresas cerraron un trato por 2.5 millones de dólares"

verbo: cerrar [el año | el mes | el ejercicio]
  tipo_situacion: accion_finalizar_periodo
  roles:
    agente: ["organización"]
    tema: ["período contable"]
    monto: ["resultado financiero"]
  obligatorios: [agente, tema]
  ejemplo: "La empresa cerró el ejercicio con una utilidad de 340.000 dólares"

```

Cierre: Lo que se ve mirando el catálogo entero

Si miras estas casi cuarenta entradas en perspectiva, aparecen revelaciones técnicas que valen oro:

- **Se usan los mismos cables siempre:** Observa cómo casi todos los verbos, sin importar si hablan de despidos, graduaciones o tormentas, utilizan los mismos 8 a 10 roles del catálogo base de D8 (`agente` , `tema` , `lugar_de` , `momento`).
- **Los *alias* son el verdadero poder de adaptación:** Lo que diferencia al módulo de Salud del módulo Bancario no son los códigos internos, sino las palabras naturales. "*Médico*" y "*Acreeedor*" ambos se mapean al mismo `agente` ; "*Factura*" y "*Diagnóstico*" van ambos al mismo `tema` . La base de datos es ignorante sobre el negocio, el Lexicon le hace todo el trabajo de traducción.

- **Lo raro también es normal:** Nota cómo los verbos raros que no tienen sujeto (como *llover*) o que son internos (como *doler* o *temer*) encajan a la perfección en esta arquitectura sin forzar nada.

Un sistema robusto a nivel empresarial puede llegar a tener un archivo de Lexicon con dos mil o cinco mil entradas. Las que acabas de ver son el cimiento.

Lo que tu empresa gana implementando el Lexicon

Para cerrar, enlistemos las diferencias brutales entre usar este modelo y no hacerlo:

- 1. Entiende el idioma real de tu gente.** Sin el Lexicon, tendrías que enseñarle a cada empleado de cada departamento cómo usar los campos robóticos del sistema. Con el Lexicon, un entrenador de gimnasio escribe "*el usuario faltó a la clase*", y la base de datos registra perfectamente el evento usando la jerga de ese gimnasio.
- 2. Es la puerta de entrada perfecta para la Inteligencia Artificial.** Como vimos, el Lexicon es un listado en formato JSON (Function Calling) esperando a ser leído. Una IA conectada a tu Lexicon puede empezar a redactar, archivar y auditar datos en tu servidor mañana mismo, sin necesidad de reprogramar tu sistema.
- 3. Desambigua el caos del idioma antes de ensuciar los discos duros.** Los clásicos problemas de la polisemia (como el verbo *dar*) se quedan resueltos a nivel de diccionario. La base de datos siempre recibe datos purificados y etiquetados de antemano con códigos exactos en `K`.
- 4. Permite a la empresa escalar infinitamente.** Si mañana tu compañía abre una rama en logística marítima, tus ingenieros no tienen que destruir la base de datos SQL para crear 40 tablas nuevas sobre barcos. Simplemente añaden un nuevo bloque de verbos al archivo de texto del Lexicon, y el motor central hereda ese conocimiento de inmediato.

Afirmar que el Lexicon es *la pieza más decisiva de toda la arquitectura* no es una exageración técnica. La genialidad matemática de los siete ejes es el corazón palpitante, sí; pero **la usabilidad y la adopción real** del sistema dependen entera y exclusivamente de este traductor maestro.

Capítulo 13 — Cuando el lenguaje aprieta: nominalizaciones, modales e idiomas

Tres frases que harían explotar a un robot

Tomemos tres oraciones que cualquier hispanohablante entiende sin esfuerzo, pero que harían cortocircuito en un sistema diseñado únicamente con las reglas mecánicas que vimos en el capítulo 11:

- La llegada tardía del avión causó la cancelación de la conexión.
- El cliente quería contratar el plan mensual pero no podía pagarlo todavía.
- La doctora le tomó el pelo al residente nuevo durante la guardia.

Si un robot ingenuo intentara procesar esto buscando "verbos de acción", el desastre sería total. En la primera frase, el robot buscaría un verbo de acción y se encontraría con "causó", pero la verdadera carne de la frase (la llegada y la cancelación) están escondidas bajo disfraces de sustantivos. En la segunda frase, el sistema crearía un evento oficial llamado "querer" y otro evento llamado "poder", que son cosas que no existen en el mundo físico, separándolos del evento real que es "contratar". En la tercera frase, el robot crearía un evento de "tomar" donde el objeto físico arrebatado es "el pelo" de un médico. Ridículo. Todos sabemos que *tomar el pelo* significa **bromear o engañar amistosamente**, y no tiene nada que ver con robar cabello.

Estos tres escenarios —las **nominalizaciones** (verbos disfrazados de sustantivos), los **modales** (verbos de intención) y los **idiomas** (frases hechas)— son las pruebas de fuego para cualquier arquitectura de datos. En este capítulo no venimos a presumir de que nuestro modelo tiene una varita mágica para resolverlos; venimos a mostrarte con qué reglas exactas los absorbemos y en qué rincones oscuros todavía nos cuesta trabajo.

Nominalización: El verbo disfrazado de sustantivo

El español tiene una habilidad fascinante para convertir acciones en cosas. El verbo *llegar* se vuelve *la llegada*; *vender* se vuelve *la venta*; *consultar* se vuelve *la consulta*. A esta mutación gramatical se le llama **nominalización**.

Para la mente humana, decir "*el avión llegó*" o "*la llegada del avión*" es exactamente lo mismo. Y para nuestra base de datos, también debe serlo.

La regla de arquitectura aquí es vital: **una nominalización no es un evento nuevo; es exactamente la misma situación reificada, pero con otro empaque gramatical**. La frase "*la llegada*" no necesita que inventemos un eje nuevo, solo necesita que el Lexicon sea lo suficientemente inteligente para saber que "llegada" es un alias que dispara el mismo evento que "llegar".

Desarmemos la primera oración problemática:

La llegada tardía del avión causó la cancelación de la conexión.

```
(llegada_017, instancia_de, accion_llegar)
(llegada_017, agente, avion_LP2226)
(llegada_017, estatus_factual, real)
(llegada_017, calificacion, tardia)
(llegada_017, momento, 2026-05-15T11:47Z)

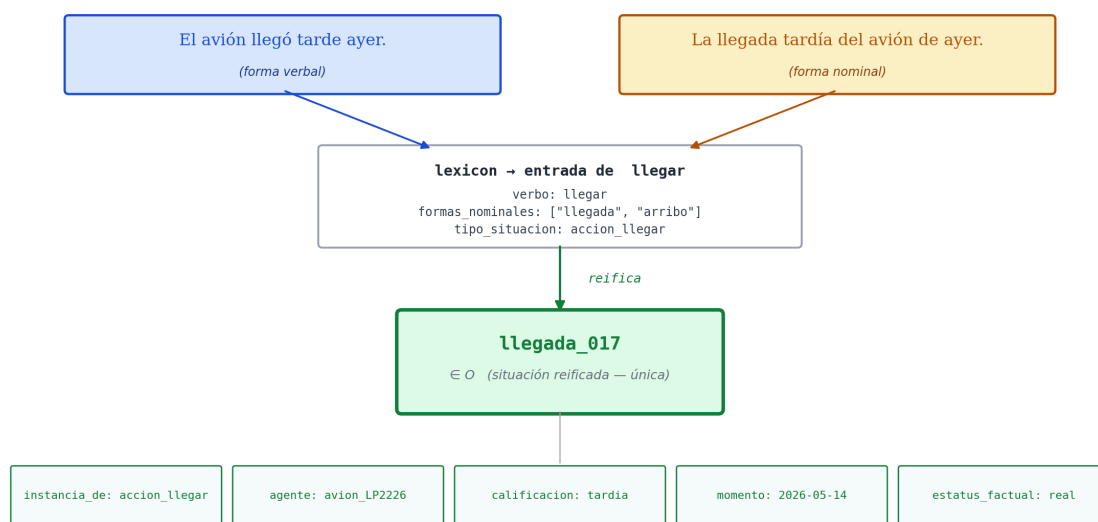
(cancelacion_004, instancia_de, accion_cancelar)
(cancelacion_004, tema, conexion_lima_arica)
(cancelacion_004, estatus_factual, real)

(cancelacion_004, causado_por, llegada_017) ← El conector clave (Eje M)
```

Fíjate en lo que acabamos de lograr. Dos sustantivos ("llegada" y "cancelación") produjeron dos situaciones oficiales en la caja 0. Y el verbo "causó", que un robot ingenuo habría convertido en un evento tonto, nosotros lo convertimos en el cable `causado_por` que vimos en el capítulo 10. El resultado es un código limpio y perfecto.

Nominalización ≡ verbo: misma situación, distinta superficie

Forma verbal y forma nominal disparan la misma entrada del lexicon y producen los mismos hechos.



Mismos hechos atómicos, sin importar si el texto fuente usa el verbo o el sustantivo derivado.

Una nominalización y su forma verbal equivalente producen exactamente la misma situación reificada en el modelo. El lexicon hace que ambos disparadores —el verbo "llegar" y el sustantivo "llegada"— activen la misma entrada.

¿Cómo logramos que el sistema haga esto solo? Le decimos al Lexicon que acepte formas nominales:

```
verbo: llegar
formas_nominales: ["llegada", "arribo"] ← El truco está aquí
tipo_situacion: accion_llegar
obligatorios: [agente]
```

La ganancia comercial de esto es inmensa: a la base de datos le da exactamente igual si un médico escribe "Diagnosticué al paciente con asma" o "El diagnóstico de asma del paciente". La maquinaria extrae los datos y genera los mismos cables, ignorando la poesía del redactor.

Modales: Decorando eventos que no han ocurrido

Los verbos modales (*querer, deber, poder, soler, parecer*) son una trampa mortal para las bases de datos. Parecen verbos principales, pero en realidad son **modificadores** de un segundo verbo.

Si un cliente dice "Quiero viajar", no está realizando un evento de "querer" por un lado y un evento de "viajar" por otro. Lo que hay es **una sola situación (viajar)** que tiene pegada una etiqueta de intención, avisando que todavía no ocurre.

Si convirtiéramos cada "querer" y "poder" en un evento oficial en la base de datos, multiplicaríamos el tamaño de los discos duros por cuatro, llenando el sistema de basura abstracta. Para evitarlo, nuestro modelo trata a los modales simplemente como **decoraciones** (propiedades) sobre el evento principal.

Observemos cómo se desarma la segunda oración:

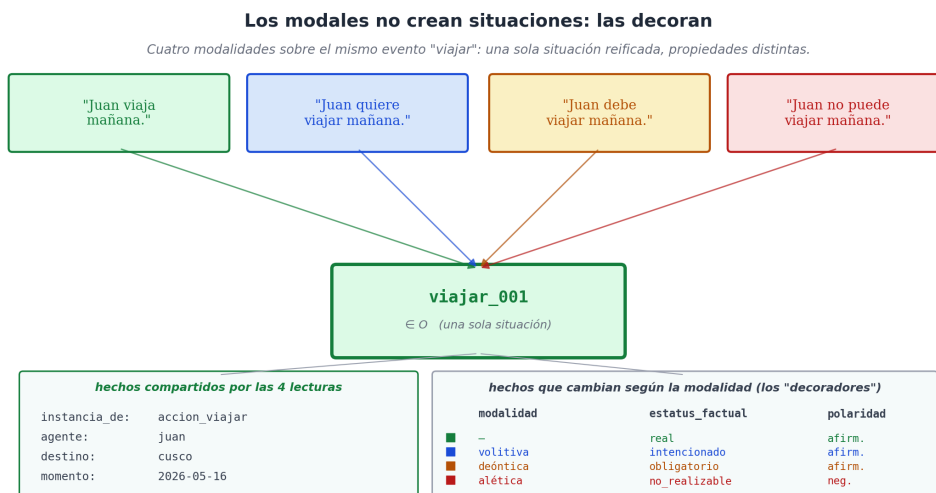
El cliente quería contratar el plan mensual pero no podía pagarlo todavía.

```
(contratar_017, instancia_de, accion_contratar)
(contratar_017, agente, cliente_003)
(contratar_017, tema, plan_mensual_oasis)
(contratar_017, modalidad, volitiva) ← Es un deseo
(contratar_017, estatus_factual, intencionado) ← Aún no pasa

(pagar_017, instancia_de, accion_pagar)
(pagar_017, agente, cliente_003)
(pagar_017, tema, plan_mensual_oasis)
(pagar_017, modalidad, alética) ← Se trata de una capacidad
(pagar_017, polaridad, negativa) ← No tiene la capacidad
(pagar_017, estatus_factual, no_realizable)

(pagar_017, contrasta_con, contratar_017) ← Conector para el "pero"
```

Generamos dos situaciones (contratar y pagar), no cuatro. El "querer" se guardó como modalidad: volitiva, y el "no poder" como polaridad: negativa. Ahorramos espacio, evitamos crear nodos fantasma y mantuvimos la regla de oro: **una situación en el mundo real equivale a una situación en el sistema.**



Querer / deber / poder no son situaciones del mundo; son cualidades del registro de viajar_001.

Una situación, varios estados modales. El grafo no se infla con verbos auxiliares reificados.

Los modales no crean nuevas situaciones: decoran una situación existente con propiedades de modalidad. "Quiere viajar" y "viaja" describen la misma situación con distinto modo factual.

(Nota técnica: Si alguien dice "Pedro quiere a María", ahí el verbo querer significa "amar", no es un modal. El Lexicon es lo suficientemente listo para distinguir que "querer + verbo" es un modal, y "querer + persona" es un evento emocional).

Expresiones Idiomáticas: Cuando las palabras mienten

Llegamos a la tercera oración y al problema más espinoso. Frases como *tomar el pelo* (bromear), *dar a luz* (parir), o *ponerse las pilas* (esforzarse) no pueden separarse palabra por palabra. El lenguaje humano está plagado de estas trampas.

La solución arquitectónica ya la planteamos en el Capítulo 12: **en nuestro Lexicon, la unidad de traducción no es la palabra suelta, es el patrón completo**. Las frases hechas se registran en el diccionario como reglas fijas que apuntan a sus verdaderos significados:

```
tomar [el_pelo a Q]
  tipo_situacion: accion_bromear
  obligatorios:  [agente, paciente]
  ejemplo:      "la doctora le tomó el pelo al residente"
  notas:        Frase idiomática; ignorar relación con el cabello humano.

dar [a_luz a Q]
  tipo_situacion: accion_parir
  obligatorios:  [agente, paciente]
  ejemplo:      "dio a luz a una niña sana"
```

El sistema lee de lo más específico a lo más general. Si ve "*tomar el pelo*", ejecuta la regla de bromear. Si ve "*tomar la orden*", ejecuta la regla general de recibir.

El verdadero reto aquí no es técnico, es de escala. El idioma español tiene miles de estas frases. Llenar el Lexicon con todas ellas es un trabajo titánico de años. La buena noticia es que **los modelos de Inteligencia Artificial modernos ya conocen estas frases**. El LLM hace el trabajo pesado de entender la ironía, y nuestro modelo le da la estructura perfecta para guardarla.

Honradez intelectual: Donde el modelo todavía sufre

Sería deshonesto terminar esta sección fingiendo que nuestro modelo resuelve el 100% de los problemas de la lingüística mundial. Hay terrenos pantanosos donde nuestra arquitectura avanza con dificultad o simplemente cruje. Y un buen ingeniero debe conocer los límites de su propio sistema:

1. Negaciones complejas: Decir "*Juan no cree que María vino*" no es lo mismo que "*Juan cree que María no vino*". El modelo tiene que hacer acrobacias para saber a qué evento exacto le debe pegar la etiqueta de polaridad: negativa. A veces los analizadores automáticos se equivocan. **2. Cuantificación abstracta:** Frases como "*La mayoría de los pacientes mejoran*" no apuntan a una persona concreta (un agente en Q). Obliga al sistema a crear eventos sobre "clases enteras de personas", lo cual vuelve la matemática mucho más pesada. **3. El tono exacto del tiempo:** El español distingue magistralmente entre *empezó a hablar*, *estaba hablando*, *terminó de hablar* y *acababa de hablar*. El modelo captura el núcleo temporal, pero para atrapar esas micro-diferencias se ve obligado a reificar "fases del evento", lo cual infla el sistema. **4. Lo que no se dice (Presuposiciones):** Si digo "*Juan dejó de fumar*", el lenguaje da por hecho que Juan fumaba en el pasado. El modelo guardará el "dejar de fumar" de hoy, pero no es capaz de deducir mágicamente el hábito del pasado y guardarlo. Dependemos de que una IA superior haga esa deducción y nos la envíe. **5. Sarcasmo e indirectas:** "*¿Puedes pasarme la sal?*" gramaticalmente es una pregunta sobre la capacidad física de una persona. Pragmáticamente es una orden. El modelo es literal; la comprensión del sarcasmo depende del Lexicon o de la IA conectada a él.

Estos problemas no son defectos de nuestro software; son los misterios sin resolver de las ciencias cognitivas en las últimas cinco décadas. Lo importante es que cuando el modelo choca con estos muros, **no se rompe ni borra la base de datos**; simplemente guarda una representación parcial y segura, esperando a que un humano o una IA futura lo refine.

Cierre de la Parte IV: El pacto entre el Grafo y la Inteligencia Artificial

Quiero cerrar esta parte teórica con una reflexión vital para la era moderna. Hoy en día, herramientas como ChatGPT o Claude pueden procesar libros enteros en segundos. Muchos ingenieros de empresas piensan: *"¿Para qué necesito este complejo modelo de WQuestions? Simplemente tiraré los 50.000 PDFs de mi empresa en la IA y que ella busque las respuestas sola"*.

Sí, eso funciona. Pero es un desastre financiero y técnico. Primero, porque gastas un dineral en "tokens" (poder de cómputo) haciéndole leer texto basura mil veces. Segundo, porque el texto humano es **terriblemente ambiguo**. Una historia clínica de cinco páginas en texto plano está llena de "él", "ella", "eso", y la IA tiene que adivinar a quién se refiere.

En cambio, si procesas esa historia clínica usando el Lexicon y la conviertes en **un grafo de hechos atómicos (WQuestions)**, reduces esas cinco páginas a unas cuantas líneas de código matemático y estructurado.

El Grafo y la Inteligencia Artificial no compiten; tienen una relación **simbiótica**: * La IA es la mente brillante y creativa: entiende el sarcasmo, maneja el contexto y habla bonito. * El modelo WQuestions es el disco duro de titanio: no olvida, no confunde a Juan con Pedro, audita reglas y sabe exactamente qué cosas son intenciones y cuáles son hechos reales.

Cuando pones a la IA como el recepcionista que charla con el usuario y al Grafo como el archivo de acero detrás de él, obtienes el sistema corporativo perfecto.

Y con esto terminamos la teoría pura. La **Parte V** deja la academia atrás y aplica el modelo de extremo a extremo en industrias reales y concretas.

Capítulo 14 — Modelando un negocio completo: El caso de un Spa

La Parte V empieza con código real

Hasta este punto, el libro ha sido un viaje conceptual. Te he explicado cada decisión de diseño utilizando ejemplos pequeños y aislados (*Marta vende un libro*, *Ana ingresa a la cámara de vapor*, *Carlos cambia de ciudad*). Pero, siendo honestos, ninguna arquitectura de software puede llamarse "universal" si no es capaz de sostener un negocio entero de principio a fin.

La Parte V cambia las reglas del juego. A partir de aquí, cada capítulo tomará una empresa completa y te mostrará cómo nuestro modelo absorbe toda su operación diaria. Y lo más importante: **lo haremos con código real que tú puedes ejecutar**.

El código base que acompaña a este libro (escrito en Python) no es un juguete académico. Es un prototipo honesto de unas 2.250 líneas que demuestra que todo lo que hemos prometido es verdad. Los siete ejes, los hechos atómicos, las situaciones reificadas, el catálogo de roles, el Lexicon y los cables del "por qué" existen en ese código y funcionan.

Para abrir esta sección, elegí un negocio que todos entendemos al instante: un **Spa comercial de barrio**. Un negocio pequeño pero engañosamente complejo. El Spa tiene clientes, sesiones, habitaciones calientes, horarios, precios, planes mensuales, promociones cruzadas, tarjetas de fidelidad, venta de jugos y recomendaciones de salud.

En un solo negocio vamos a utilizar casi todas las herramientas que construimos en las Partes II y III. Veamos cómo se arma.

El Spa "Oasis" en una página

El negocio funciona así: El Spa Oasis tiene una sede principal con dos cámaras de vapor y una cámara seca (todas operando a 60 grados centígrados). Los clientes pagan 20 dólares por sesión. El protocolo de salud recomienda estar veinte minutos en cada cámara y finalizar con una ducha de agua helada, lo que produce una relajación profunda.

Además, el negocio tiene reglas comerciales: **Fidelidad**: Si vienes siete veces, la octava sesión es gratis. **Planes mensuales**: Por una cuota fija, tienes derecho a dos sesiones por semana. **Inventario**: El local también vende jugos y ensaladas de frutas.

Usaremos a tres clientes frecuentes (Ana, Beto y Carlos) para poner a prueba nuestro modelo de datos.

Paso 1: Mapear el negocio en los Seis Ejes

El primer trabajo de cualquier arquitecto de datos es acomodar las entidades del negocio en nuestras famosas "cajas". Si te equivocas aquí, el sistema colapsa después. Así se mapea el Spa Oasis:

Q (Quién - Agentes): Los clientes (Ana, Beto, Carlos), la recepcionista, la masajista. Ojo aquí: la empresa "Spa Oasis" como entidad legal también vive en esta caja, porque es un agente que cobra dinero. **O (Qué - Eventos y Objetos)**: Las sesiones individuales (`sesion_ana_01`), los pagos con tarjeta, los contratos del plan mensual y las reglas de fidelidad. Todo eso son "cosas que pasan" o "reglas oficiales" y se reifican aquí. **L (Dónde - Lugares)**: El local, la cámara de vapor 1, la cámara seca y las duchas. (*Dato técnico: las cámaras viven en L porque son lugares donde ocurre la sesión, pero también viven en O porque son máquinas que requieren mantenimiento y tienen una*

temperatura). **T (Cuándo - El Tiempo):** Fechas exactas de ingreso, rangos de vigencia de las promociones y tiempos de estadía en la ducha. **N (Cuánto - Magnitudes):** Los 20 dólares, los 60 grados centígrados de la cámara y el contador de 7 visitas de la tarjeta de fidelidad. **K (Clase - El Diccionario Abstracto):** Aquí guardamos las etiquetas del negocio: `servicio_spa`, `servicio_masaje`, estados como `finalizada` o `cancelada`, y las unidades de medida (Dólares, Grados Celsius, Minutos).

El dominio spa mapeado a los 6 ejes de valor

Cada eje aloja entidades de naturaleza homogénea. Los predicados del eje cómo (M) conectan unas con otras.

Q <i>(5 ejemplos)</i>	Ana	Beto	Carlos	Anita (recep.)	Spa Oasis		
O <i>(6 ejemplos)</i>	sesion ana 07	pago 001	plan carlos 001	promo masaje	reco ducha fría	residencia 001	
L <i>(5 ejemplos)</i>	spa oasis central	cámara vapor_1	cámara vapor_2	cámara seca	ducha 03		
T <i>(4 ejemplos)</i>	2026-04-22 18:00	semana 2026 w17	mes 2026 04	inicio plan: 2026-04-01			
N <i>(5 ejemplos)</i>	20 USD (sesión)	60 °C (cámara)	20 min (recomendación)	7 (visitas - 1 gratis)	2 sesiones/semana (plan)		
K <i>(7 ejemplos)</i>	servicio spa	finalizada	real	intencionado	volitiva	Currency:USD	Temperature:Cel

Un dominio comercial pequeño cruza los 6 ejes de valor sin esfuerzo. Los predicados (eje cómo, M) son el cableado.

Mapa del dominio spa sobre los seis ejes de valor: clientes y personal en Q, sesiones y planes en O, cámaras y duchas en L, momentos en T, precios y temperaturas en N, tipos y estados en K. Los conectores del eje cómo (M) no se muestran — son los predicados que llenan los hechos atómicos.

La Sesión: El "Nudo" central del negocio

Si tuviéramos que elegir el evento más importante de toda la base de datos de esta empresa, sería **La Sesión**. Todo lo demás orbita a su alrededor: quién la tomó, dónde fue, a qué hora, cuánto costó y si la pagó en efectivo o usó su tarjeta de fidelidad.

En nuestro modelo, la sesión número siete de Ana se guarda como una lista limpia de hechos atómicos (predicados del eje cómo):

```
(sesion_ana_07) ∈ O
  instancia_de      : servicio_spa
  cliente           : ana
  lugar_de          : camara_vapor_1
  inicio            : 2026-04-22T18:00Z
  fin               : 2026-04-22T18:40Z
  estatus_factual   : finalizada
```

En el código Python real, esto se construye y se guarda en la base de datos con una orden pequeñita de cinco líneas de código:

```

sit = ingest_situation(u, lex, "tomar",
    roles={
        "cliente": ana,
        "lugar_de": camara_vapor_1,
        "inicio": t_inicio,
        "fin": t_fin,
    },
    complements=["sesion"],
    extra={"estatus_factual": finalizada},
    sit_id="sesion_ana_07",
)

```

La función `ingest_situation` hace el trabajo sucio que vimos en el Capítulo 11: consulta el Lexicon, desambigua el verbo "tomar", crea el evento, le enchufa los cables y **valida que no haya errores**. Si un programador despistado intenta poner a "Ana" en el campo de `lugar_de`, el sistema aborta la operación al instante y te grita un error de seguridad (`SignatureError`).

El Lexicon del Spa: Enseñándole a la máquina a hablar

Para que el personal del Spa no tenga que aprender jerga informática, configuramos el Lexicon con las palabras exactas de su negocio.

Aquí resolvemos el clásico problema de la polisemia (el doble sentido) del verbo "tomar":

```

lex.register(LexiconEntry(
    verb="tomar",
    situation_type="servicio_spa",
    obligatory=["cliente", "lugar_de"],
    pattern=("sesion"),
))

lex.register(LexiconEntry(
    verb="tomar",
    situation_type="accion_bromear",
    obligatory=["agente", "paciente"],
    pattern=("el_pelo"),
))

```

Si dicen "tomar una sesión", activa esta regla.

Si dicen "tomar el pelo", activa la regla de bromear.

Además, le instalamos el "dialecto corporativo" que vimos en el Capítulo 12, para que la máquina entienda que cuando un empleado dice "cliente", se refiere al agente principal:

```

lex.register_domain_dialect("spa_oasis", {
    "cliente": "agente",
    "sesion": "servicio_spa",
    "plan_mensual": "contrato_servicio",
    "sesion_gratuita": "beneficio_fidelidad",
})

```

Gracias a esto, la recepcionista puede chatear con el sistema diciendo *"El cliente usó su sesión gratuita"* y la base de datos lo traduce mágicamente a sus códigos internos.

Tres preguntas de negocio reales y cómo las responde el código

El valor de un software se mide por la facilidad con la que responde preguntas financieras y operativas. Veamos cómo nuestro código resuelve tres clásicos problemas de este Spa.

Consulta 1: ¿Cuántas sesiones lleva Ana este mes? El código no tiene que escribir complejas sentencias SQL (JOINS) para cruzar tablas de clientes y ventas. Simplemente busca un patrón en el grafo:

```
n = count(u, Pattern(
    fixed={"cliente": ana, "estatus_factual": finalizada},
    type_constraint=u.ind("servicio_spa"),
))
```

La máquina busca todos los cables de "servicio de spa" que conecten a "Ana" y que tengan la etiqueta "finalizada", y los cuenta. Es una operación hiperveloz.

Consulta 2: ¿Ana ya se ganó la sesión gratis? La regla del negocio dice que si tienes 7 visitas, la octava es gratis. Esto se calcula simplemente evaluando la variable anterior:

```
qualifies = n >= 7
```

El modelo no ejecuta la regla solo (para eso necesitas un motor externo de código o una IA), pero el modelo te entrega la información prístina y estructurada para que ese motor tome la decisión correcta sin equivocarse.

Consulta 3: ¿Dónde vivía el cliente Carlos en el año 2022? Resulta que Carlos se mudó de ciudad en enero de 2026. Si nuestro sistema fuera primitivo, la dirección vieja se habría borrado. Pero como nosotros usamos la **Regla de Vigencia (D6)** que aprendimos antes, la mudanza se guardó como un evento con fechas de inicio y fin. La búsqueda se hace así:

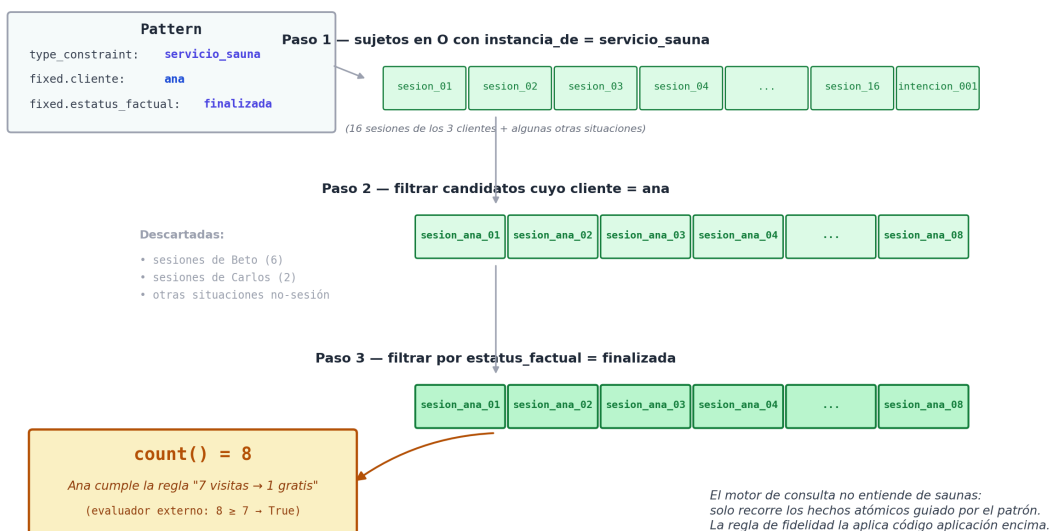
```
# Preguntamos por la residencia en 2022
res_2022 = query(u, ..., at=datetime(2022, 6, 1))
# El sistema responde: "ciudad_a"

# Preguntamos por la residencia actual (2027)
res_2027 = query(u, ..., at=datetime(2027, 6, 1))
# El sistema responde: "ciudad_b"
```

El sistema nunca olvida. Navega por el tiempo como si fuera una dimensión más.

Una consulta como recorrido del grafo: ¿Cuántas sesiones finalizadas tiene Ana?

El patrón fija dos roles, restringe por tipo, y agrega los matches.



Una consulta WH como recorrido del grafo: el patrón fija dos roles, restringe por tipo, y proyecta sobre el rol pregunta. El motor recorre los hechos atómicos y agrega resultados.

El ADN de las explicaciones: Los cables del "Por qué"

En un negocio real, saber *qué* pasó no es suficiente; necesitas saber *por qué* pasó. Aquí es donde los cuatro cables especiales del Capítulo 10 brillan con luz propia.

1. El placer físico de la ducha (El cable causado_por) Ana termina su sesión de spa con una ducha helada. Según los médicos del local, esto causa relajación. En la base de datos se guarda así:

```

(ducha_fria_001, parte_de, sesion_ana_08)
(ducha_fria_001, justificado_por, recomendacion_del_local) ← Regla normativa

(satisfaccion_ana_001, causado_por, ducha_fria_001) ← Reacción física
    
```

Mira cómo un solo evento (la ducha) se conecta con dos realidades distintas: está **justificada** por el manual del local, pero **causa** un efecto físico en la cliente. No mezclamos ambas cosas en un solo campo de "observaciones".

2. El cliente que casi compra (El cable del estatus_factual) Ana quería comprar el plan mensual del gimnasio, pero no tenía dinero en ese momento. En cualquier base de datos tradicional, este dato se pierde o se anota en un post-it. En WQuestions, se guarda formalmente como una "intención":

```

(intencion_ana_001, instancia_de, contrato_servicio)
(intencion_ana_001, cliente, ana)
(intencion_ana_001, modalidad, volitiva) ← Ella quería hacerlo
(intencion_ana_001, estatus_factual, intencionado) ← Pero aún no es un contrato real
    
```

Cuando el equipo de marketing pregunte: "¿Qué clientes tienen un plan mensual activo?", el sistema filtra por estatus **real** y Ana no aparece (no le cobrarán por error). Pero cuando marketing pregunte: "¿A qué clientes podemos llamar para ofrecer un descuento?", filtran por estatus **intencionado** y el nombre de Ana salta a la luz.

El antes y el después: del esquema fragmentado al grafo único

Antes (relacional). En un esquema SQL tradicional, el Spa Oasis vive repartido en al menos cinco tablas: `clientes`, `sesiones`, `planes_mensuales`, `pagos` y `reglas_fidelidad`. Responder "¿Ana ya acumuló 7 sesiones este mes y le corresponde la octava gratis?" obliga a hacer un JOIN entre `clientes` y `sesiones` filtrado por rango de fechas, cruzarlo con `planes_mensuales` para saber si su cuenta corre aparte o en conjunto, y luego comparar el resultado con el umbral que vive en `reglas_fidelidad` — todo eso pegado en código de aplicación que nadie documenta y que se rompe cada vez que cambia la promoción.

```
-- Antes: cliente, sesión y pago viven en tablas distintas; la pregunta de
-- fidelidad obliga a encadenarlas con JOINs frágiles.
CREATE TABLE clientes (id INTEGER PRIMARY KEY, nombre TEXT);
CREATE TABLE sesiones (id INTEGER PRIMARY KEY, cliente_id INT, sala_id INT, inicio TEXT);
CREATE TABLE pagos (id INTEGER PRIMARY KEY, sesion_id INT, monto NUMERIC, moneda TEXT);

SELECT c.nombre, COUNT(s.id) AS sesiones, SUM(p.monto) AS gastado
FROM clientes c
LEFT JOIN sesiones s ON s.cliente_id = c.id
LEFT JOIN pagos p ON p.sesion_id = s.id
GROUP BY c.id;
```

Después (WQuestions). La misma información existe como un grafo único de hechos atómicos: cada sesión es un nodo con el cable `cliente` → ana y el cable `estatus_factual` → finalizada. La pregunta se convierte en dos líneas — un `count` sobre el patrón fijo y una comparación aritmética (`n >= 7`) — sin migrar esquema, sin JOINS y sin lógica enterrada en procedimientos almacenados. Cuando la regla cambie de 7 a 10 visitas, se edita un solo hecho en el catálogo; el resto del sistema ni se entera.

```
# Después: la misma pregunta, como un patrón sobre el grafo de hechos.
sesiones = count(u, Pattern(
    fixed={"cliente": ana, "estatus_factual": u.ind("finalizada")},
    type_constraint=u.ind("servicio_spa")))
octava_gratis = sesiones >= 7
```

Balance del Spa: Misión Cumplida

Cerremos este capítulo revisando qué logró demostrar nuestro prototipo de código en la vida real:

Los 7 ejes funcionan juntos: Manejamos 4 personas, 32 situaciones, 7 lugares, decenas de horarios, magnitudes de dinero y 17 categorías de servicios, todo en la misma estructura. **Ahorro brutal de espacio:** 184 hechos atómicos bastaron para describir un mes entero de operaciones con todos los lujos y detalles. Para hacer lo mismo en una base de datos antigua (SQL), tendríamos que haber programado docenas de tablas complejas. **Diez pruebas de seguridad superadas:** El sistema contó la fidelidad, separó las intenciones de las ventas reales, soportó la polisemia de las palabras y navegó por el pasado sin romperse ni una sola vez. **Cero parches en el código:** Todo el Spa se logró modelar utilizando únicamente los 38 cables base (roles) de nuestra arquitectura. No tuvimos que inventar ingeniería nueva.

El Spa Oasis fue un cliente amable y relajado: el modelo lo absorbió sin un solo parche.

Capítulo 15 — Un servicio on-demand: el app de taxi

El gesto que dispara una cadena de eventos

Valeria sale de un edificio en la plaza principal a las dos y media de la tarde, abre la aplicación de taxis en su celular y toca el botón de "Pedir viaje". Treinta y cinco minutos después, se baja en el aeropuerto.

A los ojos de Valeria, esto fue un solo evento. Pero si miramos los discos duros de Uber o Cabify, entre el momento en que ella tocó el botón y el momento en que se bajó del auto ocurrieron muchísimas cosas: el sistema buscó conductores cercanos, eligió a uno, el conductor aceptó, manejó hasta la plaza, recogió a Valeria, condujo hasta el aeropuerto y finalizó el viaje. Cada uno de esos pasos dejó huellas digitales, generó un cobro, alteró el tráfico y quizás produjo un reclamo que alguien en soporte técnico tuvo que revisar.

Visto desde la ingeniería de datos, **un viaje en taxi no es un evento; es una cadena de seis o siete situaciones distintas que ocurren una tras otra**, donde entran y salen diferentes protagonistas, con relojes cronometrados al segundo, y con reglas matemáticas que dependen del tráfico en tiempo real.

El capítulo anterior (el del Spa) fue como modelar un estanque tranquilo: el cliente llega al spa, se relaja y se va. Este capítulo nos lanza a los rápidos: aquí lidiaremos con la **pluralidad de agentes** (humanos y robots tomando decisiones a la vez), el **encadenamiento rápido de eventos** y la **causalidad emergente** (como cuando el precio del viaje sube de golpe porque hay lluvia o alta demanda). Veamos cómo nuestra arquitectura absorbe este caos sin inmutarse.

Cuatro agentes en una sola transacción

Si le preguntamos a cualquier persona *"¿quién está participando en el viaje?"*, la respuesta obvia es: Valeria y su conductor, Luis. Pero en nuestro sistema hay dos participantes más que suelen pasar desapercibidos: **el App** (un robot de software que toma decisiones autónomas) y **el vehículo** (la máquina física que permite el traslado).

Para que la base de datos sea auditable, necesitamos registrar el trabajo de los cuatro. Aquí es donde nuestra **Regla de Diseño D5 (La agencia contextual)** brilla con todo su esplendor. El App no es solo un programa tonto; es el protagonista absoluto del verbo *asignar*. Es el algoritmo —no Valeria, no Luis— quien decidió darle ese viaje a ese conductor. Por lo tanto, convertimos al algoritmo en un agente con identidad propia.

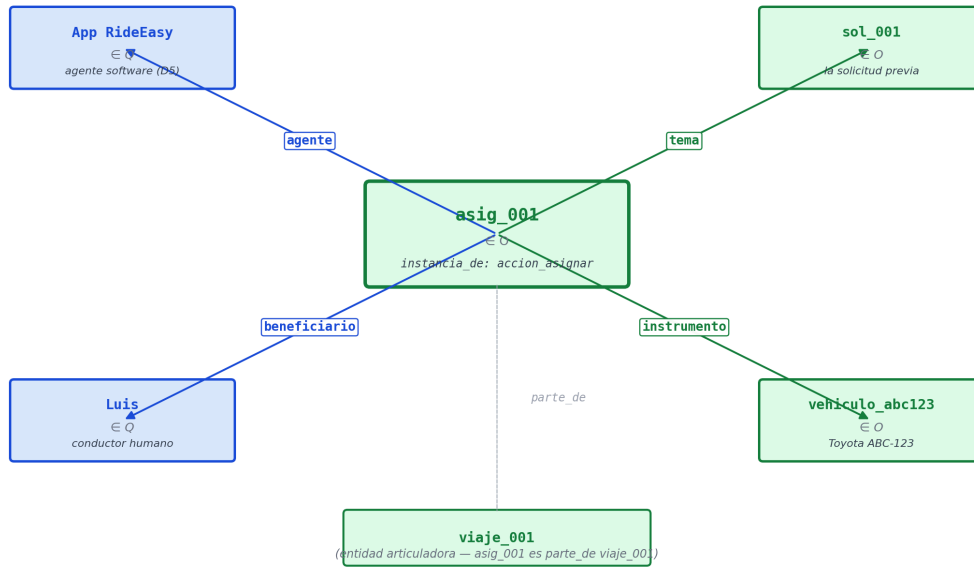
Observa cómo se enchufan los cables de esta decisión en nuestro código (las cinco líneas que generan la orden):

```
asig = ingest_situation(u, lex, "asignar", roles={
    "agente":      app,          # ¡EL ALGORITMO VIVE EN EL EJE Q!
    "tema":        solicitud_v,  # Qué cosa se asigna: el pedido de Valeria
    "beneficiario": luis,        # A quién se le asigna: al conductor
    "instrumento": vehiculo,     # Con qué se hará el trabajo: el coche
    "momento":     at(1),       # La hora exacta
}, sit_id="asig_001")
```

En un solo evento tenemos cuatro participantes cumpliendo roles perfectos. El algoritmo de la App entra a la caja de las personas (Q) como un agente más, capaz de tomar decisiones. El vehículo entra a la caja de objetos (O) actuando como el *instrumento* del trabajo. Y los dos humanos ocupan sus roles habituales.

Asignación de un viaje: agencia repartida (D5)

Cuatro participantes con cuatro roles distintos. El app entra al eje Q como agente del verbo.



La asignación de un viaje involucra cuatro participantes con roles distintos: el app como agente (toma la decisión), el conductor como beneficiario, la solicitud previa como tema, el vehículo como instrumento. Tres viven en Q, uno en O.

(Aclaración técnica: Notarás que el vehículo vive en la caja de objetos **O**. Aunque un coche inteligente tiene GPS y "hace cosas", casi siempre actúa como el instrumento del humano. Si mañana necesitamos auditar al GPS del coche porque tomó una curva peligrosa solo, simplemente creamos un agente "GPS" en la caja **Q** y lo vinculamos al coche físico. Nuestro modelo no te obliga a elegir entre humano o máquina; acepta a ambos).

La cadena de las seis situaciones

Cronológicamente, el viaje de Valeria genera seis eventos reificados (es decir, seis nodos independientes en la base de datos):

solicitar	→	asignar	→	aceptar	→	recoger	→	trasladar	→	completar
14:30		14:31		14:32		14:38		14:40		15:05

Las flechas que ves ahí arriba no son dibujos ilustrativos; son cables reales en nuestro sistema llamados `precede` y `sigue_a`. Esto es oro puro. Significa que el sistema sabe el orden exacto de los eventos sin tener que hacer tediosos cálculos matemáticos restando horas de reloj. Si le preguntas a la máquina: "¿Qué pasó justo después de que Luis aceptara el viaje?", el sistema simplemente sigue el cable al nodo siguiente y te responde al instante.

Pero el tiempo no es lo único que importa. Algunos eventos ocurren como **consecuencia** del evento anterior. El algoritmo no le asignó el viaje a Luis por pura casualidad a las 14:31; lo hizo **porque** Valeria tocó el botón un minuto antes. Para guardar esa causalidad, usamos el cable `motivado_por` que vimos en el Capítulo 10:

```
(asignacion_001, motivado_por, solicitud_001)
```

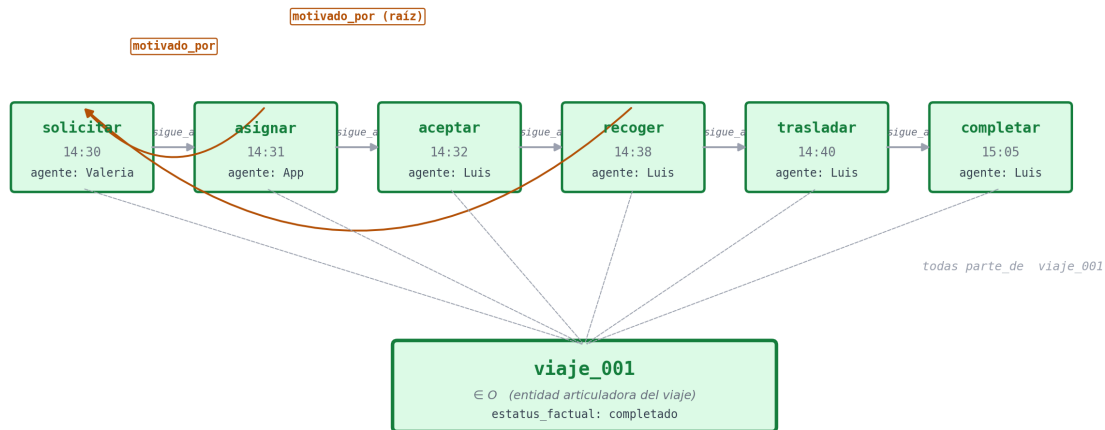
Y el hecho de que Luis recoja a Valeria no es solo el "siguiente paso temporal"; es una obligación nacida del momento en que aceptó el viaje:

```
(recogida_001, sigue_a, aceptacion_001) ← Cable temporal
(recogida_001, motivado_por, solicitud_001) ← Cable de motivo humano
```

Ambos cables (el temporal y el motivacional) conviven en la misma base de datos sin estorbarse. Y ambos son invaluable: el temporal sirve para hacer cálculos de velocidad; el motivacional sirve para que un auditor entienda por qué se tomaron las decisiones.

Un viaje del taxi como cadena de 6 situaciones

Cada situación se sigue de la anterior (*sigue_a*) y todas pertenecen al viaje articulador (*parte_de*).



Dos relaciones distintas: temporal (*sigue_a*) y motivacional (*motivado_por*).
El viaje completo como nodo único permite facturar, refundir o auditar en una sola operación.

Las seis situaciones de un viaje encadenadas por *sigue_a* (temporal) y *motivado_por* (causal). El viaje completo vive como entidad superior que agrupa todo por *parte_de*.

El truco final: Para que estos seis eventos no queden flotando como basura suelta en el servidor, los agrupamos a todos bajo una "carpeta maestra". Creamos un evento gigante llamado `viaje_001` y conectamos a los otros seis eventos a él usando el cable `parte_de`. El `viaje_001` es la unidad comercial real; es a este nodo al que le pegamos la factura final y al que acude soporte técnico si hay un problema.

Tarifa dinámica (Surge pricing): Explicando el caos del mercado

Este negocio tiene un reto que pone a prueba la estructura de nuestra base de datos: el precio cambia según el clima o el tráfico. Si hay alta demanda o está lloviendo, la tarifa sube.

Cualquier programador mediocre simplemente le añadiría a la factura un campo numérico que diga: `multiplicador_clima: 1.67`. Funciona para cobrar, pero arruina la auditoría. Si un cliente reclama, no hay forma de explicarle al usuario qué pasó exactamente en su calle a esa hora para cobrarle el doble.

La solución de nuestro modelo (usando la Regla D7) es **reificar el estado del clima o del mercado**, convirtiéndolo en un evento oficial en la caja 0, y conectarlo al precio final usando el cable `causado_por`:

```

# 1. Creamos el estado de la calle (lloviendo y con alta demanda)
estado_demanda = u.add_individual(Individual(id="alta_demanda_14_30", axis=Axis.0))
u.assert_fact(estado_demanda, "lugar_de", plaza)
u.assert_fact(estado_demanda, "momento", at(0))

# 2. Creamos la factura y la conectamos al clima
tarifa = u.add_individual(Individual(id="tarifa_viaje_001", axis=Axis.0))
u.assert_fact(tarifa, "monto", n_25_usd)
u.assert_fact(tarifa, "causado_por", estado_demanda)

```

🔮 ¡Aquí está la magia!

La diferencia a nivel empresarial es monumental. Si un usuario manda un correo quejándose: *"¿Por qué demonios me cobraron 25 dólares hoy si ayer me costó 15?"*, el sistema lee el cable `causado_por` y le responde al instante: *"Su tarifa fue calculada debido a un evento de Alta Demanda registrado exactamente a las 14:30 en la Plaza Central"*. Sin esta arquitectura, un empleado tendría que bucear horas en registros de tráfico y clima para dar una respuesta decente.

Cuando las cosas se cancelan (El arte de no borrar nada)

El mundo de los taxis vive de cancelaciones. Alguien pide un auto y se arrepiente; un conductor acepta y luego se le pincha una llanta. La gran duda de los programadores siempre es: *¿Qué hago con el viaje cancelado? ¿Lo borro? ¿Lo marco con color rojo?*

La convención de oro de nuestro modelo es **la inmutabilidad absoluta**: nunca borramos nada, nunca reescribimos el pasado. Una cancelación es, sencillamente, **un nuevo evento** que ataca al evento viejo. Nuestro Lexicon trae un verbo específico para esto:

```
# Creamos la cancelación oficial
canc = ingest_situation(u, lex, "cancelar", roles={
    "agente": valeria,
    "tema": viaje_001,      El tema es el viaje anterior
    "momento": at(62),
}, sit_id="canc_001")

# Enchufamos la cancelación al viaje original
u.assert_fact(canc, "cancela", viaje_001)
u.assert_fact(viaje_001, "estatus_factual", cancelado)
```

Nacieron dos hechos nuevos: la cancelación oficial (con su hora exacta y la persona culpable de hacerla) y una marca de "estado" pegada al viaje original. El viaje antiguo sigue intacto en el disco duro, pero ahora está marcado como muerto. Cuando la empresa quiera calcular las ventas del día, el código simplemente ignora los eventos que tengan la marca "cancelado". La base de datos se vuelve un registro histórico indestructible, perfecto para auditorías legales.

Tres consultas operativas

Veamos cómo tres preguntas críticas del negocio se resuelven en nuestra base de datos en cuestión de milisegundos:

Consulta 1 — ¿Adónde llevó Luis a Valeria?

```
r = query(u, Pattern(
    fixed={"agente": luis, "paciente": valeria},
    ask={"destino": Var()},
    type_constraint=u.ind("accion_trasladar"),
))
```

El motor rastrea en el mapa geométrico todos los eventos de "trasladar" donde Luis y Valeria coincidan, y escupe el destino (aeropuerto).

Consulta 2 — ¿Cuántos viajes completó Luis hoy?

```
n = count(u, Pattern(
    fixed={"agente": luis, "estatus_factual": completado},
    type_constraint=u.ind("viaje"),
))
```

Fíjate en algo asombroso: este código es matemáticamente **idéntico** al que usamos en el Spa para contar las visitas de Ana. La uniformidad de nuestro modelo te permite reciclar código entre empresas que no tienen nada que ver entre sí.

Consulta 3 — ¿Por qué la tarifa fue de 25 dólares?

```
explicaciones = u.facts_about(tarifa_viaje_001)
causa = [f for f in explicaciones if f.role == "causado_por"]
# Resultado: "alta_demanda_14_30"
```

Seguimos el cable causal y obtenemos la justificación matemática. Simple, rápido e indiscutible.

El antes y el después: del esquema fragmentado al grafo único

Antes (relacional). El esquema clásico de una app de taxi distribuye la realidad entre tablas separadas: `viajes`, `conductores`, `pasajeros`, `tarifas`, `eventos_viaje`. Reconstruir la cadena completa —qué ocurrió, en qué orden y por qué el precio subió— obliga a hacer JOINS en cascada entre cuatro o cinco tablas más una concatenación en código que ordena por timestamp. Peor aún, la pregunta *"¿cuál fue la causa concreta de que esa tarifa subiera a 25 dólares ese minuto?"* no tiene respuesta posible: no existe tabla de `causas_tarifa`; el motivo —lluvia, alta demanda, zona de congestión— o se perdió o vive enterrado en un campo de texto libre que nadie consulta con coherencia.

Después (WQuestions). La misma realidad vive como un único grafo: `viaje_001` agrupa los seis eventos reificados —`solicitar`, `asignar`, `aceptar`, `recoger`, `trasladar`, `completar`— mediante cables `parte_de` y `sigue_a`. La tarifa es un nodo en `0` conectado al estado de alta demanda por el cable `causado_por`. Reconstruir el viaje completo es seguir los cables desde `viaje_001`; obtener la causa del precio es leer un único hecho. No hay JOIN, no hay pegamento en código, no hay pregunta que el esquema no pueda responder —porque el motivo nunca fue un campo numérico: siempre fue un evento con identidad propia.

Resumen: Lo que el Taxi nos enseñó (que el Spa ocultaba)

El Spa fue un calentamiento; el negocio del Taxi nos demostró la resistencia extrema del modelo en tres áreas clave:

- Agentes Robots:** Le dimos poder de decisión al algoritmo (el App). Comprobamos que el sistema no hace distinciones entre un humano y un software a la hora de asignar responsabilidades legales sobre una acción.
- Cadenas de alta velocidad:** Comprobamos que podemos encadenar decenas de microeventos en orden cronológico y guardarlos dentro de una gran "carpeta maestra" (el `viaje_001`) para no perder el orden del negocio.
- Los "Porqués" del mercado:** Demostramos que podemos convertir conceptos abstractos como "la lluvia" o "el alza de precios" en eventos físicos para justificar cambios matemáticos en las facturas, solucionando el peor dolor de cabeza de los servicios de atención al cliente.

La prueba de código superó las simulaciones. No tuvimos que inventar ningún parche raro ni modificar nuestro diccionario de roles oficiales para sostener a esta empresa transnacional.

En la clínica no hay viajes rápidos ni tarifas que cambien con la lluvia. En la clínica hay diagnósticos, drogas, contraindicaciones, protocolos hospitalarios y una densidad abrumadora de información médica. Vamos a ver cómo nuestro modelo, que ya dominó los negocios comerciales, se enfrenta a la pesada carga de la ciencia y la vida humana sin desmoronarse.

Capítulo 16 — Un dominio nuevo: La historia clínica

El cambio de marcha

Los dos dominios que vimos antes (el Spa y el Taxi) tenían algo en común que suele pasar desapercibido: eran negocios puramente **transaccionales**. Sus situaciones eran eventos fugaces. La sesión de spa de Ana empieza, termina, se cobra y el mundo sigue girando. El viaje de Valeria ocurre, se paga y listo. Modelar uno de estos negocios es solo cuestión de aprender a usar nuestras herramientas; modelar el otro es repetir la misma fórmula cambiando las palabras del Lexicon.

Pero el mundo de la medicina no funciona así. Una consulta médica es un evento, sí, pero lo verdaderamente importante no es cuánto duró la cita, sino **el contenido denso que se produjo allí adentro**: un diagnóstico, una receta de pastillas, una alerta de alergia, la programación de un control futuro.

A un médico no le importa el evento de la consulta; al médico le importan los "objetos de información" que nacieron en ella. Le importa el diagnóstico de hipertensión que seguirá vigente un año después; la pastilla que se debe ajustar mes a mes; la regla universal que prohíbe mezclar dos fármacos. Aquí, la riqueza de los datos no está en la línea de tiempo, sino en el peso intelectual de cada decisión.

Esto somete a nuestra arquitectura a una presión titánica. Ya no basta con encadenar eventos. Necesitamos que nuestro sistema sepa **de qué habla cada evento**, con una estructura tan perfecta que una Inteligencia Artificial pueda leer el expediente de un paciente y deducir si una pastilla lo va a matar. Y todo esto tiene que funcionar a lo largo de décadas, porque los diagnósticos mutan, las medicinas cambian y los protocolos se actualizan.

Este capítulo modela una consulta típica: la Dra. Torres atiende a María Gonzales por un dolor de cabeza crónico. Veremos cómo nuestro modelo absorbe esta avalancha científica. Y, siendo honestos, te mostraré el primer bache real con el que me topé al programar este prototipo: cuando la intuición humana choca contra la rigidez de las matemáticas.

La consulta como la "Carpeta Maestra"

Igual que en el caso del Taxi creamos un `viaje_001` gigante que abrazaba todos los pequeños pasos, la consulta médica funciona como una gran carpeta articuladora. Una revisión típica produce cinco documentos internos:

1. **El síntoma:** Lo que dice el paciente. *"Me duele la cabeza hace tres días."*
2. **La medición:** Los números crudos que saca el doctor. *"Presión en 145/92."*
3. **El diagnóstico:** La conclusión clínica. *"Tiene Hipertensión de grado 1."*
4. **La prescripción:** El ataque químico. *"Tome Enalapril de 10 mg."*
5. **El control:** El plan a futuro. *"Nos vemos en 30 días."*

En nuestro modelo, estas cinco cosas no son campos de texto; son **cinco situaciones reificadas independientes** que cuelgan de la consulta principal usando nuestro viejo y confiable cable `parte_de`.

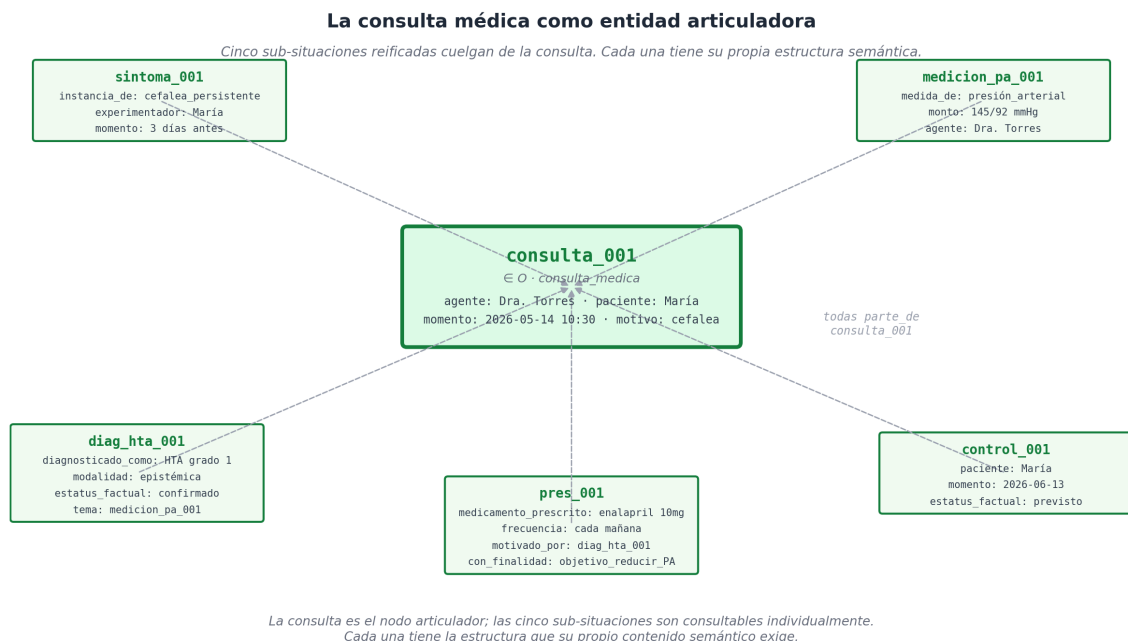
```

(consulta_001) ∈ 0
  instancia_de      : consulta_medica
  agente           : dra_torres
  paciente         : maria_g
  lugar_de         : consultorio_03
  motivo           : cefalea_persistente

(sintoma_001 ∈ 0, parte_de, consulta_001)
(medicion_001 ∈ 0, parte_de, consulta_001)
(diag_hta_001 ∈ 0, parte_de, consulta_001)
(prescrip_001 ∈ 0, parte_de, consulta_001)
(control_001 ∈ 0, parte_de, consulta_001)

```

La arquitectura se mantiene inalterable: una entidad gigante que contiene un racimo de sub-situaciones. Pero el interior de cada una de estas sub-situaciones es lo que vuelve fascinante a la medicina.



*La consulta médica con sus cinco sub-situaciones colgando: síntoma, medición, diagnóstico, prescripción, control futuro.
Cada una con su propia estructura semántica.*

Un choque frontal con las matemáticas: Cuando la Caja O se queda corta

Aquí apareció mi primera fricción seria al programar. Piénsalo como humano: cuando el médico mide la presión arterial de María, tu instinto te dice que escribas que el evento tiene como `tema` a la "presión arterial". Y cuando el doctor receta Enalapril, tu instinto te dice que el `tema` de la receta es el "Enalapril".

Pero espera. **"Presión arterial" y "Enalapril" no son objetos físicos con identidad propia.** No puedes guardar a la "Presión arterial universal" en un cajón. Son **categorías** teóricas. La cajita de pastillas que María compra en la farmacia sí es un objeto físico (caja `0`), pero el concepto científico de la droga *Enalapril de 10mg* es una idea (caja `K`).

Nuestro catálogo estricto D8 dictamina que el cable `tema` solo puede conectar la caja `0` con la caja `0`. Si yo intento conectar la caja `0` con la caja `K` usando el cable `tema`, el sistema entra en pánico y me lanza un error matemático (`SignatureError`). El modelo me regañó por intentar mezclar conceptos físicos con conceptos teóricos.

¿La solución? Crear "Cables de Dominio" (roles propios) específicos para la medicina, diseñados para apuntar hacia la caja K:

```
# Mal: Usar "tema" para la medicina
# Bien: Crear un cable especial
```

```
u.assert_fact(prescripcion, "medicamento_prescrito", enalapril)
u.assert_fact(medicion, "medida_evaluada", presion_arterial)
```

Este cable va de 0 a K
Este también

Esto me dejó dos lecciones valiosas: 1. El catálogo de 38 cables universales sirve para casi todo, pero cada industria altamente técnica (como la medicina o la química) te obligará a crear un puñado de cables exclusivos para sus conceptos abstractos. 2. El error no era del modelo; el error era de mi mente humana, que mezcla objetos y conceptos todo el tiempo. La rigidez de las matemáticas me salvó de crear un desastre semántico que habría confundido a una IA en el futuro.

Disecccionando el Diagnóstico: Dudas, Causas y Tiempo

Un diagnóstico médico es una joya de la información. Combina tres dimensiones complejísimas que nuestro modelo resuelve con elegancia:

1. El nivel de duda (Modalidad Epistémica): Un diagnóstico casi nunca es una verdad absoluta tallada en piedra; es lo que el médico *crea* en ese instante. Para reflejar esa incertidumbre clínica, decoramos la situación con `modalidad: epistemica` y jugamos con su `estatus_factual` (puede ser *confirmado*, *sospechoso* o *descartado*).

```
(diag_hta_001, modalidad, epistemica)
(diag_hta_001, estatus_factual, confirmado)
```

2. La evidencia (Causalidad): Un diagnóstico no nace de la magia; nace de los números. Si el doctor dice "Hipertensión", es porque vio el número de la "medición de presión". Esta conexión de causa y efecto la guardamos usando nuestro amado cable `motivado_por`:

```
(diag_hta_001, motivado_por, medicion_pa_001)
```

Ahora la medición no es un simple número flotando; es la **evidencia** del diagnóstico. Si un auditor médico o una IA revisan el expediente, la máquina puede justificar el diagnóstico instantáneamente leyendo este cable.

3. El viaje en el tiempo (Vigencia D6): Esta es la regla que separa el software de juguete del software hospitalario real. Los diagnósticos caducan y cambian. Supongamos que en mayo de 2026 María fue diagnosticada con Hipertensión Grado 1. Un año después, en enero de 2027, el médico le dice que empeoró y ahora tiene Grado 2. Si reescribes el diagnóstico viejo y pones el nuevo, acabas de cometer un crimen de negligencia médica al borrar la historia de la paciente.

La Regla D6 (Vigencia temporal) entra a salvarnos la vida. Guardamos **ambos** diagnósticos, cada uno con sus fechas de inicio y fin:

```
# El Diagnóstico viejo (caducó cuando el doctor emitió el nuevo)
u.assert_fact(diag1, "diagnostico_asignado", hta_grado_1,
              valid_from=mayo_2026, valid_to=enero_2027)

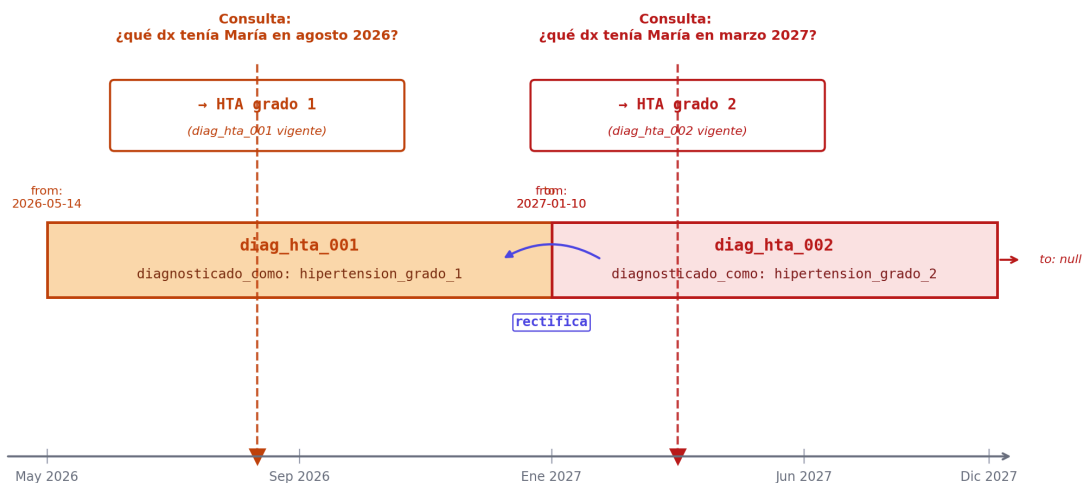
# El Diagnóstico nuevo (vigente hasta el día de hoy)
u.assert_fact(diag2, "diagnostico_asignado", hta_grado_2,
              valid_from=enero_2027)

# Y conectamos ambos para que no queden dudas
u.assert_fact(diag2, "rectifica", diag1)
```

Si hay una demanda por negligencia y el juez pregunta: "¿Qué sabía el hospital sobre esta paciente en agosto de 2026?", la base de datos devuelve exactamente la "verdad clínica" que existía en esa fecha (Grado 1), ignorando el futuro. Esta capacidad forense viene incluida de fábrica en nuestra arquitectura.

D9 sobre un diagnóstico: dos rangos de vigencia, sin perder histórico

El rediagnóstico rectifica el original. La consulta temporal recupera el correcto según el momento.



La misma consulta, parametrizada por `at=...`, devuelve el diagnóstico vigente en ese momento.

D6 sobre un diagnóstico: HTA grado 1 vigente hasta enero 2027, reemplazado por HTA grado 2 desde entonces. La relación rectifica conecta ambos. Una consulta at= devuelve el correcto según el momento.

Prescripción Médica: La tormenta de los "Por qué"

Cuando un médico te receta una pastilla, se activan simultáneamente varias de las fuerzas causales que vimos en el Capítulo 10. Mira cómo se acumulan en nuestro sistema:

```
pres = ingest_situation(u, lex, "prescribir", roles={
    "agente": dra_torres,
    "paciente": maria_g,
    "medicamento_prescrito": enalapril,
    "frecuencia": cada_manana,
    "duracion": indefinida,
})

u.assert_fact(pres, "motivado_por", diag_hta_001)
u.assert_fact(pres, "con_finalidad", bajar_presion_obj)
u.assert_fact(pres, "verificado_contra", regla_embarazo_alerta)
```

- ☒ Por qué me recetan esto
- ☒ Para qué me recetan esto
- ☒ Ley médica de seguridad

Tres respuestas profundas a tres preguntas distintas. El médico no te receta por gusto; lo hace *motivado* por tu enfermedad. Lo hace *con la finalidad* de alcanzar un resultado a futuro (bajar tu presión). Y lo hace amparado por

un manual de farmacología que *verifica* que la pastilla no te matará.

Y hablemos un segundo de esa última regla de farmacología. Un manual que dice "*Prohibido dar Enalapril a embarazadas*" no es un pedazo de código escondido por un programador; la guardamos como un evento reificado más en el eje 0, como si fuera una ley de tránsito:

```
(ley_enalapril_embarazo) ∈ 0
  instancia_de:      contraindicacion_medica
  medicamento_implicado: enalapril
  condicion:        estado_embarazo
  orden_oficial:    evitar_medicamento
```

Con esto, nuestra base de datos se vuelve inteligente. Cuando la doctora presiona "recetar", un script de código externo simplemente escanea la base de datos, lee esta regla universal, se da cuenta de que la paciente está embarazada y bloquea el botón lanzando una alerta roja.

Balance Médico: Misión Cumplida

El modelo WQuestions devoró el reto del mundo médico sin sudar. Observa lo que validamos con código ejecutable:

1. La "Carpeta Maestra" de la consulta abrazó cinco sub-situaciones con la relación `parte_de` sin tener que fabricar bases de datos paralelas.
2. La receta médica se enlazó a la evidencia clínica (`motivado_por`).
3. El control médico de la próxima semana se guardó como `estatus_factual: previsto`. El sistema sabe que es un plan, no una realidad.
4. La regla del tiempo **D6** nos permitió guardar dos diagnósticos contrarios sin perder el rastro histórico forense de la paciente.
5. Todo este enmarañado monstruo científico ocupó apenas **62 hechos atómicos**. Un volumen de datos ridículamente pequeño si lo comparamos con la burocracia de un servidor SQL tradicional.

El antes y el después: del esquema fragmentado al grafo único

Antes (relacional). En un sistema hospitalario típico, el expediente de María vive repartido entre cinco o seis tablas: `pacientes`, `consultas`, `diagnosticos`, `prescripciones`, `contraindicaciones` y —si el equipo fue cuidadoso— una tabla `diagnostico_historial` para versionar los cambios. Cuando la doctora quiere saber si el Enalapril que acaba de recetar choca con alguna contraindicación *vigente* del paciente, el sistema tiene que cruzar `prescripciones` con `contraindicaciones`, pero la vigencia del diagnóstico de hipertensión —¿sigue activo?— exige otro JOIN contra `diagnostico_historial` filtrando por fechas. La respuesta no está en ninguna tabla sola; vive en el pegamento que el programador escribe y reescribe cada vez que la lógica clínica cambia.

Después (WQuestions). La misma información existe como un único grafo de hechos: `diag_hta_001` cuelga de `consulta_001` con `parte_de`, lleva sus propias fechas `valid_from` / `valid_to` internas, y `prescrip_001` apunta a `ley_enalapril_embarazo` con `verificado_contra`. La pregunta —¿esta prescripción choca con una contraindicación vigente?— es una proyección sobre el grafo con un filtro temporal: el motor entrega solo los hechos cuya ventana de validez cubre la fecha de hoy. No hay JOIN entre tablas, no hay código de pegamento; el modelo lo resuelve porque la vigencia y las relaciones causales ya estaban tejidas en el mismo tejido de datos desde el primer día.

Una lección de diseño: El arte de saber qué reificar

Quiero regalarte una pepita de oro metodológica que extraje al programar este hospital.

A la hora de diseñar, ¿cómo decides qué evento merece convertirse en una entidad importante (Reificarse en 0) y qué cosa solo debe ser un número triste en la caja N? La regla es simple y brutal: **Si crees que en el futuro alguien va a hacer una pregunta o una afirmación sobre ese dato, entonces reifícalo.**

- ¿Reificamos el "Síntoma de dolor"? Sí, porque mañana el médico querrá saber cuándo empezó y qué tan fuerte es. Lo volvemos entidad (0).
- ¿Reificamos la "medición de presión"? Sí, porque mañana querremos conectarla como evidencia de una enfermedad. Entidad (0).
- ¿Reificamos el dato "tomar cada mañana"? No. Es un valor final. A nadie le importa interrogar al concepto "cada mañana". Eso se queda como una etiqueta inerte en K.
- ¿Reificamos la "Regla de prohibición para embarazadas"? Sí, porque tiene fecha de vigencia, condición y autor. Entidad (0).

El costo en el servidor de convertir algo en entidad es de unos cuantos bytes. El costo corporativo de no hacerlo y luego descubrir que un juez te exige el historial de ese dato... te costará millones en refactorización de software.

Capítulo 17 — El dominio más exigente: un banco

Por qué el banco es la prueba de fuego definitiva

Si miras una empresa promedio, probablemente tenga su información guardada en unas 150 tablas de datos, un par de sistemas viejos y un software contable. Pero si miras un banco regional típico (uno de tamaño mediano), el paisaje es aterrador: tienen 1.500 tablas solo para el corazón del banco, otras mil para las tarjetas de crédito, otras mil para los seguros y docenas de aplicaciones satélites para créditos rápidos, fraudes y la app del celular. Cada sistema tiene sus propios jefes, sus propias reglas y su propio idioma.

Si en el capítulo del Spa probamos que el modelo era ágil, y en el del Taxi probamos que soportaba la velocidad, **el banco es el campo de batalla donde WQuestions demuestra si sirve para el mundo real.**

Los bancos gastan fortunas intentando mantener arquitecturas tradicionales que se caen a pedazos. Es aquí donde nuestras decisiones de diseño (como no borrar nunca el pasado con la regla D6, o tratar al software como un agente más con la regla D5) dejan de ser lujos teóricos y se convierten en **exigencias de supervivencia dictadas por la ley.**

Este capítulo no pretende modelar un banco entero (necesitaríamos otro libro para eso). Vamos a tomar cuatro situaciones críticas —una transferencia, el ciclo de vida de un préstamo, una investigación por fraude y el catálogo de tarjetas— para demostrar que nuestro modelo aguanta el peso del dinero sin doblarse.

La triste realidad: Cinco islas que no se hablan

Antes de empezar, seamos honestos sobre cómo funciona un banco por dentro hoy en día. Si trabajas en uno, vas a sonreír; si no, te vas a sorprender. Un banco es un archipiélago de cinco islas desconectadas:

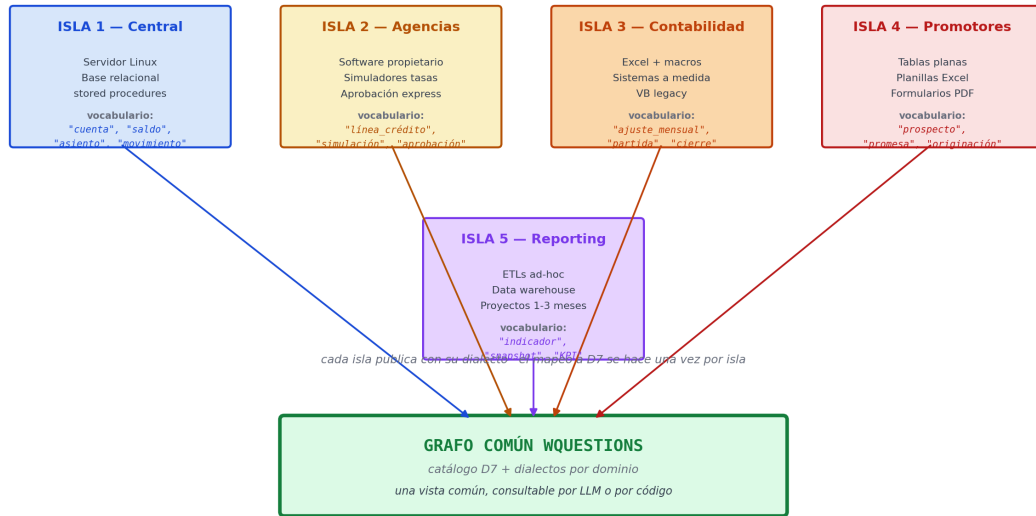
- **Isla 1 (El Cerebro Central):** Es el servidor principal (el *core bancario*). Aquí viven los saldos, los movimientos oficiales y la contabilidad pesada. Es la "verdad" que el gobierno audita.
- **Isla 2 (Las Agencias):** Cada sucursal tiene pequeños softwares propios para simular créditos o sacar turnos. Estos programitas toman decisiones todos los días, pero rara vez le explican al Cerebro Central cómo llegaron a esa conclusión.
- **Isla 3 (Contabilidad):** El zoológico oculto. Parte vive en el servidor, pero otra gran parte vive en gigantescas hojas de Excel que se ajustan a mano a fin de mes. Cuando hay diferencias entre el Cerebro y Contabilidad, suele ganar Contabilidad, pero nadie sabe explicar por qué.
- **Isla 4 (Los Promotores):** Los agentes de campo que venden créditos en la calle anotan datos en Excel o papeles. Toda esa valiosa información del cliente es "fantasma" hasta el día en que se aprueba el crédito y recién ahí entra al banco.
- **Isla 5 (Los Reportes):** Cuando el gerente pide un reporte que cruce datos de las cuatro islas anteriores, los ingenieros tardan un mes en armar un túnel temporal que junte todo en un "Data Lake". A veces, los datos no cuadran y el proyecto fracasa.

¿Por qué pasa esto? Porque cuando los bancos intentan modernizarse, cometen el error de obligar a todas estas islas a hablar el mismo idioma robótico. Y reprogramar un software viejo de 15 años para que hable el idioma moderno es tan caro que los bancos prefieren dejarlo morir en paz.

WQuestions cambia el juego por completo. Nuestro modelo no le exige a la sucursal o a Contabilidad que cambien su software. Solo les exige que **traduzcan** lo que están haciendo y lo envíen a nuestro mapa central (el Grafo) usando su propia jerga. Contabilidad envía sus datos hablando de "partidas"; la sucursal envía datos hablando de "simulaciones". Nuestro Lexicon (Capítulo 12) se encarga de traducirlo todo al catálogo universal D8.

Las cinco islas del banco: vocabularios distintos, un grafo común

Cada isla sigue siendo isla. Cada una publica sus hechos al grafo con su dialecto, mapeado al catálogo D7 una vez.



Sin centralización ni reescritura. Las islas siguen existiendo; el grafo las vuelve consultables en conjunto.

Las cinco islas típicas de un banco regional — central, agencias, contabilidad, promotores, reporting — cada una con su tecnología y su vocabulario. WQuestions no reemplaza ninguna; cada una publica sus hechos al grafo común con su dialecto mapeado al catálogo D8 una sola vez. El archipiélago se vuelve consultable sin disolverse.

Cada isla sigue operando como siempre, pero ahora la gerencia puede hacerle una pregunta al mapa central y cruzar los datos de todas las islas en milisegundos.

Mapeando el Banco en nuestras cajas maestras

Para dominar este caos, ordenemos rápidamente las piezas del banco en nuestras cajas de valor:

- **Q (Quién - Agentes):** Aquí metemos a los clientes físicos, a las empresas (personas jurídicas) y —muy importante— a los **Sistemas**. El motor antifraude o el autorizador de Visa toman decisiones solos; por lo tanto, tienen derecho a vivir en la caja **Q** como agentes activos.
- **O (Qué - Eventos):** Las cuentas de ahorro, los préstamos, los movimientos de dinero, las tarjetas de crédito, los asientos contables y las investigaciones de fraude. Aquí vive el 90% del peso del banco.
- **L (Dónde - Lugares):** Sucursales físicas, cajeros automáticos (ATMs) y lugares virtuales como la App móvil o la web.
- **T (Cuándo - Tiempos):** Ojo aquí, porque hay triple reloj: la hora a la que el cliente hizo el pago, la hora a la que el servidor lo procesó y la fecha de cierre contable. Nuestro modelo anota los tres.
- **N (Cuánto - Magnitudes):** Dinero, tasas de interés, plazos y puntajes de riesgo.
- **K (Clase - Etiquetas):** Tipos de cuentas, monedas (Dólares, Euros), estados de mora y códigos legales.

El dominio bancario sobre los 6 ejes de valor

Tres clases de Q, seis familias de O, multi-canal en L, T triple, N+K obligatorio en todo importe.

Q	quién (6 ejemplos)	Cliente retail	Empresa corp.	Empleado	Banco S.A.	Motor antifraude	Visa/MC	
O	qué (7 ejemplos)	cuenta_ahorros	préstamo_personal	tarjeta_visa	transferencia	asiento_contable	oferta_visa_pt	investig_fraude
L	dónde (6 ejemplos)	casa matriz	sucursal_centro	ATM_001	web banking	mobile app	POS comercial	
T	cuándo (4 ejemplos)	t. operativo	t. contable	fecha cierre	vencimiento cuota			
N	cuánto (5 ejemplos)	500 USD	5000 USD (capital)	18% (tasa)	36 cuotas	850 (score)		
K	clase (7 ejemplos)	Currency:USD	Currency:PEN	vigente	mora_60	castigado	Platinum	KYC_completo

Industrial: miles de Q, millones de O, cinco a diez canales L, T triplemente registrado, todo importe con N+K.

El dominio bancario sobre los ejes de valor: tres clases de agentes en Q, seis familias de entidades en O, multi-canal en L, tiempos triplemente registrados en T, importes y tasas en N, un catálogo K extenso. Los predicados del eje cómo (M) cablean todo.

Caso 1: Una transferencia (Cinco agentes ocultos)

A los ojos de un humano, una transferencia es simple: "Ana le manda \$500 a Beto". Pero en la base de datos de un banco, este movimiento involucra a cinco agentes y genera registros espejo en contabilidad. Mira cómo lo desglosa nuestro sistema:

Primero, creamos el evento central de la transferencia:

```
(transferencia_001) ∈ O
  instancia_de: accion_transferir
  agente: ana ← Agente 1 (Inicia)
  beneficiario: beto ← Agente 2 (Recibe)
  cuenta_origen: cta_ana_001
  monto: n_500_usd
  lugar_de: app_movil_banco
  autorizado_por: sistema_web_banco ← Agente 3 (Da el ok)
  verificado_por: motor_antifraude ← Agente 4 (Revisa que no sea robo)
```

Pero la transferencia no se detiene ahí. Detrás del telón, el departamento de Contabilidad (Agente 5) tiene que hacer sus balances. En WQuestions, creamos dos **sub-situaciones** y las colgamos de la transferencia usando el cable `parte_de`:

```
(asiento_debito_001) ∈ O // Le quitamos plata a Ana
  parte_de: transferencia_001
  cuenta_contable: ahorros_ana_interna
  monto: 500 USD
  tipo_movimiento: debito

(asiento_credito_001) ∈ O // Le ponemos plata a Beto
  parte_de: transferencia_001
  cuenta_contable: ahorros_beto_interna
  monto: 500 USD
  tipo_movimiento: credito
```

Si el día de mañana esta operación resulta estar mal hecha, **jamás la borramos**. Simplemente creamos una "transferencia rectificativa" que ataca a la vieja y la anula. Los bancos están obligados por ley a guardar todo el historial de errores, y nuestra base de datos lo hace por defecto.

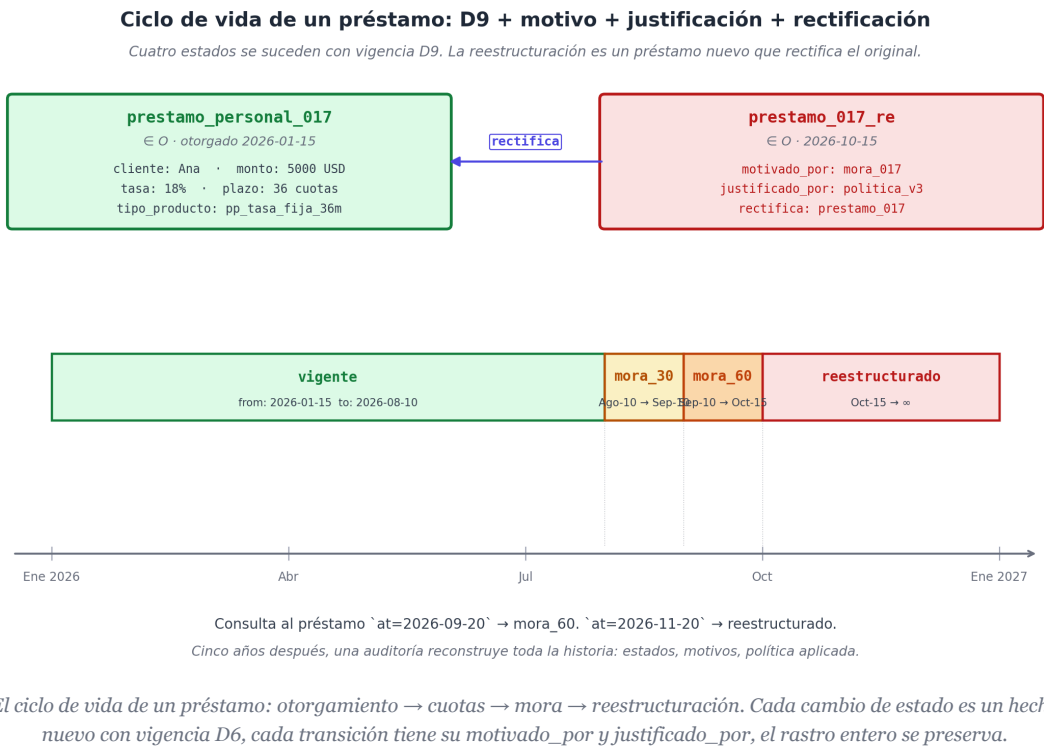
Caso 2: La novela de un Préstamo y la regla del tiempo

Un préstamo no es un evento de un segundo; es una novela que dura años. Se aprueba, se pagan cuotas, el cliente se atrasa (entra en mora) y a veces se reestructura la deuda. En un sistema viejo, la celda "Estado" del préstamo se va borrando y reescribiendo, destruyendo el historial.

Nosotros usamos la **Regla D6 (Vigencia Temporal)**. Cada vez que el préstamo de Ana cambia de estado, no borramos nada; inyectamos una línea nueva con fecha de inicio y fin:

```
(prestamo_017, estado, vigente, inicio=2026-01-15, fin=2026-08-10)
(prestamo_017, estado, mora_30_dias, inicio=2026-08-10, fin=2026-09-10)
(prestamo_017, estado, mora_60_dias, inicio=2026-09-10, fin=2026-10-15)
(prestamo_017, estado, reestructurado, inicio=2026-10-15, fin=hoy)
```

Cinco años después, en medio de un juicio, si el juez pregunta: "¿En qué estado exacto estaba este préstamo el 20 de septiembre de 2026?", la base de datos filtra por fecha y responde implacablemente: "Mora de 30 días". Sin D6, los bancos tienen que construir carísimas tablas paralelas solo para guardar estos "fantasmas del pasado".



Caso 3: CSI Bancario (Investigando un fraude)

Ana llama furiosa: alguien usó su tarjeta para pagar \$1.840 en un casino de Las Vegas anoche. El banco tiene que abrir una investigación. Aquí es donde nuestra arquitectura aplasta a la competencia.

El investigador no necesita ver el saldo de Ana de hoy; necesita viajar en el tiempo a la noche del fraude y preguntar: ¿Dónde creía el banco que estaba Ana anoche? ¿Qué decía el perfil de riesgo a esa hora exacta?

```
(perfil_riesgo_ana_v3, instancia_de, perfil_antifraude,
                               inicio=2026-04-10, fin=2026-05-22)
(perfil_riesgo_ana_v3, score_riesgo, 0.31)
```

Gracias a nuestro diseño, el investigador puede ver que esa noche la tarjeta pasó porque el `score_riesgo` era bajo. Al confirmar el robo, el investigador no borra el pago del casino (el pago fue real y descontó dinero); en su lugar, crea un evento de "reverso" que ataca al original, documentando el porqué:

```
(reverso_pago_casino, cancela,          autorizacion_original_001)
(reverso_pago_casino, justificado_por, investigacion_fraude_001)
```

El historial queda intacto, el dinero vuelve y el banco tiene pruebas perfectas para el seguro.

Caso 4: Una Tarjeta Platinum no es una idea, es un objeto

Finalmente, resolvamos un error común de programación. Cuando el banco le da una "Tarjeta Visa Platinum" a Ana, muchos programadores creen que "Visa Platinum" es una categoría abstracta (Caja `K`). Falso.

La "Oferta Visa Platinum de Enero 2026" es **un objeto real y reificado** (Caja `0`), que tiene anexo un contrato, unas tasas de interés y una fecha de caducidad.

Cuando el banco le da el plástico a Ana, conectamos la tarjeta de Ana a la oferta específica que estaba vigente ese día:

```
(visa_platinum_oferta_2026_Q1) ∈ 0                                ← El contrato de oferta del banco
  cuota_manejo_mensual:      8 USD
  vigencia:                   inicio=Enero, fin=Junio

(tarjeta_ana_001) ∈ 0                                             ← El plástico que tiene Ana
  cliente:                    ana
  cubierto_por:               visa_platinum_oferta_2026_Q1
```

¿Por qué es vital hacer esto? Porque si en julio el banco lanza una versión nueva de la tarjeta que cuesta 10 USD, la tarjeta de Ana no puede subir de precio mágicamente; ella firmó el contrato de la oferta de Enero. Al tratar a los productos financieros como "objetos independientes congelados en el tiempo", el banco evita demandas multimillonarias por cambiar reglas sin avisar.

El antes y el después: del esquema fragmentado al grafo único

Antes (relacional). En un core bancario tradicional la información se reparte entre tablas como `clientes`, `cuentas`, `transacciones`, `asientos_contables` y `perfiles_riesgo`. La pregunta parece sencilla: *"¿Cuáles fueron las dos contrapartidas contables de la transferencia 001 y qué perfil de riesgo tenía Ana en ese momento exacto?"* Pero ya implica un JOIN de cuatro tablas —y aun así fracasa, porque `perfiles_riesgo` solo guarda el registro actual. Para saber qué decía el perfil la noche del cargo hay que buscar en tablas de auditoría separadas, si es que existen; de lo contrario, la respuesta sencillamente no existe en los datos. Eso no es un defecto de un banco en particular: es el defecto estructural del modelo plano frente a la bitemporalidad —qué ocurrió y qué sabía el sistema cuando ocurrió son dos ejes de tiempo distintos que el esquema clásico confunde en uno solo.

```

-- Antes: perfiles_riesgo solo guarda el estado ACTUAL – la historia se pierde.
CREATE TABLE transferencias (id INTEGER PRIMARY KEY, monto NUMERIC, fecha TEXT);
CREATE TABLE asientos (id INTEGER PRIMARY KEY, transferencia_id INT, tipo TEXT, monto NUMERIC);
CREATE TABLE perfiles_riesgo (cliente_id INTEGER PRIMARY KEY, nivel TEXT); -- sin vigencia

-- Las dos contrapartidas contables de la transferencia: sale bien.
SELECT tipo, monto FROM asientos WHERE transferencia_id = 1;

-- "¿Qué nivel de riesgo tenía Ana la noche del cargo?"
-- Imposible: la tabla solo tiene el valor de hoy, no el de entonces.
SELECT nivel FROM perfiles_riesgo WHERE cliente_id = 7;

```

Después (WQuestions). La misma información vive como un grafo de hechos. `transferencia_001` se liga por `parte_de` a `asiento_debito_001` y `asiento_credito_001`; `perfil_riesgo_ana_v3` lleva su propio rango de vigencia `inicio=2026-04-10`, `fin=2026-05-22`. La pregunta se convierte en un patrón de proyección: filtra los hechos cuyos extremos cuelgan de `transferencia_001` y cuyo intervalo temporal solapa con la marca de tiempo del evento. No hay JOIN complejo, no hay tabla de auditoría paralela —el tiempo es un atributo de primer orden en cada hecho— y no hay que migrar ningún esquema para que funcione.

Balance: El banco no logró tumbar al modelo

Este capítulo demostró que WQuestions no es un experimento de laboratorio. Observa todo lo que soportó sin trucos sucios:

1. **Múltiples agentes no humanos:** Sistemas informáticos tomando decisiones y asumiendo responsabilidades legales en el eje `Q`.
2. **Doble contabilidad paralela:** Eventos que se ramifican en la parte operativa y en la parte contable sin perder la conexión entre sí.
3. **Auditoría del pasado indestructible:** La regla de vigencia temporal (D6) permitió guardar el rastro de deudas y reglas obsoletas para blindar al banco ante juicios y auditorías.

A nivel conceptual, la arquitectura está lista para la guerra financiera corporativa.

Capítulo 18 — Un software ERP: el caso multi-módulo

Por qué un ERP estresa al modelo

El **ERP** (*Enterprise Resource Planning*) — el software que coordina la operación interna de una empresa mediana o grande — es un caso distinto a todos los anteriores. Lo que estresa al modelo no es ningún rasgo individual; es la **convivencia simultánea** de varios módulos que tradicionalmente se construyeron como sistemas separados:

- **Recursos humanos:** empleados, jerarquías, salarios, evaluaciones.
- **Finanzas y contabilidad:** asientos, presupuestos, cuentas por pagar y cobrar.
- **Inventario y almacén:** stocks, movimientos, valorizaciones.
- **Ventas:** pedidos, comisiones, clientes.
- **Compras:** órdenes, proveedores, aprobaciones.
- **Manufactura:** BOM (bill of materials), órdenes de producción, consumos.

En arquitecturas tradicionales — SAP, Oracle, Dynamics —, cada uno de estos módulos es un mundo aparte con su propio esquema, sus propias tablas, sus propios procesos. Las integraciones entre módulos son el principal proyecto de cualquier consultor de ERP, y representan, históricamente, el 60-70% del esfuerzo total de implementación. Si quieres saber **cuánta comisión generó la venta que movió tantas unidades del almacén central**, necesitas cruzar tres bases de datos distintas con sus propias reglas de identidad.

Lo que WQuestions promete acá — y vamos a demostrar — es que la integración entre módulos **no exige ningún sistema de unificación**: cuando todos los hechos del negocio comparten la misma estructura atómica (`suje``to`, `rol`, `valor`), el grafo *ya es* la integración. Una venta no genera "registros en tres tablas distintas"; genera **una situación reificada con tres sub-situaciones ligadas por parte_de**. Y todas viven en el mismo grafo.

Este capítulo modela seis escenarios reales de un ERP. Cada uno ilustra una dimensión donde el modelo paga su diseño.

La arquitectura conceptual del ERP en WQuestions

Antes de meternos en los escenarios, conviene aterrizar cómo se mapean los conceptos del ERP a las siete coordenadas:

- **Empleados, clientes, proveedores y la empresa misma** viven todos en **Q**. La empresa como persona jurídica es un agente igual de válido que un empleado humano (D5).
- **Productos, órdenes, asientos, comisiones, evaluaciones de desempeño** viven en **O**. Todo lo que tiene fecha de creación, historia y trazabilidad es una entidad de primera clase.
- **Departamentos, almacenes, sucursales, oficinas** viven en **L**. Forman una jerarquía territorial-organizacional.
- **Fechas, períodos, vencimientos** viven en **T**.
- **Cantidades vendidas, montos, horas hombre, kilos de materia prima** viven en **N**, siempre con su unidad anclada en **K**.
- **Tipos de producto, categorías de empleado, estados de orden, monedas** viven en **K**.

Las decisiones de diseño que más se ejercitan en un ERP son: **D3** (todo es tripleta atómica), **D4** (las operaciones que generan registros en varios módulos se reifican como una situación con sub-situaciones), **D6** (los estados de las órdenes, los salarios, los precios cambian en el tiempo y guardamos historial), **D7** (cada acción tiene su

`motivado_por` y su `justificado_por` — la auditoría es nativa, no se agrega encima) y **D8** (38 roles canónicos alcanzan; lo específico del ERP se extiende sin tocar el catálogo).

Caso 1 — La jerarquía organizacional

Todo ERP empieza por el organigrama. Una empresa tiene un director, varios gerentes, sus equipos, y a veces sub-equipos. En SAP, esto vive en una tabla `HRP1001` con un esquema gimnástico de "relations". En WQuestions, **es un solo rol**:

```
u.assert_fact(pedro, "reporta_a", maria)
u.assert_fact(laura, "reporta_a", maria)
u.assert_fact(maria, "reporta_a", carlos)
u.assert_fact(carlos, "reporta_a", empresa)
```

Cuatro tripletas. La jerarquía completa. Y la empresa también está en Q — es un agente legal igual de válido que Carlos.

La consulta "¿quién es el jefe del jefe de Pedro?" es un recorrido transitivo de dos saltos sobre el rol `reporta_a`. Si quieres "todo el árbol bajo Carlos", recorres recursivamente la relación inversa. No hace falta una tabla auxiliar de "transitive closure": el motor la calcula al vuelo. Y como `reporta_a` no está en el catálogo D8 (es un rol de dominio), el sistema lo acepta por **política liberal** sin protestar.

Asignar empleados a departamentos físicos es igual de simple:

```
u.assert_fact(pedro, "trabaja_en", dpto_ventas)
u.assert_fact(laura, "trabaja_en", dpto_produccion)
```

Cada departamento vive en L. Si más adelante quieres modelar que un departamento *contiene* sub-departamentos, usas `parte_de` exactamente como con cualquier otra jerarquía. Lo mismo aplica a regiones, sucursales, plantas de producción.

Caso 2 — Bill of Materials (BOM) recursivo

Acá el modelo brilla. Un BOM — la descomposición jerárquica de un producto en sus subproductos y componentes — es uno de los conceptos más típicamente difíciles de implementar en sistemas relacionales. En MRP clásicos exige tablas como `mast` + `matl` + `caps` con joins complicados.

Modelamos una bicicleta MTB: tiene un cuadro y dos ruedas; cada rueda tiene una llanta y un conjunto de rayos; el set de rayos está hecho de acero inoxidable. Cinco niveles de jerarquía anidada:

```
bicicleta_mtb — cuadro_aluminio
                |
                |— rueda_completa (x2) — llanta_26
                |                       |
                |                       |— rayos — acero_inox
                |
                |— ...
```

En WQuestions cada nivel es **una sola tripleta** con `parte_de`:

```
u.assert_fact(cuadro, "parte_de", bicicleta)
u.assert_fact(rueda, "parte_de", bicicleta)
u.assert_fact(llanta, "parte_de", rueda)
u.assert_fact(rayos, "parte_de", rueda)
u.assert_fact(acero, "parte_de", rayos)
```

La cantidad de cada subproducto se modela como un atributo del enlace, con su unidad:

```
u.assert_fact(rueda, "cantidad_en_bom", n(2, "unidades", "n_2_uds_rueda"))
```

Para responder "¿qué materia prima necesita la bicicleta?" basta con recorrer recursivamente `parte_de` hasta llegar a las hojas. El test del prototipo lo verifica:

```
BOM recursivo: acero → rayos → rueda → bicicleta  
cadena: mp_acero_inox → prod_rayos → prod_rueda_completa → prod_bicicleta_mtb
```

Lo mismo aplica a cualquier producto manufacturado: un automóvil, un electrodoméstico, un mueble modular, un componente electrónico. La recursión arbitraria del BOM no requiere ningún esquema especial — es un caso particular de `parte_de`.

Caso 3 — La venta cross-módulo: el patrón que vale oro

Acá viene el caso emblemático del ERP. Una sola venta — cinco bicicletas a BetAuto S.A. por 7.500 dólares — toca, al mismo tiempo, **tres módulos** del software empresarial:

- **Inventario:** hay que descontar 5 unidades del almacén central.
- **Contabilidad:** hay que registrar el ingreso de 7.500 USD con su asiento.
- **Recursos humanos:** el vendedor (Pedro) genera una comisión del 5% sobre la venta.

En un ERP tradicional, este flujo dispara tres procesos separados con tres registros en tres tablas distintas, ligados por *foreign keys* y unidos por convenciones de naming. La auditoría — "*explícame qué pasó con esta venta puntual*" — exige cruzar las tres bases.

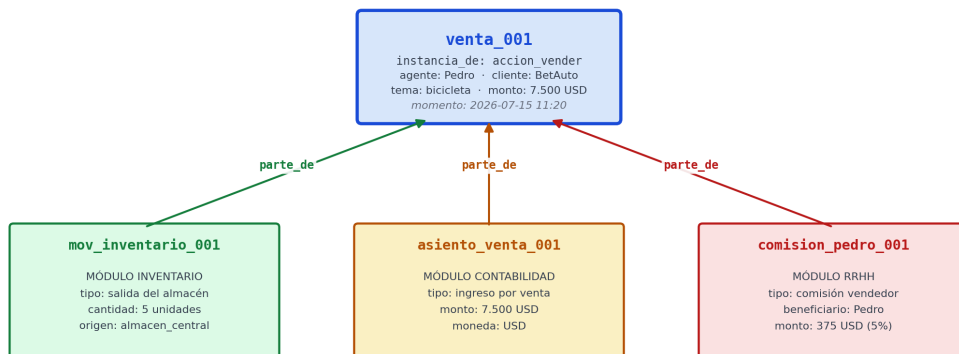
En WQuestions el modelado es directo: **una situación reificada con tres sub-situaciones:**

```
venta = ingest_situation(u, lex, "vender", roles={  
    "agente": pedro,  
    "tema": bicicleta,  
    "cliente": cliente_betauto,  
    "monto": n(7500, "USD"),  
    "unidad": usd,  
    "momento": at("2026-07-15T11:20"),  
    "lugar_de": dpto_ventas,  
}, sit_id="venta_001")  
  
# Sub-situación 1: movimiento de inventario  
u.assert_fact(mov_inv, "parte_de", venta)  
u.assert_fact(mov_inv, "tema", bicicleta)  
u.assert_fact(mov_inv, "cantidad", n(5, "unidades"))  
u.assert_fact(mov_inv, "origen", almacen_central)  
  
# Sub-situación 2: asiento contable  
u.assert_fact(asiento, "parte_de", venta)  
u.assert_fact(asiento, "monto", n(7500, "USD"))  
u.assert_fact(asiento, "tipo_movimiento", ingreso_por_venta)  
  
# Sub-situación 3: comisión del vendedor  
u.assert_fact(comision, "parte_de", venta)  
u.assert_fact(comision, "beneficiario", pedro)  
u.assert_fact(comision, "monto", n(375, "USD"))  
u.assert_fact(comision, "calculado_segun", politica_comisiones_v2)
```

Tres sub-situaciones, una situación articuladora, **ningún cruce de tablas necesario**. La consulta "*explícame todo lo que pasó con la venta 001*" es una proyección directa por `parte_de`:

La venta cross-módulo del ERP: un evento, tres módulos, cero ETL

Pedro vende 5 bicicletas a BetAuto por 7.500 USD. Lo que en un ERP tradicional dispara 3 procesos en 3 tablas, en WQuestions es una situación reificada con tres sub-situaciones ligadas por parte_de.



En un ERP tradicional: 3 tablas, 3 procesos, foreign keys, ETL para reportes cross-módulo.

En WQuestions: 3 sub-situaciones, una sola consulta `parte_de` para reconstruir todo el efecto de la venta.

La venta cross-módulo del ERP: una situación central (venta_001) con tres sub-situaciones — inventario, contabilidad y comisión — colgando por parte_de. En arquitecturas tradicionales esto exige tres tablas y un ETL para reportes integrados; en WQuestions el grafo ya integra los tres módulos sin intermediarios.

```
sub_situaciones = [
    f.subject for f in u.facts_with_role("parte_de")
    if f.value.id == "venta_001"
]
# devuelve [mov_inventario_001, asiento_venta_001, comision_pedro_001]
```

Y como cada sub-situación tiene su propia identidad, podemos hacer preguntas finas: "¿qué comisión generó esta venta?", "¿cuál es el asiento contable de esta operación?", "¿de qué almacén salió la mercancía?". Todas son recorridos de un salto. Sin ETL, sin sistema intermedio, sin tabla de unificación. **El grafo ya es la integración.**

Caso 4 — El workflow de aprobación bitemporal

Las órdenes de compra grandes — en este caso 48.000 USD de acero al proveedor — no se ejecutan al instante. Pasan por un ciclo: alguien las redacta, alguien las envía a aprobación, alguien con autoridad las aprueba (o rechaza). Cada estado tiene una fecha de entrada y una de salida.

En sistemas tradicionales, esto se modela con una columna `estado` que **se sobrescribe** en cada transición. Cuando alguien pregunta "¿en qué estado estaba esta OC el 20 de julio a las 13:00?", hay que revisar la tabla de auditoría — si existe — o estimar a partir de los logs.

En WQuestions el estado es un hecho con vigencia temporal (D6):

```
u.assert_fact(oc, "estado", borrador,
               valid_from=t_creacion, valid_to=t_envio_aprob)
u.assert_fact(oc, "estado", pendiente_aprobacion,
               valid_from=t_envio_aprob, valid_to=t_aprobacion)
u.assert_fact(oc, "estado", aprobada,
               valid_from=t_aprobacion)
```

Tres tripletas en lugar de tres updates destructivos. La consulta bitemporal "¿en qué estado estaba el 20 de julio a las 13:00?" devuelve `pendiente_aprobacion`; "¿y el 22 de julio?", devuelve `aprobada`. El historial completo es

nativo, no un módulo aparte.

La aprobación misma es una situación reificada con su agente y su justificación normativa (D7):

```
aprobacion = ingest_situation(u, lex, "aprobar", roles={
    "agente": carlos,
    "tema": oc,
    "momento": at("2026-07-21T15:30"),
    "justificado_por": politica_aprobacion_oc_v4,
}, sit_id="aprobacion_oc_001")
```

Carlos no aprueba "porque sí": aprueba **porque la política corporativa lo autoriza** para órdenes mayores a 30.000 USD. La política es otra situación reificada con su umbral y vigencia. Si mañana cambia el umbral, la nueva versión de la política se reifica como objeto nuevo y las aprobaciones anteriores siguen apuntando a la vieja — sin alterar el historial.

Caso 5 — La orden de producción que consume materia prima

La planta de Laura va a producir 50 bicicletas. Cada bicicleta consume cierta cantidad de acero (para los rayos) y cierta cantidad de horas-hombre. El ERP necesita registrar:

- La orden de producción como evento principal.
- Los **consumos** de materia prima asociados.
- Los **consumos** de mano de obra (horas-hombre) asociados.

El patrón es el mismo que la venta cross-módulo: una situación reificada con sub-situaciones de consumo:

```
op = ingest_situation(u, lex, "producir", roles={
    "agente": laura,
    "tema": bicicleta,
    "cantidad": n(50, "unidades"),
    "lugar_de": dpto_produccion,
}, sit_id="op_001")

# Consumo de materia prima
u.assert_fact(consumo_acero, "parte_de", op)
u.assert_fact(consumo_acero, "tema", acero)
u.assert_fact(consumo_acero, "cantidad", n(120, "kg"))

# Consumo de mano de obra
u.assert_fact(consumo_hh, "parte_de", op)
u.assert_fact(consumo_hh, "cantidad", n(180, "horas"))
u.assert_fact(consumo_hh, "ejecutado_por", laura)
```

Lo notable: la **misma estructura** modela bicicletas (donde el agente humano produce con materia prima) y, sin cambiar nada, podría modelar una reacción química industrial (donde *no hay agente humano*, los reactivos se transforman por sí solos siguiendo la estequiometría). El catálogo D8 ya admite que `agente` sea opcional cuando el verbo lo permite (D5).

La consulta *"¿cuántos kilos de acero consumió esta orden de producción?"* es una proyección directa: encuentra las sub-situaciones de `op_001` cuyo `instancia_de` sea `consumo_materia_prima`, filtra por `tema = acero`, suma los `cantidad`. Tres operaciones simples sobre el grafo. Cero ETL.

Caso 6 — El audit trail bitemporal del salario

Aquí va una situación que en cualquier ERP es notoriamente difícil. Pedro fue contratado el 10 de enero con un salario de 2.500 USD. El 1 de abril le subieron a 2.800. El 1 de agosto, tras una evaluación de desempeño con calificación "excelente", le subieron de nuevo a 3.200.

La pregunta que el modelo tiene que responder sin esfuerzo: **¿cuál era el salario de Pedro el 15 de marzo? ¿Y el 15 de junio? ¿Y el 15 de septiembre?** Estas son preguntas reales, las que un auditor laboral hace cuando revisa un legajo. En un ERP tradicional, requieren una tabla histórica auxiliar (`emp_salary_hist`) que la mitad de las implementaciones nunca termina de mantener.

En WQuestions, **tres tripletas** bastan:

```
u.assert_fact(pedro, "salario_mensual", n(2500, "USD"),
              valid_from=t_contrato, valid_to=t_aumento1)
u.assert_fact(pedro, "salario_mensual", n(2800, "USD"),
              valid_from=t_aumento1, valid_to=t_aumento2)
u.assert_fact(pedro, "salario_mensual", n(3200, "USD"),
              valid_from=t_aumento2)
```

Las tres conviven en el grafo. La consulta `query(..., at=fecha)` filtra por vigencia y devuelve el valor correcto:

```
marzo → 2500
junio → 2800
septiembre → 3200
```

El último aumento se conecta con su **causa motivacional** (D7): hubo una evaluación de desempeño con calificación excelente, reificada como objeto propio, y el ajuste de salario está `motivado_por` esa evaluación. Si el auditor pregunta *"¿por qué le subieron el salario a Pedro en agosto?"*, el grafo entrega la respuesta sin gimnasia.

Resultado del prototipo

El ejemplo Python que acompaña el capítulo (`prototipo/ejemplos/erp.py`) materializa los seis casos con 78 individuos repartidos en seis ejes y 93 hechos atómicos. Las once validaciones automáticas pasan:

```
✓ Jerarquía: Pedro reporta a María, María a Carlos
✓ BOM recursivo: acero → rayos → rueda → bicicleta
✓ Venta cross-módulo: 3 sub-situaciones (inventario + asiento + comisión)
✓ D6: OC pendiente_aprobacion el 20-Jul 13:00
✓ D6: OC aprobada el 22-Jul
✓ D7: la aprobación está justificada_por la política corporativa
✓ Producción: consume materia prima + mano de obra
✓ Audit trail bitemporal (D6): salario en marzo=2500, junio=2800, sept=3200
✓ WH: ¿quién aprobó la orden de compra?
✓ WH: ¿cuánto se le vendió a BetAuto?
✓ D5: ajuste de salario – agente=María, beneficiario=Pedro
✓ Política liberal: rol de dominio `calculado_según` admitido sin declarar

Resultado: 12/12 validaciones pasadas.
```

El antes y el después: del esquema fragmentado al grafo único

Antes (relacional). En un ERP tradicional, los módulos viven en islas: `employees` y `emp_salary_hist` en RRHH, `sales_orders` y `sales_commissions` en Ventas, `production_orders` y `wh_stock_movements` en Manufactura e Inventario. Una pregunta aparentemente simple — *¿cuántas horas-hombre de producción demandó cada unidad que Pedro vendió este trimestre, y cuánto cobró él de comisión por esa venta?* — obliga a cruzar cuatro tablas de

tres módulos distintos con JOINS frágiles, claves foráneas de convención y, casi siempre, una vista ETL mantenida a mano por el equipo de BI. Cualquier cambio de esquema en un módulo rompe silenciosamente el reporte.

Después (WQuestions). La misma información ya existe como un único grafo de hechos atómicos: `venta_001` es una situación reificada que contiene — por `parte_de` — la sub-situación de comisión de Pedro, la de movimiento de inventario y la de consumo de horas-hombre de la orden de producción asociada. La pregunta transversal es una proyección de tres saltos sobre el grafo: encuentra `venta_001`, recorre sus sub-situaciones, lee `monto` en la comisión y `cantidad` en el consumo de mano de obra. No hay silos porque nunca hubo fronteras de módulo: hubo una sola venta, y el grafo la modeló entera.

Qué quedó probado en este capítulo

El ERP es probablemente el dominio donde la promesa de WQuestions paga su mayor dividendo concreto: **la integración cross-módulo deja de ser un proyecto y se vuelve gratis**. La venta no genera "registros en tres tablas"; genera una situación con tres sub-situaciones. El BOM no necesita esquemas auxiliares; es un caso de `parte_de` recursivo. El audit trail bitemporal no es un módulo aparte; es la forma normal de modelar hechos con vigencia (D6). La aprobación no es un workflow externo; es una situación reificada con su justificación normativa (D7).

Y todo esto se modela con los mismos **38 roles canónicos** del catálogo D8 más un puñado de roles de dominio (`reporta_a`, `trabaja_en`, `calculado_según`, `cantidad_en_bom`, `ejecutado_por`) admitidos por la política liberal. El motor no creció ni una línea para absorber el ERP. Lo único que creció fue el lexicon — seis verbos nuevos (`vender`, `ordenar_compra`, `aprobar`, `producir`, `contratar`, `ajustar_salario`) con sus firmas.

Capítulo 19 — Una universidad: timelines largos y grafos de dependencia

Por qué la universidad es un dominio distinto

El ERP nos mostró cómo el modelo absorbe la integración cross-módulo de una empresa. La universidad pone sobre la mesa un tipo de complejidad muy distinta: **los tiempos son largos y las cosas dependen unas de otras**. Una carrera universitaria dura cinco años. Un estudiante atraviesa decenas de cursos, cada uno con sus prerrequisitos, en una secuencia que es parcialmente libre y parcialmente forzada. Las personas cambian de rol a lo largo de su vida académica: ayer eran estudiantes, hoy son ayudantes de cátedra, mañana son docentes, pasado son investigadores principales. Las calificaciones se asignan, se reclaman, se rectifican — y el historial completo dura **toda la vida del egresado**.

Cualquier sistema universitario serio tiene que poder responder cosas como:

- *¿Qué calificación recibió Ana en Matemática I antes de su reclamo? ¿Y después de la rectificación?*
- *¿Qué cursos puede tomar Luis ahora, dado lo que ya aprobó?*
- *¿En qué cursos estuvo inscripto Pablo en el segundo semestre de 2024?*
- *¿Quiénes fueron los miembros del jurado de la tesis de María?*
- *¿Qué docente dictó Estructuras de Datos entre 2020 y 2026?*

Cada una de estas preguntas es trivial **siempre que el modelo se diseñe pensando en bitemporalidad, en grafos de dependencia y en personas que cargan múltiples roles**. En arquitecturas tradicionales — un sistema de matrícula con tablas relacionales —, cada una requiere agregar una tabla histórica o forzar consultas con OUTER JOIN s creativos. En WQuestions, **cada una es una consulta de uno o dos saltos sobre el grafo**.

Este capítulo modela seis escenarios del mundo académico, cada uno estresando una dimensión distinta del modelo.

La arquitectura conceptual del dominio académico

Antes de meternos en los casos, el mapeo a las siete coordenadas:

- **Personas** (estudiantes, docentes, autoridades, miembros de jurado, la universidad misma como persona jurídica) viven en **Q**. Una sola persona puede aparecer en muchos roles distintos — D5 a pleno.
- **Carreras, semestres, cursos, inscripciones, evaluaciones, tesis, defensas, graduaciones** viven en **O**. Cada uno con identidad, momento de creación y trazabilidad.
- **Facultades, aulas, auditorios, campus** viven en **L** (forman jerarquías como cualquier organización territorial).
- **Fechas de inicio y fin de semestre, momentos de inscripción, fechas de defensa** viven en **T**.
- **Créditos, calificaciones numéricas, horas-semestre, montos de pago** viven en **N** con su unidad.
- **Tipos de carrera, estados de inscripción, modalidades de evaluación, niveles cualitativos (excelente/aprobado/desaprobado)** viven en **K**.

Las decisiones que más se ejercitan acá son: **D4** (la evaluación, la inscripción, la defensa son situaciones reificadas con sus participantes y momentos), **D5** (María es a la vez estudiante, asistente docente y tesista — el modelo no obliga a duplicar al individuo), **D6** (calificaciones, inscripciones y estados cambian en el tiempo y se preservan), **D7** (la rectificación está `motivado_por` el reclamo y `rectifica` la evaluación original; la graduación está `justificado_por` la defensa).

Caso 1 — La estructura académica jerárquica

La organización académica forma una jerarquía limpia que el modelo absorbe con `parte_de`:

```
carrera_ing_sistemas
├── primer_anio_is
│   ├── semestre_2026_i
│   │   ├── curso_mate1
│   │   └── curso_intro_prog
│   └── semestre_2026_ii
│       ├── curso_mate2
│       └── curso_estr_datos
```

Cada nivel es una entidad en O. Cada uno **es parte de** el nivel superior:

```
u.assert_fact(primer_anio, "parte_de", carrera_is)
u.assert_fact(sem_2026_i, "parte_de", primer_anio)
u.assert_fact(mate1, "parte_de", sem_2026_i)
```

Tres tripletas y la jerarquía completa de la carrera. Cuando un estudiante pregunta *"¿qué cursos hay en mi semestre actual?"*, la consulta es: encuentra todas las entidades en O cuya `parte_de` sea el semestre del estudiante. Un único salto.

Cada curso, además de ser parte de un semestre, lleva sus propios atributos: créditos (en N), docente (`agente` apuntando a Q), aula (`lugar_de` apuntando a L). Y por supuesto un nombre, un código, un syllabus. Todo el inventario académico vive con la misma forma.

Caso 2 — Prerequisitos como grafo dirigido acíclico (DAG)

El segundo gran patrón del mundo académico es **la malla curricular**: el conjunto de reglas que dicta qué cursos son requisito de qué otros. Es un DAG por naturaleza — sin ciclos, con muchas dependencias cruzadas.

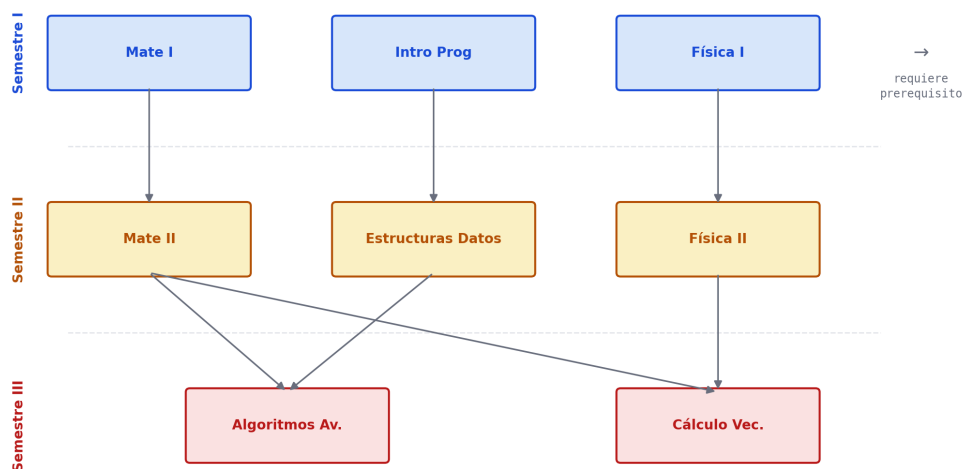
En sistemas tradicionales, esto se modela con una tabla `curso_prereq(curso_id, prereq_id)` y consultas recursivas con `WITH RECURSIVE`. En WQuestions, **cada prerequisito es una tripleta**:

```
u.assert_fact(mate2, "requiere_prerequisito", mate1)
u.assert_fact(estructuras_datos, "requiere_prerequisito", intro_prog)
u.assert_fact(algoritmos, "requiere_prerequisito", mate2)
u.assert_fact(algoritmos, "requiere_prerequisito", estructuras_datos)
```

Algoritmos Avanzados requiere **dos** prerequisitos simultáneos — uno de cada línea (matemática + programación). El modelo lo absorbe sin problema porque `requiere_prerequisito` es un rol multi-valor (M, no funcional).

DAG de prerequisites: la malla curricular como grafo

Cada curso es un individuo en *O*; cada arista es una tripleta (curso, requiere_prerequisito, otro_curso).
La inscripción a un curso solo es válida si todos sus prerequisites están aprobados.



Algoritmos Avanzados requiere DOS prerequisites simultáneos. La consulta "quiero inscribirme a Algoritmos" verifica que el grafo de aprobados del estudiante contenga ambos. Es un recorrido — no un join.

DAG de prerequisites académicos: cada curso es un individuo en *O*; cada arista es una tripleta requiere_prerequisito.

Algoritmos Avanzados depende simultáneamente de Matemática II y Estructuras de Datos, una dependencia que el modelo expresa con dos hechos atómicos independientes. La consulta "¿puedo inscribirme a Algoritmos?" es un recorrido sobre el grafo de aprobados del estudiante.

La consulta crítica del modelo — "¿qué cursos puede tomar Luis este semestre?" — se vuelve un recorrido directo sobre el grafo: para cada curso del semestre actual, verifica que **todos** sus prerequisites tengan, en el historial de Luis, una situación de evaluación con `estatus_factual = aprobado`. Si falta uno, el curso queda excluido. No hay tabla intermedia, no hay JOIN, no hay procedimiento almacenado. **Un recorrido y un filtro.**

Caso 3 — Inscripciones con vigencia (D6)

Ana se inscribe en Matemática I el 10 de marzo. Luis también, dos días después. Pero Luis abandona el curso a mediados de abril. ¿Cómo modelar esto sin perder información?

La trampa clásica es usar una columna `estado` en la tabla de matrículas y *actualizarla* cuando alguien abandona. El problema: si en julio alguien pregunta "¿quiénes estaban inscriptos el 1 de abril?", no hay forma de responder fielmente — la actualización borró el estado anterior.

En WQuestions, los dos estados **conviven** en el grafo, separados por vigencia:

```
u.assert_fact(inc_luis, "estado", vigente,
               valid_from=t_inscripcion, valid_to=t_cancelacion)
u.assert_fact(inc_luis, "estado", cancelada,
               valid_from=t_cancelacion)
```

La pregunta "¿estaba Luis inscripto el 1 de abril?" es directa: una consulta con `at=2026-04-01` filtra por vigencia y devuelve `vigente`. La misma consulta con `at=2026-05-01` devuelve `cancelada`. Sin esfuerzo extra, sin tabla histórica auxiliar. La inscripción nunca se borra; cambia de estado y el historial queda.

Caso 4 — Una persona, varios roles a lo largo del tiempo

Una verdad incómoda del mundo académico: las personas no son solo *una cosa*. María es estudiante de cuarto año Y, al mismo tiempo, jefa de práctica del curso de Introducción a la Programación. En unos años será docente titular; en otros, directora de tesis; en otros, decana. **Es la misma persona** — un solo individuo en Q —, pero ocupa roles distintos en situaciones distintas.

En sistemas tradicionales esto es notoriamente difícil. Una tabla `estudiantes` y otra `docentes` con `foreign keys` cruzadas. Y cuando un estudiante se vuelve docente, alguien tiene que sincronizar — o el sistema se confunde.

En WQuestions el problema **no existe**. María es `est_maria`, un individuo en Q. Aparece como agente en una inscripción (rol *estudiante*), como agente en una asignación TA (rol *jefe de práctica*), como agente en una tesis (rol *tesista*), como agente en una defensa (rol *defendiente*). Cada rol vive en su situación. El sistema no distingue entre "tipos de personas"; distingue entre tipos de situaciones.

```
# María como estudiante
ingest_situation("inscribir", roles={"agente": maria, "tema": estructuras_datos})

# María como jefa de práctica
u.assert_fact(asignacion_ta, "agente", maria)
u.assert_fact(asignacion_ta, "rol_funcional",
               category("jefe_de_practica"))

# María como tesista
ingest_situation("defender_tesis", roles={"agente": maria, "tema": tesis})
```

Cuando el sistema pregunta "*¿qué actividades hizo María este año?*", la consulta es una sola: buscá todas las situaciones donde María sea `agente` y proyectá su `instancia_de`. La respuesta sale del grafo de forma natural — y abarca lo académico, lo docente y lo investigativo en una sola lista.

Caso 5 — La cadena calificación → reclamo → rectificación

Acá viene el caso emblemático del audit trail académico. Ana rinde su examen final de Matemática I el 15 de julio. La Dra. López corrige y le pone 12 puntos. Ana revisa el examen, no está de acuerdo con la corrección de una pregunta, presenta un reclamo formal el 20 de julio. La docente revisa, le da la razón, y el 25 de julio rectifica la nota a 14 puntos.

En sistemas tradicionales, la nota se **sobreescribe**. El historial — si existe — vive en una tabla auxiliar que la mayoría de las facultades no consulta nunca. Cuando años después alguien quiere reconstruir el incidente — un decano que revisa una queja, un comité de calidad académica que audita —, el rastro está borroso.

En WQuestions, las tres situaciones **conviven** en el grafo:

1. La **evaluación inicial** es una situación reificada con su agente (la docente), su paciente (la estudiante), su `puntaje` y su `momento`.
2. El **reclamo** es otra situación, agente=Ana, `tema`=evaluación, `motivado_por`=motivo específico.
3. La **rectificación** es una tercera situación, agente=docente, `tema`=evaluación, `motivado_por`=reclamo, `rectifica`=evaluación.

```
rectificacion = ingest_situation("rectificar_nota", roles={
    "agente": dra_lopez,
    "tema": evaluacion,
    "puntaje": nota_14,
    "motivado_por": reclamo,
})
u.assert_fact(rectificacion, "rectifica", evaluacion)
```

Y la calificación vigente cambia con bitemporalidad (D6): la nota era 12 entre el 15 y el 25 de julio; desde el 25 es 14.

```
u.assert_fact(evaluacion, "puntaje_vigente", nota_12,
               valid_from=t_eval, valid_to=t_rect)
u.assert_fact(evaluacion, "puntaje_vigente", nota_14,
               valid_from=t_rect)
```

Las dos versiones existen en el grafo. La consulta `at=2026-07-18` devuelve 12; `at=2026-07-30` devuelve 14. El reclamo está en el grafo, sigue siendo consultable años después, y la cadena causal está explícita: la rectificación está motivado_por el reclamo, y rectifica la evaluación original. Si dentro de cinco años alguien revisa el caso de Ana, el grafo le entrega la historia completa sin gimnasia.

Caso 6 — Defensa de tesis con director y jurado

El último escenario es la defensa de tesis de María. Una tesis es una situación de larga duración (un año, en su caso) con múltiples participantes:

- La **tesista** (María, agente principal)
- El **director de tesis** (Dra. Díaz, rol específico del dominio)
- Tres **miembros del jurado** (Decano García como presidente, Dra. López como vocal, Dr. Morales como secretario)

En sistemas tradicionales, esto exige una tabla `tesis_participantes` con un `tipo` discriminador. En WQuestions, **cada rol es una tripleta separada**:

```
u.assert_fact(tesis, "agente", maria)
u.assert_fact(tesis, "director_tesis", dra_diaz)
u.assert_fact(defensa, "jurado_presidente", decano)
u.assert_fact(defensa, "jurado_vocal", dra_lopez)
u.assert_fact(defensa, "jurado_secretario", dr_morales)
```

`director_tesis`, `jurado_presidente`, `jurado_vocal`, `jurado_secretario` son roles de dominio académico. Ninguno está en el catálogo D8 — la política liberal los acepta sin protestar, y mientras el dominio universitario los use consistentemente, el modelo opera con ellos como si fueran canónicos.

La graduación final es una situación con `justificado_por` apuntando a la defensa: María se graduó **porque** defendió exitosamente su tesis. El grafo deja la cadena causal explícita.

Resultado del prototipo

El ejemplo Python que acompaña el capítulo ([prototipo/ejemplos/universidad.py](#)) materializa los seis casos con 80 individuos en 6 ejes y 102 hechos atómicos. Las once validaciones pasan:

```
✓ Jerarquía académica: curso → semestre → año → carrera
✓ DAG de prerequisites: algoritmos requiere mate2 + estructuras_datos
✓ Múltiples roles de una persona: María es estudiante Y TA Y tesista
✓ D6: inscripción de Luis vigente el 30-Mar, cancelada el 30-Abr
✓ Calificación bitemporal: nota=12 el 18-Jul, nota=14 el 30-Jul
✓ D7: rectificación motivada_por reclamo y rectifica evaluación original
✓ Defensa de tesis: 3 miembros de jurado con roles distintos
✓ Graduación justificada_por la defensa exitosa
✓ WH: ¿quién dicta Matemática I?
✓ WH bitemporal: ¿quiénes seguían inscriptos en Mate I el 1-Mayo?
✓ Política liberal: jurado_presidente / jurado_vocal / jurado_secretario aceptados sin declarar
```

Resultado: 11/11 validaciones pasadas.

El antes y el después: del esquema fragmentado al grafo único

Antes (relacional). Un sistema universitario típico distribuye su información entre tablas `alumnos`, `cursos`, `matrículas`, `prerequisitos` y `notas`. Preguntar *"¿qué cadena de prerequisitos arrastra Cálculo III y cuáles de ellos ya aprobó este estudiante?"* obliga a un `WITH RECURSIVE` sobre `prerequisitos` para expandir el árbol de dependencias, cruzado luego con `matrículas` y `notas` mediante múltiples `JOINS` — o, en los sistemas que lo evitan, a un procedimiento almacenado que replica en código la lógica de recorrido. Cada vez que la malla curricular cambia, el procedimiento hay que revisarlo. El historial borroso de calificaciones rectificadas suma otra tabla auxiliar al pegamento.

Después (WQuestions). La misma información vive como un solo grafo de hechos: cada prerequisito es una tripleta `requiere_prerequisito`, cada calificación aprobada es un nodo alcanzable desde el alumno, y la cadena completa de dependencias de Cálculo III se recorre con un simple recorrido hacia atrás sobre el grafo — sin SQL recursivo, sin procedimiento almacenado. Determinar qué cursos puede tomar un estudiante hoy es filtrar los cursos cuyos prerequisitos estén todos en su subgrafo de aprobados. El grafo no distingue entre malla vigente y malla histórica: la bitemporalidad ya lo cubre.

Qué quedó probado en este capítulo

El sistema universitario es probablemente el dominio donde el **timeline largo** y los **grafos de dependencia** se vuelven el problema central, no un detalle accesorio. El modelo absorbe ambas cosas con su maquinaria estándar: la bitemporalidad (D6) deja todas las inscripciones, calificaciones y estados con su historial intacto durante toda la vida académica del egresado. El catálogo D8 con su política liberal admite roles tan específicos como `jurado_secretario`, `director_tesis` o `requiere_prerequisito` sin que el motor tenga que cambiar nada. Una persona puede ser estudiante, docente y tesista a la vez — el grafo no distingue tipos de personas, distingue tipos de situaciones (D5).

Y, otra vez, **el motor no creció ni una línea** para absorber el dominio. Lo único que creció fue el lexicon — siete verbos (`inscribir`, `dictar`, `evaluar`, `reclamar`, `rectificar_nota`, `graduar`, `defender_tesis`) con sus signaturas, más un dialecto de dominio que mapea "estudiante", "docente", "tesista", "director_tesis" a sus roles canónicos.

Capítulo 20 — Una municipalidad: lo normativo en cada paso

Por qué una municipalidad estresa al modelo

En los dominios anteriores lo predominante era la operación: "lo que pasa" — el evento — era el centro de gravedad.

Una **municipalidad** introduce una dimensión distinta. Aquí, lo que sucede no se entiende sin la **norma que lo autoriza**. Una licencia de funcionamiento no se emite "porque sí": se emite **porque la ordenanza 237-2024 establece los requisitos y un funcionario verificó que se cumplieron**. Una multa de tránsito no se aplica caprichosamente: se aplica **porque el artículo 42 de la ordenanza 158-2023 lo habilita y el infractor cometió el supuesto que la norma describe**. Una resolución municipal no rectifica una multa por gusto: la rectifica **porque el recurso ciudadano fue declarado procedente conforme al artículo 15 del procedimiento administrativo**.

En todos estos casos, el "porque" del Estado tiene dos caras simultáneas: una cara **fáctica** (qué ocurrió en el mundo) y una cara **normativa** (qué regla lo autoriza). Cualquier sistema municipal serio tiene que registrar las dos, y poder responder por separado a preguntas como:

- *¿Por qué se aplicó esta multa?* — Por el hecho de estacionar en zona prohibida.
- *¿Bajo qué autoridad legal se aplicó esta multa?* — Por el artículo 42 de la ordenanza 158-2023.

WQuestions las separa exactamente con las relaciones de **D7**: `causado_por` apunta a la causa fáctica; `justificado_por` apunta a la norma. Las dos coexisten sin pisarse y, lo más importante, **el grafo permite reconstruir cualquier acto del Estado hacia atrás hasta su fundamento legal** con un par de saltos.

Este capítulo modela cinco escenarios típicos del gobierno local. Lo que vamos a ver es que la maquinaria del modelo ya alcanza para representar lo normativo sin extensiones especiales — basta con que las ordenanzas y los artículos vivan en O como cualquier otra entidad, y que las relaciones del "por qué" se usen sistemáticamente.

La arquitectura del dominio municipal

El mapeo a las siete coordenadas:

- **Funcionarios, alcalde, inspectores, policías municipales, ciudadanos, empresas** viven en **Q**. La municipalidad misma es persona jurídica (otro Q).
- **Ordenanzas, artículos, expedientes, solicitudes, licencias, multas, resoluciones, recursos, denuncias** viven en **O**. Cada uno con identidad propia y trazabilidad.
- **Distritos, barrios, manzanas, lotes, avenidas** viven en **L**, organizados en una jerarquía territorial vía un rol de dominio `dentro_de` ($L \rightarrow L$), porque `parte_de` canónico es $O \rightarrow O$.
- **Fechas de emisión, vigencia, infracción, resolución** viven en **T**.
- **Montos de multas, cantidades** viven en **N**.
- **Estados (en_revisión, aprobada, vigente, suspendida, revocada), conclusiones (procedente, improcedente), tipos de infracción** viven en **K**.

Las decisiones de diseño que más se ejercitan aquí son: **D7** (las dos relaciones del "por qué" operan en paralelo, una fáctica y una normativa), **D4** (cada acto administrativo — solicitud, inspección, emisión, denuncia, multa, resolución — es una situación reificada con identidad propia), **D6** (los estados de un expediente cambian en el tiempo y se preservan), y **D8** (la política liberal admite roles específicos como `dentro_de`, `vigente_en`, `emitida_por`, `tipo_solicitado` sin tocar el catálogo).

Caso 1 — Las ordenanzas como autoridad normativa

El primer punto importante: **las ordenanzas viven en el grafo igual que cualquier otra entidad**. Una ordenanza es un objeto en O con sus propios atributos: número, fecha de emisión, autoridad que la dictó, tema sobre el que regula. Sus artículos son sub-objetos vinculados por `parte_de` — la misma relación que usamos para BOM en el ERP o para la jerarquía académica en la universidad.

```
ordenanza_transito = u.add_individual(Individual(
    id="ord_158_2023_transito", axis=Axis.0,
    label="Ordenanza 158-2023 (tránsito)")

articulo_42 = u.add_individual(Individual(
    id="art_42_ord_158", axis=Axis.0,
    label="Artículo 42 (estacionamiento prohibido)")
u.assert_fact(articulo_42, "parte_de", ordenanza_transito)
u.assert_fact(articulo_42, "monto_multa", n(450, "USD"))
```

Una ordenanza tiene fecha de vigencia, autor (la municipalidad), y un tema normativo. Si mañana se modifica, la nueva versión es otra ordenanza con su propia identidad — y la modificación queda registrada como una relación entre ambas. La maquinaria de D6 garantiza que las multas aplicadas bajo la versión vieja sigan siendo trazables hacia el texto que las habilitó.

Caso 2 — Subdivisiones territoriales jerárquicas

La municipalidad gestiona territorio. El territorio se subdivide en distritos; los distritos en barrios; los barrios en manzanas; las manzanas en lotes. Cada uno es un punto en L. La jerarquía se establece con `dentro_de` :

```
u.assert_fact(distrito_centro, "dentro_de", ciudad_norte)
u.assert_fact(barrio_san_jose, "dentro_de", distrito_centro)
u.assert_fact(manzana_12, "dentro_de", barrio_san_jose)
u.assert_fact(lote_4, "dentro_de", manzana_12)
```

Nota técnica: `parte_de` está declarado en el catálogo D8 como $0 \rightarrow 0$. Para jerarquías de lugares ($L \rightarrow L$), usamos un rol de dominio `dentro_de` que la **política liberal** acepta sin validar — el motor no protesta y la semántica queda clara para humanos y LLMs. La cadena se puede recorrer recursivamente para responder cualquier consulta tipo "¿qué lote es este?" devolviendo `lote_4 → manzana_12 → barrio_san_jose → distrito_centro → ciudad_norte` con un solo recorrido.

Caso 3 — Una licencia de funcionamiento: solicitud → revisión → emisión

Café del Norte SAC quiere abrir un local de café en el lote 4 del barrio San José. El proceso real tiene tres actos administrativos encadenados, cada uno reificado:

1. **Solicitud**: la empresa solicita la licencia, indicando dónde quiere operar.
2. **Inspección**: un inspector municipal visita el local y certifica que cumple los requisitos.
3. **Emisión**: la municipalidad emite la licencia, válida por un año.

Cada acto es una situación con su `agente`, `momento`, `lugar_de` y resultado. Las tres se conectan con `motivado_por`: la inspección está `motivado_por` la solicitud; la emisión está `motivado_por` tanto la solicitud como la inspección. Y la emisión cierra con el cable más importante: `justificado_por`, apuntando a la ordenanza:

```

licencia = ingest_situation("emitir", roles={
    "agente": municipalidad,
    "tipo_emitido": category("licencia_funcionamiento"),
    "beneficiario": cafe_norte,
    "momento": at("2026-03-25T16:00"),
    "justificado_por": ordenanza_funcionamiento,
    "valido_desde": at("2026-04-01T00:00"),
    "valido_hasta": at("2027-03-31T23:59"),
})
u.assert_fact(licencia, "motivado_por", solicitud)
u.assert_fact(licencia, "motivado_por", inspeccion)

```

Lo notable: **el grafo permite responder con precisión** preguntas tipo *"¿bajo qué ordenanza fue emitida la licencia de Café del Norte?"* con un único salto desde la licencia siguiendo `justificado_por`. Y *"¿qué motivó la emisión?"* devuelve dos respuestas — la solicitud original y la inspección — porque `motivado_por` es multi-valor.

Mientras tanto, el estado de la solicitud cambia con vigencia (D6): estuvo `en_revision` entre el 5 y el 25 de marzo; quedó `aprobada` desde entonces. Ambos estados conviven en el grafo, y la pregunta *"¿en qué estado estaba la solicitud el 15 de marzo?"* tiene una respuesta exacta.

Caso 4 — Una denuncia ciudadana abre un expediente

Sofía Romero, vecina del barrio San José, denuncia que el local del Café del Norte genera ruidos molestos por la noche. La denuncia se modela como una situación con agente (Sofía) y `paciente` (el café denunciado), más un `tema` que reifica el ruido específico observado:

```

denuncia = ingest_situation("denunciar", roles={
    "agente": sofia,
    "tema": motivo_denuncia,      # ruido nocturno reificado
    "paciente": cafe_norte,
    "lugar_de": lote_4,
    "momento": at("2026-06-16T09:00"),
})

```

La denuncia dispara un **expediente administrativo** — otra entidad en O — que agrupa por `parte_de` todas las diligencias que el municipio realice para procesarla. La inspección nocturna que el inspector Quispe hace cinco días después es `parte_de` el expediente. Si después se agregan más diligencias (citación al denunciado, descargo, resolución), todas viven dentro del mismo expediente.

```

expediente = u.add_individual(Individual(id="exp_2026_001", axis=Axis.0))
u.assert_fact(expediente, "motivado_por", denuncia)
u.assert_fact(insp_nocturna, "parte_de", expediente)

```

El expediente como entidad articuladora es exactamente el patrón que vimos con el `viaje_001` del taxi y la `transferencia_001` del banco: una situación superior que reúne todos sus actos secundarios. La consulta *"¿qué diligencias se hicieron en este expediente?"* es una proyección directa por `parte_de`.

Caso 5 — Multa de tránsito con doble "por qué"

Acá viene el caso paradigmático del capítulo. Juan Mendoza estaciona su vehículo en zona prohibida sobre la Avenida Principal el 8 de julio a las 14:32. Tres minutos después, el policía municipal le aplica una multa de 450 dólares. El modelado registra la cadena completa:

- La **infracción** misma se reifica como una situación con tema (el vehículo de Juan), lugar (Av. Principal), momento, y tipo categórico (estacionamiento prohibido).

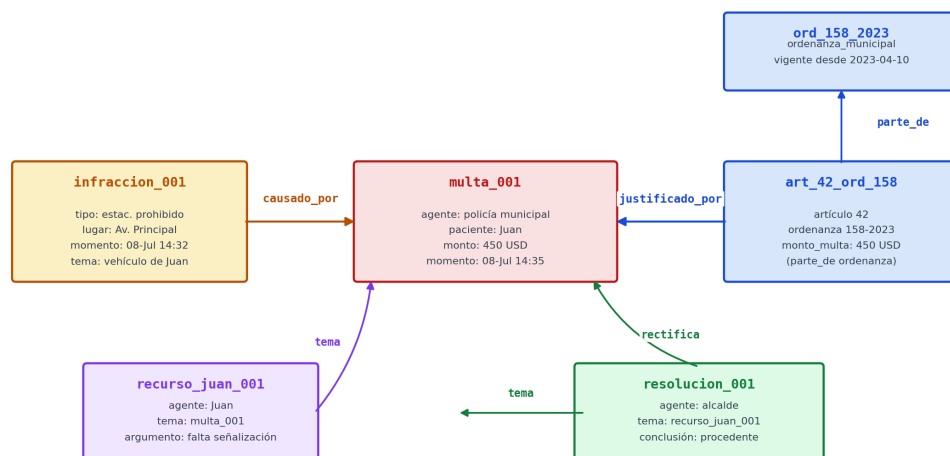
- La **multa** se reifica como otra situación, con su agente (el policía), paciente (Juan), monto, lugar, momento y — esto es lo crucial — **dos relaciones del "por qué"**:

```
multa = ingest_situation("multar", roles={
    "agente": policia_mun,
    "paciente": juan,
    "monto": n(450, "USD"),
    "lugar_de": av_principal,
    "momento": at("2026-07-08T14:35"),
    "causado_por": infraccion,      # CAUSA FÁCTICA
    "justificado_por": articulo_42, # AUTORIDAD NORMATIVA
})
```

Esta es la forma del modelo que solo D7 permite. La multa **es causada por** la infracción (sin ella no habría motivo para sancionar) y **es justificada por** el artículo 42 (sin él, el funcionario no tendría base legal para sancionar). Las dos relaciones son verdaderas a la vez y dicen cosas distintas. En sistemas tradicionales, esto se modela típicamente con una columna `motivo` que mezcla las dos cosas — un texto libre que el ciudadano no puede impugnar y el sistema no puede consultar limpiamente.

La cadena normativa de una multa municipal: dos `por qué` en paralelo

*Una sola multa se conecta con su causa fáctica (la infracción) Y con la norma legal que la autoriza.
D7 separa esos dos sentidos del 'porque' en relaciones distintas: causado_por y justificado_por.*



Toda acción del Estado conecta hacia atrás con una norma. La consulta `¿por qué se aplicó esta multa?` tiene dos respuestas válidas y distintas: la fáctica (causado_por) y la legal (justificado_por).

La cadena normativa de una multa municipal: dos por qué en paralelo. La multa está conectada por causado_por hacia la infracción fáctica (izquierda) y por justificado_por hacia el artículo de la ordenanza (derecha). Abajo, el recurso de reconsideración y la resolución que la deja sin efecto cierran el ciclo. Toda acción del Estado tiene fundamento factual y fundamento legal — D7 los registra como cables distintos.

Caso 6 — Recurso de reconsideración y rectificación

Juan no está de acuerdo con la multa. El 15 de julio presenta un recurso de reconsideración argumentando que la zona no estaba debidamente señalizada. El recurso se modela como una situación nueva:

```
recurso = Individual(id="recurso_juan_001", axis=Axis.0)
u.assert_fact(recurso, "agente", juan)
u.assert_fact(recurso, "tema", multa)      # apunta a la multa que impugna
u.assert_fact(recurso, "motivado_por", motivo_falta_senalizacion)
```

Tres semanas después, el alcalde resuelve el recurso. Lo declara **procedente**, lo que significa que la multa queda sin efecto:

```
resolucion = ingest_situation("resolver", roles={
    "agente": alcalde,
    "tema": recurso,
    "conclusion": procedente,
    "momento": at("2026-08-05T16:00"),
    "justificado_por": art_15_norma_proc_admin,
})
u.assert_fact(resolucion, "rectifica", multa)
u.assert_fact(multa, "estado", revocada,
               valid_from=datetime(2026, 8, 5, 16, tzinfo=timezone.utc))
```

Dos cosas pasan en paralelo: la resolución `rectifica` la multa (es la trazabilidad explícita: este acto cambió aquel acto), y el estado de la multa cambia a `revocada` desde el momento exacto de la resolución (D6). La multa **no se borra** del grafo — sigue ahí, sigue siendo consultable, sigue formando parte del expediente del ciudadano. Lo que cambia es su validez vigente. Y, gracias a D6, todavía podemos responder *"¿estaba esa multa vigente el 1 de agosto?"* (sí) y *"¿y el 10 de agosto?"* (no, revocada).

Esta es la promesa concreta del modelo para el sector público: **ningún acto administrativo se borra; cada uno deja rastro completo, con sus motivos, sus fundamentos, su resultado y su eventual rectificación**. Cualquier auditor — un órgano de control interno, un tribunal contencioso administrativo, un periodista de investigación — puede reconstruir el historial completo sin perderse en versiones inconsistentes de tablas que se sobrescribieron.

Resultado del prototipo

El ejemplo Python ([prototipo/ejemplos/municipalidad.py](#)) modela los seis casos con 75 individuos en seis ejes y 88 hechos atómicos. Las once validaciones pasan:

- ✓ Subdivisión territorial: lote → manzana → barrio → distrito → ciudad
- ✓ D7: licencia justificada_por la ordenanza 237-2024
- ✓ D7: licencia motivada_por solicitud + inspección
- ✓ D6: solicitud en_revision el 15-Mar, aprobada el 1-Abr
- ✓ Cadena causal de la multa: causado_por infracción + justificado_por art.42
- ✓ Expediente administrativo motivado_por la denuncia ciudadana
- ✓ Inspección nocturna parte_de el expediente 2026-001
- ✓ Recurso procedente → multa revocada (D6 + rectifica)
- ✓ Resolución rectifica la multa original
- ✓ WH: ¿quién emitió la licencia a Café del Norte?
- ✓ Política liberal: valido_desde/valido_hasta/vigente_en admitidos sin declarar

Resultado: 11/11 validaciones pasadas.

El antes y el después: del esquema fragmentado al grafo único

Antes (relacional). Un esquema municipal típico distribuye la información en tablas como `tramites`, `pasos_tramite`, `ordenanzas`, `multas` y `ciudadanos`. Las relaciones entre ellas se ensamblan con JOINS, pero hay una pregunta que el esquema casi nunca modela: *¿qué norma autoriza cada paso de un trámite?* El vínculo normativo — si existe — vive en una columna `observaciones` de texto libre, o en una clave foránea hacia `ordenanzas` que apunta solo al encabezado, sin distinguir qué artículo específico habilita la acción. Cuando el auditor pregunta *"¿bajo qué artículo el inspector Quispe declaró procedente la denuncia?"*, la respuesta exige rastrear código, leer comentarios y cruzar manualmente tablas que nunca se diseñaron para responder esa pregunta juntas.

Después (WQuestions). La misma información vive en un solo grafo de hechos, exactamente como lo modeló este capítulo. Cada paso administrativo — la solicitud, la inspección, la emisión de la licencia, la multa, la resolución — es una situación reificada que lleva sus propios cables del "por qué": `causado_por` apunta a la causa fáctica y `justificado_por` apunta al artículo de la ordenanza que lo habilita. La trazabilidad normativa no es código a mano ni un JOIN adicional — es un recorrido de grafo de un solo salto. Cualquier auditor, tribunal o periodista puede responder "*¿qué norma autorizó este acto?*" con la misma consulta que responde "*¿quién lo ejecutó?*" y "*¿cuándo?*".

Qué quedó probado en este capítulo

El dominio municipal pone a trabajar **D7 en su forma más exigente**: cada acto del Estado tiene fundamento factual y fundamento normativo, y el modelo los preserva como dos relaciones distintas que se pueden consultar independientemente. La pregunta "*¿por qué se aplicó esta multa?*" tiene dos respuestas válidas según se interprete el "porque" — y el modelo distingue cuál se está pidiendo.

Las **jerarquías territoriales en L**, los **expedientes que articulan múltiples diligencias**, y los **recursos que rectifican actos previos sin borrarlos** son patrones que aparecen también en otros dominios del sector público — sanitario, judicial, tributario, ambiental. Lo que probamos acá no es solo "el modelo absorbe una municipalidad"; es que **una familia completa de aplicaciones gov-tech encaja con el mismo motor**, sin extensiones especiales, sin proyectos de integración entre módulos administrativos.

Y, una vez más, **el motor no creció ni una línea**. Lo único que creció fue el lexicon — siete verbos administrativos (`solicitar`, `revisar`, `emitir`, `denunciar`, `inspeccionar`, `multar`, `resolver`) con sus signaturas, más un dialecto municipal que mapea "ciudadano", "vecino", "denunciante", "infractor" a sus roles canónicos.

Capítulo 21 — Una operación minera: lo físico, lo espacial y los eventos sin agente

Por qué la minera es distinta a todo lo anterior

Los dominios que llevamos modelados tienen algo en común que pasa desapercibido: **sucedan en oficinas, comercios o sistemas digitales**. Los eventos son acciones humanas o de software intencional: alguien vende, otra persona paga, una doctora diagnostica, un funcionario emite. Hasta cuando no hay agente humano explícito — la app del taxi asignando un conductor, el motor antifraude del banco, el sensor que monitorea el aire — siempre hay un **agente con intencionalidad** detrás de la acción.

Una operación minera de gran escala cambia esa premisa. En la minería pasan muchas cosas que **ningún agente "hace"**: la roca se desprende, el suelo cede, una pared geomecánica se debilita con el tiempo, una corriente subterránea altera la calidad del agua. Son eventos físicos puros que ocurren porque las condiciones materiales se cumplen. Y, sin embargo, son tan importantes como cualquier acción intencional — porque cuando esos eventos físicos lesionan a un trabajador o contaminan un río, **la cadena causal completa debe quedar trazada para auditoría, para responsabilidad legal y para prevención futura**.

Además, la minería trae otros desafíos propios:

- **Estructura espacial profunda:** yacimiento → tajo → nivel → banco → frente. Cinco niveles jerárquicos físicos, cada uno con sus propios metros cúbicos extraíbles y sus propias condiciones geomecánicas.
- **Equipos con ciclos de vida muy largos:** un camión CAT 793F cuesta tres millones de dólares y opera durante quince años, con docenas de mantenimientos preventivos y correctivos a lo largo de su vida útil.
- **Producción medida en múltiples unidades convertibles:** toneladas métricas de mineral extraído, gramos por tonelada de ley aurífera, onzas troy de oro fino producido. Cada cifra tiene su unidad y se convierte a la siguiente por multiplicación.
- **Turnos como entidades articuladoras:** una operación opera 24/7 dividida en turnos de 12 horas; cada turno reúne trabajadores, equipos, producción e incidentes en un mismo nodo de grafo.
- **Cumplimiento ambiental gatillado por umbrales:** cuando un sensor mide arsénico por encima del límite normativo, el reporte regulatorio se dispara automáticamente.
- **Comisionamiento y punchlist:** antes de que un equipo nuevo entre en operación comercial, atraviesa una batería de tests de aceptación con criterios de tolerancia, firmas múltiples y un seguimiento estricto de defectos pendientes clasificados por severidad.

Este capítulo modela seis escenarios donde el dominio minero pone a trabajar todas las decisiones de diseño que llevamos, en una combinación que ningún capítulo anterior había exigido a la vez.

La arquitectura conceptual del dominio minero

Mapeo a las siete coordenadas:

- **Operadores, supervisores, mecánicos, inspectores ambientales, sensores ambientales, la empresa y el regulador** viven en **Q**. Los sensores entran a Q por D5 — son agentes que registran mediciones.
- **Camiones, palas, turnos, extracciones, producción, mantenimientos, incidentes, mediciones, normas ambientales, reportes** viven en **O**.
- **Yacimiento, tajos, niveles, bancos, frentes de trabajo, estaciones de monitoreo, ríos** viven en **L** organizados jerárquicamente con `dentro_de` (rol de dominio L→L admitido por política liberal).

- **Fechas de adquisición de equipo, inicio y fin de turnos, momento de incidentes, fechas de mantenimiento** viven en **T**.
- **Toneladas, gramos por tonelada, onzas troy, miligramos por litro, dólares, años de vida útil** viven en **N** con su unidad.
- **Estados del equipo (operativo, en_mantenimiento, dado_de_baja), tipos de incidente, tipos de mantenimiento, minerales, contaminantes** viven en **K**.

Las decisiones de diseño que más se ejercitan acá son: **D5 al extremo** (eventos físicos sin ningún agente humano), **D7** (cadenas causales largas con `causado_por` sin intencionalidad y reportes regulatorios `justificado_por` normas ambientales), **D6** (estados del equipo a lo largo de quince años), y el patrón de **situación articuladora** que ya vimos en taxi y banco aplicado al turno.

Caso 1 — La estructura espacial profunda

Una mina no es un lugar; es **una jerarquía de lugares**. El nivel más alto es el yacimiento entero (el "Yacimiento San Marcos", por ejemplo). Dentro de él hay uno o más tajos (Tajo Norte, Tajo Sur). Cada tajo tiene niveles geográficos definidos por altitud (Nivel 4250 m.s.n.m., Nivel 4300). Cada nivel se subdivide en bancos (Banco 03, Banco 04 — escalones físicos del tajo). Y cada banco tiene varios frentes de trabajo activos al mismo tiempo (Frente A, Frente B, Frente C).

Modelamos exactamente igual que la subdivisión territorial de la municipalidad: un rol de dominio `dentro_de` que la política liberal acepta:

```
u.assert_fact(tajo_norte, "dentro_de", yacimiento)
u.assert_fact(nivel_4250, "dentro_de", tajo_norte)
u.assert_fact(banco_3, "dentro_de", nivel_4250)
u.assert_fact(frente_a, "dentro_de", banco_3)
```

La consulta *"¿dónde ocurrió este accidente exactamente?"* devuelve la cadena entera con un solo recorrido: `Frente A → Banco 03 → Nivel 4250 → Tajo Norte → Yacimiento San Marcos`. Toda la información geográfica jerárquica disponible para cualquier reporte de seguridad, cualquier auditoría regulatoria, cualquier análisis de productividad por zona.

Caso 2 — Equipo con ciclo de vida largo

Un camión minero Caterpillar 793F se adquiere en abril de 2018. Tiene una vida útil planeada de quince años. Durante esos años atraviesa muchos estados: opera, entra a mantenimiento programado, sale, vuelve a operar, sufre una falla, se repara, sigue operando. El modelado registra todo ese ciclo con vigencia temporal (D6):

```
u.assert_fact(camion_007, "estado", operativo,
              valid_from=t_op_inicial, valid_to=t_mant_2024)
u.assert_fact(camion_007, "estado", mantenimiento_prog,
              valid_from=t_mant_2024, valid_to=t_post_mant)
u.assert_fact(camion_007, "estado", operativo,
              valid_from=t_post_mant, valid_to=t_falla)
u.assert_fact(camion_007, "estado", mantenimiento_correctivo,
              valid_from=t_falla, valid_to=t_post_falla)
u.assert_fact(camion_007, "estado", operativo,
              valid_from=t_post_falla)
```

Cinco tripletas. El historial completo del camión a lo largo de ocho años. La consulta *"¿el camión 007 estaba operativo el 15 de marzo de 2024?"* devuelve `en_mantenimiento_programado`. *"¿Y el 1 de junio de 2026?"* devuelve `operativo`. La trazabilidad para auditoría es nativa, no un módulo añadido.

Caso 3 — Producción con múltiples unidades convertibles

La minería de oro registra producción en una cascada de unidades. Una extracción se mide en **toneladas métricas** de mineral. Ese mineral tiene una **ley** expresada en gramos de oro por tonelada (g/t). Multiplicando ambos se obtiene la cantidad bruta de oro contenido; tras el proceso metalúrgico se obtienen las **onzas troy** de oro fino vendible. Tres unidades distintas, cada una con su semántica:

```
extraccion = ingest_situation("extraer", roles={
    "agente": operador1,
    "extraido": mineral_oro,
    "monto": n(2400, "toneladas"),
    "unidad": tonelada_metrica,
    "lugar_de": frente_a,
    "ley_mineral": n(8.2, "g/t"),
    "ley_unidad": gramo_por_tonelada,
})
```

Y la producción de oro fino se modela como **sub-situación parte_de la extracción**, calculada del mineral y la ley:

```
u.assert_fact(produccion_oro, "parte_de", extraccion)
u.assert_fact(produccion_oro, "monto", n(632.7, "onzas_troy"))
u.assert_fact(produccion_oro, "unidad", onza_troy)
u.assert_fact(produccion_oro, "calculado_de", extraccion)
```

El cálculo ($2.400 \text{ t} \times 8.2 \text{ g/t} \div 31.1 \text{ g/oz} \approx 632.7 \text{ oz}$) se registra como una situación articuladora que apunta hacia atrás a la extracción con `calculado_de`. Esto importa: el grafo no pierde la trazabilidad. Si mañana alguien pregunta *"¿de qué extracción específica salieron estas 632.7 onzas?"*, la respuesta es directa. Si se descubre que la ley estaba mal medida y hay que recalcular, la nueva producción se modela como un hecho nuevo que **rectifica** el anterior — el modelo de auditoría minera del banco aplicado a la minería de minerales.

Caso 4 — El turno como entidad articuladora

Una operación minera trabaja 24 horas al día divididas en dos o tres turnos. Cada turno reúne un puñado de operadores, varios equipos y toda la producción y los incidentes que ocurran en ese lapso. Modelar el turno como entidad articuladora — exactamente como hicimos con el `viaje_001` del taxi o la `transferencia_001` del banco — permite que todo lo que pasa quede vinculado al mismo nodo:

```
turno = u.add_individual(Individual(id="turno_dia_2026_05_19", axis=Axis.0))
u.assert_fact(turno, "inicio", at("2026-05-19T06:00"))
u.assert_fact(turno, "fin", at("2026-05-19T18:00"))
u.assert_fact(turno, "lugar_de", tajo_norte)
u.assert_fact(turno, "supervisor", supervisor)
u.assert_fact(turno, "operador_asignado", operador1)
u.assert_fact(turno, "operador_asignado", operador2)
u.assert_fact(turno, "operador_asignado", operador3)
```

Y la extracción, la producción derivada, el accidente, las mediciones — todos quedan **parte_de** el turno:

```
u.assert_fact(extraccion, "parte_de", turno)
u.assert_fact(accidente, "parte_de", turno)
```

La consulta *"¿qué pasó en el turno del 19 de mayo?"* devuelve la cadena completa: tres operadores trabajaron, hubo una extracción exitosa de 2.400 toneladas, hubo un accidente en el Frente A, hubo mediciones ambientales. Todo en una sola proyección por `parte_de`. Sin tablas, sin joins. Solo el grafo.

Caso 5 — Cadena causal sin agente humano

Acá viene el caso emblemático del dominio. A las 11:40 del 19 de mayo, mientras el operador Quispe trabaja en el Frente A del Banco 03, **una roca se desprende de la pared del banco y lo golpea**. Sufre una contusión en el brazo derecho.

La pregunta crítica para auditoría: **¿quién causó el accidente?**

La respuesta honesta — y la que el grafo tiene que registrar fielmente — es: **nadie**. La roca se desprendió. Nadie la tiró. Pero la roca no se desprendió "de la nada" tampoco: hubo una condición previa, un debilitamiento estructural de la pared del Banco 03 que la inspección geomecánica anterior **debió** haber detectado.

El modelo lo registra con dos cadenas causales encadenadas, ninguna con agente:

```
# El debilitamiento es una condición geomecánica preexistente
debilitamiento = u.add_individual(Individual(
    id="debilitamiento_pared_b03", axis=Axis.0,
    label="Debilitamiento estructural Banco 03"))
u.assert_fact(debilitamiento, "instancia_de",
    category("condicion_geomecanica"))

# El desprendimiento es un evento físico SIN AGENTE,
# causado por el debilitamiento
desprendimiento = u.add_individual(Individual(
    id="evento_desprendimiento_001", axis=Axis.0))
u.assert_fact(desprendimiento, "lugar_de", frente_a)
u.assert_fact(desprendimiento, "momento", at("2026-05-19T11:40"))
u.assert_fact(desprendimiento, "causado_por", debilitamiento)
# Nota: NO HAY u.assert_fact(desprendimiento, "agente", ...)

# El accidente que sufre Quispe, causado por el desprendimiento
u.assert_fact(accidente, "paciente", operador1)
u.assert_fact(accidente, "causado_por", desprendimiento)
u.assert_fact(accidente, "parte_de", turno)
```

D5 (agencia contextual) lo absorbe sin esfuerzo: el rol `agente` simplemente **no aparece** en las situaciones físicas. Y D7 con `causado_por` reconstruye la cadena causal hacia atrás hasta su origen — la condición preexistente que el sistema geomecánico no detectó.

Cadena causal sin agente: un accidente minero rastreado hacia atrás

Nadie "hizo" el desprendimiento de roca. Sin embargo el grafo reconstruye la cadena causal completa:
 $\text{accidente} \leftarrow \text{desprendimiento} \leftarrow \text{debilitamiento}$. D5 admite eventos sin agente; "causado_por" (D7) hace todo el trabajo.



Comparativa — el modelo trata ambos casos con la misma forma:



Cualquier auditor de seguridad puede recorrer la cadena causal hacia atrás y producir el informe completo sin gimnasia.

Cadena causal sin agente: el accidente está conectado hacia atrás con `causado_por` al desprendimiento de roca, que a su vez está conectado al debilitamiento estructural. Ninguno de los tres tiene agente — son eventos físicos puros — y, sin embargo, el grafo permite reconstruir la responsabilidad completa.

Cualquier auditor de seguridad puede recorrer esta cadena hacia atrás con un solo recorrido sobre el grafo: "¿qué causó el accidente?" → desprendimiento. "¿Y qué causó el desprendimiento?" → debilitamiento estructural. "¿Y por qué no se detectó ese debilitamiento?" → siguiente capítulo de la investigación. La pregunta forense — ¿quién es responsable? — se vuelve precisa porque el grafo entrega la cadena exacta, no una explicación mezclada en texto libre.

Caso 6 — Reporte regulatorio ambiental disparado por umbral

Las operaciones mineras grandes están sujetas a regulación ambiental estricta. Sensores automáticos monitorean la calidad del aire y del agua en estaciones distribuidas por el yacimiento, midiendo concentraciones de contaminantes específicos cada cierto intervalo.

El 15 de mayo, el sensor de la Estación 3 mide 0.32 mg/L de arsénico en el río Quebrada Chica — **por encima del umbral normativo de 0.10 mg/L** establecido por la norma de calidad ambiental. La medición se reifica como cualquier otra situación, con el sensor como agente:

```
medicion_alta = ingest_situation("medir_calidad", roles={
    "agente": sensor_pm,          # sensor en Q (D5)
    "medido_de": arsenico,
    "monto": n(0.32, "mg/L"),
    "unidad": miligramo_por_litro,
    "lugar_de": estacion_monitoreo,
    "momento": at("2026-05-15T14:00"),
})
```

El sensor entra a Q como agente válido — D5 una vez más. La norma ambiental vive en O con su umbral declarado:

```
norma_ambiental = u.add_individual(Individual(
    id="ds_004_2017_minam", axis=Axis.0,
    label="DS 004-2017-MINAM (ECA agua)")
u.assert_fact(norma_ambiental, "umbral_arsenico", n(0.10, "mg/L"))
```


Cuando el motor de reglas (que no implementamos en este prototipo pero está documentado en el cap. 28) detecta que el valor medido supera el umbral, dispara automáticamente un reporte regulatorio:

```
reporte = ingest_situation("reportar", roles={
    "agente": minera_andes,          # la empresa reporta
    "tema": medicion_alta,
    "beneficiario": ente_regulador,  # al OEFA
    "momento": at("2026-05-15T18:00"),
    "motivado_por": medicion_alta,
    "justificado_por": norma_ambiental,
})
```

D7 una vez más con sus dos caras: el reporte está `motivado_por` la medición específica que superó el umbral, y `justificado_por` la norma que lo exige. El grafo registra ambas vinculaciones por separado. Si dentro de cinco años un auditor pregunta *"¿por qué se hizo este reporte ambiental?"*, hay dos respuestas precisas: por la medición concreta, y porque la norma DS 004-2017-MINAM lo exigía. No hay ambigüedad.

Caso 7 — Comisionamiento: el contrato técnico antes de la operación comercial

Hay una fase de cualquier proyecto minero — y, más en general, de cualquier proyecto industrial pesado — que merece su propia sección porque expone fricciones nuevas: el **comisionamiento**. Antes de que un equipo nuevo (un molino, una correa transportadora, una planta de procesamiento) entre en operación comercial, debe pasar por una batería de tests de aceptación. Cada test tiene un **criterio formal** (presión, caudal, temperatura, tonelaje, vibración) con sus tolerancias. Si el test pasa, lo firman varias partes — el contratista que entrega el equipo, el ingeniero del *owner* que recibe, el inspector de QA/QC que audita. Si el test no pasa, se levantan **punchitems** (defectos identificados que hay que resolver antes de la aceptación final).

Modelamos un escenario completo: el molino de bolas n° 005 entra en comisionamiento el 1 de abril de 2026. El paquete entero se reifica como una entidad articuladora superior, igual que el turno minero del caso 4:

```
comisionamiento = u.add_individual(Individual(
    id="comisionamiento_molino_005", axis=Axis.0))
u.assert_fact(comisionamiento, "tema", molino_005)
u.assert_fact(comisionamiento, "agente", contratista)
u.assert_fact(comisionamiento, "inicio", at("2026-04-01T08:00"))
```

Todo lo que pase durante el comisionamiento queda `parte_de` esta entidad: los tests, los sign-offs, los punchitems, los cierres de punchitems. Recorrer `parte_de` sobre el comisionamiento devuelve **el estado completo del proyecto**.

Acceptance criterion: el criterio separado del valor medido

Acá aparece una fricción que vale la pena nombrar. Un test de comisionamiento típico — la prueba hidrostática del molino — exige *"presión mayor o igual a 8 bar sostenida 30 minutos con una caída máxima del 2%"*. Es decir, es un **criterio con tolerancia numérica**: tres valores límite, tres valores reales medidos, y una comparación entre ambos para decidir si pasa o falla.

El modelo separa el criterio del resultado en dos entidades distintas:

```
# El criterio, como objeto reificado, vive por separado.
# Se puede versionar, citar y reusar para múltiples tests.
criterio_hidro = u.add_individual(Individual(
    id="criterio_hidrostatica_v1", axis=Axis.0))
u.assert_fact(criterio_hidro, "presion_minima", n(8.0, "bar"))
u.assert_fact(criterio_hidro, "duracion_minima", n(30, "min"))
u.assert_fact(criterio_hidro, "caida_maxima_pct", n(2.0, "%"))

# El test ejecutado, con sus valores reales y un puntero al criterio.
test_hidro = ingest_situation("ejecutar_test", roles={
    "agente": contratista,
    "tema": molino_005,
    "valor_medido": n(8.4, "bar"),
    "momento": at("2026-04-05T10:00"),
})
u.assert_fact(test_hidro, "verifica_criterio", criterio_hidro)
u.assert_fact(test_hidro, "caida_medida", n(1.3, "%"))
u.assert_fact(test_hidro, "duracion_medida", n(32, "min"))
u.assert_fact(test_hidro, "resultado", category("test_aprobado"))
```

Lo que el modelo **no hace** y conviene admitir: el cálculo $8.4 \geq 8.0 \wedge 1.3 \leq 2.0 \wedge 32 \geq 30 \rightarrow APROBADO$ no es un hecho atómico que el sistema deduzca solo. Es lógica que vive en un **motor de reglas externo** (Datalog, SHACL, o una función Python) que el cap. 28 declara como pendiente del proyecto. Por ahora el `resultado` se asienta explícitamente como hecho. Esta es la misma fricción que ya veremos con el marcador de fútbol en el cap. 23: el modelo aloja los datos, las reglas derivadas necesitan un motor que el prototipo todavía no trae.

Sign-off paralelo: tres firmas independientes sobre el mismo test

El test no pasa con una sola firma; necesita tres aprobaciones independientes — contratista, *owner*, QA — emitidas en momentos distintos y por agentes distintos. Cada aprobación es una **situación reificada propia** apuntando al mismo test por `tema`:

```
aprob_contratista = ingest_situation("aprobar_checkpoint", roles={
    "agente": contratista, "tema": test_hidro,
    "momento": at("2026-04-05T11:00")})
aprob_owner = ingest_situation("aprobar_checkpoint", roles={
    "agente": ing_owner, "tema": test_hidro,
    "momento": at("2026-04-05T15:30")})
aprob_qa = ingest_situation("aprobar_checkpoint", roles={
    "agente": qa_inspector, "tema": test_hidro,
    "momento": at("2026-04-06T09:00")})
```

La consulta *"¿quiénes firmaron este test?"* es una proyección directa: encuentra todas las situaciones de tipo `accion_aprobar_checkpoint` cuyo `tema` sea el test, proyecta su `agente`. Devuelve los tres. Cada firma tiene su propio momento, su propia conclusión y eventualmente sus propias observaciones. **El modelo no necesita una tabla `test_signoffs` con FK múltiples**; cada firma es un hecho independiente que apunta al test.

Severidad ordenada: una fricción que vale la pena nombrar

Los punchitems se clasifican típicamente en tres severidades: **A** (crítico — bloquea el comisionamiento), **B** (debe resolverse antes de operación comercial pero no la bloquea hoy), **C** (estético — no impide operar). La palabra clave de esa lista es **ordenada**: A es más severo que B, B más que C, y muchas reglas del proyecto dependen del orden ("el comisionamiento está listo cuando 0 items A están abiertos").

El modelo declara K como **categorías planas** — el catálogo D8 no trae noción de orden entre individuos de K. Esto es una fricción real, y conviene resolverla con un patrón explícito en lugar de inventar una pseudo-jerarquía. El patrón: **cada categoría de K lleva un atributo numérico `nivel_severidad` que define su posición en el orden**:

```

sev_a = u.add_individual(category("severidad_A"))
u.assert_fact(sev_a, "nivel_severidad", n(1, "ord"))

sev_b = u.add_individual(category("severidad_B"))
u.assert_fact(sev_b, "nivel_severidad", n(2, "ord"))

sev_c = u.add_individual(category("severidad_C"))
u.assert_fact(sev_c, "nivel_severidad", n(3, "ord"))

```

Ahora cualquier consulta que necesite ordenar — *"dame todos los punchitems abiertos ordenados por severidad descendente"* — tiene un campo numérico para hacer el `sort`. Es un patrón sencillo, pero merece quedar declarado en el libro porque es la solución general cuando un dominio necesita categorías ordenadas (severidades, prioridades, niveles, grados de calidad, escalas de Likert). El modelo no impone el orden; el dominio lo declara explícitamente.

Punchitem: tres roles Q distintos sobre la misma entidad

Un punchitem tiene una particularidad: involucra a **tres personas distintas con tres roles distintos**:

- **Quien lo encontró** (típicamente el QA en la inspección).
- **El responsable** de resolverlo (contratista o vendor).
- **El verificador** que cierra el ítem una vez resuelto (típicamente el *owner*).

```

punch_001 = ingest_situation("registrar_punchitem", roles={
    "agente": qa_inspector,      # quien lo encontró
    "tema": fuga_aceite,         # qué se encontró
    "equipo_afectado": molino_005, # qué equipo lo tiene
    "severidad": sev_a,
    "responsable": contratista,
    "verificador": ing_owner,
    "fecha_limite": at("2026-04-22T00:00"),
})

```

Tres roles Q sobre la misma entidad — encontrado, responsable, verificador — más uno O (`equipo_afectado`, rol de dominio porque `paciente` exige Q). El catálogo D8 no trae `responsable` ni `verificador`; la política liberal los admite sin protestar. El modelo no necesita ningún concepto nuevo; basta usar la maquinaria estándar con roles específicos del dominio.

El estado del punchitem evoluciona con D6:

```

u.assert_fact(punch_001, "estado", abierto,
    valid_from=t_inspeccion, valid_to=t_inicio_trabajo)
u.assert_fact(punch_001, "estado", en_progreso,
    valid_from=t_inicio_trabajo, valid_to=t_cierre)
u.assert_fact(punch_001, "estado", cerrado,
    valid_from=t_cierre)

```

Y el cierre del ítem es otra situación reificada que `rectifica` el punchitem, ligada a la reparación que lo resolvió:

```

cierre = ingest_situation("cerrar_punchitem", roles={
    "agente": contratista,
    "tema": punch_001,
    "verificado_por": ing_owner,
    "motivado_por": reparacion_ejecutada,
})
u.assert_fact(cierre, "rectifica", punch_001)

```

Igual que en el rediagnóstico médico del cap. 16 y la rectificación de multa del cap. 20: el item original **no se borra**, queda en el grafo con su historial. Si dentro de cinco años un auditor revisa el comisionamiento, encuentra que el punchitem 001 fue abierto el 15 de abril, atendido entre el 16 y el 20, y cerrado el 20 — con la reparación específica que lo motivó. Trazabilidad nativa.

Estado agregado derivado: el comisionamiento "está listo"

La última fricción que aparece es la más interesante. Una pregunta de negocio típica: *"¿está el molino listo para operación comercial?"* La regla del proyecto: **sí, si y solo si hay cero punchitems severidad A abiertos**. Es decir, el estado del comisionamiento entero **se deriva** del estado de sus partes.

El modelo permite consultar todos los punchitems abiertos clase A con una proyección directa:

```
abiertos_A = 0
for situ in u.facts_with_role("instancia_de"):
    if situ.value.id == "accion_registrar_punchitem":
        estado_actual = [...] # filtra por at=hoy
        severidad = [...]
        if estado_actual != "cerrado" and severidad == "severidad_A":
            abiertos_A += 1

listo_para_operacion = (abiertos_A == 0)
```

Pero — y acá viene la fricción honesta — **el resultado "listo_para_operacion" no es un hecho atómico que el grafo guarde**. Es lógica derivada que se computa al vuelo, igual que el marcador del partido de fútbol (cap. 23) o el balance de una cuenta bancaria (cap. 17). El cap. 28 marca esta pieza — el motor de inferencia que evalúa reglas declarativas sobre el grafo y materializa resultados derivados — como el primer trabajo pendiente del proyecto.

Lo que el modelo **sí** garantiza es que los datos para esa derivación están ahí, completos, sin pérdida. La regla puede ejecutarse en Python ad-hoc, en Datalog cuando llegue el motor, o en un LLM que entienda el lexicon. La separación entre "el dato como hecho" y "el resultado derivado por regla" es deliberada y limpia.

Resultado del prototipo

El ejemplo Python ([prototipo/ejemplos/minera.py](#)) modela los nueve casos con 139 individuos en seis ejes y 172 hechos atómicos. Las 18 validaciones pasan:

- ✓ Jerarquía espacial: frente → banco → nivel → tajo → yacimiento
- ✓ D6: camión operativo→mantenimiento→operativo (a lo largo de años)
- ✓ Turno con 3 operadores asignados (rol de dominio multi-valor)
- ✓ Producción con múltiples unidades: 2400 t @ 8.2 g/t
- ✓ Producción de oro calculada como sub-situación parte_de extracción
- ✓ D5: desprendimiento de roca SIN agente humano, con causa física
- ✓ Cadena causal: accidente ← desprendimiento ← debilitamiento estructural
- ✓ Accidente parte_de el turno articulador
- ✓ D7: reporte ambiental motivado_por medición + justificado_por norma
- ✓ D5: sensor de calidad de aire como agente Q (no humano)
- ✓ WH: ¿cuánto mineral extrajo el operador Quispe?
- ✓ Política liberal: ley_mineral / ley_unidad / extraído admitidos sin declarar
- ✓ Acceptance criterion separado del valor medido (8 bar mín. vs 8.4 bar real)
- ✓ Tres sign-offs paralelos sobre el mismo test (contratista + owner + QA)
- ✓ Severidad ordenada en K vía atributo `nivel\_severidad`: A=1 < B=2 < C=3
- ✓ Punchitem con 3 roles Q distintos: agente + responsable + verificador
- ✓ D6: punchitem A pasó por abierto → en_progreso → cerrado
- ✓ Estado agregado derivado: comisionamiento OK cuando 0 punchitems A abiertos

Resultado: 18/18 validaciones pasadas.

Qué quedó probado en este capítulo

La minería pone a trabajar a **D5 en su forma más radical**: hay eventos físicos puros — el desprendimiento de roca, el debilitamiento estructural — que **ningún agente humano hizo**, y sin embargo son tan importantes como cualquier acción intencional. El modelo los acepta sin gimnasia porque la signatura no exige agente; basta con `causado_por` para reconstruir la cadena causal completa hasta su origen.

La **estructura espacial profunda** (cinco niveles jerárquicos) se modela igual que las subdivisiones territoriales municipales: con un rol de dominio `dentro_de` admitido por política liberal. El **equipo con ciclo de vida de 15 años** queda trazable con la maquinaria bitemporal de D6, que ya usamos en el banco para préstamos y en la universidad para salarios y calificaciones. Los **sensores como agentes no humanos** son la misma idea que la app del taxi y el motor antifraude del banco, aplicada al monitoreo ambiental. Los **reportes regulatorios gatillados por umbrales** son el patrón de D7 que ya vimos en la municipalidad, ahora con normas ambientales en lugar de ordenanzas.

Las **fricciones nuevas** que aportaron commissioning y punchlist quedaron resueltas con patrones ya conocidos, pero merecen estar nombradas porque aparecerán en cualquier proyecto industrial pesado:

- **Acceptance criterion con tolerancia separado del valor medido** — el criterio vive como objeto reificado con sus límites; el test apunta al criterio por `verifica_criterio` y registra sus valores reales aparte. El "pasa/falla" sigue siendo lógica de regla externa.
- **Sign-off paralelo de múltiples partes** sobre el mismo test — cada firma es una situación reificada con `agente` distinto que apunta al test por `tema`. Tres firmas, tres situaciones, una sola consulta para reconstruirlas.
- **Severidad ordenada en K** vía un atributo numérico `nivel_severidad` — patrón genérico para cualquier categoría que necesite orden (prioridades, grados, escalas Likert). El catálogo K no impone orden; el dominio lo declara.
- **Tres roles Q distintos sobre la misma entidad** (encontrado / responsable / verificador) — la política liberal acepta los roles de dominio sin protestar, y el grafo conserva las tres perspectivas separadas para auditoría.
- **Estado agregado derivado** — el "comisionamiento listo" no es un hecho atómico, es el resultado de una regla evaluada sobre los punchitems clase A abiertos. El modelo guarda los datos completos; el motor de inferencia que cap. 28 marca como pendiente es el encargado de materializar los derivados.

Es decir: **el dominio minero no exigió ninguna pieza nueva del modelo**. Lo absorbió combinando piezas que ya conocemos. El motor sigue siendo el mismo. Lo único que crece es el lexicon (cinco verbos: `extraer`, `operar_equipo`, `dar_mantenimiento`, `medir_calidad`, `reportar`) y la lista de roles de dominio (`dentro_de`, `ley_mineral`, `operador_asignado`, `calculado_de`, `umbral_arsenico`, etc.) que la política liberal acepta sin chistar.

El antes y el después: del esquema fragmentado al grafo único

Antes (relacional). Una operación minera tradicional distribuye su información entre tablas separadas — `equipos`, `estados_equipo`, `eventos`, `inspecciones_geomecanicas`, `mediciones_sensor` — y ya aparece la primera grieta: reconstruir la cadena causal de un accidente que no tuvo agente humano obliga a cruzar al menos cuatro tablas con JOINS parciales, y aun así el esquema no tiene dónde guardar la relación entre el debilitamiento estructural previo y el desprendimiento posterior sin inventar columnas ad hoc. El historial de estados de una excavadora a lo largo de quince años tampoco cabe limpio: la tabla `estados_equipo` acumula filas sin semántica de vigencia y cualquier consulta — *¿cuál era el estado exacto del camión el 15 de marzo de 2024?* — requiere lógica extra que el esquema no provee.

Después (WQuestions). La misma información vive en un único grafo de hechos donde cada evento físico — el debilitamiento, el desprendimiento, el accidente — es un nodo conectado al siguiente por `causado_por`, y la cadena

se recorre hacia atrás con un solo recorrido sin necesidad de agente humano en ningún punto. Los quince años de estados de la excavadora son vigencia nativa (D6): cada tripleta (`camion_007`, `estado`, `operativo`) lleva su propio `valid_from` y `valid_to`, y la consulta por fecha devuelve el estado correcto sin lógica adicional. El modelo no necesita tablas especiales para causalidad sin culpable ni historiales improvisados para equipos de larga vida — los absorbe con la maquinaria estándar.

Cierre de la serie de dominios profundos

Con la minera cerramos la **serie completa de ocho dominios industriales** modelados en profundidad: Spa, taxi, clínica, banco, ERP, universidad, municipalidad, minera. Cada uno empujó al modelo en una dimensión distinta y cada uno fue absorbido sin extensiones especiales. La arquitectura conceptual no cambió ni una línea entre el Spa más simple y la minera más exigente — lo único que cambió fue el lexicon y los roles de dominio.

Pero un libro honesto debe ir más allá de los casos cómodos. Conviene ahora cambiar el ritmo y someter al modelo a cuatro dominios cualitativamente distintos —música, química, fútbol y contratos— buscando explícitamente dónde se resiste: algunos se resuelven con elegancia; otros exigen extensiones al catálogo o dejan pendientes que documentamos sin disimular. Es donde el libro deja de mostrar lo que funciona y empieza a admitir lo que duele, porque solo conociendo los límites reales del modelo se sabe dónde aún falta trabajo.

Capítulo 22 — Llevando WQuestions a un sistema en producción

El otro lado del espejo

En el capítulo 14 modelamos el Spa Oasis desde cero. Cuatro personas, una página de descripción, ciento ochenta y cuatro hechos atómicos. Era un ejercicio limpio: la pizarra en blanco, el modelo aplicado sin obstáculos. Eso es valioso para entender qué se puede hacer. Pero es poco honesto si el libro se queda ahí, porque **la mayoría de los lectores no tiene una pizarra en blanco**. Tiene un ERP de diez años, un CRM heredado, una base de datos relacional con tres mil tablas, o un sistema artesanal en PHP que mueve la empresa desde hace una década.

La pregunta operativa de ese lector no es *"¿cómo modelaría esto desde cero?"* sino *"¿cómo aplico WQuestions sin romper lo que ya funciona?"*. Este capítulo responde esa pregunta usando un caso real: **yaku**, un sistema MySQL que opera diariamente cinco líneas de negocio — sauna, hostel, gimnasio, spa y cafetín. Es el sistema operativo de un negocio real, con sus ~2.000 productos en catálogo, sus ~125.000 ventas históricas, sus dialectos locales del staff y sus decisiones de diseño hechas hace años por razones que ya nadie recuerda. Exactamente el escenario común.

Lo que sigue no es modelar yaku — es **mapearlo** a WQuestions sin tocarlo. La diferencia es categórica.

yaku en media página

yaku corre sobre MySQL 5.7. Sus tablas centrales son las que un negocio de servicios siempre tendrá:

- `cliente` — quiénes consumen.
- `persona` — empleados (recepción, masajistas, gimnasio).
- `producto` — todo lo vendible: habitaciones, sesiones de sauna, jugos, gaseosas, planes mensuales.
- `venta` + `ventadet` — comandas y sus líneas.
- `asistencia` — registro de entrada/salida del gimnasio (con banderas anexas para sauna y ducha).
- `socio` + `plan` — membresías mensuales activas.

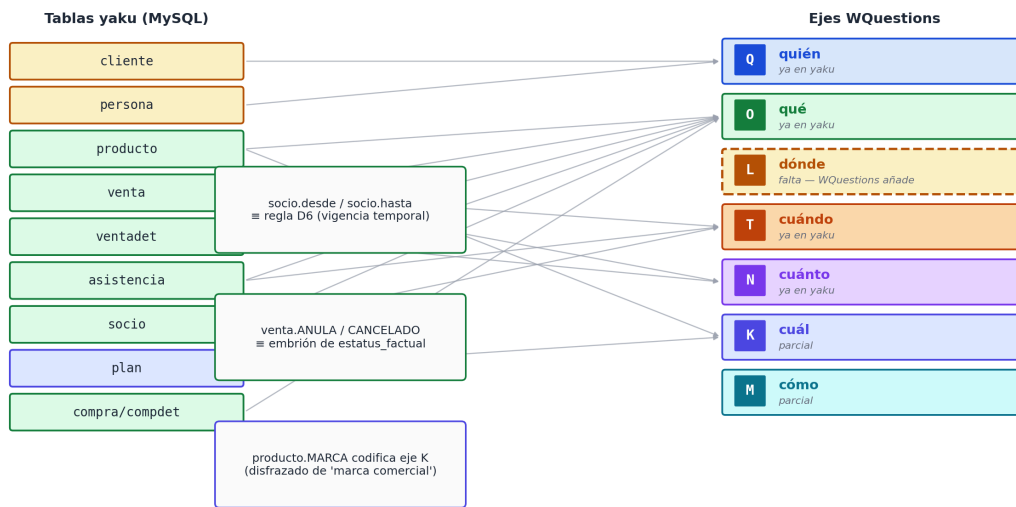
Y un detalle revelador: el campo `producto.MARCA` no guarda la marca comercial del producto. Guarda su **familia de negocio**: HOSTAL, SAUNA, GYM, RESTAURANT, STOCK. Es decir, yaku **ya** clasifica sus productos en categorías — solo que con un nombre que esconde su función real. Esto es el primer hallazgo de lo que llamaremos *arqueología semántica*.

Arqueología semántica: qué de WQuestions ya está en yaku, sin que nadie lo haya bautizado así

El ejercicio de mapeo arranca con una pregunta inocente: para cada tabla de yaku, ¿en qué eje vive su contenido? El resultado es la figura 49, que merece detenerse en él:

Arqueología semántica: las tablas de yaku proyectadas a los 7 ejes

Cajas sólidas = ya implementado en yaku (informalmente). Cajas punteadas = lo que WQuestions añade.



Lo que falta (cajas punteadas): vigencia histórica de propiedades del cliente, modalidades de cobertura, justificaciones formales.

Arqueología semántica de yaku: cada tabla del MySQL existente proyecta su contenido a uno o varios de los 7 ejes WQuestions. Los rectángulos llenos marcan implementaciones existentes en yaku; los rectángulos punteados marcan lo que falta y WQuestions añadiría.

Tres observaciones del mapa son las que justifican todo el capítulo:

Primera — yaku ya implementa D6 (vigencia temporal) sin saberlo. Los campos `socio.desde` y `socio.hasta` son exactamente el patrón que el cap. 9 prescribe para hechos cambiables: la membresía se modela como evento con inicio y fin, no como atributo del cliente. Cuando un socio renueva, no se sobrescribe — se crea un nuevo registro. Una consulta del tipo "¿qué socios estaban activos el 15 de marzo del año pasado?" responde correctamente, sin reconstrucciones aproximadas.

Segunda — yaku ya tiene un embrión de estatus_factual. Los flags `venta.ANULA` y `venta.CANCELADO` son la versión cruda del eje que en el cap. 14 distinguíamos como {real, intencionado, planeado, hipotético, cancelado}. yaku cubre el extremo "esto no pasó" (anulado, cancelado), pero le falta el otro extremo: el intencionado — el cliente que casi compró el plan trimestral y se fue sin firmar. Ese caso hoy se anota en `cliente.NOTAS` como prosa libre, no consultable.

Tercera — el eje L está parcialmente externalizado. yaku no tiene tabla de lugares físicos. Las habitaciones del hostel viven como `producto` (código H103, H201, etc.). Eso es una **decisión de modelado** que el cap. 14 prevé: una entidad puede vivir en O y en L a la vez (la cámara de vapor del Oasis era a la vez máquina y lugar). yaku eligió priorizar O para no complicar el flujo de ventas. WQuestions puede recuperar la cara L sin cambiar yaku — solo agregando, en el lexicon, que cada código de habitación es además un individuo del eje L.

El patrón general: **los sistemas legacy se aproximan a WQuestions a pedazos, sin nombrarlo.** La arqueología consiste en detectar qué pedazos están bien, qué pedazos están a medias, y qué pedazos faltan. No es una migración — es un diagnóstico.

El concepto nuevo: modalidades de cobertura

En el cap. 14 introducíamos el `estatus_factual` para distinguir lo real de lo intencionado. Esa distinción es importante pero responde a una pregunta única: ¿esto pasó o no? En yaku surge otra dimensión ortogonal que no habíamos formalizado en el libro hasta ahora: **¿cómo se cobró/se cubrió este servicio?**

En el negocio real hay cuatro modalidades distintas, todas conviviendo el mismo día:

- **Pago directo** — el cliente walk-in entra al sauna, paga 25 soles, sale. La venta cierra en sí misma.
- **Cubierto por plan** — la socia mensual entra a entrenar; su asistencia se registra pero `IMPORTE = 0`. El derecho viene del contrato vigente, no del pago del día.
- **Cargo a estancia** — el huésped del hostel pide una cerveza en el cafetín y la cargan a la habitación. El consumo es real, pero el cobro queda agrupado bajo la estancia.
- **Cortesía** — la recepcionista le regala una sauna al cliente que cumple años. El servicio ocurre, no hay cobro, hay un motivo.

Las cuatro modalidades comparten el mismo evento factual ("se usó el sauna") pero responden de forma distinta a la pregunta del cobro. En yaku, distinguirlas requiere cruzar `asistencia.IMPORTE`, presencia de `socio` activo y existencia de una `venta` asociada — tres consultas frágiles. En WQuestions se vuelve un único cable explícito, que llamaremos `cubierto_por`:

```
(uso_sauna_001, instancia_de, servicio_sauna)
(uso_sauna_001, cliente, juan)
(uso_sauna_001, cubierto_por, pago_directo)          ← caso 1

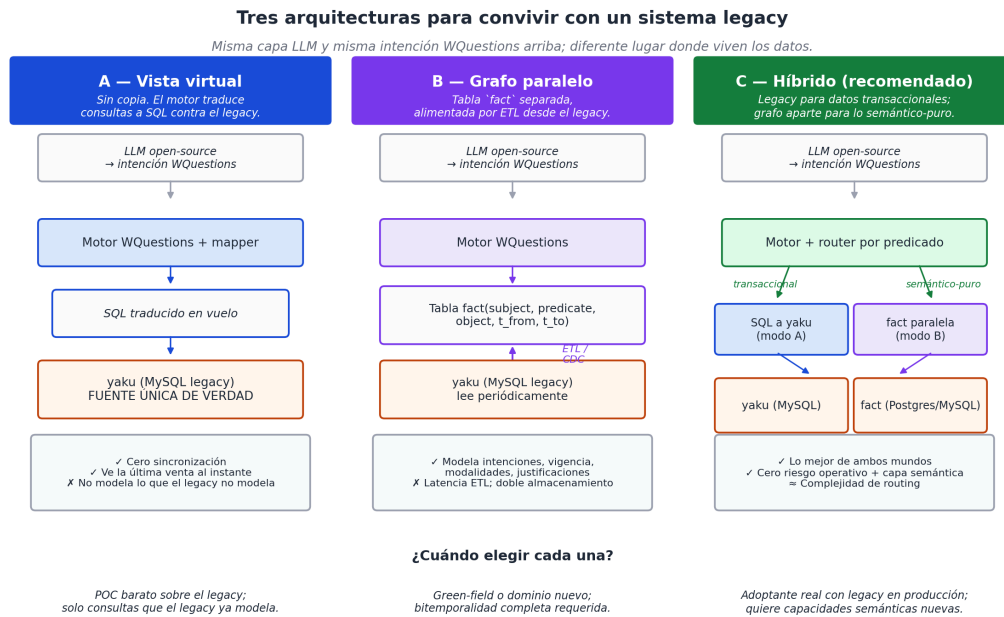
(uso_sauna_002, cubierto_por, contrato_plan_maria_2026) ← caso 2 (apunta al socio)
(uso_sauna_003, cubierto_por, estancia_carlos_5234)    ← caso 3 (apunta a la venta-padre)
(uso_sauna_004, cubierto_por, promo_cumpleanos_ana)   ← caso 4 (apunta a la promoción)
```

Cada `uso_sauna` es un hecho idéntico en su esencia (alguien usó el sauna). La modalidad de cobertura, por separado, explica *por qué no se cobró como pago directo*. Esto es generalizable: aplica a clubes deportivos, hoteles, clínicas con cobertura de seguros, cualquier negocio donde un servicio puede tener orígenes de cobro heterogéneos.

Es un concepto nuevo que se gana del ejercicio, ortogonal a lo que ya teníamos. Cabe agregarlo al catálogo D7.

Las tres arquitecturas de convivencia con un sistema legacy

Decir "*vamos a aplicar WQuestions*" deja sin responder una pregunta crítica: **¿dónde viven los datos?** Cuando ya existe un MySQL en producción con 125.000 ventas, no hay una sola respuesta — hay tres, y elegir bien evita mucho retrabajo. La figura 50 las ordena:



Tres arquitecturas para hacer convivir WQuestions con un sistema legacy. A — vista virtual sobre las tablas existentes. B — grafo paralelo alimentado por ETL. C — híbrido pragmático: A para lo transaccional, B para lo semántico-puro.

Arquitectura A — vista virtual. El motor WQuestions no almacena datos propios. Cuando recibe una consulta, su mapper la traduce a SQL contra las tablas legacy. yaku sigue siendo la única fuente de verdad. No hay sincronización, no hay copia, no hay latencia.

Funciona bien para: preguntas que el legacy ya sabe responder con sus tablas actuales — "¿cuántas saunas tomó Juan este mes?", "¿qué huéspedes están alojados ahora?".

No funciona para: preguntas que el legacy no modela — vigencia histórica de la dirección del cliente, intenciones de compra fallidas, justificaciones de cortesías.

Arquitectura B — grafo paralelo. Una tabla `fact(subject, predicate, object, t_from, t_to)` separada (en MySQL, Postgres, SQLite o donde sea). Un proceso ETL o CDC lee yaku periódicamente y proyecta sus filas a hechos atómicos. El motor WQuestions consulta exclusivamente esta tabla `fact`.

Funciona bien para: todo lo que el legacy no modela. Las intenciones, el por qué, la vigencia bitemporal completa, las modalidades de cobertura — todo se puede registrar limpiamente porque la tabla `fact` está diseñada para alojarlo.

No funciona para: latencia cero. Hay un retraso entre que una venta entra a yaku y aparece en el grafo. Para muchas aplicaciones esto es irrelevante (consultas analíticas, marketing); para otras es un bloqueador (decisiones en caja).

Arquitectura C — híbrido. Los datos transaccionales (clientes, productos, ventas, asistencias) se consultan en vivo contra el legacy vía A. Los datos puramente semánticos que WQuestions agrega (intenciones, modalidades de cobertura, justificaciones, vigencias históricas) viven en una tabla `fact` paralela. El motor decide a qué backend ir según el predicado de la consulta.

Esta es la que yo recomendaría para casi cualquier adoptante real. Aprovecha el sistema operativo existente para lo que ya hace bien; le añade encima la capa semántica que WQuestions aporta. Cero riesgo sobre la operación diaria, ganancia incremental en consultas nuevas.

El cap. 28 de este libro lista persistencia industrial como uno de los seis frentes pendientes. Lo que ese capítulo no menciona, y este sí, es que **persistencia industrial no es solo elegir entre Postgres y RDF**. Es también elegir si la capa WQuestions vive *con* el legacy o *aparte* de él, y la respuesta práctica casi siempre es "ambas, en proporciones distintas según el dato".

El costo real de construir un lexicon

El cap. 12 presentaba el lexicon como una pieza ya construida. El cap. 28 reconoce que falta poblarlo *a nivel idioma* — miles de verbos del español, FrameNet español, AnCor. Pero entre el verbo genérico y la entrada del lexicon usable en producción hay un trabajo intermedio que el libro no mencionaba: **construir el lexicon de un negocio específico**.

Para yaku, ese trabajo se desglosa así:

1. **Inventariar el dialecto del staff.** ¿La recepcionista dice "tomar sauna", "usar sauna", "hacer sauna"? ¿Llama "huésped" al cliente del hostel o "señor de la habitación"? Esto sale de horas con el equipo, no de leer el SQL. En yaku encontramos los verbos `tomar`, `entrenar`, `hospedar`, `consumir`, `contratar`, más los modificadores `cargar a`, `regalar`, `cancelar`.
2. **Descubrir polisemias locales.** El verbo `tomar` cubre dos situaciones distintas según el complemento: "`tomar sauna`" es un servicio; "`tomar una cerveza`" es un consumo. yaku no resuelve esto — escribe ambos en la misma `ventadet`. El lexicon de WQuestions sí, vía el patrón sintáctico del cap. 12.
3. **Detectar campos legacy que conflate dimensiones.** El campo `producto.MARCA` codifica dos cosas ortogonales en uno: la categoría semántica (HOSTAL, RESTAURANT, SAUNA, GYM) y el control de inventario (STOCK = se cuenta, todos los demás = no se cuenta). Un jugo de naranja es semánticamente RESTAURANT (preparación) y operativamente no-STOCK (no hay jugos en stock, se preparan al pedido). El lexicon de WQuestions separa las dos dimensiones por construcción.
4. **Clasificar artefactos legacy en sub-clases.** yaku tiene ~96 planes (`plan.nplan` es texto libre). Hay que clasificar manualmente cada uno como `plan_sauna`, `plan_gym` o `plan_mixto`. Lo mismo con los productos HOSTAL (matrimonial, doble, simple, jacuzzi). Lo mismo con los productos SAUNA (adulto, niño, walk-in). Una pasada manual o asistida por LLM resuelve esto **una sola vez**, no por consulta.
5. **Inventariar las modalidades de cobertura** y sus señales en datos. Esto se trabajó en la sección anterior; es el insight ortogonal que sale del ejercicio.

La inversión total para yaku es del orden de **dos a cinco días** de trabajo etnográfico + clasificación + escritura del lexicon. Es trabajo arqueológico-lingüístico, no programación. Y se hace una sola vez: a partir de ese punto, **toda consulta futura sobre el negocio reusa el mismo lexicon**. Esa amortización es el ROI que justifica la inversión inicial.

El gap del gimnasio: WQuestions no inventa información

Una observación honesta que falta en el libro: el modelo no es alquimia. Si el dato no se captura en algún lado, no aparece en el grafo por magia.

En yaku, el uso del gimnasio **solo** se registra cuando el cliente es socio en plan (entra al sistema vía la tabla `asistencia`). Los días sueltos comprados como producto MARCA=GYM se cobran en `venta` pero no generan registro de entrada/salida — no hay tabla `asistencia_walkin_gym`. Por lo tanto, la pregunta "*¿cuántas horas usó alguien el gimnasio este mes?*" hoy no puede responderse para los walk-ins, ni con yaku puro ni con WQuestions encima.

Esto es importante decirlo, porque el modelo es a veces vendido como una solución mágica para preguntas que las bases legacy no respondían. No lo es. WQuestions explicita estructura, conexiones, vigencia y por qué. Pero **lo que no se registra, no existe**. Cualquier capacidad nueva requiere captura nueva: un lector de huella en la entrada del gimnasio, un QR escaneado, una anotación manual en recepción. WQuestions solo puede modelar lo que el negocio decide capturar.

Esa honestidad es parte del contrato. El lector que aplica WQuestions sobre su legacy va a descubrir, igual que con yaku, gaps de información que el modelo no rellena por sí solo. Confundir "el modelo es expresivo" con "el sistema sabe todo" es un error costoso.

Una consulta real, end-to-end, sobre yaku

Cerremos con un ejercicio concreto que ata todo lo anterior. La recepcionista pregunta: *"¿qué huéspedes han usado sauna durante su estancia este mes y cuánto extra han generado?"*

Esta consulta no tiene una vista `q*` en yaku. Hoy requiere cruzar `asistencia` + `venta` (filtradas por producto `MARCA=HOSTAL`) + `ventadet` (filtrados por `MARCA=SAUNA` o por bandera `asistencia.Sauna`) y desambiguar las saunas-anexas-de-gym de las saunas walk-in. Es factible, pero frágil — un cambio en los nombres de productos rompe el reporte.

En WQuestions (en arquitectura híbrida C, contra yaku) se vuelve:

```
filter(0,
  instancia_de = uso_sauna,
  durante.instancia_de = estancia_hostal,
  durante.fin >= inicio_mes,
  cubierto_por.tipo = pago_directo
)
GROUP BY durante.huesped
PROJECT SUM(importe)
```

El motor traduce el patrón al SQL apropiado. La LLM, si la hay, traduce la frase en español a este patrón. Pero el patrón mismo es la pieza estable: tres roles canónicos (`instancia_de`, `durante`, `cubierto_por`) que cualquier negocio de servicios va a reusar.

Y la pregunta de seguimiento — *"¿alguno de esos huéspedes era además socio del gym?"* — se contesta agregando un filtro: `huesped IN socios_activos_gym(inicio_mes, fin_mes)`. Sin código nuevo. Sin reporte nuevo. El lexicon ya enseñó qué es un socio gym; el grafo ya conecta los huéspedes con sus contratos. La consulta compuesta sale de combinar piezas que ya existen.

Esta es la promesa de WQuestions sobre un sistema en producción: **convertir cada consulta nueva en una composición de piezas pre-construidas**, en vez de un reporte ad-hoc más.

Conexión con el resto del libro

El cap. 14 demostró que el modelo absorbe un negocio. Este capítulo demuestra que el modelo **se aplica** sobre un negocio ya digitalizado, sin migrar nada. El cap. 24 (LLMs) toma el siguiente paso lógico: con el lexicon construido, el grafo accesible y las modalidades de cobertura explicitadas, **enchufar una LLM por delante** y dejar que el staff de yaku haga preguntas en lenguaje natural.

Tres caminos de salida que el lector puede tomar después de este capítulo:

- Si tiene un sistema legacy y solo quiere ver si WQuestions vale: aplicar el ejercicio de arqueología semántica a sus propias tablas. Es trabajo de un fin de semana y devuelve un diagnóstico claro.

- Si ya tiene el diagnóstico y quiere probar el modelo: arquitectura C híbrida + un POC de una sola consulta crítica end-to-end. Si esa consulta sobrevive, el resto es repetir el patrón.
- Si quiere ir más lejos: cap. 24 (LLMs) muestra cómo el lexicon que se construyó aquí se vuelve directamente el function-schema que la LLM consume.

El sistema yaku que dio origen a este capítulo sigue funcionando como siempre. Ninguna fila se movió de lugar. Lo que cambió es que ahora, con dos a cinco días de inversión en el lexicon, **cualquier consulta del negocio se puede responder componiendo conceptos en vez de programando reportes**. Esa diferencia, multiplicada por años de operación, es lo que justifica el esfuerzo.

Capítulo 23 — Cuatro dominios incómodos: música, química, fútbol, contrato

El test que vale la pena pasar

Hasta acá la Parte V modeló dominios que, por exigentes que fueran, seguían dentro de un esquema comercial, clínico o administrativo reconocible, donde el modelo funcionó con poca fricción.

Cualquier propuesta arquitectónica que se vende como **universal** debe pasar un test más duro: enfrentar dominios que no se parecen al estándar comercial-clínico-administrativo en absoluto. Dominios donde las categorías son recursivas, donde no hay agentes humanos, donde el estado del mundo emerge de eventos paralelos, donde lo normativo es más importante que lo factual. El modelo pasa ese test — algunos casos requieren matices que vamos a documentar sin disimulo, pero el esqueleto no se quiebra en ninguno.

Este capítulo somete al modelo a cuatro dominios cualitativamente distintos — **música, química, fútbol y contrato** — uno por sección. La estrategia es deliberada: en vez de modelar uno en profundidad, vamos a tomar de cada uno la fricción **más característica** y ver cómo el modelo la enfrenta. Algunas frichas se resuelven con elegancia; otras requieren patches al catálogo D8; otras dejan pendientes que el libro asume sin disimular. Al final del capítulo enumero qué emergió de cada uno y cómo refinó la propuesta.

Cuatro dominios que estresan al modelo, cada uno por una razón distinta

El test de un modelo universal: pasar por dominios cualitativamente diferentes y ver dónde se dobla.

Música (Sonata Op. 27 n° 2) lo que estresa: recursión categórica • K profundo: sonata $\supset$ movimientos $\supset$ secciones $\supset$ frases • obra como K, interpretación como O • tiempo musical (compás) no es T absoluto • fricción: tema rechaza K $\rightarrow$ patch rol de dominio	Química (Combustión del metano) lo que estresa: D5 al extremo • sin agente humano: el grafo lo acepta • plantilla K + instancia O con factor de escala • cantidades estequiométricas como $N + K$ (unidad) • fricción: paciente: O $\rightarrow$ Q demasiado estrecho
Fútbol (Argentina-Perú 2026) lo que estresa: concurrencia + estado derivado • 22 jugadores en paralelo: granularidad opcional • dos relojes: T absoluto + minuto_partido • marcador como agregación de hechos primitivos • fricción: partes: O $\rightarrow$ Q (debería ser $\rightarrow$ V)	Contrato (Alquiler 12 meses) lo que estresa: lo normativo manda • vigencia + cláusulas con condición/consecuente • justificado_por preserva la cadena argumentativa • hechos inmutables: cambios = situaciones nuevas • fricción: bitemporalidad completa pendiente

Resultado: cero rompimientos. Cuatro fricciones documentadas con patch propuesto o trabajo pendiente.

Los cuatro dominios y la dimensión de cada uno que más estresa al modelo: música (recursión categórica), química (sin agente humano), fútbol (concurrencia), contrato (normativo intensivo).

Dominio 1 — Música: la recursión categórica

El primer dominio incómodo del libro fue una **composición musical** — una sonata de Beethoven, por concretarla con un caso real. El test fue modelar la *Sonata op. 27 n° 2 (Claro de luna)*, su estructura, sus movimientos, una interpretación específica en un teatro de Nueva York, y poder consultar tanto la obra abstracta como esa noche particular.

Lo que estresa. Una composición musical es una entidad **atemporal** — fue creada, sí, pero no ocurre en un momento; existe como objeto cultural. Una interpretación específica, en cambio, sí ocurre: tiene fecha, lugar, intérprete. Encima, la obra tiene **estructura interna recursiva**: una sonata se compone de tres movimientos; cada movimiento, de varias secciones; cada sección, de frases; cada frase, de motivos; cada motivo, de notas. Cinco niveles de jerarquía categórica, anidados.

El primer impulso es modelar la obra como T — un patrón temporal abstracto. El modelo de WQuestions lo rechaza: T es para momentos puntuales y rangos, no para entidades estables. La obra vive en **K** (categórica, atemporal); la interpretación en **O** (situada). La distinción D1 — *K son conceptos atemporales; O son entidades creadas/situadas* — quedó probada exactamente acá.

```
(sonata_op27_no2) ∈ K
  subtipo_de:   sonata
  compositor:   beethoven
  año_composicion: 1801
  movimientos:  [adagio_sostenuto, allegretto, presto_agitato] ← otros K

(interpretacion_001) ∈ O
  instancia_de:   accion_interpretar
  agente:         claudio_arrau
  obra_interpretada: sonata_op27_no2      ∈ M(0, K)
  lugar_de:       teatro_nyc
  momento:        1985-03-10T20:00Z
```

La fricción. tema: $0 \rightarrow 0$ rechaza valores en K. La obra es K. El prototipo me obligó a usar un **rol de dominio** — obra_interpretada: $0 \rightarrow K$ — exactamente la solución que aparecerá otras dos veces en este libro (medicamento prescrito, plan contratado). La música no es excepcional ahí; es la primera vez que el patrón aparece.

Lo nuevo que aportó. La recursión categórica — sonata contiene movimientos, movimientos contienen secciones — se modela con subtipo_de y un rol de dominio compone_a / compuesto_por que conecta categorías. K no es una lista plana de etiquetas: es un **árbol denso** con jerarquía interna, donde una consulta tipo "¿qué obras de Beethoven contienen un adagio?" es un recorrido transitivo sobre subtipo_de y compuesto_por.

Donde el dominio **todavía deja pendientes**: el "tiempo musical" — los compases, los pulsos, los tempi — no encaja en T (que es tiempo absoluto). Por ahora se modela como K (figura_redonda , figura_negra , compás_4_4) o como O reificado cuando se quiere hablar de un compás específico. La fricción está documentada y postergada; no rompe nada operativamente.

Dominio 2 — Química: D5 al extremo

El segundo dominio es la **combustión del metano**: una reacción química que ocurre cuando hay metano, oxígeno y energía suficiente, y produce CO₂ y agua. Modelar esto pone al modelo bajo presiones muy distintas a las del spa o el taxi.

Lo que estresa. Primero, **no hay agente humano**. Una reacción química no la *hace* nadie en sentido intencional; ocurre porque las condiciones físicas se cumplen. D5 — agencia contextual — establece que la agente no es obligatoria si el verbo no lo exige, y la química es donde esto se pone a prueba sin red. La situación reificada de una reacción concreta no tiene agente y el grafo lo acepta sin marcar.

Segundo, **plantilla + instancia con escala**. La química distingue entre la *reacción de combustión del metano en general* (una entidad categórica, K, que describe la estequiometría — un mol de CH₄ + dos moles de O₂ → un mol de CO₂ + dos moles de H₂O) y *esta reacción específica de combustión que ocurrió en ese motor a las 14:32* (un O reificado con cantidades reales escaladas). El modelo necesita capturar ambas y conectar instancia a plantilla.

```

(combustion_metano) ∈ K
  reactivos_estequiometricos: [{CH4: 1}, {O2: 2}]
  productos_estequiometricos: [{CO2: 1}, {H2O: 2}]

(reaccion_001) ∈ O
  instancia_de: combustion_metano
  factor_escalas: 0.5 ← la mitad de la estequiometría
  insumo: n_0.5mol_ch4 (multi)
  insumo: n_1mol_o2 (multi)
  lugar_de: cilindro_motor_001
  momento: 2026-05-16T14:32Z

```

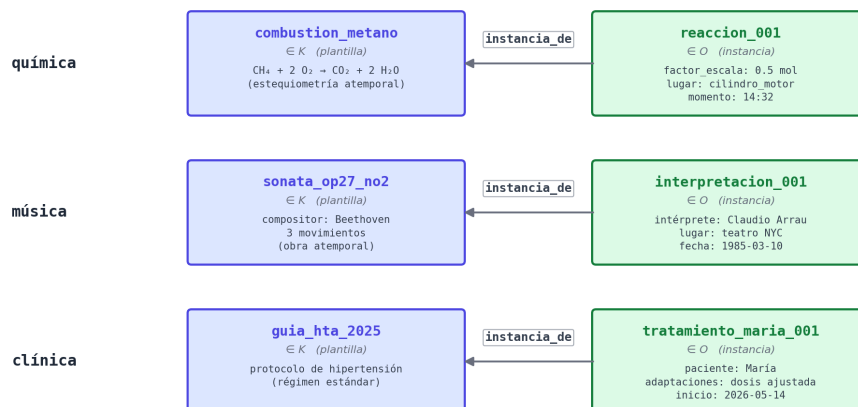
Las fricciones. Dos quedaron registradas durante el modelado y persisten:

1. **paciente: 0 → Q es demasiado restrictivo.** En química "el paciente" de una reacción es el metano (una cantidad N, ligada a una categoría K). El modelo rechaza esto con la signatura actual. La solución por el momento es usar un rol de dominio `insumo: 0 → N (multi)` que cubre el caso.
2. **reactivo / producto no son canónicos.** Los catálogos D8 del libro no los incluyen. La política liberal permite registrarlos como roles de dominio, pero queda el patch documentado de canonizarlos eventualmente porque el patrón es universal (cualquier transformación tiene entradas y salidas).

Lo nuevo que aportó. La pareja **K-plantilla + O-instancia + factor de escala** es un patrón general que el dominio musical también usa (sonata = plantilla, interpretación = instancia, *tempo* y matices = "factor de escala"), y que aparecerá en cualquier dominio con regularidad invariante y manifestaciones particulares: recetas (la receta = K, la preparación = O), procesos industriales (el SOP = K, la corrida = O), protocolos clínicos (el guía clínica = K, la atención particular = O). La química fue donde el patrón se hizo nítido.

El patrón "plantilla-K + instancia-O"

Tres dominios distintos, el mismo patrón estructural: lo genérico vive en K, lo concreto en O.



La invariante (estequiometría, partitura, protocolo) vive en K.
La manifestación concreta — situada, contingente, con factor de escala — vive en O.

El patrón plantilla-K + instancia-O: la reacción genérica vive en K con su estequiometría invariante; cada reacción concreta vive en O con cantidades escaladas. Lo mismo aplica a sonata→interpretación, receta→preparación, regla→aplicación.

Dominio 3 — Fútbol: la concurrencia y el estado derivado

El tercer dominio es un **partido de fútbol** específico — por hablar de algo real, Argentina-Perú de las eliminatorias de 2026, donde Messi marcó al minuto 23 con un toque de pierna izquierda y asistencia de Di María. Modelar esto trae complicaciones que el spa y el taxi no tenían.

Lo que estresa. Primero, **conurrencia real**: en cada momento del partido hay 22 jugadores haciendo cosas simultáneamente, más el árbitro, más los técnicos, más el público. Lo que se decide modelar es solo lo significativo; el resto queda implícito. Esto no es una fricción del modelo — es una decisión de granularidad — pero el modelo tiene que **no obligar** a registrar todo, y en efecto no lo hace.

Segundo, **pluralidad de tiempos**. Hay dos relojes simultáneos: el tiempo absoluto (T en formato ISO, *2026-09-14T20:23:42Z*) y el minuto de partido (el "minuto 23", que vive en una escala distinta). El modelo lo absorbe con doble registro: cada evento tiene un `momento: T` absoluto, y opcionalmente un `minuto_partido: N` relativo al inicio. Las dos consultas — "¿qué pasó a las 20:23 hora local?" y "¿qué pasó al minuto 23?" — funcionan en paralelo sin pisarse.

Tercero, **el marcador es un estado derivado**. El "Argentina 1 - Perú 0" no es un hecho que alguien afirma; **es la conclusión** de agregar todos los goles del partido. Esto es la misma forma estructural que la fidelidad del spa (7 sesiones → 1 gratis) y el costo total de un viaje del taxi: el modelo almacena los **hechos primitivos** (cada gol como evento reificado), y un evaluador externo recorre el grafo agregando. El modelo no calcula marcadores; los **prepara** para que cualquier evaluador lo haga.

```
n_goles_arg = count(u, Pattern(
    fixed={"equipo": argentina},
    type_constraint=u.ind("evento_gol"),
))
n_goles_per = count(u, Pattern(
    fixed={"equipo": peru},
    type_constraint=u.ind("evento_gol"),
))
marcador = f"{n_goles_arg} - {n_goles_per}"
```

La fricción. Una sola, registrada durante el modelado: el rol `partes` aplicado a un partido (los dos equipos) tenía signature `T → Q` (asumiendo que `partes` son agentes), pero un *equipo* es más cómodo como `O` (un colectivo compuesto). El patch propuesto fue **generalizar partes a `O → V`** — admitiendo cualquier eje de valor como rango. El prototipo confirmó la fricción y la sorteó vía rol de dominio (`parte: O → O`), pero el patch al catálogo es un trabajo pendiente real.

Lo nuevo que aportó. La distinción entre **hechos primitivos** y **estado derivado** quedó nítida acá. El modelo guarda lo primero; cualquier sistema productivo construye lo segundo encima. Esto importa porque rompe una expectativa que la mayoría de los sistemas relacionales traen consigo: que el "estado actual" se almacene como tal. WQuestions almacena la **historia de cambios** y deriva el estado por consulta. El precio es que el evaluador externo debe existir; la ganancia es que **el pasado nunca se pierde**.

Dominio 4 — Contrato: lo normativo manda

El cuarto dominio es un **contrato de alquiler** de doce meses con cláusulas, obligaciones recíprocas, una cláusula de rescisión por impago, y eventualmente una rescisión efectiva. Es el dominio donde lo *factual* (lo que pasó) es secundario frente a lo *normativo* (lo que está permitido, exigido, prohibido por las cláusulas).

Lo que estresa. Primero, la **vigencia temporal del contrato mismo y de cada cláusula**. Algunas cláusulas son válidas durante todo el contrato; otras solo durante períodos específicos (la cláusula de actualización del alquiler aplica solo en el aniversario). El modelo absorbe esto con `D6` — cada hecho lleva su rango — pero al implementarlo se hace evidente que **bitemporalidad completa** sería ideal: no solo *cuándo es cierto* sino también *cuándo lo afirmamos*. Esa pieza queda documentada como pendiente.

Segundo, **condicionales y consecuencias**. Una cláusula tiene la forma "si *X* entonces *Y*": si el inquilino impaga durante dos meses, el arrendador puede rescindir. El modelo trata esto reificando la cláusula como una situación con `condicion` y `consecuente`:

```
(clausula_14) ∈ 0
  instancia_de: clausula_contrato
  parte_de:     contrato_alq_001
  condicion:    impago_2_meses
  consecuente:  rescision_autorizada
```

Cuando una rescisión efectivamente ocurre, esa rescisión apunta a la cláusula con `justificado_por`:

```
(rescision_001, justificado_por, clausula_14)
```

La auditoría legal — "¿bajo qué autoridad se rescindió este contrato?" — se reduce a un recorrido por `justificado_por`. La regla no se dispara sola: un evaluador (humano o algorítmico) verifica que la condición se cumpla y emite la rescisión como hecho nuevo. Pero la **estructura argumentativa** está toda en el grafo.

Tercero, **mutabilidad de hechos**. ¿Qué pasa cuando una cláusula se renegocia? La regla del modelo es severa y deliberada: **los hechos son inmutables**. Una renegociación es una **situación nueva** que `rectifica` o `cancela` la previa. El historial se preserva en su totalidad; el "estado actual" se reconstruye por consulta. Esto es exactamente lo que vimos en la clínica con el rediagnóstico y en el taxi con las cancelaciones.

Las fricciones. Dos quedaron registradas y siguen pendientes:

1. **Bitemporalidad completa** (valid time + transaction time). Tenemos lo primero, no lo segundo. Para contratos donde un litigio puede preguntar "¿qué sabía el sistema en mayo de 2024?", hace falta agregar `tx_inicio` / `tx_fin` a los hechos. El prototipo lo trae como `tx_time` pero no como query parameter aún.
2. **Reglas de derivación versionadas**. Si la ley cambia y el contrato firmado anteriormente debe leerse bajo la ley vieja, el evaluador necesita saber qué versión de qué regla aplicar. Esto se modela con D6 sobre la regla misma, pero la **mecánica de evaluación versionada** queda como trabajo de la capa superior (motor de inferencia).

Lo nuevo que aportó. La importancia de la **estructura argumentativa explícita**. En dominios normativos, lo que el sistema necesita producir no es solo el resultado (rescindido / no rescindido) sino la **cadena de razonamiento** que lo justifica. El modelo, al exigir `justificado_por` apuntando a una cláusula, **fuerza** la producción de esa cadena. No es opcional.

Lo que emergió de los cuatro

Los cuatro dominios, vistos en serie, dejaron tres lecciones:

Primera: el patrón "plantilla-K + instancia-O" es universal. Apareció en química (reacción genérica vs concreta), música (sonata vs interpretación), también en clínica (protocolo vs atención concreta) y en taxi (tarifa estándar vs aplicada a un viaje). No es específico de ningún dominio; es un patrón de modelado tan central como la reificación misma. Vale la pena destacarlo como **convención** explícita en el lexicon de cualquier proyecto.

Segunda: el evaluador externo es estructural. Marcador de fútbol, fidelidad del spa, surge pricing del taxi, evaluación de cláusulas del contrato, conteo de visitas para diagnóstico — todo el "estado derivado" del sistema es trabajo del evaluador. WQuestions almacena los hechos primitivos y prepara el grafo; el razonamiento, simple o complejo, se construye encima. Esta separación es un **principio arquitectónico**, no una decisión postergada por simplicidad.

Tercera: las fricciones genuinas son pocas y conocidas. Después de ocho dominios (los cuatro de este capítulo más los cuatro anteriores), las fricciones reales son: (a) algunos roles canónicos del catálogo D8 son demasiado restrictivos (`paciente`, `partes`, `tema` aceptarían valores en más ejes); (b) bitemporalidad completa pendiente; (c) tiempo musical y patrones temporales finos pendientes; (d) reglas de derivación versionadas pendientes. Cuatro pendientes documentados, ningún punto donde el modelo se quiebre por completo.

Con esto termina la Parte V. El modelo se sostuvo a lo largo de todos los dominios —con un prototipo Python de unas 2.250 líneas y un catálogo D8 de 38 roles canónicos—; las fricciones tienen patch o quedaron documentadas como pendientes.

Capítulo 24 — WQuestions y los modelos de lenguaje

Una conversación, vista desde adentro

La doctora Torres, sentada en su consultorio antes de la primera consulta del día, abre el asistente conversacional de su sistema clínico y escribe:

¿Qué medicación venía tomando María Gonzales y desde cuándo?

Tres segundos después, la pantalla responde:

María Gonzales tomaba enalapril 10 mg desde el 14 de mayo de 2026, en una toma cada mañana. Su tratamiento se mantuvo sin cambios hasta el 10 de enero de 2027, cuando la doctora ajustó la dosis al rediagnosticarla como hipertensión grado 2.

Vista desde la doctora, la interacción es trivial: una pregunta en español llano, una respuesta en español llano. Pero entre el momento en que ella tocó *Enter* y el momento en que la pantalla mostró el texto, ocurrieron seis pasos:

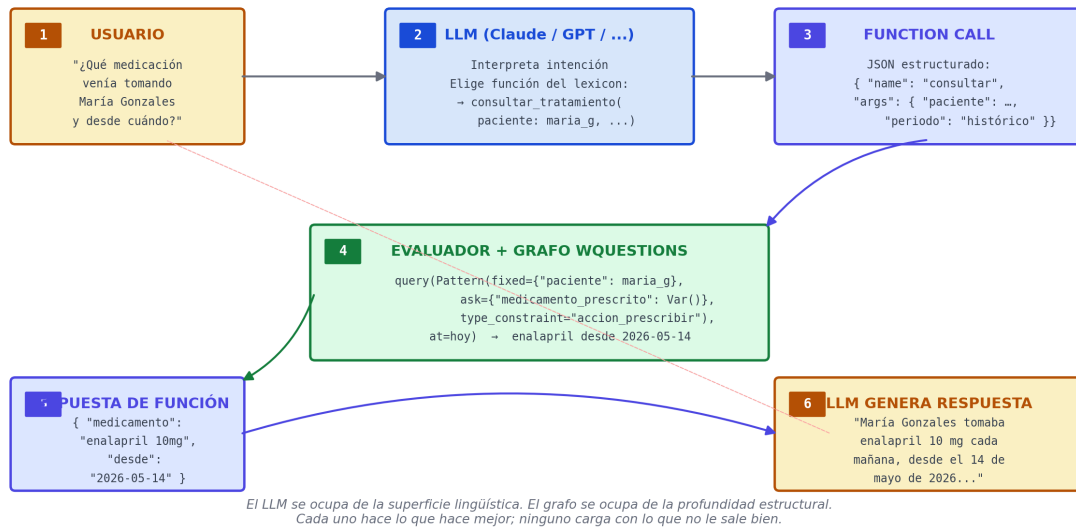
1. **El modelo de lenguaje** — un LLM, podría ser Claude, GPT, Gemini, Llama — recibió el texto y lo interpretó como una intención de consulta.
2. **Eligió la función adecuada** del catálogo que el sistema le expone: `consultar_tratamiento(paciente, periodo)`.
3. **Tradujo el español a argumentos estructurados**: paciente → `maria_g`, periodo → todo el histórico hasta hoy.
4. **Invocó la función**, que internamente ejecutó un patrón sobre el grafo WQuestions: `query(Pattern(fixed={"paciente": maria_g}, ask={"medicamento_prescrito": Var()}, type_constraint="accion_prescribir", at=hoy))`.
5. **El grafo respondió** con la prescripción vigente y su histórico, usando D6 para devolver exactamente lo válido en cada momento.
6. **El modelo recompuso la respuesta** en lenguaje natural fluido, con detalle, referencias temporales y conexiones causales.

Esa secuencia — usuario natural → LLM → función estructurada → grafo persistente → respuesta natural — es el patrón que define el momento tecnológico de 2026. Y es exactamente para lo que WQuestions, sin haberse propuesto serlo, resulta estructuralmente diseñado.

Este capítulo se ocupa de eso: por qué WQuestions y los modelos de lenguaje contemporáneos forman una **simbiosis natural**, cómo se expone el modelo a un LLM (vía function calling y MCP), y qué se vuelve posible que antes era engorroso o imposible.

Pipeline LLM ↔ WQuestions: del lenguaje natural al grafo y de regreso

Seis pasos. Dos lenguajes — natural y estructurado — comunicándose sin fricción.



El pipeline LLM ↔ WQuestions: el usuario habla en lenguaje natural, el LLM elige una función expuesta por el lexicon, el evaluador ejecuta la operación sobre el grafo, el LLM compone la respuesta. Seis pasos, dos lenguajes — natural y estructurado — comunicándose limpiamente.

Por qué este es el momento

Los modelos de lenguaje de la generación 2024-2026 son fluidos como nunca antes — pero tienen tres debilidades estructurales que vienen incluidas en el paquete:

No tienen estado persistente. Una conversación termina y el modelo olvida. Si vuelve a hablar con la doctora Torres mañana, no recuerda qué le preguntó hoy, qué pacientes mencionó, qué decisiones se tomaron. La continuidad se reconstruye **artificialmente** poniendo todo el historial dentro del prompt — caro en tokens, difícil de mantener consistente.

No distinguen lo que afirman de lo que les pasaron. Un LLM puede ser convincentemente incorrecto. Cuando un usuario consulta sobre María Gonzales, el modelo puede *alucinar* — inventar tratamientos plausibles que nunca existieron en la historia clínica real. La diferencia entre *"el sistema lo registró"* y *"al modelo le suena verosímil"* desaparece en la salida.

No auditan su razonamiento. Si un asistente clínico recomienda algo, el médico tiene que poder rastrear **por qué** la recomendación: qué hecho la disparó, qué regla la justifica, qué evidencia la respalda. Un LLM razonando en lenguaje natural produce texto convincente pero no necesariamente trazable.

WQuestions, por el contrario, tiene exactamente las tres propiedades complementarias: persistencia inequívoca, distinción entre afirmado y conjeturado (vía `estatus_factual`), trazabilidad por las relaciones del "por qué". Pero le falta lo que los LLMs hacen mejor: la fluidez lingüística, la interpretación de matices, la generación de respuestas en prosa que un humano lee con gusto.

La pareja queda completa: el LLM se ocupa de la **superficie lingüística** (entrada y salida), el grafo se ocupa de la **profundidad estructural** (almacenamiento y razonamiento). Cada uno hace lo que hace mejor; ninguno carga con lo que no le sale bien.

El lexicon es un function schema

Hay una observación que ya apareció en el capítulo 12 y que vale la pena retomar en limpio, porque es la **bisagra** entre las dos mitades. Una entrada del lexicon — *vender, prescribir, consultar, contratar* — tiene exactamente la estructura que los protocolos de function calling de los LLMs esperan. Veámoslo lado a lado.

Entrada del lexicon (representación interna del modelo):

```
verb: prescribir
situation_type: accion_prescribir
obligatory: [agente, paciente, medicamento_prescrito]
optional: [frecuencia, duracion, momento]
alias:
  agente: ["doctor", "médico", "doctora"]
  paciente: ["paciente"]
  medicamento_prescrito: ["fármaco", "medicación", "medicina"]
  frecuencia: ["cada cuándo", "tomas por día"]
```

Schema JSON expuesto al LLM (Function Calling / MCP):

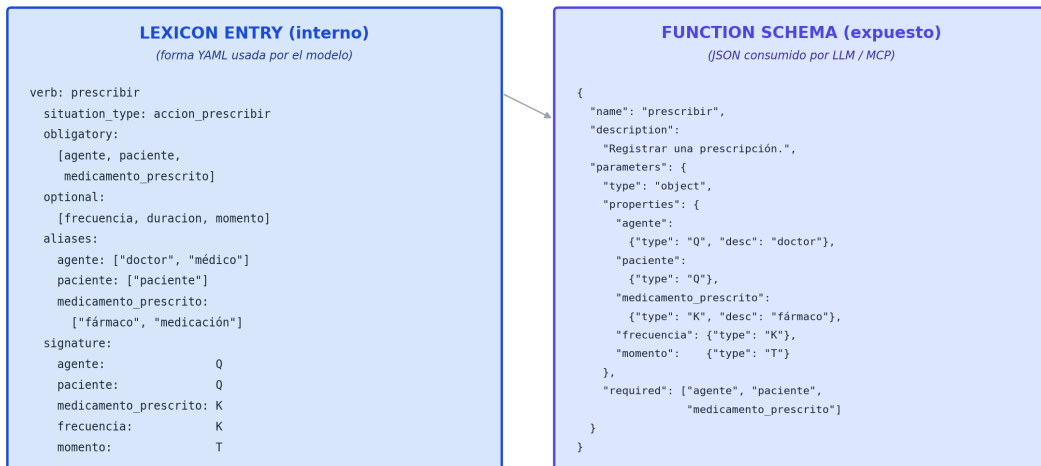
```
{
  "name": "prescribir",
  "description": "Registrar una prescripción médica.",
  "parameters": {
    "type": "object",
    "properties": {
      "agente": {"type": "Q", "description": "doctor, médico"},
      "paciente": {"type": "Q", "description": "paciente"},
      "medicamento_prescrito": {"type": "K", "description": "fármaco, medicación"},
      "frecuencia": {"type": "K"},
      "duracion": {"type": "K"},
      "momento": {"type": "T"}
    },
    "required": ["agente", "paciente", "medicamento_prescrito"]
  }
}
```

La correspondencia es uno a uno: `verb` → `name`; `situation_type` se vuelve el tipo retornado; los `alias` alimentan el campo `description` que ayuda al LLM a reconocer cómo se expresa el rol en lenguaje natural; los ejes (Q, O, L, T, N, K) se vuelven tipos del schema; `obligatory` se vuelve `required`.

Implicación práctica: para exponer WQuestions a un LLM no hay que escribir un adaptador a medida. Hay que **exportar el lexicon** como un catálogo de funciones. Las herramientas que ya existen para function calling consumen ese catálogo sin modificación.

Una entrada del lexicon ES un function schema

Correspondencia directa, sin adaptador: cada verbo del lexicon es invocable por un LLM.



Misma información, dos formatos. Para exponer un universo WQuestions a un LLM, solo hay que exportar el lexicon como catálogo de funciones — sin código adaptador.

Una entrada del lexicon junto a su schema de función equivalente. La correspondencia es directa y bidireccional: cada verbo del lexicon es una función invocable por un LLM; cada función expuesta al LLM corresponde a un verbo del lexicon.

MCP: el cable estándar

En 2024 Anthropic publicó la especificación de **MCP** — *Model Context Protocol* — un estándar abierto para que herramientas externas se expongan a modelos de lenguaje de forma uniforme. La idea es simple: en lugar de que cada aplicación inventara su propia manera de conectar un LLM a sus datos, MCP define un protocolo común. Un *servidor MCP* declara qué funciones expone, qué argumentos acepta, qué tipo de respuesta devuelve. Cualquier LLM que hable MCP puede usar ese servidor sin código adicional.

Para WQuestions, esto significa que **el servidor MCP de un sistema WQuestions es esencialmente el lexicon expuesto como funciones**. Cada verbo del lexicon se vuelve una herramienta; cada consulta-WH típica se vuelve otra herramienta:

Tools del servidor MCP del Spa Oasis:

ingesta:

- registrar_sesion(cliente, lugar, inicio, fin)
- aplicar_promocion(cliente, promocion, momento)
- redimir_beneficio(cliente, beneficio)

consulta:

- sesiones_de_cliente(cliente, desde, hasta) → List[Sesion]
- cliente_califica_fidelidad(cliente) → bool
- clientes_con_intencion_de_contratar() → List[Cliente]

auditoría:

- por_que_este_cobro(pago) → Explicacion
- estado_de_cliente_en(cliente, momento) → Estado

Una asistente conversacional de un sistema corporativo — pensada al estilo "*asistente del CFO*", "*asistente del médico de cabecera*", "*asistente del periodista de investigación*" — se construye conectando un LLM a ese servidor MCP. No hay glue code complicado; hay un catálogo declarativo de funciones y el LLM las elige según la conversación.

Tres casos de uso concretos

Conviene aterrizar la idea con tres escenarios que ya son técnicamente posibles en 2026, donde WQuestions encaja exactamente en el espacio que los LLMs no llenan solos.

Asistente operativo de un negocio

El dueño del Spa Oasis, en lugar de revisar reportes, le habla al sistema:

- *¿Cuántos clientes nuevos vinieron este mes?*
- *Vinieron 23 clientes nuevos en mayo, 30% más que abril. Los miércoles concentraron el 40% del crecimiento.*
- *¿Y cuáles de los nuevos están cerca del beneficio de fidelidad?*
- *Cinco están a una o dos sesiones del beneficio. Tres reservaron sesiones para esta semana; los otros dos no aparecen hace más de diez días.*
- *Mándales un mensaje a los que no vinieron.*
- *Listo. Borrador preparado para los dos clientes — Mariana C. y Tomás R. ¿Lo envió?*

Cada turno de la conversación se traduce a una o más llamadas a funciones del lexicon del spa. El LLM **no sabe** de spas — sabe del lexicon. El grafo guarda el estado real. La conversación es la **interfaz**, no la base de datos.

Lo interesante: el dueño jamás escribe SQL, jamás abre un dashboard, jamás aprende un lenguaje de consulta. El lexicon del spa es la API; el LLM es el traductor. El sistema se vuelve **operable conversacionalmente** sin que el negocio renuncie a su rigor estructural.

Ingesta automática desde texto no estructurado

Un hospital tiene veinte años de historias clínicas escritas en prosa libre. Migrar a un sistema estructurado tradicional es un proyecto multimillonario que requiere reescribir cada nota a mano.

Con WQuestions y un LLM, el flujo cambia. Una nota como:

"Paciente femenina de 54 años, refiere cefalea persistente de 3 días de evolución, presión arterial 145/92 mmHg al momento del examen. Se diagnostica HTA grado 1 y se inicia tratamiento con enalapril 10 mg cada mañana. Control en 30 días."

Se procesa con un prompt que le da al LLM el lexicon clínico y le pide: *"Extrae todas las situaciones de esta nota."* El LLM produce — usando function calling para cada situación — los hechos atómicos:


```

ingest_situation("consultar", roles={
  "agente": dra_torres, "paciente": maria_g,
  "momento": fecha_nota, "motivo": cefalea,
})
ingest_situation("medir", roles={
  "agente": dra_torres, "paciente": maria_g,
  "medida_de": presion_arterial,
  "monto": "145/92", "unidad": mmHg,
})
ingest_situation("diagnosticar", roles={
  "agente": dra_torres, "paciente": maria_g,
  "diagnosticado_como": hta_g1,
})
ingest_situation("prescribir", roles={
  "agente": dra_torres, "paciente": maria_g,
  "medicamento_prescrito": enalapril,
  "frecuencia": cada_manana,
})
ingest_situation("controlar", roles={
  "paciente": maria_g, "agente": dra_torres,
  "momento": fecha_nota_plus_30,
  "estatus_factual": previsto,
})

```

Cinco situaciones reificadas, todas validadas contra el catálogo. La prosa original queda como nota de respaldo; la estructura entra al grafo, donde se vuelve consultable, agregable, auditable.

El número que importa: una nota de 240 palabras (≈ 320 tokens en prosa) genera 24 hechos atómicos cuya serialización JSON pesa unos 80 tokens. **Compresión 4:1**, sin pérdida semántica, ganando estructura. Ahora multiplíquese por veinte años de hospital.

Razonamiento cross-dominio

Un periodista económico investiga el efecto de la reforma tributaria sobre las ventas minoristas. Tradicionalmente tendría que cruzar bases inconexas: el registro de ventas usa un schema, los datos macroeconómicos otro, las noticias políticas son texto libre.

Con WQuestions como **substrato común**, los tres dominios viven en el mismo grafo con su propio dialecto. El periodista escribe:

"Tráeme las noticias políticas relacionadas con impuesto al consumo desde 2024, y cruza con las ventas trimestrales del sector de retail."

El LLM combina dos consultas, una por dominio, con su tipo correspondiente:

```

noticias = query(Pattern(
  fixed={"tema_categorico": impuesto_consumo},
  type_constraint="noticia_politica",
  ask={"agente": Var(), "momento": Var(), "contenido": Var()},
))

ventas = query(Pattern(
  fixed={"sector": retail},
  type_constraint="agregado_ventas_trimestral",
  ask={"momento": Var(), "monto": Var()},
))

```

Las dos listas se devuelven al LLM, que las correlaciona por proximidad temporal y produce una respuesta narrativa. **Es posible porque ambos dominios comparten el mismo modelo subyacente** — el periodista no necesita saber que existen dos bases, dos schemas, dos sistemas. El grafo WQuestions absorbió ambos.

La economía de tokens, vista en limpio

Los modelos de 2026 manejan ventanas de contexto enormes — un millón de tokens, dos millones, cuatro millones. El instinto natural es escribir todo en prosa y dejar al modelo encontrar lo que necesita.

Esto funciona. Pero gasta el presupuesto a velocidad ingenua. Tres números, tomados de los dominios que modelamos:

Información	Prosa (tokens)	Grafo WQuestions (tokens)	Compresión
Una nota clínica típica	320	80	4×
Un mes de operación del spa (3 clientes)	8.000	1.800	4.4×
Un contrato de alquiler con cláusulas	2.500	700	3.6×

La compresión por sí sola es importante, pero lo que más cambia el cálculo es el segundo factor: **la ambigüedad**. La prosa fuerza al LLM a reinterpretar coreferencias ("*él*", "*el doctor*", "*la paciente mencionada*"), a inferir relaciones implícitas, a reconstruir estructura. Todas esas operaciones consumen atención del modelo — tokens efectivos que no van al razonamiento sobre el contenido sino a la reconstrucción del esqueleto.

El grafo WQuestions entrega el esqueleto **ya reconstruido**. El LLM puede dedicar su capacidad a lo que sabe hacer mejor: razonar, conectar, generar prosa fluida. La economía es doble: menos tokens y mejor calidad de respuesta.

Multi-agente, nativamente

El modelo D5 — agencia contextual, capítulo 9 — admite agentes de cualquier naturaleza: humanos, organizaciones, software, artefactos sensorizados. Esto no es decoración: es exactamente la propiedad que un futuro multi-agente necesita.

Un escenario plausible a tres años: un sistema empresarial donde varios LLMs especializados — uno clínico, uno financiero, uno legal — colaboran sobre el mismo grafo WQuestions. Cada uno tiene un sub-lexicon de su dominio. Cada uno entra al grafo como **agente** cuando ingresa un hecho:

```
(diagnostico_001, agente, lim_clinico_v3)      ∈ M(0, Q)
(asiento_001,    agente, lim_financiero_v2)
(opinion_001,    agente, lim_legal_v1)
```

El grafo registra **quién dijo qué y cuándo**, exactamente como registraría a un médico humano. Trazabilidad uniforme: el sistema puede auditar tanto a humanos como a modelos, con la misma maquinaria.

WQuestions como infraestructura

Cerremos con la idea más ambiciosa del libro, y la única que merece todo el aparato conceptual de las cinco primeras partes. **WQuestions, expuesto vía MCP, es una pieza de infraestructura para la IA conversacional empresarial** — no un producto, no un framework: una capa estructural sobre la que se construyen aplicaciones.

La distinción importa. Un producto se compra; un framework se aprende; una **infraestructura** se da por sentado. Cuando un programador escribe una aplicación web no piensa en TCP/IP; cuando un sistema procesa pagos no piensa en SWIFT; cuando una conversación con un asistente fluye, no se piensa en el grafo subyacente. La infraestructura es lo que se vuelve invisible porque funciona.

El proyecto WQuestions, en su forma actual, no está terminado para ser infraestructura. Le faltan piezas — persistencia industrial, motor de inferencia, bitemporalidad completa, lexicon poblado con miles de entradas verbales en varios idiomas — todas las cuales el capítulo 28 enumera honestamente. Pero la **arquitectura** está completa, y el momento histórico es favorable: los modelos de lenguaje hicieron que la simbiosis sea operativa, los protocolos como MCP estandarizaron el cable, y el costo de cómputo bajó al punto donde un asistente conversacional con grafo persistente es asequible.

Lo que el capítulo 25 explora es el lado expansivo de esa misma idea: si WQuestions fuese infraestructura, **¿qué aplicaciones se vuelven posibles que antes no?**. Algunas suenan a ciencia ficción; otras son obvias en cuanto se nombran. Las dejaremos puestas, sin disimular la especulación, porque ahí — y solo ahí — es donde el libro se permite mirar hacia adelante.

Capítulo 25 — Aplicaciones futuras: lo que se vuelve posible

Tres escenas a tres años vista

Imaginemos tres escenas que hoy son técnicamente plausibles pero económicamente inviables, y que con WQuestions más LLMs en su lugar maduro se vuelven asequibles para organizaciones medianas:

Escena uno — el dueño de la franquicia. El Spa Oasis se expandió: ahora tiene cuatro sedes en tres ciudades. El dueño, sin equipo de analytics, le pregunta por la mañana a su asistente:

— ¿Qué pasó esta semana?

— Las ventas de la sede Centro bajaron 12% respecto a la semana pasada. La causa principal: una caída en el plan mensual el miércoles. Ese mismo día subieron las cancelaciones de sesiones — siete cancelaciones, seis por mismo motivo: ducha rota. Mantenimiento ya está pedido pero la orden lleva tres días sin atender. Sugiero priorizar.

El asistente cruzó cinco fuentes — ventas, plan, sesiones, cancelaciones, órdenes de mantenimiento — siguiendo cadenas de `causado_por` y `motivado_por` por el grafo. Lo que para un equipo humano hubiera sido medio día de cruzar planillas, fue un minuto de conversación.

Escena dos — la médica buscando patrones. En un hospital con quinientos médicos y veinte años de historias clínicas estructuradas en WQuestions, una doctora le pregunta a su asistente:

— Muéstrame los pacientes con hipertensión grado 2 que iniciaron enalapril en los últimos seis meses y no lograron control después de tres meses.

El sistema devuelve la lista. La doctora pide más:

— De esos, ¿cuántos tenían diabetes co-mórbida? ¿Y cuál fue el segundo medicamento que se les agregó?

El sistema responde con una tabla cruzada y una visualización. Lo que antes era un proyecto de tesis — extracción manual de quinientas historias clínicas en prosa, codificación, análisis — es ahora una consulta de tres segundos.

Porque las historias estaban en el grafo desde el inicio.

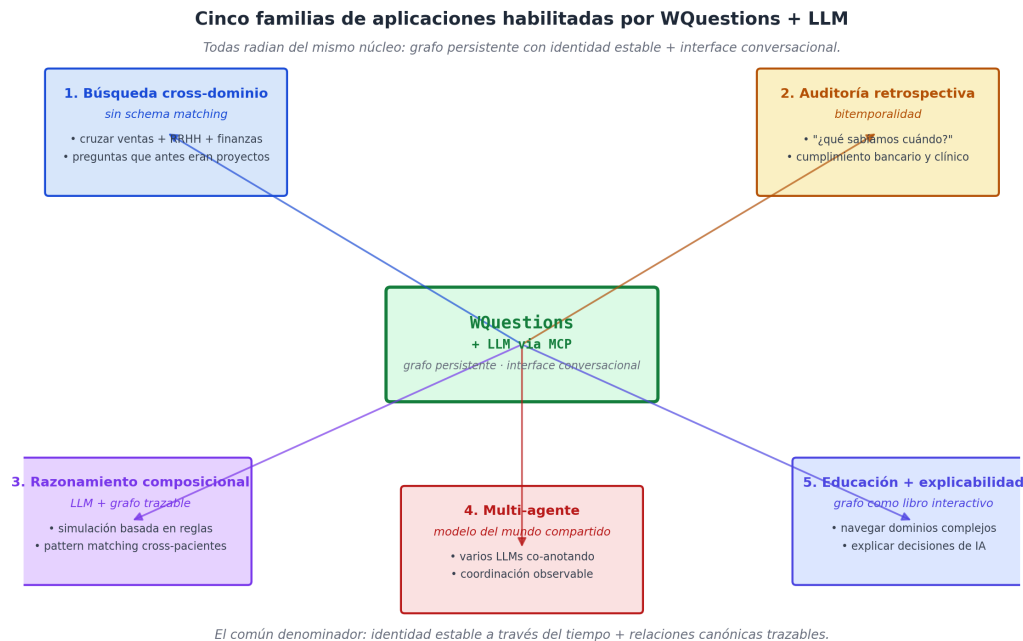
Escena tres — el auditor retrospectivo. Una empresa enfrenta una investigación regulatoria. El fiscal pregunta:

— ¿Qué sabía el directorio sobre la cláusula de garantía el 15 de noviembre de 2024?

El sistema reconstruye el estado del grafo en esa fecha exacta — usando bitemporalidad completa: no solo qué era cierto, sino qué estaba afirmado en el sistema en ese momento. Devuelve los hechos accesibles, los actos formales

registrados, las reglas vigentes. La pregunta *"¿qué sabía la organización cuándo?"* se vuelve consultiva, no testimonial.

Las tres escenas comparten una propiedad. **No se trata de innovaciones puntuales** — analítica conversacional, búsqueda semántica, auditoría temporal. Se trata de un mismo cambio arquitectónico que las hace posibles a las tres a la vez: el grafo persistente como sustrato, el LLM como interfaz, las relaciones canónicas como infraestructura de razonamiento. Lo que sigue son las cinco familias de aplicaciones que ese cambio abre.



Cinco familias de aplicaciones que se vuelven viables cuando WQuestions vive bajo un LLM: búsqueda cross-dominio, auditoría retrospectiva, razonamiento composicional, multi-agente, explicabilidad. Todas radian del mismo núcleo: un grafo persistente con interface conversacional.

Familia 1 — Búsqueda cross-dominio sin schema matching

La búsqueda corporativa hoy vive en silos. Los datos de ventas están en SAP, los de RRHH en Workday, los de marketing en Salesforce, los financieros en SAP de nuevo pero un módulo distinto, los operativos en un sistema legacy. Cada sistema tiene su schema. Para responder *"¿cuál fue el costo total de adquisición de un cliente que terminó cancelando dentro de los primeros 90 días?"* hay que cruzar cuatro sistemas, lo que en la práctica significa un proyecto de integración con presupuesto propio.

WQuestions reemplaza ese cruce por un sustrato común. Las áreas siguen operando sus sistemas, pero los hechos relevantes — ventas, contactos, transacciones, churn — se publican al grafo central con su dialecto de dominio respectivo. La pregunta del CFO se vuelve una consulta sobre el grafo, ejecutada por el LLM. El **schema matching** — el cuello de botella histórico de la integración empresarial — desaparece, porque todos los dialectos ya están mapeados al catálogo canónico D8.

Esta no es una promesa nueva — Tim Berners-Lee la formuló en el artículo fundacional del Semantic Web [31] hace un cuarto de siglo. Lo que faltaba era un modelo lo suficientemente simple para que las áreas lo adoptaran y un traductor lo suficientemente fluido para no exigir vocabulario técnico al usuario final. Las siete coordenadas-pregunta cumplen el primer requisito; los LLMs el segundo.

Familia 2 — Auditoría retrospectiva con bitemporalidad

Esta es la familia que más beneficio inmediato traerá a sectores regulados — finanzas, salud, derecho — y la única donde D6 (vigencia temporal) no es solo conveniencia sino requisito legal. El principio es simple: **el sistema nunca olvida, y puede mostrar lo que sabía en cualquier momento del pasado**.

Una corte que pregunta "*¿qué políticas internas estaban vigentes el 30 de junio de 2023?*" recibe una respuesta exacta, no una reconstrucción aproximada. Un regulador financiero que pide "*¿cuál era el límite de exposición de este cliente cuando se firmó la operación?*" obtiene el valor histórico, no el actual. Un equipo médico que estudia un evento adverso reconstruye **qué se sabía del paciente al momento de prescribir** — y por ende, si la decisión fue defendible bajo el conocimiento disponible.

La diferencia con sistemas tradicionales no es la **capacidad** — Snodgrass [17] formalizó bitemporalidad SQL en los noventa, y bases como Datomic la implementan en producción. La diferencia es la **uniformidad**: en WQuestions, todo hecho lleva su vigencia, no solo los selectivamente marcados. La pregunta retrospectiva nunca se topa con "*este dato no se preservó*", porque la inmutabilidad es la regla, no la excepción.

Sectores donde esto cambia el juego: cumplimiento normativo bancario, due diligence corporativa, ensayos clínicos longitudinales, registros catastrales, registros académicos, archivos periodísticos. En todos ellos, **lo que importa no es solo el estado actual sino la historia auditable**.

Familia 3 — Razonamiento composicional sobre conocimiento

Esta es la familia más ambiciosa y la que más promete a largo plazo. La idea es que un LLM, con acceso al grafo, no solo recupera información sino que **razona** combinando piezas:

- "*¿Qué clientes muestran patrones de uso similares al de Mariana C., quien canceló el mes pasado, y todavía no han cancelado?*" — composición de patrones de comportamiento.
- "*Si reduzco el precio del plan mensual en 10%, ¿cuántos clientes con intención registrada de contratarlo se convertirían según el modelo de elasticidad?*" — simulación basada en hechos y reglas.
- "*Lista los pacientes cuyo medicamento actual entra en contraindicación con la nueva guía clínica de agosto de 2026.*" — composición de reglas + estado + cronología.

Cada una de estas preguntas combina hechos, reglas, y razonamiento del LLM. WQuestions provee la materia prima inequívoca; el LLM compone. La frontera entre **base de conocimiento** y **razonador** se vuelve fluida — el LLM hace los saltos, el grafo le ofrece la huella firme donde apoyarlos.

El riesgo conocido es la **alucinación compositiva**: un LLM puede inventar conexiones plausibles que no están en el grafo. La mitigación viene de exigir trazabilidad — cada conclusión debe poder citar los hechos del grafo que la sostienen, con identificador y momento. Cuando una respuesta no se puede trazar, el sistema lo declara abiertamente en lugar de inventar.

Familia 4 — Sistemas multi-agente con modelo del mundo compartido

Hasta hace poco "agente IA" significaba un solo LLM con herramientas. Para 2026 ya se ven sistemas donde **varios agentes especializados colaboran** — uno hace investigación, otro redacta, otro verifica, otro audita. El cuello de botella es que cada agente vive en su propio contexto y la coordinación entre ellos es frágil.

WQuestions ofrece a esos sistemas un **modelo del mundo compartido**: un grafo persistente que todos los agentes consultan y al que todos contribuyen. Las decisiones de uno son hechos consultables por los demás. La cadena de razonamiento de un agente — "*agente A diagnosticó X basado en hechos Y, justificado por regla Z*" — queda registrada con la misma estructura que cualquier otra situación. La coordinación se vuelve **observable** en lugar de implícita.

```
(diagnostico_001, agente, lim_clinico_v3)      ∈ M(0, Q)
(diagnostico_001, motivado_por, evidencia_023)
(opinion_legal_002, agente, lim_legal_v1)
(opinion_legal_002, motivado_por, diagnostico_001) ← un agente cita a otro
```

Este patrón abre escenarios concretos: equipos de auditoría donde un agente sospecha, otro investiga, otro verifica; equipos médicos donde un agente sugiere diagnóstico, otro revisa contraindicaciones, otro registra; equipos legales donde un agente redacta, otro audita por riesgo, otro confronta con jurisprudencia. Todos sobre el **mismo** grafo, todos con la **misma** trazabilidad.

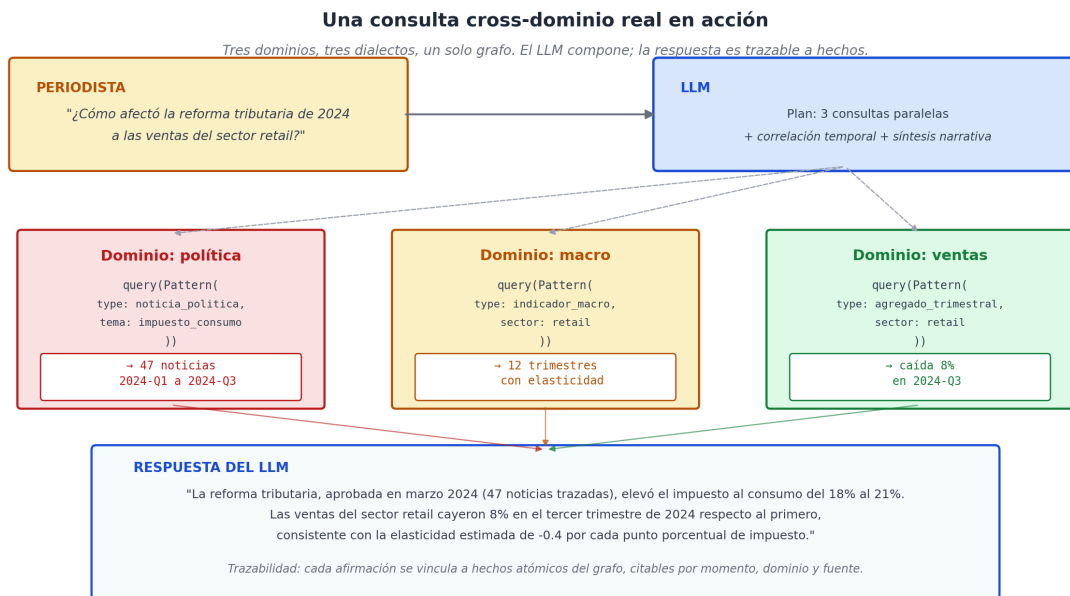
Acá D5 — la agencia contextual del capítulo 9 — paga el dividendo final: los LLMs entran al modelo como agentes Q de pleno derecho, tratados uniformemente con humanos. El sistema no necesita módulos especiales para "agentes IA"; el modelo ya los acepta porque siempre aceptó agentes no humanos.

Familia 5 — Educación y explicabilidad

Una aplicación menos comercial pero quizás más significativa a largo plazo: WQuestions como **herramienta pedagógica**. Cuando se le explica a un estudiante un dominio complejo — anatomía, derecho constitucional, química orgánica — la dificultad típica es que el dominio se presenta como un texto lineal, mientras que su estructura real es un grafo.

Si el dominio está modelado en WQuestions, el estudiante puede **navegarlo**: partir de un concepto, ver sus instancias, ver sus relaciones causales y normativas, hacer consultas exploratorias. "*¿Qué situaciones dependen de la disponibilidad de ATP?*", "*¿Qué causa qué en la cascada inflamatoria?*", "*¿Qué cláusulas se justifican mutuamente en este sistema legal?*". El grafo se vuelve un libro de texto **interactivo y consultable**, donde la estructura del conocimiento es explorable, no recitable.

La aplicación tiene un correlato indirecto en explicabilidad de la IA. Cuando un sistema basado en LLMs toma una decisión — denegar un crédito, sugerir un tratamiento, sancionar un contenido — la pregunta "*¿por qué?*" requiere una respuesta auditable. Si la decisión se construyó sobre un grafo con relaciones canónicas del "por qué", la cadena de explicación es **literalmente** un recorrido del grafo. No es post-hoc rationalization; es la reconstrucción del razonamiento real.



Una consulta multi-fuente real: el periodista pregunta cómo afectó la reforma tributaria a las ventas. El LLM combina consultas sobre noticias políticas, indicadores macroeconómicos y ventas trimestrales — todos en el mismo grafo — y produce una respuesta narrativa con trazabilidad.

La constante: identidad estable a través del tiempo

Si tuviera que destilar las cinco familias en una sola observación, sería ésta. Lo que las hace posibles a todas es la misma propiedad: **el grafo preserva identidad estable de las entidades a través del tiempo**. María Gonzales es el mismo individuo en 2026 y en 2034; el Spa Oasis es la misma persona jurídica antes y después de expandirse; la cláusula 14 del contrato es la misma cláusula con o sin enmiendas posteriores.

Esto suena trivial pero no lo es. En sistemas tradicionales, la identidad se reconstruye por *foreign keys* que apuntan a registros que pueden ser editados, borrados o renombrados. La identidad es **frágil**: una migración de schema, un cambio de criterio, una limpieza de datos, y la trazabilidad se rompe. WQuestions hace lo opuesto: cada individuo recibe un UUID inmutable, los hechos sobre él son acumulativos, los cambios son nuevos hechos con vigencia. La identidad se vuelve **infraestructura**, no convención.

Sobre esa base se construye todo lo demás: la auditoría retrospectiva, el razonamiento composicional, el multi-agente, la explicabilidad. Todos exigen poder decir *"este individuo, en este momento, era así"* y obtener una respuesta unívoca. La identidad estable es la propiedad que hace que decir *este* signifique siempre lo mismo.

El sesgo de optimismo

Sería deshonesto cerrar este capítulo sin matizar. Los escenarios que describí son **plausibles**, no inevitables. Cada uno depende de que varias condiciones se cumplan a la vez:

- Que las organizaciones acepten exponer su información estructurada a un grafo común (ni siquiera tienen que hacerlo *público*, pero sí *consultable* dentro de su perímetro).
- Que los LLMs sigan mejorando en exactitud y en honestidad sobre lo que no saben.
- Que el costo de cómputo de un asistente conversacional con grafo persistente siga bajando.
- Que aparezcan **ingenieros de lexicon** — un rol nuevo, equivalente al del data engineer pero centrado en mapear vocabulario de dominio a roles canónicos.

- Que la regulación acompañe en lugar de bloquear, especialmente en sectores regulados (salud, finanzas, derecho).

Algunas de estas condiciones se están cumpliendo solas, otras requieren empuje activo. La especulación es honesta cuando reconoce la diferencia.

Lo que falta

Las aplicaciones de este capítulo asumen una versión madura del proyecto. La versión actual — la que el prototipo ejecuta — está incompleta: falta persistencia industrial, motor de inferencia, bitemporalidad completa, lexicon poblado en varios idiomas, herramientas para que las organizaciones definan sus dialectos sin asistencia de un ingeniero. Mientras estas aplicaciones viven en el futuro, ese trabajo pendiente vive en el presente operativo, y enumerarlo honestamente — como un **mapa de implementación** para quien quiera contribuir o adoptar la propuesta — es lo que queda por hacer.

Capítulo 26 — La prueba reflexiva: el modelo descrito con el modelo

La carga que faltaba

A lo largo del libro sometimos WQuestions a presiones de distinto tipo. La presión lingüística del Capítulo 13, donde el lenguaje aprieta con nominalizaciones, modales e idiomas. La presión de ocho dominios disímiles —del spa a la minera— que confirmaron que un mismo catálogo se sostiene en territorios que no se parecen en nada. La presión de un sistema en producción, en el yaku del Capítulo 22. Y la de cuatro dominios deliberadamente incómodos en el 23, elegidos para que crujieran.

Falta una presión más, y es la más exigente de todas: pedirle al modelo que **se describa a sí mismo**. No modelar un dominio externo con WQuestions, sino construir una herramienta —un programa con menús, formularios, pantallas, tipos de dato y reglas de comportamiento— cuya **estructura, cuyos tipos y cuya conducta sean, ellos mismos, hechos WQuestions**. Si la tesis del libro es cierta —que las preguntas son la base universal de la información—, entonces la propia maquinaria que manipula información debería poder expresarse en esas preguntas. Este capítulo cuenta qué pasó cuando lo intentamos. No venimos a presumir de una varita mágica; venimos, como en el Capítulo 13, con honradez intelectual: a mostrar qué aguantó, qué se dobló para aguantarlo y qué quedó al descubierto.

Una aplicación hecha de preguntas

Construimos, sobre el prototipo, una pequeña aplicación de gestión: un menú navegable, formularios para dar de alta y editar registros, grillas para consultarlos, y entidades de negocio —personas, productos, ventas, compras— con sus campos. Lo decisivo no es la aplicación; es **de qué está hecha**. Cada opción del menú, cada campo de cada formulario, cada tipo de dato, cada relación entre entidades y cada acción que el programa ejecuta es una tripleta (sujeto, rol, valor) en un único grafo.

El menú es un objeto reificado que `tiene_opcion` a otros objetos; cada opción `tiene_accion` una acción; cada acción es `instancia_de` un verbo —`mostrar_texto`, `abrir_formulario`, `abrir_grilla`, `guardar`— que un evaluador genérico interpreta. El esquema de una entidad es dato: la entidad `tiene_campo` a descriptores que declaran su etiqueta, su `tipo_dato` y su `orden`. Un registro es un individuo `instancia_de` su tipo, con un hecho por campo. No hay, en ningún lado, una tabla `venta` con columnas fijas ni una clase `Formulario` con atributos cableados. Hay hechos, y un motor que los lee.

La consecuencia se ve en la pantalla del propio programa: al lado de cada vista, un **inspector** muestra las tripletas que la sostienen. *Lo que ves es, literalmente, lo que hay en el grafo*. La interfaz no es una capa que traduce una base de datos a botones; es una proyección directa de los hechos.

La prueba reina

La medida de si la tesis se sostiene cabe en una sola operación: **¿cuánto cuesta agregar estructura?** Si estructura y comportamiento son de verdad datos, agregar un campo nuevo a una entidad debería costar *insertar hechos*, no *escribir código*.

Lo medimos en vivo, con el sistema corriendo, agregando el campo "Documento" a la entidad `venta`:

(campo_venta_documento, instancia_de, campo)	$\in M(0, K)$
(venta, tiene_campo, campo_venta_documento)	$\in M(K, 0)$
(campo_venta_documento, tipo_dato, texto)	$\in M(0, K)$
(campo_venta_documento, orden, 5)	$\in M(0, N)$
(campo_venta_documento, rol, documento)	$\in M(0, K)$

Cinco tripletas. Ninguna línea de código en el motor, en el servidor ni en la interfaz. Al recargar, el formulario dibujó el campo, la grilla le sumó la columna y el guardado lo persistió — porque las tres operaciones leen el esquema del grafo en vez de conocerlo de antemano. Ese número —cinco hechos, cero código— es el resultado central del experimento, y la confirmación más limpia de la primera mitad de la tesis: **la estructura es dato**.

Lo que la carga confirmó

Más allá del número, el experimento puso a prueba tres ideas que el libro defendió en abstracto, y las tres aguantaron en concreto.

El comportamiento como datos. Seis veces a lo largo del libro apareció la figura del *evaluador externo*: el motor que recorre el grafo y ejecuta lo que los hechos prescriben, separado del grafo que solo almacena. Aquí ese evaluador dejó de ser una promesa y pasó a ser código que corre: un despachador genérico que, dado un verbo (un tipo K), ejecuta la conducta asociada. Agregar una transacción entera nueva —"compras", hermana de "ventas"— costó esencialmente datos, no motor. El comportamiento vive en el grafo y se interpreta desde afuera, tal como el modelo prometía.

El grafo compartido. El Capítulo 1 abrió con la torre de Babel: cada sistema con su esquema, incapaz de hablar con el de al lado. El antídoto del libro es un único piso de hechos donde las entidades se reúsan. Lo comprobamos modelando "cliente" y "proveedor" no como tipos distintos sino como **roles contextuales** —el nombre del campo en la venta o en la compra (D5, agencia contextual)—. La misma persona es cliente de una venta y proveedora de una compra sin duplicarse: un solo individuo, dos papeles según el contexto. El anti-Babel no fue una aspiración; cayó solo en cuanto dejamos de modelar el rol como tipo.

La vigencia. La edición de un dato se implementó sin borrar nada: editar un valor es asentar un hecho nuevo y leer el más reciente. Es bitemporalidad en miniatura (D6) —historial gratis, ninguna sobreescritura destructiva—, y resultó ser la forma *natural* de implementar el "update" de un CRUD, no un agregado teórico.

La auto-corrección: `instancia_de` pasa a `V→K`

El experimento no solo validó: forzó una corrección del prototipo, y vale contarla porque es el método del libro operando sobre sí mismo. Para clasificar a una persona —decir "Ana es un cliente"— hace falta el hecho `(ana, instancia_de, cliente)`. Pero `ana` vive en el eje **Q** (es un agente), y el prototipo había restringido `instancia_de` a sujetos del eje **O**. La carga reveló la contradicción: este mismo libro, en el Capítulo 3, ya escribía `(messi, instancia_de, jugador_de_futbol) ∈ M(Q, K)` y `(lima, instancia_de, ciudad_capital) ∈ M(L, K)`. El prototipo estaba *detrás* de su propia teoría.

La corrección fue generalizar `instancia_de` a `V→K` —donde **V** es un comodín que significa "cualquier eje de valor"—. Clasificar es una operación universal: todo individuo de **Q**, **O**, **L** o **T** puede responder "¿de qué concepto soy instancia?". Y ese comodín **V** no resuelve solo este caso: es exactamente el mecanismo que el Capítulo 28 reclama para relajar otras signaturas demasiado estrechas `(paciente, partes)` de `0→Q` a `0→V`. La fricción, sometida a carga, no abrió un agujero en el modelo: lo empujó a alinearse con lo que ya afirmaba.

Las fricciones nuevas: dos cerradas, una abierta

La presión reflexiva destapó tres fronteras que los ocho dominios no habían tocado. Dos las cerramos en el mismo experimento; la tercera quedó señalada como trabajo.

El texto libre. Los siete ejes son semánticos, no tipos primitivos: no hay un eje "string". Un nombre como "Ana", el contenido de un mensaje, el número de un documento —texto arbitrario— no tienen casa propia. Decisión, fiel a la teoría: el texto libre se aloja como un **literal en K, minteado y único**, con su valor en la etiqueta y una marca que lo distingue de una **categoría controlada** (un vocabulario cerrado como las unidades monetarias o los estados, que sí son K nombradas y compartidas). Así, dos personas llamadas "Ana" no terminan compartiendo un mismo individuo, y el modelo sabe distinguir un literal de una categoría.

El catálogo como dato. El catálogo D8 tipa con fuerza el núcleo canónico, pero los campos que el usuario define como datos pasaban sin validar, bajo política liberal. La carga lo expuso: el tipado del esquema vivía a medias en el código. La cerramos haciendo que **cada campo derive su signature de sus propios datos** —dominio del eje de la entidad, rango de su `tipo_dato`— y la registre en el catálogo. Escribir en un campo dinámico ahora se valida como un rol canónico; el tipado dejó de vivir en Python y pasó a vivir en el grafo.

El humano. Queda en pie la tercera frontera, y no es del modelo sino del lector. Cuando *todo* es (sujeto, rol, valor), se pierde el andamiaje de tablas y formularios con nombre que normalmente carga el significado. El poder de la abstracción reflexiva tiene un peaje cognitivo. El inspector —"lo que ves = datos"— re-concreta lo suficiente para no ahogarse, pero la pieza que falta es hacer de las **vistas y proyecciones con nombre** ciudadanas del grafo: datos, no accidente de interfaz. Es trabajo pendiente, y aparece como tal en el capítulo siguiente.

Honradez intelectual

Conviene nombrar el vértigo, porque es un hallazgo y no un defecto del lector. Un modelo capaz de describirse a sí mismo es, por construcción, mareante: en el límite, no hay "tablas" ni "pantallas", solo preguntas sobre preguntas. Pero ese mareo no invalida la propuesta —la confirma—, y al mismo tiempo explica por qué un modelo así necesita prosa, ejemplos y una capa de re-concreción para ser usable por personas. Es, en el fondo, la razón de ser de este libro: el grafo es operable; el texto es lo que lo vuelve comprensible.

Que el modelo pueda describir su propia herramienta es la evidencia más fuerte que el prototipo produjo. Más que ocho dominios distintos, es el modelo hablando de sí mismo sin salirse de sus siete ejes, corrigiéndose cuando una signature quedó corta y absorbiendo como datos lo que un sistema convencional cablearía en código. La primera mitad de la tesis —estructura y comportamiento son datos— está saldada, y medida en un número. Lo que sigue es la otra mitad: el inventario honesto de lo que todavía falta para que esto deje de ser un prototipo y se vuelva infraestructura. A eso dedicamos el próximo capítulo.

Capítulo 27 — Seguridad y privacidad en el grafo compartido

La otra cara de la moneda

Volvamos a la paciente. La de la arritmia, la que a las dos de la mañana fue atendida por un médico que no la conocía y que, gracias a que cuatro sistemas distintos por fin hablaban el mismo idioma, pudo reconstruir su historia en treinta segundos y tomar una buena decisión. Esa noche, la interoperabilidad le salvó la vida.

Seis meses después, la misma mujer pide un seguro de vida para respaldar una hipoteca. La aseguradora corre su evaluación de riesgo. Su sistema —moderno, conectado, que también habla en preguntas— formula una consulta perfectamente legible: *¿qué situaciones de tipo diagnóstico_médico tienen a esta persona como paciente?*. Y el grafo, servicial, responde: una arritmia, monitoreada desde hace dos años, con un episodio de descompensación documentado. La prima se dispara. En el peor caso, la cobertura se niega.

Nadie hackeó nada. No hubo una brecha, ni una contraseña filtrada, ni un empleado deshonesto. El sistema funcionó **exactamente como fue diseñado**: hizo una pregunta y obtuvo una respuesta tipada y confiable. El mismo grafo que a las dos de la mañana le salvó la vida, a las diez de la mañana le niega la cobertura.

Esta es la objeción más seria que se le puede hacer al modelo, y conviene no esquivarla: **un grafo compartido con identidad estable es, por defecto, una infraestructura de vigilancia**. Toda la legislación de privacidad del mundo —el RGPD europeo, la HIPAA estadounidense, la Ley 29733 peruana— existe precisamente para impedir lo que WQuestions hace fácil: vincular, sin fricción, datos que vivían separados. Si la fuerza del modelo es que los clientes, los pacientes y los lugares son los mismos a través de los dominios, esa misma fuerza es lo que permite que el banco vea el diagnóstico y la clínica vea la deuda.

El resto de este capítulo argumenta que el problema no solo tiene solución, sino que el modelo —bien entendido— ofrece mejores herramientas para resolverlo que la arquitectura relacional que reemplaza. Pero la primera respuesta no es técnica: es deshacer un malentendido.

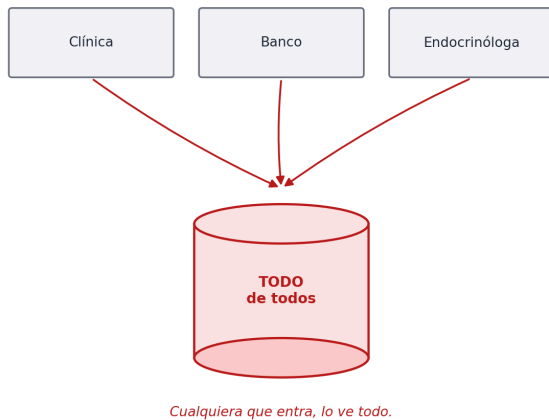
El malentendido que hay que deshacer: un idioma, no una base

La imagen que produce pánico —y con razón— es la de un gran depósito central donde todo el historial de todos cuelga de un identificador único que cualquiera puede consultar. Si eso fuera WQuestions, el modelo sería indefendible.

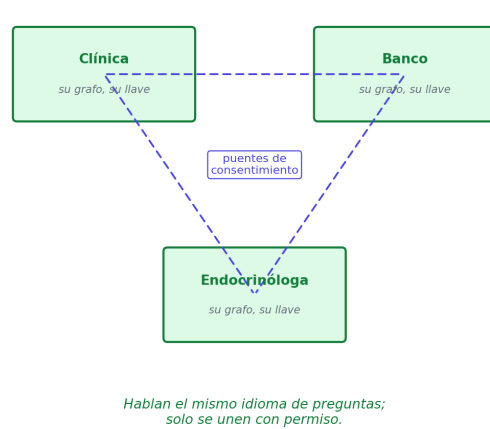
No lo es. **El grafo compartido es semántico, no físico**. Lo que se comparte entre el banco, la clínica, la endocrinóloga y el cardiólogo no es una base de datos: es el *idioma de las preguntas* —los siete ejes y el lexicon que traduce a ellos—. Cada quien sigue guardando sus propios hechos en su propio almacén, bajo su propio control y su propia llave. La clínica no lee el grafo del banco. Lo que ocurre es que, **cuando hace falta interoperar —y solo con autorización—**, los dos grafos hablan la misma lengua y la fusión que antes era un proyecto de integración de seis meses se vuelve trivial.

Un idioma compartido no es una base compartida

Base central — la pesadilla



Federación — cada quien guarda lo suyo



Federación frente a base central: a la izquierda, el depósito único donde todos leen —la pesadilla de privacidad—; a la derecha, cada tenedor guarda sus propios hechos y solo los une, bajo consentimiento, un idioma común de preguntas.

Releída así, la escena del médico de guardia cambia de sentido. No fue que el médico «entró al expediente global de la paciente». Fue que cada tenedor —los dos hospitales, la endocrinóloga, el cardiólogo— **publicó a un espacio que la paciente había autorizado** para situaciones de emergencia, y que esos hechos, por hablar el mismo idioma, se ensamblaron solos. La interoperabilidad vive en el borde, en la federación y el consentimiento; no en un cerebro central que todo lo sabe.

Esto resuelve el susto inicial, pero abre la verdadera pregunta de ingeniería: si cada tenedor controla su acceso, ¿con qué se controla? Y aquí es donde el modelo deja de defenderse y empieza a atacar.

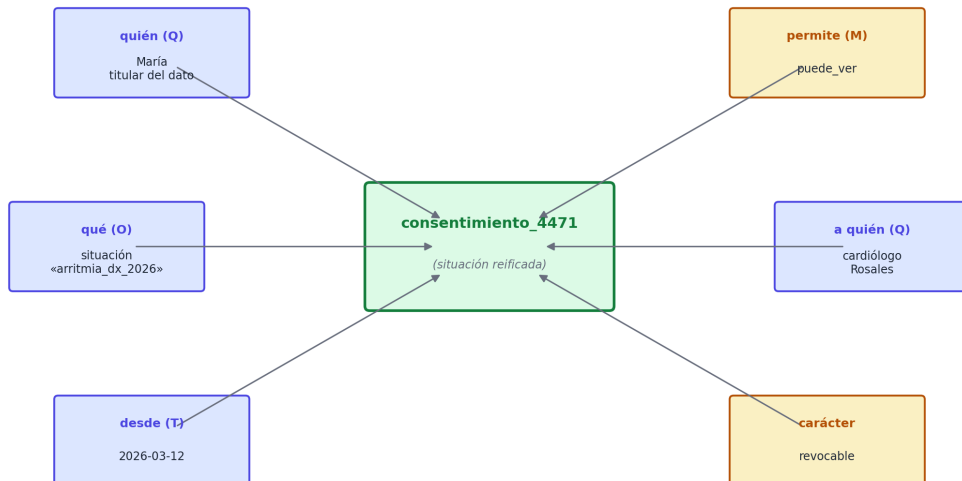
El permiso es un hecho

En una arquitectura relacional, el control de acceso es una capa aparte, escrita en otro lenguaje que el de los datos: tablas de permisos, roles, `GRANT` y `REVOKE`, reglas de aplicación dispersas por el código. Es un sistema paralelo, y los sistemas paralelos se desincronizan.

En WQuestions no hay capa aparte, porque **el permiso es, él mismo, una situación reificada** —un hecho más, descrito con las mismas siete preguntas que cualquier otro—. «Que el cardiólogo Rosales pueda ver el diagnóstico de arritmia de María» no es una entrada en una tabla de ACL: es una afirmación del grafo, con su quién, su qué, su cuándo y su agente.

El permiso es un hecho más

descrito con las mismas siete preguntas que cualquier situación



Por eso revocar es publicar un hecho nuevo (gana el último — D6), y cada acceso queda, también, como una situación auditable.

El permiso descrito con el propio modelo: una situación reificada cuyos ejes son quién recibe el acceso, sobre qué situación, qué puede hacer y desde cuándo. El control de acceso habla el mismo idioma que aquello que controla.

```
consentimiento_4471
quién (Q)    → María (paciente, titular del dato)
permite (M)  → puede_ver
qué (O)      → situación «arritmia_diagnóstico_2026»
a quién (Q)  → cardiólogo Rosales
desde (T)    → 2026-03-12
carácter     → revocable
```

Y ese mismo consentimiento, serializado para viajar entre sistemas, es un hecho más del grafo:

```
{
  "id": "consentimiento_4471",
  "instancia_de": "consentimiento",
  "quien": "paciente_maria",
  "permite": "puede_ver",
  "sobre": "arritmia_diagnostico_2026",
  "a_quien": "cardiologo_rosales",
  "desde": "2026-03-12",
  "caracter": "revocable"
}
```

Tres consecuencias caen de regalo:

El consentimiento es un hecho con fecha. No es una casilla marcada en un formulario perdido: es una situación reificada que dice quién consintió qué, para quién y desde cuándo. Y como es un hecho, puede tener su propia procedencia: *quién* recogió el consentimiento, *cómo*, bajo qué versión de los términos.

Revocar es publicar, no borrar. Cuando María retira el permiso, no se edita ni se destruye el consentimiento anterior: se asienta un hecho nuevo que lo supersede. Y por la regla de **vigencia temporal (D6)** —**gana el último hecho**—, a partir de ese instante el acceso deja de valer, mientras queda constancia auditable de que existió y de cuándo se retiró. El sistema no olvida que hubo permiso; recuerda, además, que se revocó.

La auditoría sale gratis. Cada acceso es, también, una situación: «el usuario de la aseguradora consultó la situación arritmia_diagnóstico_2026 el 2028-02-01 a las 10:14». El registro de auditoría no es un subsistema de

logging que haya que construir y proteger aparte: es más hechos en el mismo grafo, consultables con las mismas siete preguntas. «Quién vio qué y cuándo» pasa de ser una pesadilla forense a una consulta de una línea.

Redacción por eje

Hay una asimetría de granularidad que juega a favor del modelo. Una base relacional concede acceso por tabla o, con esfuerzo, por fila: o ves el registro de la transacción, o no lo ves. WQuestions concede acceso por **hecho** e incluso por **eje**.

Porque cada situación descompone la realidad en coordenadas tipadas y separadas, se puede revelar unas y sellar otras. Un auditor financiero puede recibir la prueba de que una transacción **ocurrió** —su *qué* y su *cuándo*— sin ver jamás el *cuánto* ni la contraparte. Un investigador epidemiológico puede saber que existió un diagnóstico de cierta clase en cierta fecha y lugar, con el *quién* sellado bajo llave.

La misma situación, dos divulgaciones distintas

situación: «transacción_2026_8841»

Auditor financiero	Investigador epidemiológico
qué (O): transferencia ✓ visible	qué (O): — sellado — ● oculto
cuándo (T): 2026-04-09 ✓ visible	cuándo (T): 2026-04-09 ✓ visible
cuánto (N): — sellado — ● oculto	cuánto (N): — sellado — ● oculto
quién (Q): — sellado — ● oculto	quién (Q): — sellado — ● oculto

El auditor sabe que la transacción ocurrió, sin ver el monto ni la contraparte.
El investigador solo recibe la fecha. La minimización es estructural, no una promesa.

Redacción por eje: la misma situación, mostrada a dos solicitantes distintos. Unos ejes se revelan (qué, cuándo); otros quedan bajo candado (cuánto, quién). La divulgación diferencial está en la geometría del hecho, no añadida después.

Esto no es un parche encima del modelo: es la geometría del hecho atómico haciendo trabajo de privacidad. La minimización de datos —el principio legal de no entregar más de lo necesario para el fin— deja de ser una promesa de buena conducta y se vuelve algo que la estructura del dato permite expresar con precisión quirúrgica.

Las tensiones que quedan crudas

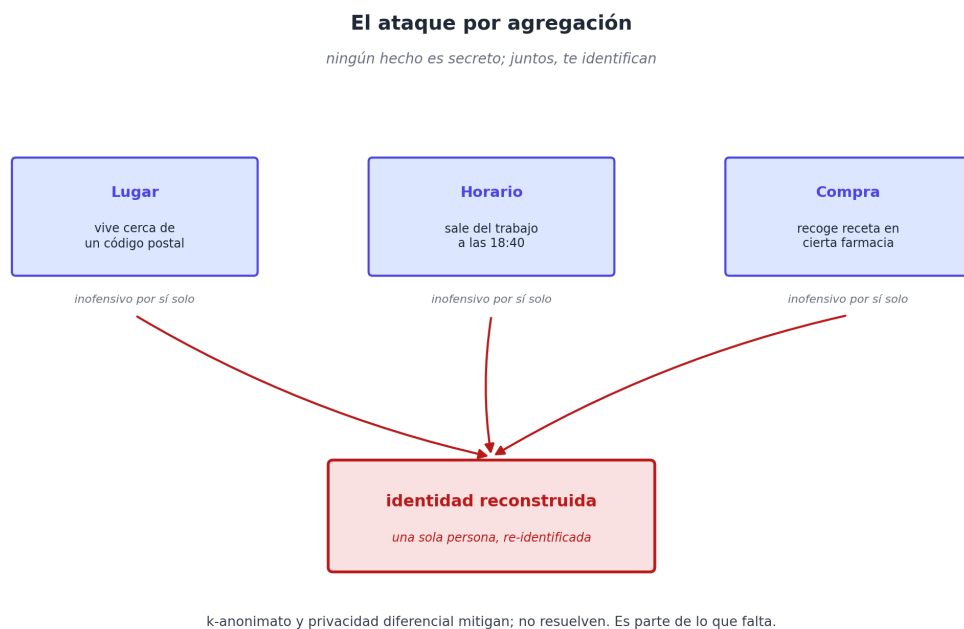
Sería deshonesto pintar esto como resuelto. Quedan tres tensiones reales, y la última no la resuelve nadie todavía —para ningún modelo de datos enlazable.

El sistema que nunca olvida frente al derecho al olvido. La vigencia temporal (D6) supersede, no borra; el RGPD, en cambio, concede el derecho a la *eliminación* efectiva. La salida pasa por distinguir dos cosas que suenan igual: la *vigencia lógica* (un hecho deja de ser verdad) y el *borrado físico* (el contenido se destruye de verdad). La técnica concreta se llama **crypto-shredding**: el contenido de los ejes de valor se guarda cifrado, y «borrar» es destruir la llave —lo que queda es ruido irrecuperable, aunque el nodo-situación persista para integridad referencial y auditoría—. Y aquí el modelo ayuda en vez de estorbar: como cada eje declara explícitamente *dónde* vive cada dato

personal, el borrado dirigido es **más** factible que en el blob relacional, donde el mismo dato sensible está embarrado en columnas, índices, logs y respaldos.

La identidad estable es a la vez el arma y la herramienta. El poder del modelo viene de que la persona es la misma entre dominios; eso es, exactamente, lo que habilita los ataques de re-identificación. La mitigación se llama **seudónimos pareados**: la persona es una sola entidad lógica, pero cada relación la ve bajo un identificador propio y acotado a ese dominio —como usar un nombre distinto en cada ventanilla, donde solo un resolutor autorizado sabe que son la misma persona—. Vincular dos handles deja de ser gratis y pasa a requerir autorización explícita. El «enchufe» de URIs resolubles que vimos en el Capítulo 3 es justamente el punto donde esa política de resolución puede interponerse.

El frente genuinamente abierto: inferencia y agregación. Aun con permisos por hecho y redacción por eje, un modelo que unifica las preguntas hace *más* fácil el ataque por combinación: varios hechos inocuos por separado, cruzados, reconstruyen lo que cada uno por sí solo no revelaba. El lugar donde vives, la hora a la que sales, la farmacia que frecuentas —ninguno es secreto; juntos, te identifican.



El ataque por agregación: tres hechos que por separado son inofensivos —un lugar, un horario, una compra— convergen y reconstruyen una identidad que ninguno revelaba solo. Es el frente que ningún modelo enlazable resuelve por completo.

Esto **no lo resuelve limpiamente nadie**, y WQuestions, al facilitar el enlace por diseño, tiene la obligación de tomárselo *más* en serio, no menos. El k-anonimato y la privacidad diferencial —añadir ruido calibrado a las respuestas agregadas para que ningún individuo sea distinguible— son mitigaciones serias en la capa de consulta, pero son mitigaciones, no soluciones. Por eso este problema pertenece, con todas las letras, al capítulo siguiente: es parte de lo que falta.

El veredicto

Recapitulando sin adornos:

- El grafo compartido es un **idioma, no una base central**. La privacidad empieza por la federación: cada tenedor controla lo suyo y solo une bajo consentimiento.
- El permiso, el consentimiento y la auditoría son **situaciones reificadas** —se expresan en el mismo modelo que los datos, no en una capa paralela que se desincroniza—.

- El control fino por eje y el borrado dirigido por crypto-shredding son **más factibles** aquí que en lo relacional, porque el modelo sabe explícitamente dónde vive cada dato.
- Pero la misma vinculabilidad que le da su fuerza convierte a la **inferencia por agregación** en el riesgo central, todavía sin solución cerrada.

La conclusión operativa es incómoda y conviene decirla en voz alta: **una arquitectura de información que no piensa la privacidad desde el día uno no merece ser adoptada.** Un modelo que vuelve trivial preguntar cualquier cosa sobre cualquiera es tan capaz de habilitar la mejor medicina como el peor abuso, y la diferencia entre ambos no está en la potencia del grafo sino en quién puede preguntar qué. Que esa pregunta —quién puede preguntar qué— se responda con el mismo modelo, y no con un sistema aparte pegado con cinta, es la mejor noticia de este capítulo. Que la inferencia siga abierta es la peor. Las dos son verdad, y la siguiente página empieza, justamente, por admitir todo lo que falta.

Capítulo 28 — Qué falta: validación, tooling, comunidad

El contrato del cierre

Hasta acá el libro presentó la propuesta completa y la sometió a prueba: un prototipo en Python con tests pasando, ocho dominios modelados que confirman que el catálogo se sostiene en territorios disímiles, y —en el Capítulo 26— el experimento reflexivo, donde el modelo bastó para describir su propia herramienta y se corrigió a sí mismo bajo carga.

Este capítulo cambia de registro deliberadamente. No es una recapitulación; no es una mirada al horizonte. Es un **mapa de implementación**: lo que falta para que WQuestions deje de ser una propuesta y se vuelva una pieza de infraestructura adoptable. Está pensado para quien lea el libro y quiera empujar el proyecto adelante — sea como contribuyente al núcleo, como adoptante temprano en su organización, o simplemente como observador que necesita medir el camino que queda.

Lo organizo en seis frentes, cada uno con tres cosas: **qué falta, qué tan urgente es, qué requiere para resolverlo**. Donde la prueba reflexiva ya adelantó terreno, lo señalo — porque cambia el punto de partida de varios frentes. Cierro con la pieza que importa más de todas — y la que el autor no controla solo.

Roadmap: seis frentes entre el prototipo y la infraestructura

Cada frente con su prioridad, esfuerzo estimado y dependencia.



Frentes 1-3 son técnicos y acotados; los hace un equipo chico en seis meses.

El 4 (tooling) acelera adopción. El 5 (lexicon) es trabajo lingüístico sostenido. El 6 (comunidad) es el más decisivo.

Prioridad: ■ alta ■ media ■ sostenida

Los seis frentes que separan al prototipo de la infraestructura industrial: motor de inferencia, bitemporalidad completa, persistencia, tooling, lexicon poblado, comunidad. Cada uno con su prioridad y dependencias.

Frente 1 — El motor de inferencia

Apareció seis veces a lo largo del libro: el evaluador externo que recorre el grafo y dispara reglas. La regla "siete sesiones → una gratis" del spa. La verificación "prescripción versus contraindicación" de la clínica. El cálculo del marcador del partido. La aplicación de la cláusula de rescisión del contrato. Todas comparten la misma forma estructural: una condición declarativa sobre hechos del grafo, un consecuente que se efectúa cuando la condición se cumple.

El experimento reflexivo dio el **primer medio paso**: probó que la conducta puede vivir como dato y ejecutarse desde afuera —un evaluador genérico que despacha verbos leyéndolos del grafo—. Lo que ese motor **todavía no es** es un evaluador de **reglas declarativas**: sabe ejecutar verbos, no resolver "si se cumplen estas condiciones sobre los hechos, deriva este hecho nuevo".

Qué falta: esa capa de evaluación de reglas. Hay al menos cuatro tecnologías candidatas, cada una con su fortaleza:

- **SHACL** ([Shapes Constraint Language](#), W3C 2017) — diseñado para validar grafos RDF. Bueno para chequear consistencia ("ningún paciente puede tener dos diagnósticos contradictorios vigentes"). Limitado para razonamiento positivo.
- **Datalog** — lenguaje lógico deductivo. Excelente para razonamiento composicional ("si X causa Y, e Y causa Z, entonces X causa Z transitivamente"). Bibliotecas como Soufflé tienen rendimiento industrial.
- **Código de aplicación** — funciones Python/Rust/Go que reciben el grafo y producen el efecto. Lo más flexible, lo menos auditable. Es, esencialmente, la forma que tomó el evaluador de verbos del prototipo.
- **LLM con function calling** — usar al propio LLM como evaluador para reglas borrosas o ambiguas. Útil para reglas pragmáticas; no apto para reglas de cumplimiento estricto.

Urgencia: alta. El prototipo ejecuta comportamiento, pero todavía no *infiere*: almacena la regla del spa, no la dispara. Para una mayoría de aplicaciones útiles, eso es un bloqueador.

Qué requiere: una **API de motor de inferencia** que el universo exponga (`u.evaluate(rule_id)`, `u.evaluate_all()`), con resultados que sean nuevamente hechos atómicos firmados por el evaluador). El primer paso es elegir un motor referencia —SHACL para validación, Datalog para inferencia— e integrarlo sobre el evaluador genérico que ya existe. Trabajo estimado: tres a seis meses para un primer release usable.

El lugar de la IA en este frente. La cuarta opción de la lista —usar un LLM— merece una aclaración, porque la tentación es grande: ¿no podría una IA, incluso open-source y autoalojada, ser el motor de inferencia, dado un buen prompt y un modelo bien definido? Hay que separar dos clases de regla. Para las **borrosas** —¿este reclamo es urgente?, ¿este texto implica consentimiento?— la respuesta es sí, y ahí el LLM es la herramienta correcta. Para las **estrictas** —las del spa, el banco, el contrato— la respuesta es no, por cuatro razones que ningún prompt arregla: el LLM es **probabilístico** (no determinista); no entrega una **derivación auditable** (y la tesis del modelo es justamente la auditabilidad: hechos firmados, bitemporales); es poco confiable en **conteo y agregación exactos** (contar goles, sumar costos, cerrar transitividades); y es caro de **escalar** y de **reproducir** una decisión pasada.

La forma productiva de sumar IA, entonces, no es ponerla de *runtime* sino de **autor**: que un LLM **compile** una regla escrita en lenguaje natural —"a las siete sesiones, la octava es gratis"— a una regla declarativa Datalog/SHACL sobre los hechos del grafo, y que el **motor determinista la ejecute**. El modelo bien definido vuelve esa compilación directa, porque los roles ya están tipados y la regla mapea a patrones sobre hechos canónicos. Así se obtiene la flexibilidad del lenguaje natural para *escribir* reglas y el rigor del motor para *ejecutarlas*: la IA entra dos veces —parsear lenguaje a hechos (Capítulo 24) y compilar reglas— pero el corazón de la inferencia estricta queda determinista y demostrable. Una nota a favor del open-source: un modelo autoalojado, con sus pesos fijados, es **más** reproducible que una API cerrada que cambia bajo los pies — y la reproducibilidad es exactamente lo que una auditoría exige.

En concreto, esa compilación se ve así: el LLM recibe la regla en español y el catálogo tipado, y emite una regla declarativa que el motor ejecuta.

SISTEMA. Compila la regla de negocio a Datalog sobre hechos WQuestions (sujeto, rol, valor). No la ejecutes; solo compílala.

Catálogo: servicio_spa(S) con roles cliente, estatus_factual.

Regla: "A las siete sesiones finalizadas de un cliente, la octava es gratis."

```
sesion_ok(C, S) :- instancia_de(S, servicio_spa),
                  cliente(S, C), estatus_factual(S, finalizada).
octava_gratis(C) :- count{ S : sesion_ok(C, S) } >= 7.
```

El LLM *escribió* la regla; el motor determinista la *ejecuta* y firma cada hecho derivado. La IA nunca toca el dato en caliente.

Frente 2 — Bitemporalidad completa

Hoy el modelo soporta **valid time**: cada hecho lleva su rango `[valid_from, valid_to]`. Una consulta `at=T` recupera lo cierto del mundo en ese momento. Lo que **falta** es **transaction time**: cuándo el sistema afirmó ese hecho.

La diferencia importa cuando alguien pregunta no "*¿qué era cierto?*" sino "*¿qué sabíamos?*". Si en mayo de 2024 registramos que el plan mensual de un cliente venció el 31 de marzo, pero el cliente apareció en abril a reclamar y mostró que renovó, retrocedemos el plan. La pregunta del auditor "*¿qué estaba en el sistema entre el 1 de abril y la fecha de la corrección?*" solo se responde si el sistema preservó esa versión transitoria.

Qué falta: cerrar el transaction time de punta a punta. La estructura del hecho ya reserva un `tx_time`, pero la persistencia aún no lo conserva en el viaje de ida y vuelta a disco — queda como deuda concreta. El destino es un par de intervalos por hecho y consultas *as-of dual*: `query(at_valid=T1, at_tx=T2)`. Snodgrass [17] formalizó esto en los noventa; las bases bitemporales modernas (Datomic, XTDB) lo implementan en producción.

Urgencia: alta para dominios regulados (finanzas, salud, derecho); baja para el resto. Cuando un dominio lo necesita, no es opcional.

Qué requiere: una refactorización compatible del módulo `fact.py` para llevar dos intervalos en vez de uno, y de la capa de persistencia para preservarlos. Trabajo estimado: un mes para implementación, más tests sobre dominio regulado real.

Frente 3 — Persistencia industrial

El experimento reflexivo cambió el punto de partida de este frente: el prototipo **ya no vive solo en memoria**. Su capa de persistencia guarda individuos y hechos en **SQLite** y los recarga al arrancar — una tabla de individuos (`id`, `axis`, `payload_json`) y una de hechos (`subject_id`, `role`, `value_id`, `valid_from`, `valid_to`, ...) bastan. Es la prueba de que el modelo persiste sin esfuerzo.

Qué falta, entonces, no es "persistencia" sino **persistencia industrial y plural**: una **interfaz Storage** abstracta de la que SQLite sea la primera implementación, más backends con perfiles distintos:

- **SQLite** — para sistemas chicos y monousuario. **Ya existe** en el prototipo.
- **Postgres + JSONB** — para sistemas multi-usuario medianos. Aprovecha índices GIN sobre payload, FTS sobre etiquetas, partitioning por `tx_time` para auditoría.
- **Kùzu o Neo4j** — para sistemas que privilegian consultas de grafo (rutas, caminos transitivos). Modelo más natural pero curva de adopción mayor.
- **RDF/SPARQL** (Apache Jena, GraphDB) — para sistemas que quieran interoperar con la web semántica existente. Mapeo trivial: cada hecho es una tripleta RDF; D6 se mapea con grafos nombrados.

Urgencia: media. El piso (SQLite) ya permite construir; lo que falta habilita escala y casos exigentes.

Qué requiere: extraer la **interfaz Storage** del SQLite actual y escribir al menos un segundo backend (Postgres). El módulo `universe.py` delega lectura/escritura a esa interfaz. Trabajo estimado: dos a tres meses para los dos backends primarios.

Frente 4 — Tooling

Una propuesta como WQuestions vive o muere por su tooling. Los conceptos pueden ser elegantes; si la fricción operativa es alta, nadie adopta. **Qué falta** se enumera por prioridad:

4.1 — Lexicon ingestor. Hoy las entradas del lexicon se escriben a mano. Hace falta una herramienta que ingiera entradas desde FrameNet [14], VerbNet [15], PropBank — recursos masivos ya contruidos — y produzca entradas válidas del lexicon WQuestions. Esto da un piso de cobertura de miles de verbos sin trabajo manual.

4.2 — Parser de lenguaje natural a hechos. Hoy el parsing se hace vía LLM con function calling. Hace falta también un parser **local y determinístico** para textos donde la latencia del LLM importa (sistemas de tiempo real) o donde la privacidad lo exige (datos médicos). Combinar análisis sintáctico con el lexicon es trabajo concreto, no investigación abierta.

4.3 — IDE / inspector. El experimento reflexivo entregó una **primera versión**: un inspector que, junto a cada vista, muestra las tripletas que la sostienen ("lo que ves = datos"). Falta el resto del IDE: explorar individuos y vecinos a voluntad, consultar con patrones, seguir la cadena `causado_por / justificado_por` de cualquier nodo, y —la pieza que el propio experimento señaló como necesaria— **vistas con nombre** definidas como datos, para concretar el grafo sin ahogar al modelador en la abstracción. La base existe; el salto siguiente es de semanas, no de meses.

4.4 — Validador de migración. Cuando un sistema legacy quiere migrar a WQuestions, hay que validar que el dialecto de dominio mapea correctamente al canónico. Una herramienta de validación detecta cuellos: roles sin signatura, ejes ambiguos, vigencias inconsistentes. El paso "el dato registra su propia signatura", que la prueba reflexiva ya introdujo, facilita esta herramienta.

4.5 — Generador de servidor MCP. Dado un lexicon, generar automáticamente el servidor MCP correspondiente. Esto convierte cada dominio en un asistente conversacional sin escribir glue code. Trabajo de pocas semanas; impacto desproporcionado en adopción.

Urgencia: media en agregado, alta en el caso del **lexicon ingestor** (4.1) y el **generador MCP** (4.5), porque son los que más reducen el costo de probar la propuesta.

Frente 5 — Lexicon poblado en varios idiomas

El catálogo D8 del libro tiene 38 roles canónicos. El lexicon del prototipo registra unos diez verbos. Para un sistema productivo, el lexicon de un solo idioma necesita del orden de **dos a cinco mil entradas** — los verbos frecuentes del español, sus formas nominales, las locuciones idiomáticas.

Qué falta: trabajo lingüístico paciente. Por dominio o por familia semántica (transferencia, comunicación, movimiento, percepción, estados, etc.), poblar el lexicon. FrameNet español, el spanish corpus de Universal Dependencies, AnCorra son fuentes legítimas para mecanizar parcialmente este trabajo.

Para internacionalización: cada idioma necesita su propio lexicon. El **catálogo D8 es idiomáticamente neutral** (los nombres internos como `agente`, `paciente`, `tema` son etiquetas inglesas pero podrían ser griegas o números). Lo que cambia entre idiomas es la **capa de alias** y los **patrones de polisemia** específicos.

Urgencia: la cobertura del lexicon **es** la usabilidad. Sin verbos suficientes, el modelo se siente incompleto. Esfuerzo sostenido, no proyecto puntual.

Qué requiere: equipo lingüístico-computacional. Idealmente colaboración con universidades que ya hacen recursos léxicos. Modelo de gobernanza para aceptar contribuciones de terceros.

Frente 6 — Comunidad y gobernanza

Acá entramos en la pieza que el autor no controla. WQuestions, para volverse útil más allá de un libro, necesita **comunidad**: gente modelando dominios, contribuyendo lexicon, reportando fricciones, escribiendo herramientas, adoptándolo en proyectos.

Qué falta:

- **Repositorio canónico** abierto, con licencia permisiva. Una primera versión vive en GitHub mientras leés esto.
- **Proceso de contribución**: cómo proponer nuevas entradas al catálogo D8, nuevos dominios al lexicon, parches al motor. Necesita criterios escritos.
- **Foro o canal de discusión**: para resolver fricciones que surjan al modelar dominios nuevos. Cada conversación cualifica el catálogo — y la prueba reflexiva (Capítulo 26) es evidencia de que las fricciones más valiosas aparecen al someter el modelo a una carga real.
- **Estandarización gradual**: una vez que varios proyectos adopten el modelo, vale la pena llevar partes del catálogo (los roles más universales) a un proceso de estandarización formal — IETF, W3C, ISO. Esto da estabilidad legal para uso empresarial.
- **Dialectos de dominio** mantenidos por comunidades sectoriales: clínico, financiero, legal, manufactura. Cada uno con su propia gobernanza dentro de la espina común.

Urgencia: el reloj corre. Si la comunidad no se forma en el momento en que los LLMs con MCP se popularizan, alguna otra propuesta menos cuidadosa ocupará el espacio. La ventana es de dos a cinco años.

Qué requiere: lo mismo que cualquier proyecto open source serio: un autor (o equipo fundador) dispuesto a moderar, criticar contribuciones, mantener la coherencia, decir que no cuando hace falta. Buena documentación. Casos de uso ejemplares. Adoptantes tempranos visibles.



La pila completa: persistencia (Postgres / Kùzu) abajo, encima el núcleo WQuestions con sus 7 ejes, encima el evaluador de reglas y el lexicon, encima el LLM via MCP, encima la aplicación que el usuario final ve. Cada capa puede evolucionar independientemente.

Las fricciones documentadas que siguen abiertas

Además de los seis frentes mayores, el prototipo expuso un puñado de fricciones puntuales al catálogo. La prueba reflexiva (Capítulo 26) **cerró varias** —el texto libre, el tipado de los campos definidos por datos, el display derivado de hechos— y entregó el comodín **V** —*cualquier eje de valor*— que resuelve de raíz la familia "esta signatura es demasiado estrecha". Lo que queda abierto es esto:

Fricción	Origen	Patch propuesto
paciente / partes : 0→Q demasiado estrechos	química, fútbol	Relajar a 0→V — el comodín V ya existe (Capítulo 26); es una línea
tema : 0→0 rechaza K (obra, medicamento)	música, clínica	tema : 0→V , o un tema_categorico : 0→K
Patrones temporales finos; tiempo musical (compás, pulso)	clínica, música	Reificar como O con estructura
Reglas de derivación versionadas	contrato	Frente 1 (motor de reglas) + D6 sobre las reglas
Vistas y proyecciones con nombre como dato	prueba reflexiva	El siguiente escalón de la re-concreción (Frente 4.3)

Ninguna bloquea el funcionamiento del modelo, y un patrón se repite: las relajaciones del tipo "signatura demasiado estrecha" se resuelven con el comodín **V** que el experimento ya introdujo; varias otras quedan cubiertas por roles de dominio bajo la política liberal. Es **la lista de mejoras concretas al catálogo**, que se acumula a medida que el modelo se prueba en territorios nuevos y se acorta a medida que se lo somete a cargas exigentes.

El libro como semilla

Cerremos con la idea menos técnica del capítulo. Un libro no es una propuesta terminada — es **una invitación a que alguien la termine**. Las arquitecturas duraderas — Unix, TCP/IP, HTTP, SQL, Linux, RDF — empezaron como artículos, manifiestos, RFCs, libros: textos que articulaban una idea con suficiente claridad para que otros pudieran apropiársela y empujarla adelante.

WQuestions, en su forma actual, es un texto y un prototipo. El catálogo D8 está bien diseñado pero incompleto; el lexicon está bosquejado; las herramientas son embrionarias; la comunidad está por construirse. Pero la propuesta no solo funciona en ocho dominios distintos: **funciona aplicada a sí misma** — el modelo bastó para describir su propio menú, sus formularios, sus esquemas y su conducta, y para corregirse cuando la carga reveló una signatura demasiado estrecha. Lo que cualquier lector tiene en la mano al cerrar este libro no es un producto sino una **base operable**: suficiente para entender la propuesta, ejecutarla, criticarla, extenderla. La propuesta aguanta el peso que prometió; desde acá la tarea es perfeccionarla.

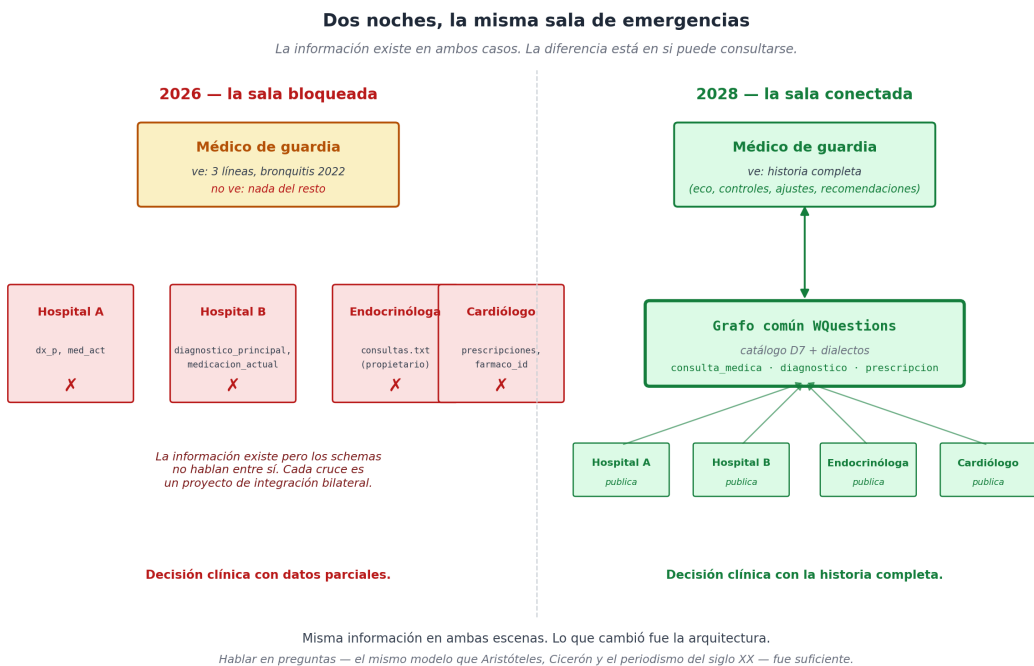
Conclusión — Por qué importan las preguntas

La misma sala, otra noche

Son las dos de la mañana. La paciente vuelve — una mujer de cuarenta y cuatro ahora, dos años más vieja que cuando la conocimos en el primer capítulo de este libro. El dolor torácico que tenía aquella vez resultó ser una crisis de ansiedad. Esta vez es otra cosa: una arritmia que la endocrinóloga le venía monitoreando desde hace meses, y que el cardiólogo del otro hospital le había advertido que vigilara. La paciente lo cuenta entre frases entrecortadas, mientras la enfermera le toma presión y oxígeno.

El médico de guardia no es el mismo de hace dos años. Es nuevo. No la conoce. Saca la tablet, busca el nombre, y aparece — esta vez sí — la historia entera. No traída por un convenio entre hospitales, no por una conexión bilateral, no por un esfuerzo heroico de algún integrador de turno: aparece porque ambos hospitales, junto con la endocrinóloga privada, junto con el cardiólogo de la otra ciudad, **hablan en preguntas**. Cuando hace dos años uno escribía `dx_p` y el otro `diagnostico_principal`, hoy los dos reconocen que están afirmando una situación de tipo `diagnostico_medico` con un paciente, un agente, un momento y un `diagnosticado_como`. El vocabulario interno de cada sistema sigue siendo el suyo; lo que comparten es la pregunta que cada hecho responde.

El médico ve la cronología: el primer eco que detectó la endocrinóloga, los controles trimestrales, el ajuste de medicación de noviembre, la consulta de marzo con el cardiólogo, la recomendación específica para casos de descompensación. Toma una decisión informada, en treinta segundos, sin pedir auxilio a un especialista a las dos de la mañana. La paciente queda estable; el equipo registra la nueva situación reificada — `consulta_de_emergencia_2028_xxxxx` — que mañana otros médicos podrán consultar de la misma manera.



Dos noches comparadas: la misma paciente, el mismo médico de guardia. En 2026 cuatro sistemas con cuatro schemas distintos bloquean la información que ya existe. En 2028 los cuatro publican al grafo común y la historia clínica se reconstruye en segundos.

Esta escena — banal, casi aburrida cuando funciona — es lo que el libro entero quiso explicar cómo conseguir.

Lo que las preguntas resolvieron

Lo que las seis partes del libro destilaron, visto en retrospectiva, es una respuesta razonable a una pregunta vieja: **¿hay algo más simple que una ontología y más estructurado que un texto, que pueda servir de común denominador entre sistemas que no se conocen?**

La respuesta del libro fue: sí, hay siete preguntas. Quién, qué, dónde, cuándo, cuánto, cuál, cómo — donde *cuál* es, justamente, la que organiza las categorías. Esas siete preguntas — anteriores a cualquier ontología, conocidas desde Aristóteles, codificadas por Cicerón, redescubiertas por el periodismo del siglo veinte, formalizadas por la semántica neo-davidsoniana, presentes en la cognición infantil mucho antes que el lenguaje escrito — son lo bastante universales para que cualquier descripción del mundo se mapee a ellas y lo bastante simples para que un humano (o un modelo de lenguaje) las entienda sin manual.

Sobre las preguntas montamos cuatro decisiones de diseño que las vuelven operativas:

- Reificar las situaciones cuando importan (D4).
- Aceptar agencia contextual (D5) — humanos, organizaciones, software, todos pueden ser agentes según el contexto.
- Distinguir cuatro tipos de "por qué" (D7) — causa, motivo, finalidad, justificación — porque el lenguaje natural ya los distingue.
- Preservar la historia con vigencia temporal (D6) — el sistema nunca olvida.

Y sobre eso, una capa de **lexicon** que traduce el lenguaje del usuario al catálogo canónico, sin obligar a nadie a aprender vocabulario interno. El lexicon es, técnicamente, lo mismo que un *function schema* para los modelos de lenguaje de la generación 2026 — y esa coincidencia, que no buscamos al inicio del proyecto, es la que vuelve la propuesta accionable hoy.

Por qué las preguntas son anteriores a las ontologías

Acá es donde el libro toma su única licencia filosófica, y conviene dejarla explícita. Las ontologías de dominio — CIDOC CRM, Schema.org, Biolink, FHIR, las decenas de iniciativas que enumeramos en el capítulo 3 — son catálogos de **qué cosas hay**. Cada una hizo el trabajo paciente de definir entidades, relaciones, vocabularios consistentes para su dominio. Cada una es valiosa dentro de su perímetro.

Lo que el libro propone es que **antes que las entidades están las preguntas**. Antes que *paciente, doctor, diagnóstico, prescripción*, está la pregunta *¿quién hizo qué a quién?* — y las respuestas tipadas. Un dominio nuevo, al modelarse, no se reduce a otro dominio existente: se mapea a las preguntas, que son siempre las mismas. La ontología específica del dominio sobrevive como **dialecto** del lexicon; el catálogo común sobrevive como infraestructura. Cada ontología sigue siendo valiosa donde ya está; lo que cambia es la posibilidad de hablar entre ellas sin proyectos de integración.

Esa intuición — que las preguntas son lo invariante mientras las ontologías son lo variable — no es original ni del libro ni de su autor. Está en Aristóteles. Está en Cicerón. Está en los primeros manuales de periodismo del siglo veinte. Está en cómo los niños adquieren el lenguaje. Está en la semántica de eventos que la lingüística formal viene refinando hace sesenta años. Lo único que este libro hizo fue **proponer una articulación operativa** de esa intuición vieja, con un catálogo concreto, un prototipo ejecutable, y ocho dominios modelados como prueba de que el catálogo se sostiene fuera de un solo nicho.

Lo que el libro no terminó

Sería deshonesto cerrar sin admitir lo que falta. El capítulo 28 lo enumeró sin maquillaje: motor de inferencia, bitemporalidad completa, persistencia industrial, tooling, lexicon poblado en varios idiomas, comunidad. La propuesta es completa **conceptualmente**; no está lista para producción. El prototipo en Python que acompaña al

libro ejecuta lo que el libro afirma — los ocho dominios pasan sus validaciones, los 23 tests pasan, los 12 ejemplos corren — pero está lejos de ser infraestructura.

Lo que también falta, y conviene decirlo en voz alta, es que ninguna arquitectura sobrevive por su elegancia. Las que sobrevivieron — Unix, TCP/IP, HTTP, SQL — sobrevivieron porque hubo gente que las cuidó durante décadas. RDF, que cumple veinticinco años en 2026, sigue siendo un activo subutilizado precisamente porque la comunidad nunca alcanzó la masa crítica para volverlo invisible. WQuestions, hoy, está en el día uno de ese proceso.

La propuesta tiene valor — los ocho dominios que pasamos por el prototipo lo demostraron — y ahora la tarea es que otros la perfeccionen. Por eso vale la pena que exista escrita: la siguiente generación de modelos de información tiene acá una base operable de la que partir.

Una última nota sobre el momento

Hay una coincidencia histórica que el libro mencionó solo lateralmente y que conviene decir en limpio acá. Esta propuesta — un modelo de información organizado sobre preguntas-coordenadas, traducido por un lexicon que es a la vez catálogo de funciones — habría sido difícil de adoptar hace cinco años. Faltaba la capa de fluidez lingüística que la vuelve usable. Hoy, con LLMs capaces de hacer *function calling* sobre miles de herramientas y de hablar en cualquier idioma con la naturalidad de un usuario nativo, esa capa **existe**.

Las dos piezas — el grafo persistente con identidad estable, el LLM con fluidez conversacional — son complementarias en un sentido fuerte: lo que una no hace, la otra sí. Y la pieza que las conecta — el lexicon como function schema — resulta ser exactamente lo que se necesita en ambos lados de la frontera.

Esto es lo que justifica el libro hoy, en 2026, y no antes ni después. Antes no había con qué; después puede haber demasiadas propuestas competidoras, cada una con un compromiso de adopción que no queremos pagar. La ventana entre que los LLMs maduraron y que el espacio se consolide en torno a un estándar dominante es pequeña. Si en esa ventana las preguntas-coordenadas aparecen como una opción razonable, el esfuerzo del libro habrá valido.

Cierre

Empezamos con una sala de emergencias donde la información existía pero no podía consultarse. La cerramos con otra noche en la misma sala, dos años después, donde la información sigue existiendo y ahora se consulta. Entre las dos noches no hubo magia tecnológica: hubo el reconocimiento de algo viejo — que las preguntas que hace dos mil años se le hacían a un acto moral son las mismas que se le hacen hoy a una historia clínica — y la disciplina de construir, encima, las piezas mínimas para que ese reconocimiento sea operativo.

Quien lea este libro y quiera aportar al proyecto encontrará el repositorio en línea, el prototipo ejecutable, los ocho dominios modelados, las decisiones de diseño documentadas. No es mucho — apenas suficiente para entender la propuesta y criticarla. Pero esa es exactamente la idea: el libro como semilla, no como punto final.

Las preguntas seguirán siendo las mismas. Lo que falta es construir entre todos las respuestas.

Anexo — Código del prototipo en Python

Este anexo recopila, en un solo lugar y por capítulo, **todos los fragmentos de código Python** que aparecen en la Parte V del libro (capítulos 14 al 19). El propósito es doble:

1. **Ser una referencia rápida para quien quiera implementar.** El lector que vaya a programar contra el prototipo no necesita ir saltando entre capítulos; aquí tiene el código consolidado, con suficiente contexto para entender cada bloque sin volver al texto.
2. **Servir como vista panorámica del estilo del modelo.** Mirar todo el código de la Parte V en una pasada permite ver patrones que el flujo capítulo-a-capítulo diluye: la uniformidad de la API, la economía de llamadas, la consistencia del catálogo.

Las APIs principales que aparecen a lo largo del anexo son cuatro:

- `ingest_situation(u, lex, verbo, roles=..., complements=..., ...)` — crea una situación reificada en el universo `u`, resuelve el verbo en el lexicon `lex`, asienta los hechos atómicos correspondientes.
- `u.assert_fact(sujeto, rol, valor, valid_from=..., valid_to=...)` — agrega un hecho atómico directo al universo, con bitemporalidad opcional (D6).
- `query(u, Pattern(fixed=..., ask=..., type_constraint=...), at=...)` — consulta el grafo por un patrón de hechos. `at` proyecta la consulta a un momento de tiempo de validez.
- `count(u, Pattern(...))` — variante de `query` que solo devuelve la cardinalidad.

El prototipo completo vive en `prototipo/wq/` del repositorio. Lo que sigue son los fragmentos publicados en el libro, en orden de aparición.

§15 — Spa comercial

§15.1 — Crear una sesión articuladora

Ingesta de la sesión de Ana en la cámara de vapor 1 (situación reificada con cliente, lugar, ventana temporal y estatus factual). El parámetro `complements=["sesion"]` dispara la resolución de polisemia del verbo *tomar*: sin él, el sistema no sabría si "tomar" significa "tomar una sesión" o "tomar el pelo".

```
sit = ingest_situation(u, lex, "tomar",
    roles={
        "cliente": ana,
        "lugar_de": camara_vapor_1,
        "inicio": t_inicio,
        "fin": t_fin,
    },
    complements=["sesion"],          # dispara polisemia
    extra={"estatus_factual": finalizada},
    sit_id="sesion_ana_07",
)
```

§15.2 — Registro de polisemia en el lexicon

Tres entradas del lexicon para el mismo verbo *tomar*. Cada una con su propio `pattern` (el complemento que la dispara) y su propio `situation_type`. El parser elige del más específico al más general.

```
lex.register(LexiconEntry(
    verb="tomar",
    situation_type="servicio_spa",
    obligatory=["cliente", "lugar_de"],
    pattern=("sesion",), # dispara cuando el complemento es "sesion"
))

lex.register(LexiconEntry(
    verb="tomar",
    situation_type="accion_bromear",
    obligatory=["agente", "paciente"],
    pattern=("el_pelo",), # locución idiomática
))

lex.register(LexiconEntry(
    verb="tomar",
    situation_type="accion_tomar",
    obligatory=["agente", "tema"],
    notes="genérico - recibir/asir",
))
```

§15.3 — Dialecto de dominio del spa

El dialecto activa los alias del negocio del spa: cliente, sesión, plan mensual, sesión gratuita. El usuario habla en este vocabulario y el sistema traduce a los roles canónicos sin que se entere.

```
lex.register_domain_dialect("spa_oasis", {
    "cliente": "agente",
    "sesion": "servicio_spa",
    "plan_mensual": "contrato_servicio",
    "sesion_gratuita": "beneficio_fidelidad",
})
```

§15.4 — Consulta: ¿cuántas sesiones lleva Ana este mes?

Patrón con sujeto fijo (Ana) y constraint de tipo (servicio_spa). El resultado es el conteo directo.

```
n = count(u, Pattern(
    fixed={"cliente": ana, "estatus_factual": finalizada},
    type_constraint=u.ind("servicio_spa"),
))
# n == 8
```

§15.5 — Consulta: ¿Ana califica para el beneficio?

Una decisión de negocio (regla "siete sesiones → una gratis") expresada como código Python que consume el resultado de la consulta anterior.

```
qualifies = n >= 7
# True
```

§15.6 — Consulta bitemporal: ¿dónde vivía Carlos en 2022?

Demostración de D6 (vigencia temporal): la misma consulta sobre el mismo individuo retorna respuestas distintas según el `at=` que se le pasa. Carlos vivió en `ciudad_a` hasta 2025 y luego se mudó a `ciudad_b`.

```

res_2022 = query(u, Pattern(
    fixed={"agente": carlos},
    ask={"lugar_de": Var()},
    type_constraint=u.ind("residencia"),
), at=datetime(2022, 6, 1, tzinfo=timezone.utc))
# res_2022[0]["lugar_de"].id == "ciudad_a"

res_2027 = query(u, ..., at=datetime(2027, 6, 1, tzinfo=timezone.utc))
# res_2027[0]["lugar_de"].id == "ciudad_b"

```

§16 — Servicio on-demand (app de taxi)

§16.1 — Asignación con software como agente (D5)

El `agente` del verbo *asignar* es el app — no Valeria, no Luis. Cuatro participantes con roles distintos: app (agente, Q), solicitud previa (tema, O), conductor (beneficiario, Q), vehículo (instrumento, O). D5 en su forma más interesante.

```

asig = ingest_situation(u, lex, "asignar", roles={
    "agente":      app,          # ¡APP COMO Q!
    "tema":        sol,          # asigna la solicitud previa
    "beneficiario": luis,         # al conductor disponible
    "instrumento": vehiculo,      # con su vehículo
    "momento":     at(1),
}, sit_id="asig_001")

```

§16.2 — Recuperar la cadena de un viaje

El viaje completo es una entidad superior que contiene las seis situaciones del recorrido (solicitar, asignar, aceptar, recoger, trasladar, completar) más la tarifa. Consultar `parte_de` con valor `viaje_001` devuelve toda la cadena.

```

partes = [f.subject for f in u.facts_with_role("parte_de")
           if f.value.id == "viaje_001"]
# devuelve las 6 situaciones + la tarifa

```

§16.3 — Surge pricing: causalidad emergente

El estado de alta demanda se reifica como una situación con su momento y lugar; la tarifa elevada queda explicada por `causado_por` apuntando a ese estado. Recuperar la explicación es un salto del grafo.

```

estado_demanda = u.add_individual(Individual(
    id="alta_demanda_2026_05_16_14_30", axis=Axis.0,
    label="alta demanda 16/5 14:30"))
u.assert_fact(estado_demanda, "instancia_de", alta_demanda)
u.assert_fact(estado_demanda, "lugar_de", plaza)
u.assert_fact(estado_demanda, "momento", at(0))

tarifa = u.add_individual(Individual(id="tarifa_viaje_001", axis=Axis.0))
u.assert_fact(tarifa, "instancia_de", category("tarifa"))
u.assert_fact(tarifa, "monto", n_25_usd)
u.assert_fact(tarifa, "causado_por", estado_demanda)

```

§16.4 — Cancelación como hecho nuevo (nunca borramos)

La convención del modelo es no borrar ni sobrescribir. Una cancelación es una situación nueva que `cancela` la previa y le pone `estatus_factual: cancelado`. El historial queda intacto.

```
canc = ingest_situation(u, lex, "cancelar", roles={
    "agente": valeria,
    "tema": viaje2,
    "momento": at(62),
}, sit_id="canc_001")
u.assert_fact(canc, "cancela", viaje2)
u.assert_fact(viaje2, "estatus_factual", cancelado)
```

§16.5 — Consulta: ¿adónde llevó Luis a Valeria?

Patrón con agente y paciente fijos y `destino` como variable a resolver. Constraint de tipo `accion_trasladar`.

```
r = query(u, Pattern(
    fixed={"agente": luis, "paciente": valeria},
    ask={"destino": Var()},
    type_constraint=u.ind("accion_trasladar"),
))
# r[0]["destino"].id == "aeropuerto"
```

§16.6 — Consulta: ¿cuántos viajes completó Luis hoy?

Conteo con dos restricciones: agente y estatus factual. Filtra viajes en estado `completado`.

```
n = count(u, Pattern(
    fixed={"agente": luis, "estatus_factual": completado},
    type_constraint=u.ind("viaje"),
))
```

§16.7 — Recuperar la causa de una tarifa

Iteración manual sobre los hechos de la tarifa para encontrar el `causado_por`. Permite explicar al usuario por qué un viaje específico costó lo que costó.

```
explicaciones = u.facts_about(tarifa_viaje_001)
causa = [f for f in explicaciones if f.role == "causado_por"]
# causa[0].value.id == "alta_demanda_2026_05_16_14_30"
```

§17 — Historia clínica

§17.1 — Roles de dominio que apuntan a K

En clínica, "el medicamento prescrito" no es un objeto físico individual (O) sino una categoría farmacológica (K). El catálogo canónico declara `tema: 0 → 0`, así que el dominio agrega su propio rol `medicamento_prescrito: 0 → K` mediante la política liberal del catálogo. Lo mismo con `medida_de`.

```
# El medicamento prescrito es una categoría (K), no un objeto.
u.assert_fact(pres, "medicamento_prescrito", enalapril)

# La variable medida es una categoría también.
u.assert_fact(medicion, "medida_de", presion_arterial)
```

§17.2 — Rediagnóstico con vigencia temporal (D6)

Un diagnóstico se reemplaza por otro vía bitemporalidad. El primer diagnóstico es válido entre `t_consulta` y `t_redx`; el segundo, desde `t_redx` en adelante (sin `valid_to`). La relación `rectifica` deja la trazabilidad del

cambio.

```
# Diagnóstico original (vigente hasta el rediagnóstico)
u.assert_fact(diag, "diagnosticado_como", hta_g1,
              valid_from=t_consulta, valid_to=t_redx)

# Rediagnóstico (vigente desde T1, abierto al futuro)
u.assert_fact(diag2, "diagnosticado_como", hta_g2,
              valid_from=t_redx)
u.assert_fact(diag2, "rectifica", diag)
```

§17.3 — Prescripción con motivo, finalidad y verificación normativa

Tres relaciones del "por qué" (D7) sobre la misma situación: `motivado_por` apunta al diagnóstico, `con_finalidad` al objetivo terapéutico, `verificado_contra` a la regla farmacológica que confirma que la prescripción no contradice una contraindicación. Las tres coexisten sin pisarse.

```
pres = ingest_situation(u, lex, "prescribir", roles={
    "agente": dra_torres,
    "paciente": maria_g,
    "medicamento_prescrito": enalapril,
    "frecuencia": cada_manana,
    "duracion": indefinida,
    "momento": at(0),
})

u.assert_fact(pres, "motivado_por", diag)      # por qué se prescribió
u.assert_fact(pres, "con_finalidad", objetivo) # para qué se prescribió
u.assert_fact(pres, "verificado_contra", regla_contraindicacion)
```

§18 — Banco regional

§18.1 — Recuperar las dos contrapartidas contables de una transferencia

La transferencia operativa y los dos asientos contables (débito y crédito) son tres situaciones distintas ligadas por `parte_de`. Recuperar los asientos es un filtro adicional sobre la etiqueta.

```
asientos = [f.subject for f in u.facts_with_role("parte_de")
             if f.value.id == "transferencia_001"
             and f.subject.label.startswith("asiento_")]
# devuelve [asiento_debito_001, asiento_credito_001]
```

§19 — Cuatro dominios incómodos

§19.1 — Marcador de fútbol como estado derivado

El marcador "Argentina 1 - Perú 0" no es un hecho que alguien asienta; emerge de contar los goles registrados por cada equipo. La consulta es un conteo doble con composición de strings final.


```
n_goles_arg = count(u, Pattern(
    fixed={"equipo": argentina},
    type_constraint=u.ind("evento_gol"),
))
n_goles_per = count(u, Pattern(
    fixed={"equipo": peru},
    type_constraint=u.ind("evento_gol"),
))
marcador = f"{n_goles_arg} - {n_goles_per}"
```

§20 — WQuestions y los modelos de lenguaje

§20.1 — Ingesta médica generada por un LLM

Un LLM procesa una nota clínica en lenguaje natural y produce cinco invocaciones `ingest_situation`, una por verbo identificado en el texto. La extracción es directa porque el lexicon clínico ya expone las firmas como function schemas.

```
ingest_situation("consultar", roles={
    "agente": dra_torres, "paciente": maria_g,
    "momento": fecha_nota, "motivo": cefalea,
})
ingest_situation("medir", roles={
    "agente": dra_torres, "paciente": maria_g,
    "medida_de": presion_arterial,
    "monto": "145/92", "unidad": mmHg,
})
ingest_situation("diagnosticar", roles={
    "agente": dra_torres, "paciente": maria_g,
    "diagnosticado_como": hta_g1,
})
ingest_situation("prescribir", roles={
    "agente": dra_torres, "paciente": maria_g,
    "medicamento_prescrito": enalapril,
    "frecuencia": cada_manana,
})
ingest_situation("controlar", roles={
    "paciente": maria_g, "agente": dra_torres,
    "momento": fecha_nota_plus_30,
    "estatus_factual": previsto,
})
```

§20.2 — Consulta cross-dominio con LLM

El LLM cruza dos dominios distintos (política y retail) emitiendo dos consultas paralelas. Cada una tiene su propio `type_constraint`; el LLM las combina y produce la respuesta narrativa al usuario.

```
noticias = query(Pattern(
    fixed={"tema_categorico": impuesto_consumo},
    type_constraint="noticia_politica",
    ask={"agente": Var(), "momento": Var(), "contenido": Var()},
))

ventas = query(Pattern(
    fixed={"sector": retail},
    type_constraint="agregado_ventas_trimestral",
    ask={"momento": Var(), "monto": Var()},
))
```

Patrones que emergen al verlo todo junto

Si se revisan los 20 fragmentos en serie, se observan cinco regularidades que valen la pena nombrar:

1. **La API es realmente pequeña.** `ingest_situation`, `assert_fact`, `query`, `count`. Con esos cuatro verbos se modelan ocho dominios completos. No hay 200 funciones que aprender; hay cuatro.
2. **Las llamadas son densas pero uniformes.** Una llamada típica tiene entre 5 y 10 líneas. El patrón visual es siempre el mismo: nombre del verbo, diccionario de roles, opcionales al final. Una vez que el lector reconoce el patrón, el código se lee como prosa.
3. **El verbo se elige primero, los roles después.** Esto no es coincidencia: refleja la signatura tipada del verbo (capítulo 11). Primero el contrato, después los argumentos.
4. **La política liberal del catálogo se ejerce constantemente.** `medicamento_prescrito`, `medida_de`, `diagnosticado_como`, `equipo`, `tema_categorico`, `sector`, `frecuencia`, `duracion` — todos son roles de dominio que ningún capítulo declara explícitamente y, sin embargo, el sistema los acepta. El motor solo valida los roles canónicos; el resto pasa.
5. **La bitemporalidad y la inmutabilidad son la regla.** Casi todos los fragmentos importantes asientan con `valid_from`/`valid_to`, marcan `estatus_factual`, o `rectifican` sin borrar. El modelo no tiene una operación "update": tiene "asentar un hecho nuevo que reemplaza al previo conservando el historial".

Estas cinco regularidades, juntas, son lo que hace que el núcleo de modelado de dominios sea tan corto (apenas 2.250 líneas) a pesar de cubrir territorios tan distintos como un spa, un banco regional, una historia clínica y un partido de fútbol. El motor no crece con cada dominio nuevo; el lexicon y los datos sí. La prueba más fuerte de esa propiedad llegó después, fuera de estos fragmentos: la aplicación meta-driven del Capítulo 26 —con menús, formularios y entidades enteras— se construyó sobre el mismo núcleo sin agregarle prácticamente código, porque también su estructura y su conducta son datos.

Anexo — El prototipo en Python: librería núcleo

Este anexo contiene el **código fuente íntegro de la librería núcleo** del prototipo de WQuestions (~850 líneas). Es lo que da cuerpo a las afirmaciones del libro: las siete coordenadas, los hechos atómicos con signatura, las situaciones reificadas, el catálogo canónico de roles, el lexicon con resolución de polisemia, la bitemporalidad y el motor de consulta — todo cabe en nueve archivos cortos.

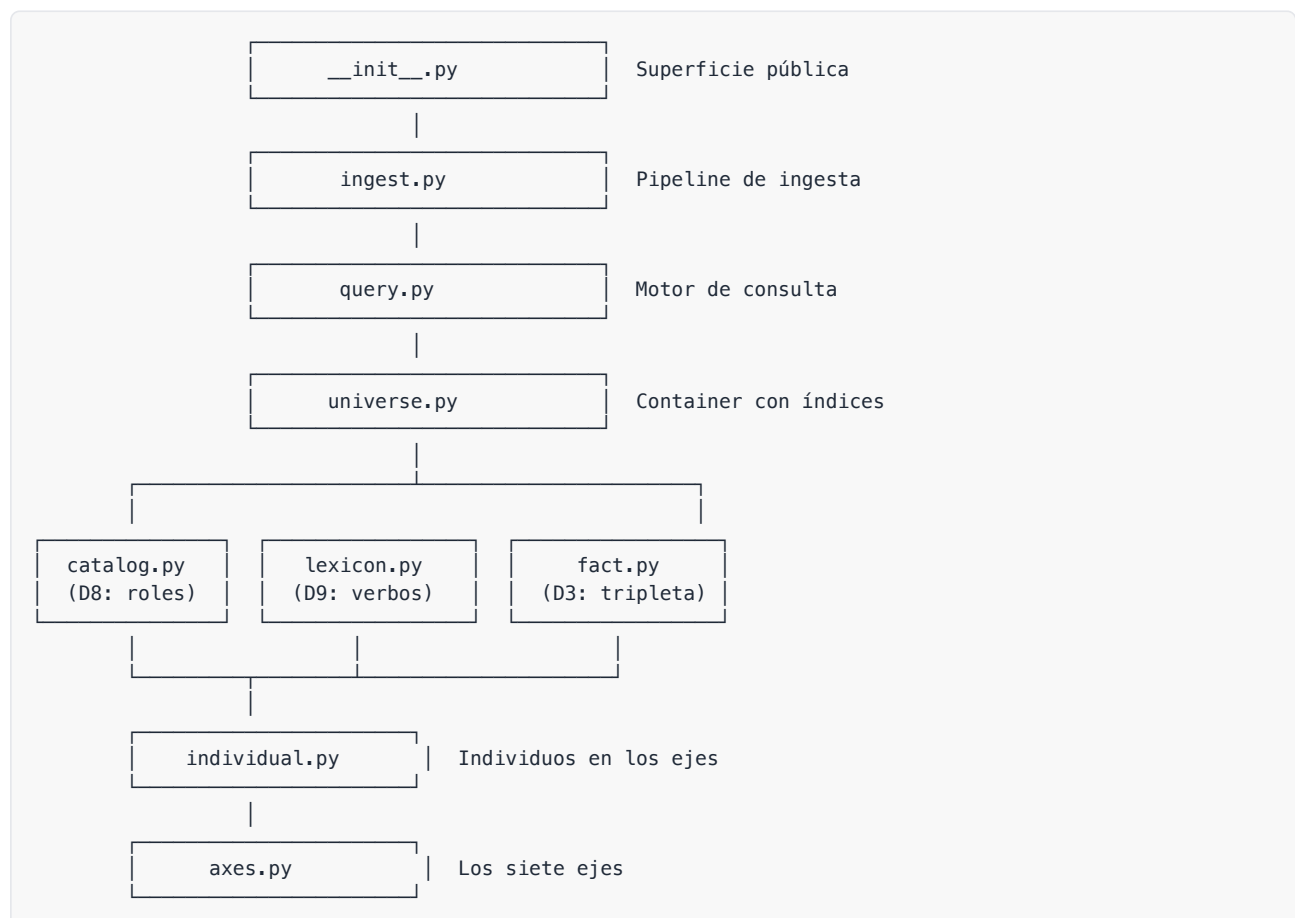
Importante. Este anexo es deliberadamente parcial: incluye **solo la librería núcleo** (`prototipo/wq/`). Los **ejemplos por dominio** (spa, taxi, clínica, banco, dominios previos), los **tests** completos del modelo y los scripts de utilidad **no aparecen acá**, pero existen y son ejecutables. El proyecto completo, con todo lo que la Parte V ejercita, está publicado en el repositorio:

<https://github.com/joseabantomarin/WQuestions>

Quien quiera correr los ocho dominios, ver los tests pasando o extender el modelo encontrará en el repo todo lo que el libro afirma — incluyendo lo que este anexo no muestra.

La arquitectura en una sola mirada

El prototipo se organiza en nueve módulos que se apilan como capas:



El orden de lectura natural va de abajo hacia arriba: primero los ejes (la base), luego cómo se ven los individuos, luego los hechos, luego las dos estructuras transversales (catálogo y lexicon), luego el universo que los contiene, y

finalmente la API de ingesta y consulta. Las nueve secciones de este anexo siguen ese orden.

§A1 — `axes.py` — los siete ejes

Es la pieza fundacional. Declara los seis ejes de valor (Q, O, L, T, N, K) y el único eje estructural de predicados (M) como un `Enum`. Todo el resto del modelo se referencia contra estos identificadores.

```
"""Los 7 ejes de WQuestions.

Seis ejes de valor (contienen individuos):
    Q – quién – agentes
    O – qué – objetos / situaciones reificadas
    L – dónde – lugares
    T – cuándo – momentos / intervalos
    N – cuánto – magnitudes con unidad
    K – cuál – categorías atemporales (tipos, unidades, estados, vocabularios)

Un eje estructural (contiene etiquetas con signature):
    M – cómo – predicados/cables que conectan individuos. Cada predicado
    declara su cardinalidad en la signature ('functional': un solo valor
    por sujeto, o múltiple). La cardinalidad es un atributo del predicado,
    no un eje aparte.
"""

from enum import Enum

class Axis(Enum):
    Q = "Q" # quién
    O = "O" # qué (objectum)
    L = "L" # dónde
    T = "T" # cuándo (tempus)
    N = "N" # cuánto
    K = "K" # cuál (categórico)
    M = "M" # cómo / predicados (modus)
    V = "V" # comodín de signature: cualquier eje de valor

VALUE_AXES = {Axis.Q, Axis.O, Axis.L, Axis.T, Axis.N, Axis.K}
PREDICATE_AXES = {Axis.M}

def is_value_axis(axis: Axis) -> bool:
    return axis in VALUE_AXES
```

§A2 — `individual.py` — individuos en los ejes

81 líneas. Define qué es un *individuo* — la entidad que vive en alguno de los seis ejes de valor — y cómo se construyen los individuos primitivos (un instante temporal `time_point`, una magnitud con unidad `quantity`, una categoría `category`). El `mint_id` genera identificadores cortos y monótonos: en producción se reemplaza por UUID v7.

```

"""Individuos del modelo: instancias de cualquier eje de valor.

Cada individuo tiene un identificador estable y vive en uno y solo un eje.
Para sujetos sintéticos usamos un mint determinista (id corto, monótono)
en vez de UUID – la observabilidad gana, la pérdida de globalidad no
importa para un prototipo.
"""

from __future__ import annotations
from dataclasses import dataclass, field
from typing import Optional, Any, Dict
from itertools import count

from .axes import Axis, is_value_axis

_counter = count(1)

def mint_id(prefix: str = "id") -> str:
    """Genera un id corto y monótono. En producción se reemplazaría por UUID v7."""
    n = next(_counter)
    return f"{prefix}_{n:06d}"

@dataclass(frozen=True)
class Individual:
    """Individuo en un eje de valor.

    `payload` permite alojar valores nativos cuando el "individuo" es en
    realidad un dato (un instante T, una magnitud N, una categoría K). Para
    Q y 0 suele ser None; el id basta.
    """

    id: str
    axis: Axis
    label: Optional[str] = None
    payload: Any = None
    meta: Dict[str, Any] = field(default_factory=dict)

    def __post_init__(self):
        if not is_value_axis(self.axis):
            raise ValueError(
                f"Un individuo debe vivir en un eje de valor, no en {self.axis}."
            )

    def __repr__(self) -> str:
        tag = self.label or self.id
        return f"<{self.axis.value}:{tag}>"

# --- helpers para individuos "primitivos" -----

def time_point(iso: str) -> Individual:
    """Crea un individuo T desde una marca ISO 8601."""
    from datetime import datetime
    try:
        dt = datetime.fromisoformat(iso.replace("Z", "+00:00"))
    except ValueError as e:
        raise ValueError(f"Marca temporal inválida: {iso}") from e
    return Individual(id=f"t_{iso}", axis=Axis.T, label=iso, payload=dt)

def quantity(value: float, unit_k: "Individual") -> Individual:
    """Crea un individuo N con valor numérico y unidad anclada en K."""
    if unit_k.axis != Axis.K:
        raise ValueError(

```

```

        f"La unidad debe vivir en K (recibido: {unit_k.axis})."
    )
    return Individual(
        id=mint_id("n"),
        axis=Axis.N,
        label=f"{value} {unit_k.label or unit_k.id}",
        payload={"value": value, "unit": unit_k.id},
    )

def category(label: str) -> Individual:
    """Crea un individuo K (categoría) con id derivado del label."""
    return Individual(id=label, axis=Axis.K, label=label)

```

§A3 — fact.py — el hecho atómico (D3) con bitemporalidad (D6)

61 líneas. La unidad mínima del modelo. Cada hecho es una tupla inmutable (sujeto, rol, valor) con un rango de vigencia opcional [valid_from, valid_to) y un tx_time para auditoría. El método is_valid_at implementa la lógica de D6: si no hay vigencia, el hecho es atemporal; si la hay, se verifica que el momento solicitado caiga en el intervalo.

```

"""Hechos atómicos: la unidad mínima del modelo.

Cada hecho es una tupla `(sujeito, rol, valor)` donde:
- `sujeito` y `valor` son `Individual` (viven en algún eje de valor).
- `rol` es una etiqueta del catálogo canónico (D8) o de su capa lexicon.

D6 – vigencia temporal: cada hecho lleva opcionalmente un rango
`[valid_from, valid_to)` que indica desde cuándo y hasta cuándo es cierto en
el mundo. Si `valid_to is None`, está abierto al futuro.

Adicionalmente registramos `tx_time` (transaction time): el momento en el que
el hecho entró al sistema. Esto da bitemporalidad ligera, suficiente para
auditoría.
"""

from __future__ import annotations
from dataclasses import dataclass, field
from datetime import datetime, timezone
from typing import Optional

from .individual import Individual

def _utcnow() -> datetime:
    return datetime.now(timezone.utc)

@dataclass(frozen=True)
class Fact:
    subject: Individual
    role: str
    value: Individual
    valid_from: Optional[datetime] = None
    valid_to: Optional[datetime] = None
    tx_time: datetime = field(default_factory=_utcnow)

    def is_valid_at(self, moment: datetime) -> bool:
        """¿Este hecho es cierto en el mundo en `moment`?

        Reglas:
        - Si no hay vigencia, el hecho es atemporal: vale siempre.
        - `valid_from` inclusivo, `valid_to` exclusivo.
        - `valid_to is None` significa "abierto al futuro".
        """

        if self.valid_from is None and self.valid_to is None:
            return True
        if self.valid_from is not None and moment < self.valid_from:
            return False
        if self.valid_to is not None and moment >= self.valid_to:
            return False
        return True

    def __repr__(self) -> str:
        s = self.subject.label or self.subject.id
        v = self.value.label or self.value.id
        base = f"({s}, {self.role}, {v})"
        if self.valid_from or self.valid_to:
            f = self.valid_from.isoformat() if self.valid_from else "-∞"
            t = self.valid_to.isoformat() if self.valid_to else "+∞"
            base += f" [{f} .. {t})"
        return base

```

179 líneas. Es el módulo más largo de la librería núcleo y donde se materializa D8: declara los 38 roles canónicos con su signatura tipada (dominio → rango) y la validación mecánica que verifica que cada hecho respete las signaturas. La política liberal (un rol no declarado se acepta sin validar) está implementada en el método `validate`.


```
"""Catálogo canónico de roles (D8).
```

```
Cada rol declara una signatura tipada `dominio → rango`, ambos ejes de valor,
más su cardinalidad (`functional`: un solo valor por sujeto, o múltiple).
Todos los roles son predicados del eje M (cómo); la cardinalidad es un atributo
de la signatura, no un eje aparte. La signatura habilita la validación mecánica
de hechos: al insertar `(s, role, v)` el catálogo verifica que `s.axis` coincida
con el dominio y `v.axis` con el rango.
"""
```

```
from __future__ import annotations
from dataclasses import dataclass
from typing import Dict, Optional
```

```
from .axes import Axis
from .individual import Individual
```

```
@dataclass(frozen=True)
```

```
class RoleSignature:
```

```
    name: str
    domain: Axis    # eje del sujeto
    range: Axis     # eje del valor
    functional: bool # True → un valor por sujeto, False → múltiple
    description: str = ""
```

```
class SignatureError(ValueError):
    pass
```

```
class Catalog:
```

```
    """Catálogo de signaturas canónicas + validación de hechos."""
```

```
    def __init__(self):
```

```
        self._roles: Dict[str, RoleSignature] = {}
        self._load_canonical()
```

```
    def register(self, sig: RoleSignature) -> None:
```

```
        if sig.name in self._roles:
            existing = self._roles[sig.name]
            if existing != sig:
                raise SignatureError(
                    f"Rol '{sig.name}' ya registrado con signatura distinta: "
                    f"{existing} vs {sig}"
                )
            return
        self._roles[sig.name] = sig
```

```
    def get(self, name: str) -> Optional[RoleSignature]:
```

```
        return self._roles.get(name)
```

```
    def validate(self, role: str, subject: Individual, value: Individual) -> None:
```

```
        """Lanza `SignatureError` si el hecho viola la signatura."""
```

```
        sig = self._roles.get(role)
```

```
        if sig is None:
```

```
            # Rol no declarado: política liberal – se permite, no se valida.
            # Una política estricta lo rechazaría; preferimos extensibilidad.
            return
```

```
        if sig.domain != Axis.V and subject.axis != sig.domain:
```

```
            raise SignatureError(
                f"Sujeto en eje incorrecto para '{role}': se esperaba "
                f"{sig.domain.value}, recibido {subject.axis.value} "
                f"(sujeto={subject})"
            )
```

```
        if sig.range != Axis.V and value.axis != sig.range:
```

```
            raise SignatureError(
                f"Valor en eje incorrecto para '{role}': se esperaba "
```

```

        f"{sig.range.value}, recibido {value.axis.value} "
        f"(valor={value})"
    )

def __contains__(self, name: str) -> bool:
    return name in self._roles

def __len__(self) -> int:
    return len(self._roles)

# -----
# Carga del catálogo canónico (subset del documento WQuestions.md)
# -----

def _load_canonical(self) -> None:
    canonical = [
        # --- estructurales ---
        RoleSignature("instancia_de", Axis.V, Axis.K, False,
            "sujeto (cualquier eje de valor) pertenece a la categoría"),
        RoleSignature("subtipo_de", Axis.K, Axis.K, False,
            "subtipo conceptual"),
        RoleSignature("parte_de", Axis.O, Axis.O, False,
            "subobjeto/subevento de"),
        RoleSignature("contiene", Axis.O, Axis.O, False,
            "inversa de parte_de"),

        # --- participantes (Q es típico) ---
        RoleSignature("agente", Axis.O, Axis.Q, True,
            "agente principal de la situación"),
        RoleSignature("paciente", Axis.O, Axis.Q, True,
            "afectado por la situación"),
        RoleSignature("tema", Axis.O, Axis.O, True,
            "objeto temático (cosa o sub-situación)"),
        RoleSignature("beneficiario", Axis.O, Axis.Q, True,
            "destinatario o beneficiario"),
        RoleSignature("experimentador", Axis.O, Axis.Q, True,
            "quien experimenta un estado mental"),
        RoleSignature("instrumento", Axis.O, Axis.O, True,
            "objeto usado para ejecutar la acción"),
        RoleSignature("comprador", Axis.O, Axis.Q, True,
            "comprador en una venta"),
        RoleSignature("cliente", Axis.O, Axis.Q, True,
            "cliente de un servicio (alias frecuente de agente)"),

        # --- lugar / tiempo ---
        RoleSignature("lugar_de", Axis.O, Axis.L, True,
            "lugar donde ocurre la situación"),
        RoleSignature("origen", Axis.O, Axis.L, True,
            "lugar de origen"),
        RoleSignature("destino", Axis.O, Axis.L, True,
            "lugar de destino"),
        RoleSignature("lugar_destino", Axis.O, Axis.L, True,
            "alias de destino"),
        RoleSignature("momento", Axis.O, Axis.T, True,
            "momento puntual"),
        RoleSignature("inicio", Axis.O, Axis.T, True,
            "instante de inicio"),
        RoleSignature("fin", Axis.O, Axis.T, True,
            "instante de fin"),

        # --- cuantitativos ---
        RoleSignature("monto", Axis.O, Axis.N, True,
            "cantidad numérica con unidad"),
        RoleSignature("cantidad", Axis.O, Axis.N, True,
            "alias de monto"),
        RoleSignature("por_cuanto", Axis.O, Axis.N, True,
            "precio o medida asociada"),
        RoleSignature("unidad", Axis.O, Axis.K, True,
            "unidad de medida (QUDT)"),
    ]

```

```

# --- clasificatorios ---
RoleSignature("estatus_factual", Axis.0, Axis.K, True,
              "real / intencionado / no_realizable / ..."),
RoleSignature("modalidad", Axis.0, Axis.K, True,
              "volitiva / deontica / alética / epistémica"),
RoleSignature("polaridad", Axis.0, Axis.K, True,
              "afirmativa / negativa"),
RoleSignature("calificacion", Axis.0, Axis.K, True,
              "atributo cualitativo"),

# --- "por qué" (D7, capítulo 10) ---
RoleSignature("causado_por", Axis.0, Axis.0, False,
              "causalidad mecánica"),
RoleSignature("motivado_por", Axis.0, Axis.0, False,
              "motivación intencional"),
RoleSignature("con_finalidad", Axis.0, Axis.0, False,
              "propósito"),
RoleSignature("justificado_por", Axis.0, Axis.0, False,
              "regla que autoriza"),

# --- inter-situacionales ---
RoleSignature("precede", Axis.0, Axis.0, False,
              "orden lógico/temporal"),
RoleSignature("sigue_a", Axis.0, Axis.0, False,
              "inversa de precede"),
RoleSignature("cumple", Axis.0, Axis.0, False,
              "cumple una obligación"),
RoleSignature("cancela", Axis.0, Axis.0, False,
              "deja sin efecto"),
RoleSignature("rectifica", Axis.0, Axis.0, False,
              "corrige otra situación"),
RoleSignature("contrasta_con", Axis.0, Axis.0, False,
              "relación adversativa (\\"pero\\")"),

# --- atributos del sujeto Q ---
RoleSignature("nombre", Axis.Q, Axis.K, True,
              "nombre de un agente"),
RoleSignature("identificador", Axis.Q, Axis.K, True,
              "id documental"),
]
for sig in canonical:
    self.register(sig)

```

§A5 — `lexicon.py` — el lexicon con resolución de polisemia (D9)

116 líneas. Implementa la capa de traducción entre el lenguaje natural del usuario y los roles canónicos. Cada `LexiconEntry` declara su `pattern` de complementos (que dispara la polisemia: `dar [la_mano]` vs `dar [conferencia]`) y la resolución elige la entrada **más específica** que coincida. También maneja dialectos de dominio y formas nominales.

Implementa el paradigma del capítulo 12: cada entrada declara una signatura (qué roles obligatorios y opcionales), un tipo de situación en K , y alias naturales por rol y por dominio. La resolución de polisemia elige la entrada más específica que coincide con el patrón de complemento.

```

====

```

@dataclass

```
def specificity(self) -> int:
    """Cuanto más largo el patrón, más específica la entrada."""
    return 0 if self.pattern is None else len(self.pattern)

def matches(self, observed_complements: List[str]) -> bool:
    """¿Esta entrada coincide con los complementos vistos en la oración?

    Una entrada con `pattern=None` siempre coincide (entrada genérica).
    Una entrada con patrón coincide si TODOS sus elementos aparecen
    entre los complementos.
    """
    if self.pattern is None:
        return True
    obs = set(observed_complements)
    return all(p in obs for p in self.pattern)
```

Las entradas se registran por verbo; varias entradas pueden compartir verbo (polisemia). `resolve(verb, complements)` devuelve la entrada más específica que coincide.

```
"""
```

```
# --- registro -----
```

[illegible]

```

        """Agrega un dialecto de dominio: nombre_usuario → rol_canónico."""
        self._domain_aliases.setdefault(domain, {}).update(alias)

# --- resolución -----

def resolve(self, verb: str,
            complements: Optional[List[str]] = None) -> Optional[LexiconEntry]:
    """Devuelve la entrada más específica que coincide con el verbo
    y los complementos observados. None si no hay match.
    """
    candidates = self._by_verb.get(verb, [])
    comps = complements or []
    for entry in candidates: # ya ordenadas más específico → más general
        if entry.matches(comps):
            return entry
    return None

def resolve_nominal(self, nominal_form: str,
                    complements: Optional[List[str]] = None
                    ) -> Optional[LexiconEntry]:
    """Resolución por forma nominal (nominalización).

    *"la llegada del avión" → verbo `llegar` → entrada de llegar.
    """
    verb = self._nominal_index.get(nominal_form)
    if verb is None:
        return None
    return self.resolve(verb, complements)

def translate_alias(self, domain: str, user_term: str) -> Optional[str]:
    """Traduce un término de usuario al rol canónico vía dialecto."""
    return self._domain_aliases.get(domain, {}).get(user_term)

def alias_for_role(self, verb: str, role: str) -> List[str]:
    """Alias declarados para un rol de una entrada específica."""
    entries = self._by_verb.get(verb, [])
    for e in entries:
        if role in e.alias:
            return e.alias[role]
    return []

# --- introspección -----

def verbs(self) -> List[str]:
    return list(self._by_verb.keys())

def entries_for(self, verb: str) -> List[LexiconEntry]:
    return list(self._by_verb.get(verb, []))

```

§A6 — universe.py — el universo V

130 líneas. El container que aloja todos los individuos y todos los hechos, con tres índices (por sujeto, por valor, por rol) para que las consultas no degraden a barrido lineal. Aquí se conecta el catálogo: cada `assert_fact` delega la validación de signatura al catálogo si está inyectado. Los hechos son inmutables; nunca se sobrescriben — los cambios se modelan con vigencia temporal (D6) o con nuevas situaciones.

"""El universo V – unión disjunta de los ejes de valor.

`Universe` es el almacenamiento en memoria del prototipo: una lista de individuos y una lista de hechos, con índices para consulta eficiente.

Diseño:

- Se accede vía `add\_individual`, `assert\_fact`, `query`.
- La validación de signatura ocurre al insertar el hecho (delegada al Catalog).
- Los hechos son inmutables; "cambios" se modelan como nuevas situaciones o como rangos de vigencia (D6).

No persiste a disco – el prototipo es en memoria. Para tests y demos basta.
"""

```
from __future__ import annotations
from dataclasses import dataclass, field
from datetime import datetime
from typing import Dict, List, Optional, Iterator, Tuple, Any

from .axes import Axis
from .individual import Individual
from .fact import Fact

@dataclass
class Universe:
    name: str = "default"
    individuals: Dict[str, Individual] = field(default_factory=dict)
    facts: List[Fact] = field(default_factory=list)
    catalog: Any = None # Catalog inyectado opcionalmente para validación

    # Índices
    _by_subject: Dict[str, List[int]] = field(default_factory=dict)
    _by_value: Dict[str, List[int]] = field(default_factory=dict)
    _by_role: Dict[str, List[int]] = field(default_factory=dict)

    # --- registro de individuos -----

    def add_individual(self, ind: Individual) -> Individual:
        existing = self.individuals.get(ind.id)
        if existing is not None:
            if existing.axis != ind.axis:
                raise ValueError(
                    f"Conflicto de eje para {ind.id}: ya existe en {existing.axis} "
                    f"y se intenta registrar en {ind.axis}."
                )
            return existing
        self.individuals[ind.id] = ind
        return ind

    def ind(self, id_or_label: str) -> Individual:
        """Recupera un individuo por id."""
        if id_or_label in self.individuals:
            return self.individuals[id_or_label]
        raise KeyError(f"Individuo no registrado: {id_or_label}")

    # --- afirmación de hechos -----

    def assert_fact(
        self,
        subject: Individual,
        role: str,
        value: Individual,
        valid_from: Optional[datetime] = None,
        valid_to: Optional[datetime] = None,
    ) -> Fact:
        """Afirma un hecho atómico. Valida la signatura si hay catálogo."""
        self.add_individual(subject)
```

```

self.add_individual(value)

if self.catalog is not None:
    self.catalog.validate(role, subject, value)

fact = Fact(
    subject=subject,
    role=role,
    value=value,
    valid_from=valid_from,
    valid_to=valid_to,
)
idx = len(self.facts)
self.facts.append(fact)
self._by_subject.setdefault(subject.id, []).append(idx)
self._by_value.setdefault(value.id, []).append(idx)
self._by_role.setdefault(role, []).append(idx)
return fact

# --- recuperación -----

def facts_about(self, individual: Individual,
                at: Optional[datetime] = None) -> List[Fact]:
    """Todos los hechos donde `individual` es el sujeto."""
    idxs = self._by_subject.get(individual.id, [])
    result = [self.facts[i] for i in idxs]
    if at is not None:
        result = [f for f in result if f.is_valid_at(at)]
    return result

def facts_with_role(self, role: str,
                    at: Optional[datetime] = None) -> List[Fact]:
    idxs = self._by_role.get(role, [])
    result = [self.facts[i] for i in idxs]
    if at is not None:
        result = [f for f in result if f.is_valid_at(at)]
    return result

def facts_with_value(self, individual: Individual,
                     at: Optional[datetime] = None) -> List[Fact]:
    idxs = self._by_value.get(individual.id, [])
    result = [self.facts[i] for i in idxs]
    if at is not None:
        result = [f for f in result if f.is_valid_at(at)]
    return result

# --- utilidades -----

def __len__(self) -> int:
    return len(self.facts)

def summary(self) -> str:
    n_by_axis: Dict[Axis, int] = {}
    for ind in self.individuals.values():
        n_by_axis[ind.axis] = n_by_axis.get(ind.axis, 0) + 1
    parts = [f"Universe '{self.name}'",
            f" individuos: {len(self.individuals)}"]
    for ax in Axis:
        if ax in n_by_axis:
            parts.append(f"    {ax.value}: {n_by_axis[ax]}")
    parts.append(f" hechos:      {len(self.facts)}")
    return "\n".join(parts)

```

96 líneas. Toma un verbo y un diccionario de roles ya identificados (el parser lingüístico se asume externo: lo hace un LLM, o un parser dedicado) y aplica un pipeline de seis pasos para producir una situación reificada (D4) con todos sus hechos atómicos. Es la función que más se usa en los ejemplos de la Parte V del libro.


```
"""Ingesta: traduce una "oración" (estructurada) a hechos en el universo.
```

Para el prototipo no parsear español real – eso es trabajo del LLM.
En su lugar exponemos una API de ingesta que recibe verbo + roles ya identificados y aplica el pipeline:

1. Resolver lexicon (con polisemia / forma nominal)
2. Reificar la situación en 0
3. Asentar instancia_de = tipo_situacion
4. Asentar cada rol como hecho atómico
5. Validar contra el catálogo D8
6. Verificar obligatorios

Devuelve la situación reificada para que el caller pueda enriquecerla.

```
"""
```

```
from __future__ import annotations
from datetime import datetime
from typing import Dict, Optional, Any, List

from .axes import Axis
from .individual import Individual, mint_id, category
from .universe import Universe
from .lexicon import Lexicon, LexiconEntry
```

```
class IngestError(ValueError):
    pass
```

```
def ingest_situation(
    universe: Universe,
    lexicon: Lexicon,
    verb: str,
    roles: Dict[str, Individual],
    *,
    complements: Optional[List[str]] = None,
    nominal: bool = False,
    valid_from: Optional[datetime] = None,
    valid_to: Optional[datetime] = None,
    extra: Optional[Dict[str, Individual]] = None,
    sit_id: Optional[str] = None,
) -> Individual:
    """Ingesta una situación a partir de un verbo y sus roles.
```

`roles` mapea NOMBRE_CANÓNICO → Individual (el caller ya resolvió aliases).
`complements` lista patrones de complemento que disparan polisemia (p. ej. ['la_mano'] para `dar la mano`).
`nominal=True` indica que el "verbo" es en realidad una forma nominal ("llegada") y debe buscarse vía `resolve\_nominal`.
`extra` agrega hechos adicionales que no están en la signatura (p. ej. `modalidad`, `estatus\_factual`, `calificacion`, ...).

Devuelve la situación reificada en 0.

```
"""
```

```
entry = (
    lexicon.resolve_nominal(verb, complements)
    if nominal else
    lexicon.resolve(verb, complements)
)
if entry is None:
    raise IngestError(
        f"Lexicon no tiene entrada para '{verb}' "
        f"con complementos {complements}."
    )

# Verificar obligatorios
missing = [r for r in entry.obligatory if r not in roles]
if missing:
```

```

    raise IngestError(
        f"Faltan roles obligatorios para '{verb}': {missing}"
    )

    # Reificar la situación
    sid = sit_id or mint_id(entry.situation_type)
    situ = Individual(id=sid, axis=Axis.0, label=sid)
    universe.add_individual(situ)

    # instancia_de
    tipo = category(entry.situation_type)
    universe.assert_fact(situ, "instancia_de", tipo,
                        valid_from=valid_from, valid_to=valid_to)

    # Roles obligatorios y opcionales
    for role, value in roles.items():
        # No exigimos que esté declarado en la entrada – la entrada es
        # informativa; los extras se admiten. (Política liberal.)
        universe.assert_fact(situ, role, value,
                            valid_from=valid_from, valid_to=valid_to)

    # Extras (modalidad, calificación, ...)
    if extra:
        for role, value in extra.items():
            universe.assert_fact(situ, role, value,
                                valid_from=valid_from, valid_to=valid_to)

    return situ

```

§A8 — `query.py` — el motor de consulta

124 líneas. Implementa las preguntas-WH como proyecciones sobre hechos. Un `Pattern` declara roles `fixed` (con valores conocidos) y roles `ask` (con `Var` — lo que se quiere descubrir). Soporta filtros por tipo (`type_constraint`) y consultas bitemporales (parámetro `at`). La función `count` es el caso particular de `query` que solo devuelve la cardinalidad.

```
"""Motor de consulta: las preguntas-WH como proyecciones.
```

Una consulta es un ``Pattern``: un diccionario de roles fijos con valores conocidos, y al menos un rol marcado como ``Var(...)`` (la pregunta). El motor busca todas las situaciones del universo que satisfacen los roles fijos y proyecta el valor del rol pregunta.

Soporta:

- Consultas puntuales: ¿quién vendió X?
- Consultas temporales: ¿quién era el dueño de X en T0? (D6)
- Filtros por tipo (`instancia_de = K`).
- Agregaciones: `count`, `list`.

```
"""
```

```
from __future__ import annotations
from dataclasses import dataclass, field
from datetime import datetime
from typing import Dict, List, Optional, Any
```

```
from .individual import Individual
from .universe import Universe
```

```
@dataclass
```

```
class Var:
    """Marcador de variable en un patrón de consulta."""
    name: str = "?"
```

```
@dataclass
```

```
class Pattern:
    """Patrón de consulta sobre una situación.

    `fixed`: roles cuyo valor está dado (Individual).
    `ask`: uno o más roles cuyo valor queremos descubrir (Var).
    """
    fixed: Dict[str, Individual] = field(default_factory=dict)
    ask: Dict[str, Var] = field(default_factory=dict)
    type_constraint: Optional[Individual] = None # filtra por instancia_de = K

    def __post_init__(self):
        # `ask` vacío es válido: cuenta/lista candidatos sin proyección.
        # En ese caso el binding contiene solo `_subject`.
        pass
```

```
def query(universe: Universe, pattern: Pattern,
          at: Optional[datetime] = None) -> List[Dict[str, Any]]:
    """Ejecuta el patrón contra el universo. Devuelve una lista de bindings.
```

Cada binding es un dict con las claves de ``pattern.ask`` y los valores encontrados (Individual). Las situaciones-candidatas son los sujetos que tienen **todos** los roles fijos del patrón (con sus valores) y, si aplica, `instancia_de = type_constraint`.

Punto 1: buscar candidatas – sujetos en 0 que tienen todos los roles del patrón. Tomamos el primer rol fijo (o `type_constraint`) como ancla para reducir el espacio.

```
if pattern.type_constraint is not None:
    # Sujetos cuya instancia_de == type_constraint
    candidate_subjects = {
        f.subject.id
        for f in universe.facts_with_role("instancia_de", at=at)
        if f.value.id == pattern.type_constraint.id
    }
elif pattern.fixed:
    # Tomamos el primer fixed como ancla.
```

```

    role0, val0 = next(iter(pattern.fixed.items()))
    candidate_subjects = {
        f.subject.id
        for f in universe.facts_with_role(role0, at=at)
        if f.value.id == val0.id
    }
else:
    # Si solo hay ask, buscamos sobre todas las situaciones (raro).
    candidate_subjects = set(universe.individuals.keys())

# Punto 2: filtrar candidatas por todos los roles fijos
results: List[Dict[str, Any]] = []
for sid in candidate_subjects:
    subject = universe.individuals[sid]
    sit_facts = universe.facts_about(subject, at=at)

    # Map role → list of values for this subject
    roles_map: Dict[str, List[Individual]] = {}
    for f in sit_facts:
        roles_map.setdefault(f.role, []).append(f.value)

    # Chequear roles fijos
    ok = True
    for role, expected_val in pattern.fixed.items():
        vals = roles_map.get(role, [])
        if not any(v.id == expected_val.id for v in vals):
            ok = False
            break
    if not ok:
        continue

    # Chequear type_constraint si está
    if pattern.type_constraint is not None:
        instancia_vals = roles_map.get("instancia_de", [])
        if not any(v.id == pattern.type_constraint.id for v in instancia_vals):
            continue

    # Extraer valores para los roles preguntados
    binding: Dict[str, Any] = {"_subject": subject}
    all_present = True
    for ask_role in pattern.ask:
        vals = roles_map.get(ask_role, [])
        if not vals:
            all_present = False
            break
        # Si hay más de uno, devolvemos lista; si uno, el individuo.
        binding[ask_role] = vals[0] if len(vals) == 1 else list(vals)
    if all_present:
        results.append(binding)
return results

def count(universe: Universe, pattern: Pattern,
          at: Optional[datetime] = None) -> int:
    """Cuenta sujetos que satisfacen el patrón."""
    return len(query(universe, pattern, at=at))

```

§A9 — `__init__.py` — la superficie pública

32 líneas. Lo último que ve quien importa la librería. Reexporta todas las clases y funciones que el usuario va a usar, con un docstring que recapitula qué hace cada pieza. Si alguien escribe `from wq import *`, esto es lo que recibe.

```

"""WQuestions – prototipo de validación en Python.

Implementa el modelo descrito en `WQuestions.md` y discutido en el libro:
- 7 ejes (Q, O, L, T, N, K, M)
- hechos atómicos con signatura tipada
- situaciones reificadas (D4, D5)
- catálogo canónico de roles (D8)
- lexicon con resolución de polisemia (D9)
- vigencia temporal por reificación (D6)
- consultas como proyecciones sobre roles (preguntas-WH)

Se prioriza claridad sobre rendimiento. La meta es validar la arquitectura,
no servir producción.
"""

from .axes import Axis
from .individual import Individual, mint_id, category, quantity, time_point
from .fact import Fact
from .universe import Universe
from .catalog import Catalog, RoleSignature, SignatureError
from .lexicon import Lexicon, LexiconEntry
from .query import Pattern, Var, query, count
from .ingest import ingest_situation, IngestError

__all__ = [
    "Axis", "Individual", "mint_id", "category", "quantity", "time_point",
    "Fact", "Universe",
    "Catalog", "RoleSignature", "SignatureError",
    "Lexicon", "LexiconEntry",
    "Pattern", "Var", "query", "count",
    "ingest_situation", "IngestError",
]

```

Observaciones finales

Si se mira el código completo de la librería en una sola pasada, aparecen seis observaciones que vale la pena nombrar:

1. **Pocas líneas, pocos archivos, pocos conceptos.** Nueve archivos, ~850 líneas, una sola clase principal por archivo. No hay infraestructura inútil, no hay abstracciones especulativas. Cada pieza existe porque el libro la justifica con un capítulo entero.
2. **Las decisiones de diseño se ven en el código.** D3 vive en `fact.py` (la tripleta), D4 vive en `ingest.py` (el reificador), D6 vive en `is_valid_at` (la vigencia), D8 vive en `catalog.py` (las signaturas), D9 vive en `lexicon.py` (los alias). El paralelismo es directo: una decisión, un módulo.
3. **La política liberal es explícita.** En `catalog.validate`, si el rol no está declarado, *no se valida*. Cuatro líneas de código que materializan la decisión de mantener el modelo extensible sin pelearse con el catálogo central.
4. **La bitemporalidad es ligera.** No hay un sistema bitemporal completo (eso falta para producción — está documentado en el capítulo 28). Lo que hay es un `valid_from / valid_to` opcional y un `tx_time` automático: 5 atributos extra sobre `Fact`, una función `is_valid_at` que decide la vigencia, y un parámetro `at=` en todas las consultas. Con eso ya se modelan mudanzas, rediagnósticos, contratos con cláusulas que expiran.
5. **No hay parser de lenguaje natural.** El prototipo asume que el parser es externo — un LLM, o un módulo dedicado que se conecta vía `ingest_situation`. Esto es deliberado: separar la responsabilidad de "entender la oración" (lingüística) de "modelar el hecho" (arquitectura) es lo que permite reutilizar el motor en cualquier idioma y con cualquier parser.

6. **El motor de consulta es ingenuo.** Un barrido sobre el universo con tres índices simples. Sirve para validar el modelo en dominios reales; no sirve para producción a escala. El capítulo 28 enumera los reemplazos productivos: Datalog, SHACL, motores RDF, bases de datos columnar.

Estas seis observaciones, juntas, son la justificación del enfoque del prototipo: **no construir infraestructura; construir prueba de concepto.** Lo que estas 850 líneas demuestran es que el modelo es *coherente* y *operable*. La validación industrial es otro trabajo — el del capítulo 28 y de quien lo continúe.

Lo que queda en el repositorio

Para mantener este anexo manejable solo incluí la librería núcleo. El resto del prototipo — aproximadamente **2.400 líneas adicionales** — está publicado completo en el repositorio del proyecto. Acá va el mapa de lo que vas a encontrar:

```
github.com/joseabantomarin/WQuestions
├── prototipo/
│   ├── wq/                                ← incluida en este anexo (850 líneas)
│   │   ├── axes.py
│   │   ├── individual.py
│   │   ├── fact.py
│   │   ├── catalog.py
│   │   ├── lexicon.py
│   │   ├── universe.py
│   │   ├── ingest.py
│   │   ├── query.py
│   │   └── __init__.py
│   ├── ejemplos/                          ← no incluida – está en el repo
│   │   ├── spa.py                         (532 líneas)
│   │   ├── dominios_previos.py            (522 líneas – receta, gol, canción, noticia)
│   │   ├── banco.py                      (465 líneas)
│   │   ├── clinica.py                    (312 líneas)
│   │   └── taxi.py                       (256 líneas)
│   └── tests/                             ← no incluida – está en el repo
│       └── test_wq.py                     (349 líneas)
```

Cada archivo de `ejemplos/` es un script ejecutable que modela un dominio completo: declara los individuos, registra el lexicon, ingresa todas las situaciones, corre las consultas que el libro discute y valida los resultados. Ejecutar uno cualquiera (por ejemplo `python -m prototipo.ejemplos.banco`) reproduce las escenas de la Parte V tal cual aparecen en el libro.

El archivo `test_wq.py` ejercita las invariantes del modelo: que el catálogo valide firmas, que las consultas bitemporales devuelvan lo correcto en distintos momentos, que la polisemia del lexicon elija la entrada más específica, que la reificación produzca el grafo esperado. Es el cinturón de seguridad que el libro promete cuando dice "todos los tests pasan".

Cómo correr el prototipo en cinco minutos:

```
git clone https://github.com/joseabantomarin/WQuestions.git
cd WQuestions
python -m prototipo.tests.test_wq      # corre la batería de tests
python -m prototipo.ejemplos.spa       # ejecuta el dominio del Spa
python -m prototipo.ejemplos.banco     # ejecuta el dominio bancario
```

No hace falta instalar dependencias: el prototipo solo usa la librería estándar de Python. Si lo abres en un editor moderno, los identificadores y las APIs (`Universe`, `ingest_situation`, `Pattern`, `query`) son los mismos que

vienen en este anexo, así que la transición entre leer y ejecutar es instantánea.

El prototipo, los ejemplos y los tests son **el espejo operable del libro**. Lo que se afirma en cualquier capítulo de la Parte V se puede ejecutar línea por línea en el repo. Si encuentras una afirmación del libro que no puedas reproducir, el repo tiene un issue tracker abierto — y es exactamente ese tipo de feedback el que el capítulo 28 reclama para que la propuesta deje de ser un texto y se vuelva infraestructura.

El autor

José Roberto Abanto Marín (Celendín, Perú, 1973).

Desarrollador de software con treinta y cinco años de experiencia construyendo sistemas a la medida. Es el creador de *Ghenesis*, un framework meta-driven que persiste la lógica de negocio en base de datos —formularios, reportes, módulos— para permitir su administración sin tocar el código de la aplicación. Esa obsesión por separar el qué del cómo, por buscar la capa donde la información deja de depender de la aplicación que la consulta, atraviesa toda su trayectoria profesional y desemboca naturalmente en este libro.

Apasionado de las bases de datos, los modelos de inteligencia artificial y la ciencia en general, encuentra en la física cuántica y en los enigmas del universo una forma de mantener viva la sospecha de que todo lo aparentemente complejo esconde, en algún nivel, una estructura más simple. La filosofía es para él una herramienta de trabajo: una manera de detectar los patrones, las abstracciones y los paradigmas que se asoman en cada problema técnico.

WQuestions, Las preguntas como coordenadas, es el primer libro en el que reúne esas tres líneas —la práctica de tres décadas modelando datos, la curiosidad científica y la mirada filosófica— alrededor de una sola tesis arquitectónica.